Sample Path Analysis and Simulation of Stochastic Systems

Nicky D. van Foreest

November 6, 2023

This work is licensed by the University of Groningen under a Creative Commons Attribution-ShareAlike 4.0 International License.

CONTENTS

1 Introduction 1
- 1.1 Motivation and overview 1
- 1.2 Exponential and Poisson Distribution 3
- 1.3 Fighting the Exponential Distribution is Futile 7

2 Construction of Simple Discrete time Stochastic Processes 13
- 2.1 Waiting for a Psychiatrist 13
- 2.2 Simulating the Psychiatrists Case 19
- 2.3 Control of Single-item Inventory Systems 23
- 2.4 Simulating Inventory Systems 28
- 2.5 General Behavior of Queueing Systems 30

3 Construction of Simple Continous time Stochastic Processes 39
- 3.1 Queueing Process in Continuous Time 39
- 3.2 Simulations in continuous time 42
- 3.3 Probabilistic Arithmetic 46
- 3.4 Discrete Event Simulations 56

4 Fundamental tools 67
- 4.1 Kendall's Notation 67
- 4.2 Rate, Stability and Load 68
- 4.3 (Limits of) Empirical Performance Measures 71
- 4.4 Renewal Reward Theorem 72
- 4.5 Little's Law 74
- 4.6 Poisson Arrivals See Time Averages 76
- 4.7 Level Crossing and Balance Equations 79
- 4.8 Graphical Summaries 84

5 Exact Models 86
- 5.1 $M/M/1$ queue and its variations 86
- 5.2 Applications of Level-crossing, Little's law and PASTA 90
- 5.3 $M^B/M/1$ Queue: Expected Waiting Time 93
- 5.4 $M/G/1$ Queue: Expected Waiting Time 96
- 5.5 $M^B/M/1$ Queue Length Distribution 97
- 5.6 N-policies for the $M/G/1$ queue 100
- 5.7 Inventory control, analytic results 104

6 Approximate Models 108
- 6.1 $G/G/c$ Queue: Sakasegawa's Formula 109
- 6.2 Server Setups 110
- 6.3 Server Adjustments 113
- 6.4 Server Failures 114
- 6.5 $G/G/c$ Queues in Tandem 116

7 Hints 117

8 Solutions 121

Bibliography 152

Notation 154

1 INTRODUCTION

In the first section of this chapter we provide some high level motivation why to study queueing systems and an overview of the book. In the second we recall some notation and results of analysis and probability that we use throughout the book.

1.1 MOTIVATION AND OVERVIEW

Queueing systems abound, and the analysis and control of queueing systems are major topics in the control, performance evaluation and optimization of production and service systems.

AT MY LOCAL SUPERMARKET, for instance, any customer gets his/her groceries for free when there is a queue of 4 or more customers and there is an unoccupied cashier desk. The manager that controls the occupation of the cashier positions is keen on keeping the queue short. Now this is easy enough: just hire many cashiers. However, the cost of personnel may then outweigh the yearly average cost of paying the customer penalties. Thus, the manager's problem is to control the service capacity such the penalties and the personnel cost are about equally large.

Fast food restaurants also deal with many interesting queueing situations. Consider, for instance, the preparation of hamburgers. Typically, hamburgers are made-to-stock, in other words, they are prepared before the actual demand has arrived. Thus, hamburgers in stock can be interpreted as customers in queue waiting for service, where the service time is the time between the arrival of two customers that buy hamburgers. The hamburgers have a typical lifetime, and they have to be scrapped if they remain on the shelf longer than a specified amount of time. Of course, it is easy to achieve zero scrap cost, simply by keeping no stock at all. However, to prevent lost-sales, it is important to maintain a certain amount of hamburgers in stock. In this case, the manager has to balance the scrap cost against the cost of lost sales.

Service systems, such as hospitals, call centers, courts, and so on, have a certain capacity available to serve customers. The performance of such systems is, in part, measured by the total number of jobs processed per year and the fraction of jobs processed within a certain time frame between receiving and closing the job. Here the problem is to organize the capacity such that the sojourn time, i.e., the typical time a job spends in the system, does not exceed some threshold.

Clearly, in all these examples, the performance of the queueing system has to be monitored and controlled. Typically the following performance measures are relevant.

1. The fraction of time $p(n)$ that the system contains n customers. In particular, $1 - p(0)$, i.e., the fraction of time the system contains jobs,

is important, as this is a measure of the time-average occupancy of the servers, hence related to personnel cost.

2. The fraction of customers $\pi(n)$ that 'see upon arrival' the system with n customers. This measure relates to customer perception and lost sales, i.e., fractions of arriving customers that do not enter the system.

3. The average, variance, and/or distribution of the waiting time.

4. The average, variance, and/or distribution of the number of customers in the system.

In this book, we will be primarily concerned with making models of queueing systems such that we can compute or estimate these performance measures.

CHAPTER 2 STARTS WITH constructing queueing and inventory systems in discrete time, both examples of stochastic processes. This serves three goals. First, construction is concrete, so that by specifying the rules to characterize the behavior of the system, you (the reader) develop essential modeling skills. Second, these rules can often be easily implemented in computer code and used to simulate and control actual stochastic systems. Simulation is in general the best way to analyze practical stochastic systems, as realistic systems seldom yield to mathematics. Third, we will use sample-path arguments to develop the theoretical results of the rest of the book, and these sample paths are precisely what simulators produce.

In Chapter 3 we move on to systems in continuous time and show how to set up simulation environments to construct sample paths. Once such environment is a discete event simulation. Nearly all professional simulation tools are built on the same principles.

Once it is clear what queueing and inventory theory is about, the stage is set for a more mathematical treatment of such systems. In Chapter 4 we develop some necessary key results. For this, we use again sample paths of stochastic processes, and by assume that such sample paths capture the 'normal' stochastic behavior, we can use the sample paths to estimate the most important performance measures to analyze how the system performs.

In Chapter 5 we use these tools to develop exact models for single-station queueing systems. In our discussions we mostly focus on obtaining an intuitive understanding of the analytical tools.

For most of the proofs and/or more extensive results we refer to the bibliography at the end of the book.

Notwithstanding the power of simulation, it is often hard to obtain structural understanding of the behavior of queueing systems. Instead, mathematical models, whether exact or approximate, are useful to help reason about and improve queueing systems. In Chapter 6 we use approximations and general results of probability theory to understand how production and service situations are affected by the system parameters such as service speed, batching rules, and outages.

WHILE THE MAIN text contains many examples and derivations, a considerable number of examples are delegated to exercises. Also, some of these exercises are consistency checks between results derived for different queueing models, and thereby provide important relations between various parts of the text. We note in passing that, while such checks are trivial in principle, the algebra can be quite difficult at times. The exercises are not meant to be really easy; they should require (some) work. Hints and solutions to all problems are available at the end of the book.

1.2 EXPONENTIAL AND POISSON DISTRIBUTION

A queueing system contains one or more servers to serve jobs or customers that arrive to the system. In this section we make a model for the arrival process that is the most useful for queueing systems. In particular, we show how the exponential and gamma distribution relate to the Poisson distribution, and we use the memoryless property of job inter-arrival times to motivate these distributions.

THE TIMESCALES that are relevant for queueing systems range roughly from minutes to a week[0]. On such timescales it seems is reasonable to model inter-arrival times of jobs as memoryless. For instance, if X is the inter-arrival time between two patients with a broken arm at the emergency room of a hospital, what can we say about the time the next patient with a broken arm arrives if somebody tells us that an hour earlier a similar patient arrived? Not much, as most of us will agree. Similarly, suppose that the average time between the sales of some shirt in a clothes shop is 5 minutes, and the shop owner informs us that no shirt has been sold the past 7 minutes. Does that knowledge imply that the next shirt must be sold within the next minute? Surely not.

[0] *With exceptions, naturally.*

In more formal terms we claim that the probability that the next arrival will not occur within t time units from now does not depend on somebody telling us that there wasn't an arrival in the past s time units. In other words, if the person does not tell us anything, we write the probability that no arrival occurs before t as $\mathsf{P}\{X > t\}$. However, if the person tells us at time s there was no arrival during $[0, s]$, we write that the probability that there will not be an arrival for an additional amount time t as $\mathsf{P}\{X > s + t | X > s\}$. And if we believe that this information is immaterial to our probabilistic model, then we must have that

$$\mathsf{P}\{X > s + t \,|\, X > s\} = \mathsf{P}\{X > t\}. \tag{1.2.1}$$

A rv X whose cdf (cumulative distribution function) satisfies this property is said to be *memoryless*.

It is simple to see that when $X \sim \text{Exp}(\lambda)$, i.e., X is exponentially distributed, it is memoryless. If fact, as

$$\mathsf{P}\{X \le t\} = 1 - e^{-\lambda t},$$

$\mathsf{P}\{X > t\} = e^{-\lambda t}$. Now fill this in (1.2.1), use the standard formula for conditional probability, and note that $e^{-\lambda t+s}/e^{-\lambda s} = e^{-\lambda t}$. Notably, the reverse is

also true: when a continuous memoryless rv is memoryless it is necessarily exponential[1]. As a counter example, the life span of human beings is not memoryless: take X as the life span of an arbitrary person born in 1880, and $s = 100$ and $t = 99$ years. Then $\mathsf{P}\{X > 99\}$ æ was small, but not zero, but $\mathsf{P}\{X > 199 \mid X > 100\} = 0$.

[1] *For a proof, see Yushkevich and Dynkin [1969, Appendix 3]*

Note that for $X \sim \text{Exp}(\lambda)$, the following properties hold:[2]

[2] *[1.2.1]*

$$\mathsf{E}[X] = \lambda^{-1}, \qquad \mathsf{V}[X] = \lambda^{-2}.$$

If we have a sequence of inter-arrival times $\{X_i\}_{i=1}^{\infty}$, then we construct job[3] arrival times according to the rule $A_k = A_{k-1} + X_k$, starting at $A_0 = 0$. When inter-arrival times are iid (independent and identically distributed) rvs (random variables) and distributed as the common rv $X \sim \text{Exp}(\lambda)$, it is clear that the density of the first arrival time is

[3] *or customer, patient*

$$f_{A_1}(t) = f_X(t) = \lambda e^{-\lambda t} = \lambda \frac{(\lambda t)^0}{0!} e^{-\lambda t},$$

where we add the last term to see how a pattern emerges. Using convolution, for the second arrival time,

$$f_{A_2}(t) = \int_0^t f_{A_1}(s) f_X(t-s)\,\mathrm{d}s = \int_0^t \lambda e^{-\lambda s} \lambda e^{-\lambda(t-s)}\,\mathrm{d}s = \lambda \frac{(\lambda t)^1}{1!} e^{-\lambda t}.$$

With induction is now straightforward to extend the pattern and prove that A_k has a gamma distribution, i.e.,

$$f_{A_k}(t) = \int_0^t f_{A_{k-1}}(s) f_X(t-s)\,\mathrm{d}s = \lambda \frac{(\lambda t)^{k-1}}{(k-1)!} e^{-\lambda t}. \tag{1.2.2}$$

WHEN THE ARRIVAL times are gamma distributed, the cdf of the number $N(t)$ of jobs arriving during $[0, t]$ follows readily by observing that

$$\mathsf{P}\{N(t) = k\} = \mathsf{P}\{A_k \le t\} - \mathsf{P}\{A_{k+1} \le t\}, \tag{1.2.3}$$

that is, to have precisely k arrivals during $[0, t]$, the arrival time A_k of job k must lie in $[0, t]$, but job $k+1$ must not have yet arrived. Using the cdf of A_k and partial integration,

$$\begin{aligned}\mathsf{P}\{A_{k+1} \le t\} &= \lambda \int_0^t \frac{(\lambda s)^k}{k!} e^{-\lambda s}\,\mathrm{d}s = \lambda \frac{(\lambda s)^k}{k!} \frac{e^{-\lambda s}}{-\lambda}\Big|_0^t + \lambda \int_0^t \frac{(\lambda s)^{k-1}}{(k-1)!} e^{-\lambda s}\,\mathrm{d}s \\ &= -\frac{(\lambda t)^k}{k!} e^{-\lambda t} + \mathsf{P}\{A_k \le t\}.\end{aligned}$$

Combining this with the previous expression we see that

$$\mathsf{P}\{N(t) = k\} = \frac{(\lambda t)^k}{k!} e^{-\lambda t}, \tag{1.2.4}$$

that is, $N(t) \sim \text{Pois}(\lambda t)$, i.e., *Poisson distributed* with parameter λt.

It is simple to see from (1.2.4) that[4]

[4] *[1.2.3]*

$$\mathsf{E}[N(t)] = \lambda t \qquad \mathsf{V}[N(t)] = \lambda t.$$

Interestingly, we can use (1.2.3) to derive the pdf of A_k from the Poisson distribution. For $k = 0$, $\mathsf{P}\{N(t) = 0\} = e^{-\lambda t}$, and as $A_0 = 0$ by definition, the

RHS of (1.2.3) becomes $1 - \mathsf{P}\{A_1 \le t\}$. Consequently, $\mathsf{P}\{A_1 \le t\} = 1 - e^{-\lambda t}$, hence, by differentiation, $f_{A_1}(t) = \lambda e^{-\lambda t}$. For $k \ge 1$, taking the derivative with respect to t of the RHS of (1.2.4) and at both sides of (1.2.3),

$$\lambda \frac{(\lambda t)^{k-1}}{(k-1)!} e^{-\lambda t} - \lambda \frac{(\lambda t)^{k-1}}{(k-1)!} e^{-\lambda t} = f_{A_k}(t) - f_{A_{k+1}}(t).$$

With recursion, we retrieve (1.2.2).

Thus, if you find it reasonable to model inter-arrival times as memoryless rvs, then the number of arrivals in an interval is necessarily Poisson distributed. And, if you find it reasonable that the occurrence of an event in a small time interval is constant over time and independent from one interval to another, then the arrival process is Poisson, and the inter-arrival times are exponential.

THE FAMILY of random variables $N_\lambda = \{N(t), t \ge 0\}$ is a *Poisson process* with *arrival rate* λ. (We note that a *random process* is a much more complicated mathematical object than a random variable: a process is a (possibly uncountable) set of random variables indexed by time, not just *one* random variable.) Once we have $N(t)$, we can define $N(s,t] := N(t) - N(s)$ as the number of customers that arrive in a general time period $(s,t]$.[5]

[5] *Note that $[0,t]$ is closed at both ends, but $(s,t]$ is open at the left.*

It is important to know that N_λ has *stationary and independent increments*. Stationarity means that the distributions of the number of arrivals are the same for all intervals of equal length, that is, $N(s,t]$ has the same probability distribution as $N(u,v]$ if $t - s = v - u$. Independence means, roughly speaking, that knowing that $N(s,t] = n$, does not help to make any predictions about the value of $N(u,v]$ if the intervals $(s,t]$ and $(u,v]$ do not overlap. (1.2.4) implies a number of useful properties of the Poisson process. Next, with small o notation[6], if $t \ll 1$,

[6] *A function $f(h) = o(h)$ when $f(h)/h \to 0$ as $h \to 0$, e.g., $x^2 = o(x)$.*

$$\mathsf{P}\{N(t) = 0\} = e^{-\lambda t} = 1 - \lambda t + o(t), \tag{1.2.5a}$$

$$\mathsf{P}\{N(t) = 1\} = \lambda t e^{-\lambda t} = \lambda t(1 - \lambda t + o(t)) = \lambda t + o(t), \tag{1.2.5b}$$

$$\mathsf{P}\{N(t) \ge 2\} = 1 - \mathsf{P}\{N(t) = 0\} - \mathsf{P}\{N(t) = 1\} \tag{1.2.5c}$$

$$= 1 - (1 - \lambda t) - \lambda t + o(t) = o(t). \tag{1.2.5d}$$

It is also important to know that the reverse of the above holds, that is, a stochastic process with stationary and independent increments and satisfying (1.2.5) is necessarily a Poisson process.[7]

[7] *We refer to the literature on (mathematical) probability theory for further background.*

WHEN WE MERGE two independent Poisson processes N_λ and N_μ, e.g., adults and children entering a shop[8], we obtain a new Poisson process $N_{\lambda+\mu}$ with rate $\lambda + \mu$. To see this, we use the above. For $t \ll 1$,

[8] *Or some other feature by which you want to distinguish between customers.*

$$\begin{aligned}
\mathsf{P}\left\{N_{\lambda+\mu}(t) = 0\right\} &= \mathsf{P}\{N_\lambda(t) = 0\}\, \mathsf{P}\left\{N_\mu(t) = 0\right\} \\
&= (1 - \lambda t)(1 - \mu t) + o(t) = 1 - (\lambda + \mu)t + o(t), \\
\mathsf{P}\left\{N_{\lambda+\mu}(t) = 1\right\} &= \mathsf{P}\{N_\lambda(t) = 1\}\, \mathsf{P}\left\{N_\mu(t) = 0\right\} + \mathsf{P}\{N_\lambda(t) = 0\}\, \mathsf{P}\left\{N_\mu(t) = 1\right\} \\
&= \lambda t(1 - \mu t) + (1 - \lambda t)\mu t + o(t) = (\lambda + \mu)t + o(t),
\end{aligned}$$

and the third relations in (1.2.5) follow similarly. Moreover, as N_λ and N_μ have stationary and independent increments, $N_{\lambda+\mu}$ has these properties too.

We can also *split*, or *thin*, a stream into several sub-streams. For instance, consider a stream of people passing by a shop as a Poisson process N_λ, and suppose that a customer decides to enter the shop as a Bernoulli distributed rv B, independent of N_λ, with success probability p and failure probability $q = 1 - p$. Then the process of customers entering the shop is a Poisson process $N_{\lambda p}$ with rate λp, because, from (1.2.5), for $t \ll 1$,

$$\begin{aligned}
\mathsf{P}\left\{N_{\lambda p}(t) = 0\right\} &= \mathsf{P}\{N_\lambda(t) = 0\} + \mathsf{P}\{N_\lambda(t) = 1\}\,\mathsf{P}\{B = 0\} \\
&= 1 - \lambda t + \lambda t q + o(t) = 1 - \lambda p t + o(t), \\
\mathsf{P}\left\{N_{\lambda p}(t) = 1\right\} &= \mathsf{P}\{N_\lambda(t) = 1\}\,\mathsf{P}\{B = 1\} = \lambda t p + o(t), \\
\mathsf{P}\left\{N_{\lambda p}(t) \geq 2\right\} &\leq \mathsf{P}\{N_\lambda(t) \geq 2\} = o(t).
\end{aligned}$$

And $N_{\lambda p}$ has stationary and independent increments because N_λ is a Poisson process.

A final interesting, and useful, property of the Poisson process is the distribution of the arrivals an interal $[0, t]$ given that $N(t) = n$, say. In fact, if this event is true, the arrivals are uniformly distributed over the entire interval $[0, t]$.

Theorem 1.2.1. Conditional on the event $N(t) = n$, $t > 0$, the distribution of the arrival times $\{A_i\}_{i=1}^n$ is the same as the distribution of the order statistic of n iid rvs $\sim$ Unif($[0, t]$).

Proof. We write for convience $\mathrm{d}t_i = [t_i, t_i + \mathrm{d}t]$, and similarly $\mathrm{d}(t_i - t_{i-1}) = [t_i - t_{i-1}, t_i - t_{i-1} + \mathrm{d}t]$. With this notation, $\mathsf{P}\{X_i \in \mathrm{d}t_i\} = \lambda e^{-\lambda t_i}\,\mathrm{d}t$ when $X_i \sim \mathrm{Exp}(\lambda)$ and $\mathsf{P}\{U_i \in \mathrm{d}t_i\} = \mathrm{d}t/t$ when $U_i \sim \mathrm{Unif}([0, t])$. We provide the proof for $N(t) = 2$; the proof for the general case is nearly identical, but needs more notation.

Consider first the Poisson process with $0 \leq t_1 < t_2 \leq t$. Then,

$$\mathsf{P}\{A_1 \in \mathrm{d}t_1, A_2 \in \mathrm{d}t_2 | N(t) = 2\} = \frac{\mathsf{P}\{A_1 \in \mathrm{d}t_1, A_2 \in \mathrm{d}t_2, A_3 > t\}}{\mathsf{P}\{N(t) = 2\}},$$

because $N(t) = 2 \iff A_2 \leq t < A_3$. For the numerator,

$$\begin{aligned}
\mathsf{P}\{A_1 \in \mathrm{d}t_1, A_2 \in \mathrm{d}t_2, A_3 > t\} &\overset{1}{=} \mathsf{P}\{X_1 \in \mathrm{d}t_1, X_2 \in \mathrm{d}(t_2 - t_1), X_3 > t - t_2\} \\
&\overset{2}{=} \lambda e^{-\lambda t_1} \lambda e^{-\lambda(t_2 - t_1)} e^{-\lambda(t - t_2)}\,\mathrm{d}t^2 \\
&\overset{3}{=} \lambda^2 e^{-\lambda t}\,\mathrm{d}t^2,
\end{aligned}$$

where 1 follows from the recursive relation $A_i = A_{i-1} + X_i$; 2 because the X_i are iid $\sim$ Exp(λ); 3 is algebra. Because $\mathsf{P}\{N(t) = 2\} = (\lambda t)^2 e^{-\lambda t}/2!$, we find that $\mathsf{P}\{A_1 \in \mathrm{d}t_1, A_2 \in \mathrm{d}t_2 | N(t) = 2\} = 2!/t^2$.

As for the iid uniform rvs, $\mathsf{P}\{U_1 \in \mathrm{d}t_1, U_2 \in \mathrm{d}t_2\} = \mathrm{d}t^2/t^2$. However, there are 2! ways that the uniform rvs hit the set $\mathrm{d}t_1\,\mathrm{d}t_2$, hence the probability that the *order statistic* of U_1, U_2 lies in this set is $2!/t^2$. This equals the earlier conditional probablity. □

SUMMARIZING, THE POISSON process N_λ and the exponential distribution are very closely related: a counting process $\{N_\lambda(t)\}$ is a *Poisson process* with rate λ if and only if the inter-arrival times $\{X_k\}$ are iid and $X_k \sim \mathrm{Exp}(\lambda)$. Thus, if you find it reasonable to model inter-arrival times as memoryless,

then the number of arrivals in an interval is necessarily Poisson distributed. And, if you find it reasonable that the occurrence of an event in a small time interval is constant over time and independent from one interval to another, then the arrival process is Poisson, and the inter-arrival times are exponential.

Ex 1.2.1. Show that when $X \sim \text{Exp}(\lambda)$ its moment-generating function (MGF)[9] is $M_X(t) = \lambda/(\lambda - t)$. Use this to compute $\mathsf{E}[X]$ and $\mathsf{V}[X]$[10].

[9] $M_X(t) := \mathsf{E}\left[e^{tX}\right]$, $\mathsf{E}[X] = M'_X(0)$, $\mathsf{E}\left[X^2\right] = M''_X(0)$.

[10] $\mathsf{V}[X] = \mathsf{E}\left[X^2\right] - (\mathsf{E}[X])^2$.

Ex 1.2.2. Use MGFs to prove that A_i has density $f_{A_i}(t) = \lambda e^{-\lambda t}\frac{(\lambda t)^{i-1}}{(i-1)!}$. Use that for independent rvs X, Y, $M_{X+Y}(t) = M_X(t)M_Y(t)$.

Ex 1.2.3. When $N \sim \text{Pois}(\lambda)$ and $\alpha > 0$, show that $\mathsf{E}\left[\alpha^N\right] = e^{\lambda(\alpha-1)}$. Use this to see that $M_{N(t)}(s) = \exp(\lambda t(e^s - 1))$. Then find $\mathsf{E}[N(t)]$ and $\mathsf{V}[N(t)]$.

Ex 1.2.4. If $X \sim \text{Exp}(\lambda)$, $S \sim \text{Exp}(\mu)$ and independent, show that $Z = \min\{X, S\} \sim \text{Exp}(\lambda + \mu)$, hence $\mathsf{E}[Z] = (\lambda + \mu)^{-1}$.

This result can be anticipated when you think about merging Poisson processes.

Ex 1.2.5. If the Poisson arrival processes N_λ and N_μ are independent, show that

$$\mathsf{P}\left\{N_\lambda(t) = 1 \,\middle|\, N_\lambda(t) + N_\mu(t) = 1\right\} = \frac{\lambda}{\lambda + \mu}.$$

In words, given that a customer arrived in $[0, t]$, the probability that it is of the first type is $\lambda/(\lambda + \mu)$.

Ex 1.2.6. If $X \sim \text{Exp}(\lambda)$, $S \sim \text{Exp}(\mu)$ and independent, show that

Now think about splitting Poisson processes.

$$\mathsf{P}\{X \le S\} = \frac{\lambda}{\lambda + \mu}.$$

If the somewhat heuristic derivations above do not satisfy your sense for rigor, then you should solve the next set of exercises; otherwise you can skip them.

Ex 1.2.7. If the Poisson arrival processes N_λ and N_μ are independent, use moment-generating functions to show that $N_\lambda + N_\mu$ is a Poisson process with rate $\lambda + \mu$.

Ex 1.2.8. Show with moment-generating functions that thinning the Poisson process N_λ by means of Bernoulli random variables with success probability p results in a Poisson process $N_{\lambda p}$.

1.3 FIGHTING THE EXPONENTIAL DISTRIBUTION IS FUTILE

Perhaps the mathematical arguments presented in the preceding section have not fully convinced you of the universality of the exponential distribution and its discrete counterpart, the Poisson distribution. To enhance your understanding, let's use simulation to see how the exponential distribution emerges as a sort of law of nature.

The simulation is based on an internship of my master's students with the task to evaluate the scheduling system for patients with inflammatory bowel disease (IBD) at a hospital. The IBD patients can be characterized as regular and urgent. Regular patients required check-ups approximately

every six months, whereas urgent patients needed to be seen within one week, i.e., five working days. [0] Each consultation takes 10 minutes. Immediately following a consultation, regular patients scheduled their next visit for approximately six months later. There are about 1000 patients in the system, and a working year consist of 200 working days. Given that each patient is seen biannually, a physician needed to consult with an average of 10 patients daily. The problem was to design a planning procedure that allowed enough flexibility to meet the time constraints for the urgent patients without leading to idle times for the physicians. Of course, new patient intakes occurred, but for ease we assume that the patient population remains constant.

[0] *This procedure has been changed recently; however, the ideas of this case apply much more generally.*

Planning regular patients six months in advance seems to lead a stable planning system, but this is not true. First, there are many practical disturbances. A significant number of patients rescheduled their appointment date due to, e.g., altered holiday plans. Then, there were no-shows, yet other patients relocated to other cities, some died. However, even without such practical problems and even when each patient individually shows relatively predictable behavior, at the population level the inter-arrival times are exponential distributed. The simulation results strongly suggest that trying to counteract the exponential distribution is as futile as attempting to alter the laws of nature.

In conclusion, whether the planner plans six months in advance, with the idea to reduce variability, or would not plan at all, there will not be much difference. The verdict must be clear then: when rules don't have an effect, it is best to drop them altogether. It is simply better to send patients a reminder for a new appointment after five months after the visit, instead of planning six months in advance. Much less replanning actions are needed, and the probability for no-shows decreases.

IN THE NEXT MODEL we focus on just the regular patients. with an extremely simple model. Assume that the hospital has just 50 regular patients. Each patient plans the time to the next appointment uniformly on the interval $[0.9, 1.1]$, independent of previous times and other patients. Like this, we model a patient's inter-arrival time between two consecutive appointments as half a year (100 days) plus or minus 10 days ($[0.9, 1.1] \times 100 = [90, 110]$). The question of interest is to estimate the distribution of the inter-arrival times between any two patients *as seen by the planner* of the hospital. Of course, when there would be just one patient, this is easy, the planner sees $\sim [0.9, 1.1]$. But what happens for many patients?

We will use simulation to obtain insight into this question. The answer will be given in Fig. 1.3.1. Here we present the code to make this figure; understanding the code is necessary to interpret the figure. Hence, I expect you to really read the code. If you don't understand certain commands of python, just consult ChatGPT.

IN PYTHON WE nearly always start with loading a set of modules. These contain specific functionality that not every Python program needs, hence these should only be loaded if required. The `numpy` module offers support for numerical work, `matplotlib` is a plotting library. Here, and elsewhere,

we place comments *above* the code to which it relates. We will not explain commands that are trivial to understand.

```
import numpy as np
import matplotlib.pyplot as plt
```

To make a pdf file with a plot that looks nice in LaTeX we use the `seaborn` library and provide it with some extra options, such as good font sizes.[1].

[1] *While designing a figure, I comment out this code because running LaTeX slows down the script.*

```
def initialize_plots():
    import seaborn as sns

    sns.set(
        style="whitegrid",
        rc={
            "text.usetex": True,
            "font.family": "fourier",
            "axes.labelsize": 10,
            "font.size": 10,
            "legend.fontsize": 8,
            "xtick.labelsize": 8,
            "ytick.labelsize": 8,
        },
    )


initialize_plots()
```

The simulation environment will contain `num_patients` patients and last for `num_periods` half years. With `default_rng` we can set up a random number generator and by setting a seed (here to 3, but any number would do) we ensure to always get the same random deviates[2]

[2] *A random deviate is a sample taken from a distribution; it is not the same as a random variable.*

```
num_periods = 10
num_patients = 50
rng = np.random.default_rng(2)
```

We next generate a number of inter-arrival times in the following way. The matrix X has `num_patients` rows and `num_periods` columns. Row k of X corresponds to the inter-arrival times of patient k. Each inter-arrival time is $\text{Unif}([0.9, 1.1])$. With these inter-arrival times, the arrival times for patient k are $A_i^k = A_{i-1}^k + X_i^k$, with $A_0^k = 0$.

The `cumsum` along a row, i.e., along `axis=1`, of a matrix adds up elements one by one along a row. For instance, the `cumsum` of the sequence of numbers $1, 3, 5$ would be $1, 4, 9$. As such, the elements of the first column of `A` would be equal to the first column of `X`. To ensure that the first row of `A` is zero, so that all patients start at time 0, we set the first column of `X` to 0. However, by setting the first column of inter-arrivals X to zero, we effectively remove one arrival. Thus, we should generate `num_periods + 1` of inter-arrival times.

```
X = rng.uniform(low=0.9, high=1.1, size=(num_patients, num_periods + 1))
X[:, 0] = 0  # set first column of X to 0
A = X.cumsum(axis=1)
```

We need an environment to control the plots. The `fig` object can be thought of as a canvas on which you will draw your plots. The `axes` object contains a list of `ax` objects, each representing a single plot within the figure. As is apparent from Fig. 1.3.1, we have a matrix of 4 `ax` objects. For easy reference in the code below, we flatten[3] the `axes` object and unpack it to the four separate objects `ax1`, ..., `ax4`.

[3] *Once again, ask ChatGPT for further explanation.*

```
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(6, 3))
ax1, ax2, ax3, ax4 = axes.flatten()
```

Next we build the left upper panel of Fig. 1.3.1. Each row contains a number of dots, each of which corresponds to an arrival event of a patient. In other words, the coordinate of the ith dot of patient k is (A_i^k, k). As we have 50 patients, we have 50 such rows of dots. The for loop adds the dots of each patient to the upper left panel referenced to by `ax1`. To beautify the plot, we give a few options to the plot command. We don't want the dots to be connected by lines, so that we set the linestyle `ls='None'`. The marker style is a dot, the markersize `ms=0.2` and the color is black.[4]

[4] *I often use the abbreviations offered by matplotlib as it saves time.*

```
ax1.set_xlim(0, 11)
for i in range(num_patients):
    x = A[i, 1:]
    y = i * np.ones(num_periods)
    ax1.plot(x, y, ls='None', marker="o", ms=0.2, c="k")
```

We plot the arrivals as seen by the planner as a jitterplot, that is, we add small random noise to the data points along the y-axis. By spreading out the points slightly, the jitter plot allows for a clearer visualization of the distribution of individual points. We plot the dots at $y = 0$ plus a jittering distributed as $\sim \text{Unif}([-1/2, 1/2])$. Without the jitter we would just see small line segments instead of the somewhat rectangular shapes.

```
for i in range(num_patients):
    x = A[i, 1:]
    y = rng.uniform(-0.5, 0.5, size=num_periods)  # add jitter
    ax1.plot(x, y, ls='None', marker="o", ms=0.2, c="k")
```

OUR NEXT TASK is to estimate the density of inter-arrival times as observed by the planner, and plot this in the right panel of Fig. 1.3.1. Recall that each row of the matrix A contains the arrival times of one specific patient. By flatting A we obtain all arrival times in one long list. By sorting, we obtain the arrival times in order of time. Finally, inter-arrival times for the planner are the times between the elements of this list of arrivals.

```
A = np.sort(A.flatten())
X = A[1:] - A[:-1]
```

We assemble all inter-arrival times in X in bins. As we have $\lambda = 50$ arrivals per day, the mean inter-arrival time is $1/\lambda = 1/50$. The pdf of

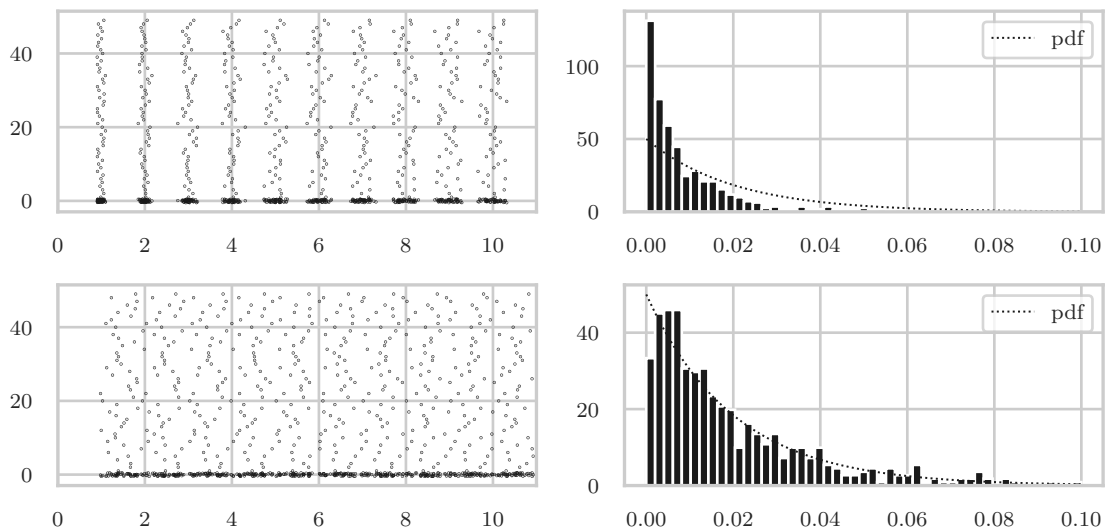


Figure 1.3.1: How the uniform distribution leads to the exponential distribution.

exponential distribution is $\lambda e^{-\lambda x}$. To see the tail of the pdf in the plot, we take 5 times the mean, i.e, $5/50 = 0.02$, as the support for the bins. We then plot the histogram, i.e., the number of hits in each bin, on `ax2`, and the pdf.

```
bins = np.linspace(0, 5 / num_patients, 50)
exp_density = num_patients * np.exp(-num_patients * bins)
ax2.hist(X, bins, density=True, color="k")
ax2.plot(bins, exp_density, ":", c="k", lw=0.75, label="pdf")
ax2.legend()
```

Before explaining how to make the second row in Fig. 1.3.1, here is the code to save the figure. The `tight_layout()` function ensures that the labels of the axes are nicely formatted.

```
fig.tight_layout()
fig.savefig("../figures/IBD_exponential.pdf")
```

WHAT CAN WE see in Fig. 1.3.1? In the left upper panel, the arrival times of each patients are at first tightly concentrated around 1, but as time progresses, the arrival times become more and spread out. The jitter plot around $y = 0$ also demonstrates this. The rectangles become wider and approach each other. However, we see also that, in comparison to the exponential density, there are relatively many short inter-arrival times. This is as expected, because in particular the first couple of inter-arrival times are much too near to provide a good model for the hospital that have been serving patients for years. But still, even when starting with very unrealistic arrival times, we see that the exponential model is quite reasonable.

To repair for these unnatural starting times, we set the initial arrival time of each patient uniformly on $[0, 1]$.

```
X[:, 0] = rng.uniform(low=0, high=1, size=num_patients)
```

With this change, we run the above code again, but write the plots ax3 and ax4 to obtain the lower left and right panels of Fig. 1.3.1. Now we see that the exponential distribution is an excellent fit, and the jitter rectangles overlap right from the start.

In conclusion, if many patients arrive to the hospital and each adds some small variation to the inter-arrival times, the hospital sees exponential inter-arrival times for the population. Clearly, this is not specific to hospitals: it applies to any system that serves a large pool of customers, each arriving with some variation. Thus, the exponential distribution is an important modeling device for nearly any service and production systems; we use it often in the sequel.

2

CONSTRUCTION OF SIMPLE DISCRETE TIME STOCHASTIC PROCESSES

The first step to analyze stochastic processes is to model it. And for this, there is often not a better start than to build a simulation model for such processes. For this reason, the aim of this first chapter is to teach you how to construct and simulate simple queueing and inventory processes.

In Section 2.1 we build discrete-time models of queueing systems, which means that we use the number of jobs that arrive and can be served in fixed periods of time to construct the queueing process. Such a period can be an hour, or a day; in fact, any amount of time that makes sense in the context in which the model will be used. Section 2.3 presents suitable recursions for single-item inventories and demonstrates in particular how to design good rules to control stochastic inventory systems. Once we have some nice models for two common stochastic processes, we demonstrate in Section 2.5 in detail how to set up simulations in python, compute important performance measures and make graphs of the process as a function of time. We use these simulation to obtain some general insights into how queues depend.

2.1 WAITING FOR A PSYCHIATRIST

We start with a case to provide motivation to study queueing systems. Then we develop a set of recursions of fundamental importance by which we can simulate the case and evaluate the efficacy of several policies to improve the system.

AT A MENTAL HEALTH department, five psychiatrists do intakes of future patients to determine the best treatment process once patients 'enter the system'. There are complaints about the time patients have to wait for the intake; the desired waiting time is around two weeks, but the realized waiting time is sometimes more than three months. This is unacceptably long, but ... what to do about it?

The five psychiatrists put forward various suggestions to reduce the waiting times.

1. Give all psychiatrists the same weekly capacity for doing intakes. Motivation: Currently one psychiatrist does 9 intakes per week, one does 3, and the three other psychiatrists do only 1. This is not a problem during weeks when all psychiatrists are present; however, psychiatrists take holidays, visit conferences, and so on. So, if the psychiatrist with the most intakes per week goes on leave, this affects the intake capacity considerably.

2. Synchronize holidays.[0] Motivation: to reduce the variation in the service capacity, the psychiatrists plan their holidays consecutively. However, perhaps it is better to work at full capacity or not at all.

[0] *Later, with the models of Chapter 6, we can immediately see that this will not work at best, but otherwise detrimental.*

3. Control[1] the intake capacity as a function of the waiting time. Motivation: in analogy with supermarkets, scale up (down) capacity when the queue is long (short).

We next develop a method to simulate the behavior of the system over time so that we can evaluate the effect of the above suggestions. We use simulation, because the mathematical analysis is too hard.[2]

LET US START with discussing the essentials of simulating of a queueing system. The easiest way to construct queueing processes is to 'chop up' time in periods[3] and develop recursions for the behavior of the queue from period to period. Using fixed-sized periods has the advantage that we do not have to specify inter-arrival times or service times of individual customers; only the number of arrivals in a period and the number of potential services are relevant.

Let us define:

$a_k =$ the number of jobs that arrive *in* period k,

$c_k =$ the capacity, i.e., the maximal number of jobs that can be served, during period k,

$d_k =$ the number of jobs that depart *in* period k,

$L_k =$ the *system length*, i.e., the number of jobs in the system at the *end* of period k.

In the sequel we also call a_k the *size of the batch* arriving in period k. Note that the definition of a_k is a bit subtle: we may assume that the arriving jobs arrive either at the start or at the end of the period. In the first case, the jobs can be served in period k, in the latter case, they *cannot* be served in period k.

Since L_{k-1} is the system length at the *end* of period $k-1$, it must also be the number of customers at the *start* of period k. Assuming that jobs arriving in period k cannot be served in period k, the number of customers that depart in period k is therefore

$$d_k = \min\{L_{k-1}, c_k\},$$

Now that we know the number of departures, the number at the end of period k is given by

$$L_k = L_{k-1} - d_k + a_k.$$

Like this, if we are given L_0 and numbers $\{a_k\}_{k=1}^n$ and $\{c_k\}_{k=1}^n$, we can obtain the numbers $\{L_k\}_{k=0}^n$ from this recursion.[4]

Of course we are not going to carry out the computations for $\{L_k\}$ by hand. Typically, we use company data to model the arrival process $\{a_k\}$ and the capacity $\{c_k\}$, and feed this data into a computer to carry out the recursions (2.1.1). If we do not have sufficient data, we make a probability model for these data and use the computer to generate random numbers with, hopefully, similar characteristics as the real data. At any rate, from this point on, we assume that it is easy, by means of computers, to obtain numbers $a_1, \ldots, a_n$ for $n \gg 1000$, and likewise for other sequences of numbers we may need.

[1] *People often object to such policies because they believe that they have to do more work. However, this is wrong. Suppose that 1000 patients per year need treatment. This number does not depend on whether they spend time in queue or not.*

[2] *In case you doubt this, try to analyze transient multiserver queueing systems with vacations, of which this is an example.*

[3] *The length of these periods depends on context. For the present case, it is appropriate to take weeks. For supermarkets perhaps a length of 5 minutes is more appropriate.*

[4] *In a sense, (2.1.1) is the $F = ma$ of a queueing system: Given initial conditions, we apply the rule at time $k-1$ to get the state at time k, and so on.*

CONTINUING WITH THE case of the five psychiatrists, with the above recursions we can now analyze the proposed rules to improve the performance of the system. We mainly want to reduce the long sojourn times.

As a first step in the analysis, we model the arrival process of patients as a Poisson process. The duration of a period is taken to be a week. The average number of arrivals per period, based on data of the company, was slightly less than 12 per week; in the simulation we set it to $\lambda = 11.8$ per week.

NEXT, WE MODEL the capacity in the form of a matrix such that row i corresponds to the weekly capacity of psychiatrist i:

$$\begin{pmatrix} 1 & 1 & 1 & \dots \\ 1 & 1 & 1 & \dots \\ 1 & 1 & 1 & \dots \\ 3 & 3 & 3 & \dots \\ 9 & 9 & 9 & \dots \end{pmatrix}. \tag{2.1.2}$$

Thus, psychiatrists 1, 2, and 3 do just one intake per week, the fourth does 3, and the fifth does 9 intakes per week. The sum over column k is the total service capacity for week k of all psychiatrists together.

With such a matrix it is simple to make other capacity schemes. A more balanced scheme would be like this:

$$\begin{pmatrix} 2 & 2 & 2 & \dots \\ 2 & 2 & 2 & \dots \\ 3 & 3 & 3 & \dots \\ 4 & 4 & 4 & \dots \\ 4 & 4 & 4 & \dots \end{pmatrix}. \tag{2.1.3}$$

We include the effects of holidays by setting the capacity of one or some psychiatrists to 0 in a certain week. Let us assume that every week just one psychiatrist is on leave such that the psychiatrists' holiday schemes rotate. To model this, we set the entries of the matrix as $C_{1,1} = C_{2,2} = \cdots = C_{1,6} = C_{2,7} = \cdots = 0$, i.e.,

$$C = \begin{pmatrix} 0 & 2 & 2 & 2 & 2 & 0 & \dots \\ 2 & 0 & 2 & 2 & 2 & 2 & \dots \\ 3 & 3 & 0 & 3 & 3 & 3 & \dots \\ 4 & 4 & 4 & 0 & 4 & 4 & \dots \\ 4 & 4 & 4 & 4 & 0 & 4 & \dots \end{pmatrix}. \tag{2.1.4}$$

Hence, the total average capacity is $4/5 \cdot (2+2+3+4+4) = 12$ patients per week. There is another simple holiday scheme: when all psychiatrists take holiday in the same week–corresponds we set entire columns to zero, i.e., $C_{i,5} = C_{i,10} = \cdots = 0$ for week 5, 10:

$$C = \begin{pmatrix} 2 & 2 & 2 & 2 & 0 & 2 & 2 & \dots \\ 2 & 2 & 2 & 2 & 0 & 2 & 2 & \dots \\ 3 & 3 & 3 & 3 & 0 & 3 & 3 & \dots \\ 4 & 4 & 4 & 4 & 0 & 4 & 4 & \dots \\ 4 & 4 & 4 & 4 & 0 & 0 & 4 & \dots \end{pmatrix}. \tag{2.1.5}$$

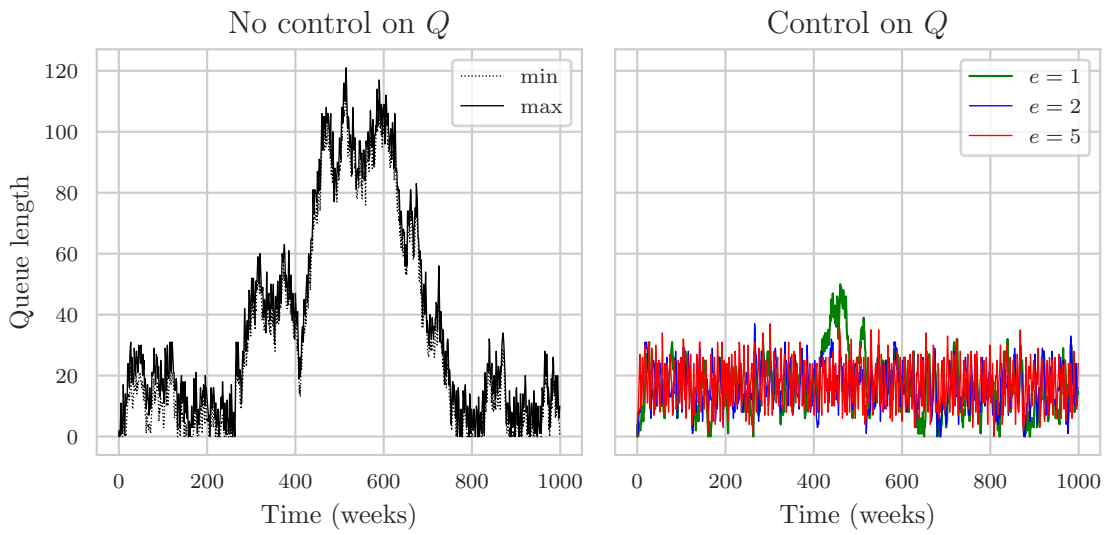


Figure 2.1.1: Effect of capacity and holiday plans on the queue length L. The left panel shows the maximum and the minimum queue length under the different holiday and balancing policies. The right panel shows the queue length under the control rule (2.1.6).

Of course, random schemes are more likely, but these two are perhaps the simplest that result in the same average capacity. Note that we also apply these the unbalanced capacity plans to obtain four different possibilies.

With these models for the arrivals and the capacities, we use the recursions (2.1.1) to simulate the queue length process for the four different scenarios proposed by the psychiatrists: unbalanced versus balanced capacity, and spread out holidays versus simultaneous holidays.

The results are shown in Fig. 2.1.1. We plot, for each period, the largest and the smallest queue that occurred under all four capacity plans that result from following the first and second suggestions of the psychiatrists. The graphs make clear that these suggestions hardly affect the behavior of the queue length process.

NOW WE CONSIDER Suggestion 3, which comes down to doing more intakes when it is busy, and fewer when it is quiet. A simple rule is to let the capacity for week k depend on the queue length of the week before, for instance,

$$c_k = \begin{cases} 12 + e, & \text{if } L_{k-1} \geq 24, \\ 12 - e, & \text{if } L_{k-1} \leq 12. \end{cases} \tag{2.1.6}$$

We can take $e = 1$ or 2, or perhaps a larger number; the larger e, the larger the control we exercise. We can of course also adapt the thresholds 12 and 24.

Let's consider three different control levels, $e = 1$, $e = 2$, and $e = 5$; when $e = 5$, each psychiatrist does one extra intake. The results, see the right panel of Fig. 2.1.1, show a striking difference indeed. The queue does not explode anymore; already taking $e = 1$ has a large influence.

This simulation experiment shows that changing holiday plans or spreading the work over multiple servers, i.e., psychiatrists, may not signifi-

cantly affect the queueing behavior. However, controlling the service rate as a function of the queue length can improve the situation dramatically.

THE ABOVE EXAMPLE hopefully claries how we can use simulation to obtain insight into a stochastic system behaves and how to control it. We next discuss some general points that need further attention.

In (2.1.1) we assume that jobs that arrive in period k cannot be served in period k. If the situation is such that jobs *can* be served in the same period as they arrive, then (2.1.1) should be changed to[5]

[5] [2.1.1]

$$d_k = \min\{L_{k-1} + a_k, c_k\}. \tag{2.1.7}$$

Which of (2.1.1) or (2.1.7) to choose depends on what we need to model; in general, no rule is 'best', and what is 'good' depends essentially on the context. For instance, if we like to be 'on the safe side', then it is perhaps best to use (2.1.1) because with this rule, we overestimate the queue lengths, while with (2.1.7) we underestimate the queue lengths.

Note next that in the computation of d_k we make a fundamental modeling choice: if there are jobs in the system and the server capacity $c_k > 0$, the server will serve jobs. There is, however, no formal need to serve jobs even when there is service capacity available. A simple reason not to serve jobs is when it is too expensive, for example, if there is very little demand for a flight on some day then the flight can be canceled. In the models we consider, we will *not* allow to let jobs wait when there is capacity available; in other words, our service process are *work-conserving*.

The above recursions obviously only construct $\{L_k\}$, i.e., the dynamics of the number of jobs in the system. If we also need information about the *sojourn times*, i.e., the time jobs spend in the system, it is necessary to specify the *service discipline*, i.e., a scheduling rule that decides on the order in which jobs in queue are taken into service. In this book we assume henceforth that jobs move to the server in the order in which they arrive. This is known as *First-In-First-Out (FIFO)*.[6] There are many other scheduling rules, such as Last-In-First-Out; we do not discuss these here.

[6] First-Come-First Serve (FCFS) *is also often used.*

It is quite remarkable that the computation of the system length process $\{L_k\}$ is very simple with (2.1.1), but it is much harder to compute the sojourn time J_k for the jobs arriving in period k. In fact, the best we can obtain are bounds on the sojourn time J_k such that $J_k^- \le J_k \le J_k^+$, where

$$J_k^- = \min\left\{m : \sum_{i=k}^{k+m} c_i > L_{k-1}\right\}, \quad J_k^+ = \min\left\{m : \sum_{i=k}^{k+m} c_i \ge L_{k-1} + a_k\right\}.$$

To see how this works, suppose $L_{k-1} = 20$, $a_k = 6$, and $c_k = 2$ for all k. Any job that arrives in the kth period, must wait at least $20/2 = 10$ periods just to get access to the server under FIFO rule, and we can be sure that the last of these jobs is served after $(20+6)/2 = 13$ periods.

We note that there is a difference between *waiting time* W and sojourn time J. The former is the time jobs spend in queue, the latter the time in the system, which is the waiting plus the time at the server L_s. Relatedly, in the model (2.1.1) there is not a job in service, we only count the jobs in the system at the end of a period. Thus, in this model the number of jobs in the system and in queue coincide.

Finally, once the simulation is finished, we compute some performance measures to analyze how the (simulation of the) queueing system behaved. Besides making graphs, we can be interested in the arrival rate, the capacity (also known as the service rate) and mean number in the system,

$$a = \frac{1}{n}\sum_{k=1}^{n} a_k \qquad c = \frac{1}{n}\sum_{k=1}^{n} c_k \qquad L = \frac{1}{n}\sum_{k=1}^{n} L_k.$$

Similary, we can take averages over d_k, J_k^- and J_k^+. The variances these (simulated) rvs can also be of interest.

IN GENERAL, WITH recursions like (2.1.1) we can carry out numerous hat-if-analyses. As another example, a hospital considers to buy a second MRI scanner, because the current one is saturated, as it is used from 8 am to 6 pm, and can certainly not serve the forecasted demand. However, if the hospital can identify some extra capacity, it might postpone the purchasing of an additional MRI scanner for a few years. Suppose that a percentage of staff is willing to work from 8 pm to 11 pm, say, and that 30% of the patients[7] is prepared to come to the hospital for a scan between 8 pm and 11 pm, then the capacity would increase with about 30% = $(3/(18-8))$. To see whether this idea is interesting, we can use (2.1.1) to let the computer make a graph of the influence on L. Thus, the recursions such as (2.1.1) and control rules such as (2.1.6) are the essential elements of any queueing model. Once we have the recursions,[8] the computer can compute the performance measures, and then we (humans) have to interpret the results.

[7] *Perhaps some staff and patients even prefer to work/have scans in the evening.*

[8] *An intellectually challenging task sometimes.*

Such recursion are not limited just to queueing systems. For instance, a common model in population dynamics has this form

$$N_{t+1} = N_t + B_t + I_t - D_t - E_t,$$

where N_t is the population size at year t, B_t the number of births, I_t the immigration, D_t the deaths, and E_t the emmigation.

Yet another setting is the reflection of infra-red light by carbon-dioxide. The atmosphere is modeled as a sequence of layers, such that per layer the temperature, air density and chemical composition are considered constant. These properties determine how much infrared light, which is initially reflected by the surface of the earth, makes it to outer space. In each of the layers we need to specify how many infrared photons are transmitted to a higher layer, absorbed, or reflected again to a lower level. Thus, people model physical process as a queueing system in which photons (customers) go from one layer (station) to another layer (station), or are absorbed (leave the network).[9]

[9] `https://github.com/scienceetonnante/RadiativeForcing/blob/main/RadiativeForcing.py`

GIVEN THAT FORMULATING recursions is the key step in making useful models for data science, you should practice with this a lot. Here are a large number exercises to help you make a start with this extremely useful skill.

Ex 2.1.1. Prove that the recursion $L_k = [L_{k-1} + a_k - c_k]^+$ generates a system in which jobs can be served in the period they arrive.

Serve jobs in the period in which they arrive.

Ex 2.1.2. To ensure that the server capacity is always fully used, a server works at rate c in period k if $L_{k-1} \geq N$ jobs, and not otherwise. Set up the recursions.

Server controlled by threshold policy

Ex 2.1.3. A machine serves with two queues such that jobs in the first queue get priority over jobs in the other queue. Find the recursions.

Priority queueing

Ex 2.1.4. One server serves two queues. Each queue receives service capacity in proportion to the queue length. Derive the recursions.

Proportionally fair queueing

Ex 2.1.5. Consider a single-server that serves two parallel queues. Queue i has minimal guaranteed service capacity r^i each period, such that $c_k \geq r^1 + r^2$. Extra capacity beyond the reserved capacity is given to queue 1 with priority. (An example is a psychiatrist who reserves capacity for different patient groups.) Formulate the recursions.

Queues with reserved service capacity

2.2 SIMULATING THE PSYCHIATRISTS CASE

In the previous section we showed how to analyze a case in four psychiatrists act as four parallel servers that process a queue of potential patients waiting for an intake. Moreover, we suggested a good rule to control the queue length. In this section we provide the python code we used for the simulation, in other words, we will make Fig. 2.1.1.

WE NEED THE next modules.

```
import numpy as np
from numpy.random import default_rng
import matplotlib.pyplot as plt
```

Recall that we compute the queue length recursively by first computing the number of departures $d = \min\{L_{n-1}, c_n\}$ and then $L_n = L_{n-1} + a_n - d$. The next function follows this logic. It first forms an empty array `L` to store all the queue lengths. Then it applies the recursion by means of a for loop. The loop has to start at $n = 1$ because it 'looks back' at L_{n-1}. For generality, we can provide an optional queue level `L0` if we want the queue to start at another level than 0.

```
def compute_L(a, c, L0=0):
    # L0 is the initial queue length
    L = np.empty(len(a))
    L[0] = L0
    for n in range(1, len(a)):
        d = min(L[n - 1] + a[n], c[n])
        L[n] = L[n - 1] + a[n] - d
    return L
```

Now we make the capacity schemes. The first step is to form an array with 5 rows such that each row corresponds to the weekly capacities of each psychiatrist. Since python starts numbering at 0, instead of 1, the row with index 0 corresponds to the first psychiatrist. We can set all values of the entire row with index 0 with notation like `p[0, :]` where the semicolon

refers all indices of the columns.[0] Next, `np.ones(n)` makes a row array of just ones. For each psychiatrist, we fill the related row in `p` with the appropriate capacity of that specific psychiatrist. The result is a matrix that looks like the one in (2.1.2).

[0] *It evokes the dots in mathematical notation like* $i = 1, 2, \ldots$

```
def unbalanced_load(n):
    p = np.empty([5, n])
    p[0, :] = 1 * np.ones(n)
    p[1, :] = 1 * np.ones(n)
    p[2, :] = 1 * np.ones(n)
    p[3, :] = 3 * np.ones(n)
    p[4, :] = 9 * np.ones(n)
    return p
```

The next function builds the matrix (2.1.3)

```
def balanced_load(n):
    p = np.empty([5, n])
    p[0, :] = 2 * np.ones(n)
    p[1, :] = 2 * np.ones(n)
    p[2, :] = 3 * np.ones(n)
    p[3, :] = 4 * np.ones(n)
    p[4, :] = 4 * np.ones(n)
    return p
```

Now we change a capacity matrix such that it includes a certain holiday pattern. This function spreads the holidays over week so that we obtain (2.1.4). The operator % computes the remainder after division, for example $17\%5 = 2$. In the for loop, the variable j moves one column per iteration, and it cycles over the rows from 0 to 4. The `shape[0]` returns the number of columns of a matrix. And, finally, the function does not need to return `p` because the change is *in-place*.[1] Here we return it so that all functions work in the same way.

[1] *Look on the web what is meant by a change 'in place'.*

```
def spread_holidays(p):
    n_cols = p.shape[1]
    for j in range(n_cols):
        p[j % 5, j] = 0
    return p
```

In the case of synchronized holidays, we set all capacities in one column to zero, cf. (2.1.5). The `range` function moves from 0 to the end, in steps of 5, so it gives $0, 5, 10, \ldots$.

```
def synchronized_holidays(p):
    for j in range(0, p.shape[1], 5):
        p[:, j] = 0
    return p
```

NOW ALL IS prepared to start the simulation. We set the seed of the random generator, and simulate `num_weeks` of Poisson distributed weekly arrivals. The matrix `L` will contain the queue lengths, one row for each different scenario.

```
rng = default_rng(3)
num_weeks = 1000
a = rng.poisson(11.8, num_weeks)
L = np.zeros((4, len(a)))
```

We have four capacity scenarios. In the first scenario, the capacity is balanced, and the holiday plans are spread over the weeks. The weekly capacity c_i is equal to the sum over all rows in `p` that correspond to week i. This is established by summing over `axis=0` in `p`. For instance, in (2.1.4), the capacity for the first week is $0+2+3+4+4$. Once we have the weekly capacities stored in `c`, we can use the recursion for l to compute the queue lengths at the end of each week. The other scenarios work likewise.

```
p = balanced_load(len(a))
p = spread_holidays(p)
c = np.sum(p, axis=0)
L[0, :] = compute_L(a, c)

p = balanced_load(len(a))
p = synchronized_holidays(p)
c = np.sum(p, axis=0)
L[1, :] = compute_L(a, c)

p = unbalanced_load(len(a))
p = synchronized_holidays(p)
c = np.sum(p, axis=0)
L[2, :] = compute_L(a, c)

p = unbalanced_load(len(a))
p = spread_holidays(p)
c = np.sum(p, axis=0)
L[3, :] = compute_L(a, c)
```

The code above suffices to make the left panel of Fig. 2.1.1. For the right panel we need control rule (2.1.6). This works nearly the same as the regular recursion for L, but now we add or subtract an extra capacity `e` to the weekly capacity given in the array c.

```
def compute_L_with_control(a, c, e, L0=0):
    L = np.empty(len(a))
    L[0] = L0
    for n in range(1, len(a)):
        if L[n - 1] < 12:
            C = c[n] - e
        elif L[n - 1] >= 24:
            C = c[n] + e
        L[n] = max(L[n - 1] + a[n] - C, 0)
    return L
```

We simulate the effect on L for various control rules. We take $e = 1$, $e = 2$ and $e = 5$.

```
c = 12 * np.ones_like(a)
LL = np.zeros((4, len(a)))
```

```
LL[0, :] = compute_L(a, c)
LL[1, :] = compute_L_with_control(a, c, 1)
LL[2, :] = compute_L_with_control(a, c, 2)
LL[3, :] = compute_L_with_control(a, c, 5)
```

FINALLY, WE CAN make a figure. It's easy to find on the web what is `ax1` and the other commands. Note, `L.min(axis=0)` computes the minimum queue length per week, and `L.max(axis=0)` the maximum.

```
fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(6, 3), sharey=True)
ax1.set_title("No control on $Q$")
ax1.set_ylabel("Queue length")
ax1.set_xlabel("Time (weeks)")

ax1.plot(L.min(axis=0), ":", label="min", color='k', lw=0.5)
ax1.plot(L.max(axis=0), "-", label="max", color="k", lw=0.5)
ax1.legend()

ax2.set_title("Control on $Q$")
ax2.set_xlabel("Time (weeks)")
ax2.plot(LL[1], label="$e = 1$", color='green', lw=0.8)
ax2.plot(LL[2], label="$e = 2$", color='blue', lw=0.5)
ax2.plot(LL[3], label="$e = 5$", color='red', lw=0.5)
ax2.legend()

fig.tight_layout()
fig.savefig("../figures/psychiatrists.pdf")
```

HERE ARE SOME more exercises that invite you to set up recursions and model systems with stochasticity. You should have learned from the code examples above that most code is just boilerplate code to run the recursions and make the graphs. The heart of the simulations are the recursions, so making these is what you should practice.

Queue with protected service capacity and lost capacity

Ex 2.2.1. Consider a single-server that serves two parallel queues. Queue i receives a minimal service capacity r^i every period. Reserved capacity unused for one queue cannot be used to serve the other queue. Any extra capacity beyond the reserved capacity, i.e, $c_k - r^2$, is given to queue 1 with priority. Formulate the recursions.

An example is the operation room of a hospital. There is a weekly capacity, part of which is reserved for emergencies. It might not be possible to assign this reserved capacity to other patient groups, because it should be available at all times for emergency patients. A result of this is that unused capacity is lost. In practice it may not be as extreme as in the model, but still part of the unused capacity is lost. 'Use it, or lose it' often applies to service capacity.

Cost of an N policy.

Ex 2.2.2. A machine can switch on and off. Suppose that the jobs that arrive in period k can be served in that period. If the system length hits N (becomes empty) in period k, the machine switches on (off) in period $k+1$. Make the recursions. Assume the machine is off at $k = 0$.

Ex 2.2.3. In the setting of the previous exercise, suppse it costs K to switch on the machine. There is also a cost β per period when the machine is

on, and it costs h per period per customer in the system. Can you make a model to compute the total cost for the first T periods?

Queues with setup times. Hard exercise.

Ex 2.2.4. A nurse takes blood samples at two departments in a hospital. It takes time to walk from one location to another. The nurse serves all patients in one location, then moves to the other location, serves all patients there, and walks the first location, and so on. (This is somewhat simplyfied case of one of my bachelor students who worked at a hospital.)

Here is the abstract description in queueing terms. One server serves two parallel queues, one at a time. After serving one queue until it is empty, the server moves to the other queue, which requires one period setup time. Find the recursions.

An interesting extension would be to model a case in which the move time from the nurse would take 2 periods instead of 1.

2.3 CONTROL OF SINGLE-ITEM INVENTORY SYSTEMS

In this section we analyze, so called, single item inventory systems under *periodic review*. As a simple example, think of boxes of macaroni on a supermarket shelf. At the end of the period, here a day the supermarket uses the amount of boxes left to decide whether a replenishment of macaroni is necessary or not. If so, it orders in the eventing a batch containing ten, say, macaroni boxes to refill the stock. The batch will be delivered by truck at the start of the next day after which a shelf stocker unpacks the batch and puts all ten macaroni boxes on the shelf.

In general there are many different types of items in inventory, ranging from cheap to expensive, so we need different rules to control inventories.[0] We assume that demand is discrete, stochastic, and items are not perishable so that items unsold in one period carry over to the next period. With these assumptions we can develop a few simple, but elegant, formulas by which we can construct, hence simulate, the discrete-time dynamics of the most important inventory control systems. Then we define a number of performance measures that allow us to assess the performance of these rules. In a later chapter, after developing some appropriate further mathematical concepts, we can derive closed form expressions for a number of performance measures.

[0] As such, inventory systems are examples of stochastic processes subject to control.

We use the following notation, the meaning of which we explain below,[1]

[1] Inventory theory uses the word 'level' for many different concepts, which should not be confused. Below we introduce inventory level I, order-up-to level S, reorder level s, and service levels.

$$
\begin{aligned}
Q_t &= \text{order size that arrives at the } \textit{start} \text{ of period } i,\\
D_t &= \text{demand arriving } \textit{during} \text{ period } i,\\
I_t &= \text{inventory level at the } \textit{start} \text{ of period } i,\\
I_t^+ &= \text{inventory on hand at the } \textit{start} \text{ of period } i,\\
I_t^- &= \text{backlogged demand at the } \textit{start} \text{ of period } i.\\
P_t &= \text{inventory position at the } \textit{start} \text{ of period } i,\\
L &= \text{fixed lead time.}
\end{aligned}
$$

Note that we are specific about the *timing* of each of the variables; we distinguish between the start and the end of a period. When refering to

a quantity measured at the *end* of a period, we mark it with a prime, for instance I'_t.[2]

[2] *The leadtime L is not the same as the average number L of jobs in the system.*

THE DYNAMICS OF the inventory can be simply constructed if we are given a set of period demands $\{D_t, t = 1,2,\ldots\}$ and order sizes $\{Q_t, t = 0,1,\ldots\}$, but first we need to address a subtle point. In many inventory systems there is a delay between *issuing* a replenishment order and *receiving* this order: only after receiving the order, the order can be used to replenish on-hand stock and serve demand. The time between the placement of the order and its arrival at the inventory is called the *leadtime* L, and the number of orders under way are called *outstanding* orders. In the sequel we take L to be constant over time and independent of the number of outstanding orders. Then, when placing an order of size Q_{t-L} at the start of period $t-L$ so that it arrives at the start of period t, the *inventory level* follows the rules

$$I_t = I'_{t-1} + Q_{t-L}. \qquad\qquad I'_t = I_t - D_t. \tag{2.3.1}$$

The sequence is important. The replenishment order arrives at the start of the period so that it is available to serve demand.

In general, the inventory level I_t can be positive and negative. In the former case, $I_t^+ = \max\{I_t, 0\}$ is the *inventory on hand* as the items can be used to meet demand directly from stock. In the latter case, $I_t^- = \max\{-I_t, 0\}$ is the *backlogged* demand, because it is demand (or a customer) that has to wait for the arrival of future replenishment order(s).

Clearly, if we would use the inventory level to determine when and how much the order, we might be too late in placing orders, due to the leadtime. To compensate for this, we use the *inventory position* which updates according to the rule

$$P_t = P_{t-1} + Q_t, \qquad\qquad P'_t = P_t - D_t. \tag{2.3.2}$$

Thus, the inventory position includes the outstanding orders, hence accounts for items under way to cover future demand. In other words, the difference between the inventory position and the inventory level is that in the former we immediately include all orders 'in the books', but these orders physically arrive L periods later, and only then they can be used to meet demand.

Let us next study a number of rules that use the inventory position to compute order sizes; these policies differ only in the decision when to trigger an order and how much to order. Since these rules check the inventory position at the end of a period, we say that the inventory is under periodic review.

THE BASESTOCK POLICY seems to be the simplest rule as it asks for a replenishment for each item demanded. Specifically, under a basestock policy, the size of the order is given by[3]

[3] *Why is the second equality true?*

$$Q_t = (s + 1 - P'_{t-1})\,\mathbb{1}_{P'_{t-1} \le s} = s + 1 - P'_{t-1},$$

where s is known as the *reorder level*, and $s+1$ as the *order-up-to* level. Note that the amount ordered depends on the *end-of-period* inventory position, and that the inventory control policy uses the inventory position, not the

inventory level. By combining this ordering rule with (2.3.2), we see that the $P_t = s+1$ for all t.

A basestock policy is often used to control inventories of expensive items in which the order cost K is small relative to the price of the item or can be easily included in the selling price of the item. For instance, when somebody buys a washing machine, the customer is often prepared to pay to have the washing machine delivered and installed in place. Also, customers do not buy multiple washing machines as the same time, hence, any setup or ordering costs must be covered by each single demand.

AN (s,S) POLICY is appropriate in situations in which the size of the deliveries can vary and the order cost is significant. The update rule is as follows. When the inventory position is at or below s, we order up to level S, specifically,

$$Q_t = (S - P'_{t-1})\,\mathbb{1}_{P'_{t-1}\le s}.$$

As in the basestock case, the parameter s is called the *reorder level*, while S is the *order-up-to* level. The basestock policy is special case when we set $S = s+1$. Sometimes supermarkets use (s,S) policies to control the inventory of packaged meat, like bacon. The inventory levels are tracked real time, and there are daily deliveries. As long as the inventory level is sufficiently high, there is no need to replenish (hence a basestock policy is not useful), and, as the items are perishable, it seems best to only replenish when there are very little items left. Then the shelvers should put the newest items under the older items, so that customers (hopefully) pick the oldest items first.

THE (Q,r) POLICY is useful when the ordering cost is large, or can be easily spread over multiple items. The order quantity is taken as a multiple of a minimal order quantity Q such that $P(t)$ always lies above the reorder level r and below $r+Q+1$. In a formula,

$$Q_t = Q\left\lceil \frac{r+1-P'_{t-1}}{Q} \right\rceil \mathbb{1}_{P'_{t-1}\le r}.$$

So, a basestock policy is a (Q,r) policy with $Q=1$ and $r=s$. The (Q,r) policy is mostly used by supermarket supply chains to replenish groceries. The factory packets the items in batches, and transports these batches on pallets to distribution centers. At a distribution center, batches are picked from the pallets and send to supermarkets. Finally, at the supermarket a shelver removes the cellophane or box and places the single items on the shelf. Thus, to reduce ordering costs, the entire supply chain orders and transports batches of items, rather than single units. Moreover, as for supermarkets leadtimes are in the order of a day, the order size Q is typically larger than the average daily demand.

Admittedly, it is a bit strange that the (Q,r) policy is not called the (Q,s) policy, but this is the names that is commonly used.

FINALLY, THE (T,S) POLICY specifies to order up to S every T periods. In a formula:

$$Q_t = (S - P'_{t-1})\,\mathbb{1}_{\mathrm{mod}\{i,T\}=0},$$

where mod$\{i, T\}$ is the remainder of i/T, In settings in which it is convenient to supply stores in a fixed order, this policy makes sense. The route length determines the number of periods T between the deliveries. Then, at a visit, the inventory is refilled to level S. An example can be highway service areas whose fuel tanks are refilled during the night.

Clearly, when we have a (simulated) sequence of period demands $D_1, D_2, \ldots$ and we have an inventory control rule, we can run the above recursions to compute how the inventory level and position behave over time. It remains to develop a number of measures to assess how the inventory control rule performs, in particular to see how costs depend on the policy parameters.

FOR COST COMPUTATIONS we need to make a choice when to account for the period cost of inventory and backlog. Here we choose to charge b Euro for each demand in backlog at the end of the period and h per item on hand. If there is a cost K per time we place an order, the average cost up to time n becomes

$$Kq' + hI_t'^{+} + bI_t'^{-},$$

where

$$q' = \frac{1}{n}\sum_{t=1}^{n} \mathbb{1}_{Q_t'>0} \qquad I'^{+} = \frac{1}{n}\sum_{t=1}^{n} I_t'^{+} \qquad I'^{-} - = \frac{1}{n}\sum_{t=1}^{n} I_t'^{-},$$

which, implicitly, depend on the inventory policy. Clearly, we can search for the policy by running (many) simulations for various policies and compute the average cost for each scenario.

SERVICE LEVELS form a set of measures to quantify the extent to which demand is met. Here we discuss three different such measures. The *ready rate* is the fraction of periods in which the end-of-period inventory is not negative, i.e.,

$$\alpha := \frac{1}{n}\sum_{t=1}^{n} \mathbb{1}_{I_t'\geq 0}.$$

The *fill rate* β relates to the fraction of demand satisfied from on-hand stock. Observe first that at period i as much as possible of the demand D_t is satisfied. It seems that this should be equal to $\min\{D_t, I_t\}$, but I_t might be negative. Thus, we should take $\min\{D_t, I_t^+\}$ instead, and with this we define the fill rate as

$$\beta := \frac{\sum_{t=1}^{n} \min\{D_t, I_t^+\}}{\sum_{t=1}^{n} D_t}.$$

We can rewrite this into a nice formula with a few interesting tricks.

Lemma 2.3.1. The fill rate equals

$$\beta = 1 - \frac{\sum_{t=1}^{n} \min\{D_t, I_t'^{-}\}}{\sum_{t=1}^{n} D_t}.$$

Proof. Using in step 1 that $\min\{x, y\} = x - [x - y]^+$, in step 2 that $x - z^+ = \min\{x, x - z\}$, in step 3 (2.3.1), in step 4 that $[\min\{x, y\}]^+ = \min\{x^+, y^+\}$ and $[-I_t]^+ = I_t^-$, we get

$$\begin{aligned}\min\{D_t, I_t^+\} &\overset{1}{=} D_t - [D_t - I_t^+]^+ \\ &\overset{2}{=} D_t - [\min\{D_t, D_t - I_t\}]^+ \\ &\overset{3}{=} D_t - [\min\{D_t, -I_t'\}]^+ \\ &\overset{4}{=} D_t - [\min\{D_t, I_t'^-\}.\end{aligned}$$

Summing and dividing gives the result. □

The third measure is the *cycle service level* which is defined as the fraction of cycles in which the inventory was not negative; a cycle is the time between the arrival of two consecutive replenishments. To capture this in a formula, note first that a cycle ends in period t when $Q_{t-L} > 0$ (because then a replenishment arrives). Second, there is still on-hand inventory at the start of period t when $I'_{t-1} \geq 0$. Thus, the number of cycles in which the inventory was non-negative must be $\sum_{t=L+1}^{n} \mathbb{1}_{Q_{t-L}>0} \mathbb{1}_{I'_{t-1}\geq 0}$. The cycle service level then follows from averaging over the total number of cycles $\sum_{t=L+1}^{n} \mathbb{1}_{Q_{t-L}>0}$:

$$\alpha_c = \frac{\sum_{t=L+1}^{n} \mathbb{1}_{Q_{t-L}>0} \mathbb{1}_{I'_{t-1}\geq 0}}{\sum_{t=L+1}^{n} \mathbb{1}_{Q_{t-L}>0}}.$$

The cycle service measure is much used in practice because it is simple to execute: when a replenishment arrives, just register whether there is still on-hand stock. However, it is often not accurate, hence not helpful to understand how many customers have been served from on-hand stock (which is actually what we want to know).

INVENTORY AND QUEUEING systems are related throught three types of 'buffer': *inventory*, *capacity*, and *time*. These three related concepts are exceedingly useful[4] in the analysis and improvement of nearly any logistic system as they can traded against each other to satisfy demand. Suppose demand can be backlogged, that is, a company does not have meet (all) demand from stock, but customers agree that they (sometimes) have to wait for their demand to be satisfied. In this case, the company needs *more* time, but *less* inventory to meet demand, which can reduce overall cost. In the limiting case in which it is impossible to stock the product, e.g., operations in a hospital, or very expensive, e.g., customer-specific machines, all demand is backlogged, thereby reducing the inventory system to a queueing system with finite capacity. The company can still increase capacity, as this typically shortens queueing times. However, increasing capacity comes at a cost. All in all, many businesses struggle with how to organize *capacity* and *inventory* levels such that the *time* limitations as imposed by customers are met.[5]

[4] *If you consider to become consultant, then memorize this as you can use it time and again.*

[5] *Hopp and Spearman [2008] offers very nice additional insights on this material.*

Ex 2.3.1. Here is check to see whether you understand the definitions. Does the inventory level or the inventory position trigger the orders? Does the inventory level or the inventory position depend on the lead time? Is there a difference inventory position and inventory level when $L = 0$?

Ex 2.3.2. In production environments it is important to decide which products have to be make-to-order (MTO) and which make-to-stock (MTS). Why is a queueing system a model for MTO production? What inventory control model would best model a queueing system, and what are the parameters?

Ex 2.3.3. Suppose $h \gg b$, i.e., inventory on hand is much more expensive than demand in backlog, what service level would you like?

Ex 2.3.4. Which variables (period, order quantity) are fixed, which can vary, in each of the inventory policies we discussed above? Which policy should be able to achieve the lowest cost?

Ex 2.3.5. Above we assumed that demand in excess of the stock level is backlogged. Modify the recursions of the basestock model such that it can cope with lost demand. For instance, if right after the start of period i, the inventory level $I_t = 5$, and $D_t = 9$, then 4 items are lost, and 5 are accepted.

2.4 SIMULATING INVENTORY SYSTEMS

In this section we first develop a simulation environment to analyze an inventory system.

THE LIGHTHOUSE COMPANY sells ceiling lamps and considers to use an (s, S) policy to control the inventory since the replenishment cost, $K = 50$ euro is relatively high. The company wants to use simulation to find sound values for s and S.

The details of the case are like this. Ceiling lamps sell at 100 Euro while the buying price is 40 Euro. The monthly holding cost is estimated to be 50% of the buying price. To prevent customers from buying a lamp at a competitor (e.g., Internet sales), the Lighthouse Company offers a reduction of 20% per day of the regular selling price when customers cannot be satisfied from on-hand stock. The pmf f of the daily demand is given by

$$f_0 = 1/6, \quad f_1 = 1/5, \quad f_2 = 1/4, \quad f_3 = 1/8, \quad f_4 = 11/120, \quad f_5 = 1/6.$$

The leadtime $L = 2$ days.

With the code below we set up a simulation to estimate the performance measures for one specific (s, S) policy. We go step by step, and provide explanations above the code to which it relates.[0]. You can copy the code to a file and it should run without problems (provided you copy all lines correctly).

[0] *You should use Google, or ChatGPT, to look up further information if necessary.*

Most of the next modules should by now be familiar to you. `scipy.stats` offers a nice class `rv_discete` that we can use, among much other functionality, to generate random deviates of specific distribution. Finally, `icecream` provides some more overview of what is printed than the regular `print()` function.

```
import numpy as np
from numpy.random import default_rng
from scipy.stats import rv_discrete
```

```
5  from icecream import ic  # simple printing
6
7  rng = default_rng(3)  # set the seed
```

The documentation of `rv_discrete` that to make a distribution we have to provide two arrays of numbers, the first is the support of the rv, the second the pmf.

```
11 xk = range(6)
12 pk = [1 / 6, 1 / 5, 1 / 4, 1 / 8, 11 / 120, 1 / 6]
13 demand = rv_discrete(values=(xk, pk))
```

Here are the other model parameters. Check in particular how we compute the cost parameters.[1]

[1] *Getting the units right is not always easy.*

You should also check the line numbers at the left margins in the code blocks. Where the line numbers skip from one block to the next, the code in my python file has empty lines.

```
17 L = 2
18 h = 40 * 0.5 / 30  # daily holding cost
19 b = 100 * 0.2  # daily backlog cost
20 K = 50
21
22 s, S = 3, 20  # The policy parameters
```

We next compute the inventory positions and inventory levels. For ease we combine (2.3.1) into single recursion to update P'_t, and likewise for I'_t. In the code, we write `Pp` to say 'Pprime', to recall that we mean the end-of-period inventory position. With regard to line 27, you should know that expressions like `(3 <= 4)` evalutate to `True` which in turn become 1 in computations. Thus, `Pp[t - 1] <= s)` implements the indicator functions.[2] Thus, We generate `N` random deviates for the demand and use the random generator to ensure we get the same numbers every time we run the simulation. The first `for` loop for the inventory levels computes I for the first couple of period, because $t - L < 0$ for $t < L$. In the second loop we should protect against a simple error: it might happen that the number of periods over which we simule is smaller than L.

[2] *Ask ChatGPT to explain such code, it's a great help.*

```
26 N = 100
27 D = demand.rvs(size=N, random_state=rng)
28 Pp = np.zeros(N)
29 Qp = np.zeros(N)
30 Ip = np.zeros(N)
31 Pp[0] = Ip[0] = S
32
33 for t in range(1, N):
34     Qp[t] = (S - Pp[t - 1]) * (Pp[t - 1] <= s)
35     Pp[t] = Pp[t - 1] + Qp[t] - D[t]
36
37 # Mind the leadtime L
38 for t in range(1, min(L, N)):
39     Ip[t] = Ip[t - 1] - D[t]
```

```
for t in range(min(L, N), N):
    Ip[t] = Ip[t - 1] + Qp[t - L] - D[t]
```

The last block of code computes the measures. The cycle service level α_c is worth to study.[3] We use slicing, which is a technique that is much used in python, but also in matlab.

[3] *Ask ChatGPT for explanation if you find this line hard.*

```
ic(D.mean(), Pp.mean(), Ip.mean(), Qp.mean())

Iplus = np.maximum(Ip, 0)
Imin = np.maximum(-Ip, 0)

ic(Iplus.mean(), Imin.mean())

cost = K * (Qp > 0).mean() + h * Iplus.mean() + b * Imin.mean()
ic(cost)

alpha = (Ip >= 0).mean()
beta = 1 - np.minimum(D, Imin).sum() / D.sum()
alpha_c = ((Qp[:-L] > 0) * (Ip[L - 1 : -1] >= 0)).sum() / (Qp[:-L] > 0).sum()
ic(alpha, beta, alpha_c)
```

It remains to find good policy parameters, i.e., s and S. In principle this is simple: just change s and S over a range of values and see how that affects the performance measures. However, finding the overal *optimal* parameters *efficiently* is not easy. To minimize the cost, I have published an article on how to do that, but I did not study how to minimize average cost under some constraint on one of the service level measures, for instance.

2.5 GENERAL BEHAVIOR OF QUEUEING SYSTEMS

In this section we develop a simulation environment to obtain insight into general behavior of queueing systems and make graphs of the evolution of queue length processes.

WE NEXT TURN our attention to simulating a queueing system and obtain some general understanding of how such systems behave. The first point of a queueing system we should address is its stability. Intuitively, when work arrives faster than it can be served, on average, the queue must increase as a function of time, while when work arrives slower, the queue must stay limited (most of the time). To check this we show how to make Fig. 2.5.1 which contains the graph of the queue length process for a number of different arrival rates and capacities.

We import the standard modules, cf., Section 2.2. The next python function below simulates the queue length process for a given sequence of arrivals $a = (a_0, a_1, \ldots, a_n)$ and service capacities $c = (c_0, c_1, \ldots, c_n)$. We set L_0 to `L0` and start simulating from time 1 onward. Thus, the values a_0 and c_0 remain unused. Note that for `d` we assume that the jobs arriving in period i can be served in that same period.

```
def compute_queue_length(a, c, L0=0):
    """Compute queue length L for  arrivals a,  capacities c
```

```
    and starting level L0"""
    L = np.zeros_like(a)
    L[0] = L0
    for i in range(1, len(a)):
        d = min(c[i], L[i - 1] + a[i])
        L[i] = L[i - 1] + a[i] - d
    return L
```

Next, we set the parameters for the simulation.

```
labda, mu = 6, 7  # arrival and service rates
L0 = 40  # starting level of queue
num = 50  # run length
x = np.arange(0, num)  # x values for the plot
```

To get a feel for the variability inherent to stochastic systems, we plot a few different sample paths of the queue length process. Thus, we give 10 different seeds to the random number generator, simulate the queue length for each different seed, and add each sample path to `ax1`. For the simulation, we take a_i uniformly distributed on the integers $\lambda-1, \lambda, \lambda+1$ with $\lambda = 6$, so that 6 jobs arrive on average in a period. As the function `rng.integers(l, h)` generates iid uniformly distributed random deviates on $l, l+1, \ldots, h-1$, we should set $h = \lambda+2$ instead of $h = \lambda+1$. The service capacity is constant and equal to $\mu = \lambda+1 = 7$, so the system must be stable.

```
fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(6, 3), sharey=True)
ax1.set_xlabel("time")
ax1.set_ylabel("Queue length")
ax1.set_title("Stable system")

for seed in range(10):
    rng = np.random.default_rng(seed)
    a = rng.integers(labda - 1, labda + 2, size=num)
    c = mu * np.ones_like(a)
    L = compute_queue_length(a, c, L0)
    ax1.plot(x, L, linewidth=0.75)
```

We repeat the simulation for a system in which $\mu = 5$, hence we expect this to be unstable. The last step is to write the figure to a file. Fig. 2.5.1 shows the result of our work.

```
mu = 5
ax2.set_xlabel("time")
ax2.set_title("Unstable system")
for seed in range(10):
    rng = np.random.default_rng(seed)
    a = rng.integers(labda - 1, labda + 2, size=num)
    c = mu * np.ones_like(a)
    L = compute_queue_length(a, c, L0)
    ax2.plot(x, L, linewidth=0.75)


fig.tight_layout()
fig.savefig('../figures/queue-discrete-time-stability.pdf')
```

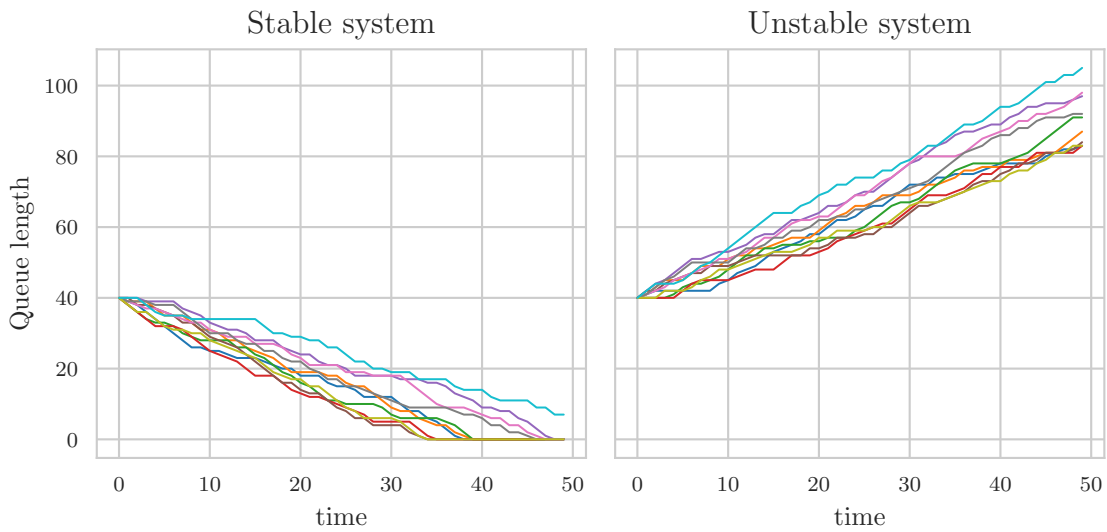


Figure 2.5.1: Drift of queue length process. In the left panel, the queue length starts at 40, and the service capacity is $\mu = 7$ while the arrival rate is $\lambda = 6$, so the initial queue drains at about 1 per period. In the right panel, $\mu = 5$, $Q_0 = 40$, and the queue increases at a rate 1 job per period. Clearly, in one case the queue length process decreases, in the other it increases.

HOW ABOUT REDUCING the difference between λ and μ? When there is little extra capacity, i.e. μ is just a little larger than λ, it must take longer to drain large fluctuations of the queue length than when μ is quite a bit larger than λ. To see this in quantitative terms, we run a simulation, one with $\mu = 6.5$ and another with $\mu = 6.2$ while keeping $\lambda = 6$, and we compute the mean and variance of the number of jobs in the system.

To do so, we need a sequence of random integers for c such that the mean is $mu = 6.2$, say. One way to make such a sequence is to set $c_i = 6 + b_i$ where b_i is a Bernoulli rv with success probability $p = \mu - 6 = 0.2$.

The code for this is simple because we don't need to make a graph this time. We just add the next code to the code above.

```
labda, mu, L0 = 6, 6.1, 0
num = 5000

a = rng.integers(labda - 1, labda + 2, size=num)
c = int(mu) * np.ones_like(a)
p = mu - int(mu)  # fractional part of mu
c += rng.binomial(1, p, size=len(c))
print(c.mean())  # just a check
L = compute_queue_length(a, c, L0)
print(L.mean(), L.var())
```

The results are $\mathsf{E}[L] = 0.7$, $\mathsf{V}[L] = 1.1$ for $\mu = 6.3$, and $\mathsf{E}[L] = 2.2$, $\mathsf{V}[L] = 5$ for $\mu = 6.1$. (Here $\mathsf{E}[L]$ stands for the sample average $n^{-1}\sum_{i=1}^{n} L_i$, and likewise for $\mathsf{V}[L]$.[0]) Indeed, the mean queue length increases when service capacity becomes tighter.

Suppose we cannot serve jobs in the period in which they arrive. Then we use the code `d = min(c[i], L[i - 1])` to update `d`. In this case, $\mathsf{E}[L] = 8.1$,

[0] *We are in a state of sin here: the mean of a rv is not the same as its sample average. The strong law of large numbers makes a statement only about the limit; for small sample sizes, the difference can be quite large.*

$\mathsf{V}[L] = 6$ when $\mu = 6.1$. Not serving jobs in the period they arrive has a non-negligible effect on the queue length. Clearly, being a bit flexible when dealing with customers can the queue quite considerably.

WOULD VARIABILITY AFFECT the average queue length? We can suspect so, because when $a_i = 1$ and $c_i = 1$ for all i, and $L_0 = 0$, then $L_i = 0$ for all i. However, if $c_1 = 5$ whenever i is a multiple of 5 and $c_i = 0$ elsewhere, then we do see queues appearing and disappearing. Let's use simulation to see how variability can influences the queue length process. The goal is now to make Fig. 2.5.2.

Before doing this, we need a way to quantify *relative* variability. To see the point, suppose the standard deviation of the time to serve a job is 1 minute. When the mean service time is 1 hour, we are inclined to call the service times very regular, while if the mean is just 1 minute, we would call it irregular. To capture this difference, we define the fundamentally important concept of *square coefficient of variation (SCV)* of a random variable X as

$$C_X^2 := \frac{\mathsf{V}[X]}{(\mathsf{E}[X])^2}.$$

Continuing with the simulation, we like to find a probability distribution for the services $\{c_i$ such that it has a specific mean μ and a SCV c^2. A simple way to achieve this is to use a rv X such that $\mathsf{P}\{X = b\} = p$ and $\mathsf{P}\{X = 0\} = 1 - p$, and require that $\mathsf{E}[X] = pb = \mu$ and $C_X^2 = c^2$. Using that $\mathsf{V}[X] = \mathsf{E}\left[X^2\right] - (\mathsf{E}[X])^2$, we see that $C_X^2 = \mathsf{E}\left[X^2\right]/(\mathsf{E}[X])^2 - 1$. Rewriting this, we get $\mathsf{E}\left[X^2\right] = \mu^2(c^2+1)$, but we also know that $\mathsf{E}\left[X^2\right] = pb^2$. Solving for b and p in terms of μ and c^2 gives that $p = 1/(1+c^2)$ and $b = \mu/p = \mu(1+c^2)$.

With this construction, we can fix $\mathsf{E}[c_i] = \mu$, but vary the SCV from one simulation to another. This next function implements the construction of a vector of rvs with the required properties.

```
def make_vector_mean_scv(mu, scv, num):
    """Make a vector of length num with a given mean mu
    and square coefficient of variation scv"""
    p = 1 / (1 + scv)
    b = mu / p
    vec = b * rng.binomial(1, p, size=num)
    return vec
```

Now we can run the simulations and make graphs for $c^2 = 0.5, 1, 2$ for the periodic service capacities. The arrivals are constant.

```
num = 5000
x = np.arange(0, num)
fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(6, 3), sharey=True)

ax1.set_title("Constant arrivals")
ax1.set_xlabel("time")
ax1.set_ylabel("Queue length")
a = make_vector_mean_scv(mu=6, scv=0, num=num)  # constant arrivals
for scv in (2, 1, 0.5):
    c = make_vector_mean_scv(mu=6.5, scv=scv, num=num)
```

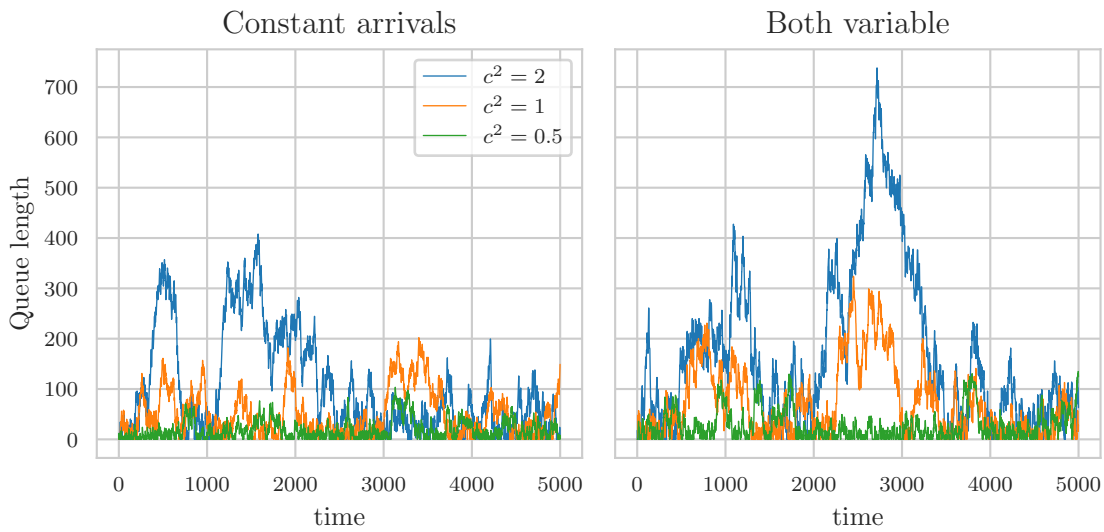


Figure 2.5.2: The influence of variability on queue length. In the left panel the arrivals are constant, but the number of services per period is variable with SCVs c^2 as indicated in the legend. In the right panel, we have variability in both arrivals and services. Note that the scales on the y-axes are not the same in the left and right panel. Thus, when there is variability in both arrivals and services, the queue length seems to become yet more variable.

```
    L = compute_queue_length(a, c, L0=0)
    ax1.plot(x, L, linewidth=0.5, label=f"$c^2 = {scv}$")
ax1.legend()
```

To see the effect of variability in both the arrivals and services, we make a second plot with the next code, and save the result to file. Fig. 2.5.2 shows clearly that variability has quite an influence on the queueing behavior: the larger the SCV, the larger the excursions of the queue length.

```
ax2.set_xlabel("time")
ax2.set_title("Both variable")
for scv in (2, 1, 0.5):
    a = make_vector_mean_scv(mu=6, scv=scv, num=num)
    c = make_vector_mean_scv(mu=6.5, scv=scv, num=num)
    L = compute_queue_length(a, c, L0=0)
    ax2.plot(x, L, linewidth=0.5, label=f"$c^2 = {scv}$")

fig.tight_layout()
fig.savefig('../figures/queue-discrete-time-scv.pdf')
```

In conclusion, when capacity is tight, or when relative variability in arrivals or services is large, queues are large. Hence, if we want to improve the performance of a queueing system, we basically have four options, increase the capacity, reduce the load (or demand), reduce the SCV of the arrivals, or reduce the SCV of the services, or of both. Of course, these options are not exclusive, for instance, we can try to reduce the variability of the service process and add some capacity at the same time.

IN OUR LAST example we consider some queues in sequence. For instance, to get your grade for this queueing course, several queueing stations are involved. First, I grade the exams (and handle all changes that might occur during the perusal). Then I mail an csv file with the grades to the secretary to have it uploaded to some database. The secretary also makes a print of this file which I have to sign because I am the examiner of the course. This paper is sent to a general office that logs them to show that all internal processes have been in line with the legal obligations of the university. In abstract terms, there are three stations (me, the secretary, and back office), and at least I am involved twice, once for the grading and a second time for the signing. More generally, many, if not most, service systems consists of a sequence of servers that have to be passed (multiple times) to complete a service from the perspective of a customer.

To get some insight into such systems, we can simulate a chain of 4 stations in tandem, which means that jobs arrive at station 1, are sent to station 2, and so on, until they leave the network after station 4. Jobs don't visit the same server, nor are lost or dropped somewhere half way the network.

Since in a tandem network the departures of station i are fed as arrivals into station $i+1$, we have to update the function to compute the queue lengths a bit.

```
def compute_queue_length(a, c, L0=0):
    """Compute departures d and queue length L for
    arrivals a,  capacities c and starting level L0"""
    L = np.zeros_like(a)
    d = np.zeros_like(a)
    L[0] = L0
    for i in range(1, len(a)):
        d[i] = min(c[i], L[i - 1] + a[i])
        L[i] = L[i - 1] + a[i] - d[i]
    return d, L  # return d too
```

We next generate the arrivals at station 1, and the service capacities at each of the stations. Pay attention to the capacities: observe that station 2 has the slowest server. Note that if we use Poisson distributed service capacities, an integer amount of jobs is served each period, but the expected number served can be fractional.

```
labda, L0 = 6, 0
num = 5000
x = np.arange(num)

rng = np.random.default_rng(3)
a = rng.integers(labda - 1, labda + 2, size=num)
c1 = rng.poisson(labda + 1, num)
c2 = rng.poisson(labda + 0.1, num)
c3 = rng.poisson(labda + 1.5, num)
c4 = rng.poisson(labda + 1.0, num)
```

It remains to make the plots.

```
fig, ax = plt.subplots(figsize=(5, 3))
ax.set_xlabel("time")
ax.set_ylabel("Queue length")

d1, L1 = compute_queue_length(a, c1)
ax.plot(x, L1, linewidth=0.5, label="Q1")

d2, L2 = compute_queue_length(d1, c2)
ax.plot(x, L2, linewidth=0.5, label="Q2")

d3, L3 = compute_queue_length(d2, c3)
ax.plot(x, L3, linewidth=0.5, label="Q3")

d4, L4 = compute_queue_length(d3, c4)
ax.plot(x, L4, linewidth=0.5, label="Q4")

plt.legend()
fig.tight_layout()
fig.savefig('../figures/queue-discrete-network.pdf')
```

From Fig. 2.5.3 we learn that the station with the slowest server typically has the most jobs in queue in front of it. This brings us to an interesting idea. If we want to improve a queueing network, we should try of course to spend our (hard-earned) money on the bottleneck. Now, in many practical situations it is not directly evident what station is the bottleneck, because it can be difficult to obtain real good estimates of the number of services that a server can do per day. (There are many reasons why this is difficult. The capacity may depends on the person at the station on a certain day. There are disturbances due to raw materials missing or whether some machine at the station broke down or not. In short, in practical situations, there are many messy details that disturb the assembly of high quality data.) However, one clue to find the bottleneck station is by looking at the queue lengths in front of each station: the station with the largest queue is a good candidate for being a bottleneck.

HERE ARE SOME further exercises to practice with setting up and implementing recursions in code.

Ex 2.5.1. Which inventory rule does the next code implement? What are the policy parameters?

```
import numpy as np

gen = np.random.default_rng(seed=42)

L = 2
n = 100
Q_size = 10
s = 20
labda = 3
D = gen.poisson(labda, size=n)

Pp = np.zeros_like(D)
Qp = np.zeros_like(D)
for t in range(1, len(D)):
```

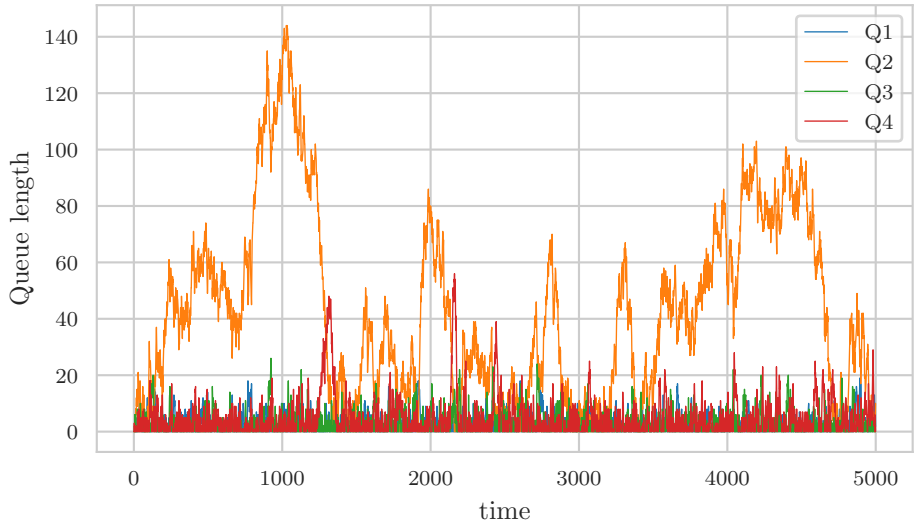


Figure 2.5.3: A queueing network of 4 stations in tandem.

```
    Qp[t] = Q_size * (Pp[t - 1] <= s)
    Pp[t] = Pp[t - 1] + Q[t] - D[t]

Ip = np.zeros_like(D)
for t in range(L, len(D)):
    Ip[t] = Ip[t - 1] + Q[t - L] - D[t]
```

Ex 2.5.2. A queueing system is under periodic review, i.e., at the end of each period the queue length is measured. Jobs arriving at period k cannot be served in period k and the system cannot contain more than K jobs. Develop code to simulate $\{L_k\}$ and compute the amount of jobs lost per period, and the fraction of jobs lost after simulating for T periods. *Queue with blocking*

Ex 2.5.3. All items that are produced by a machine are tested. With probability p an item turns out to be faulty, and is returned to the queue for repair. The production time of new and faulty items is equal. Supposing items cannot be served in the period they arrive, develop a set of recursions to simulate $\{L_k\}$. *Systems with yield loss*

Ex 2.5.4. A fraction p of items do not meet the quality requirements after the first pass at a machine, but require a second pass to repair the problems. Assume that repair jobs need half the service time of new jobs and are served with priority over new jobs. Develop the recursions. *Rework*

Ex 2.5.5. Consider a production network with two production stations in tandem and blocking: the server at station 1 is not allowed to produce more than the amount M station 2 can maximally contain. We assume also that work jobs moves first from station 1 to station 2 before jobs from station 2 leave. Thus, besides the regular conditions, we need to impose $c^1_{k+1} \leq M - L^2_k$. Find recursions. *A tandem queue with blocking*

Ex 2.5.6. Consider a production situation with two stations A and B that send their products to station C. Derive the recursions. todo make the code *Merging streams*

Ex 2.5.7. Consider the discrete-time model specified by (2.1.1). We assume that a_k is a *batch* of jobs arriving in period k *after* the departures d_k have left. Provide a simulation to estimate $\mathsf{P}\{L \le m\}$ for a situation in which the first job of the batch sees $L_{k-1} - d_k$ jobs in the system, the second sees $L_{k-1} - d_k + 1$ jobs, and so on.

Performance measures obtained by sampling in discrete-time queueing models require some extra attention.

3

CONSTRUCTION OF SIMPLE CONTINOUS TIME STOCHASTIC PROCESSES

In Section 3.1 we focus on constructing queueing processes in continuous time. Section 3.2 demonstrates how to implement the recursions in python and make plots. The models of these two sections are clean and nice, but do not allow to model and simulate complicated systems. If we want to include more complicated rules to control queueing systems, we need discrete event simulation. That will be the topic of Section 3.4.

3.1 QUEUEING PROCESS IN CONTINUOUS TIME

In Section 2.1 we modeled time as progressing in discrete chunks. However, we can also model queueing systems in continuous time, so that jobs can arrive at any moment in time and have arbitrary service times. In this section, we develop a set of recursions to construct the waiting times of jobs served in the sequence in which the jobs arrive, i.e., according to the FIFO scheduling rule. First we concentrate on a situation in which there is single server available; at the end we extend this to multiple servers.

LET'S IMAGINE THAT a machine starts working on a one-hour job at 9 am, and there is no job in queue. When the next job arrives before 10 am, this second job has to wait in queue until the first job finishes at 10 am. If, however, this second job arrives after 10 am, it finds the server free, so that its service can start right at the moment of arrival.

More generally, suppose we are given an (ordered) sequence of *arrival times* $\{A_k\}$ and a set of iid *service times* $\{S_k\}$, such that job k arrives at time A_k and needs an amount S_k of service. Then we can compute the sequence of departure times $\{D_k\}$ from the recursion[0]

$$D_k = \max\{A_k, D_{k-1}\} + S_k. \tag{3.1.1}$$

[0] *Note that we assume that job k 'reveals' its service time at the moment it arrives.*

Why is this true? The service of job k cannot start before it arrives, hence, the job cannot leave before $A_k + S_k$ which is the same as saying that $D_k \geq A_k + S_k$. Moreover, the service of job k cannot start before job $k-1$ left the server. Thus job k can also not start before D_{k-1}.

Once A_k and D_k are known, we can compute the *sojourn time* and *waiting time* as

$$J_k = D_k - A_k, \qquad W_k = J_k - S_k,$$

thus, J_k is the total amount of time job k spends in the system, and W_k is the time the job spends in queue (but not at the server). Fig. 3.1.1 shows how the above concepts relate to each other.

WHILE IT IS often simple to measure arrival times in practical situations, it is a bit harder to generate $\{A_k\}$ directly with a simulator. Instead, in simulation we let a random number generator produce a set of iid rvs $\{X_k\}$

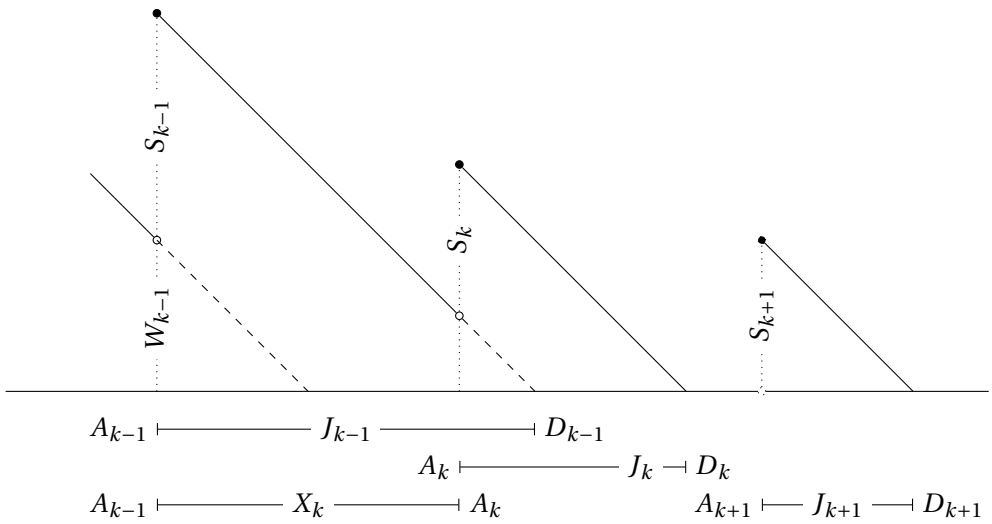


Figure 3.1.1: Construction of the single-server queue in continuous time. The sojourn time is the time a job remains in the system, hence equals the sum of the waiting time and the service time. The virtual waiting time process V is shown by the solid lines with slope -1. Note that right after the arrival of job k, the virtual waiting time $V(A_k) = J_k$.

that represent the *inter-arrival times* between jobs. The arrival times can then be computed recursively with the rule

$$A_k = A_{k-1} + X_k, \qquad A_0 = 0. \tag{3.1.2}$$

Interestingly, if $\{X_k\}$ and $\{S_k\}$ are given, we don't need to first compute $\{A_k\}$ and $\{D_k\}$ to find the sojourn times and waiting times. In fact, suppose that job $k-1$ has to wait a time W_{k-1} in queue and then adds its service time S_{k-1} to the waiting time, so that its sojourn time $J_{k-1} = W_{k-1} + S_{k-1}$. If a time X_k elapses between the arrival time of job $k-1$ and k, then job k has to wait in queue,[1]

[1] $[x]^+ := \max\{x, 0\}$

$$W_k = [J_{k-1} - X_k]^+, \qquad J_k = W_k + S_k, \qquad W_0 = 0. \tag{3.1.3}$$

If we only need J_k then the next rule suffices:

$$J_k = [J_{k-1} - X_k]^+ + S_k. \tag{3.1.4}$$

Henceforth, we make the implicit assumption that $\{X_k\}$ are iid, $\{S_k\}$ are iid, and $\{X_k\}$ are independent of $\{S_k\}$.

WE WILL SEE later that we need the state of the system at arbitrary moments in time, not just at arrival moments. The number of arrivals $A(t)$ as a *function of time* t can be computed from $\{A_k\}$ as

$$A(t) = \max\{k : A_k \le t\} = \sum_{k=1}^{\infty} \mathbb{1}_{A_k \le t}. \tag{3.1.5}$$

Observe that the function $t \to A(t)$ is right-continuous[2].

[2] *A function f if right-continuous if, for all x,* $f(x) = f(x+) := \lim_{y \downarrow x} f(y)$.

In turn, we can retrieve the arrival times $\{A_k\}$ from the sequence of numbers $\{A(t)\}$. For instance, if we know that $A(s) = k-1$ and $A(t) = k$, then the arrival time A_k of the kth job must lie somewhere in $(s, t]$. Specifically,[3]

[3] *[3.1.1]*

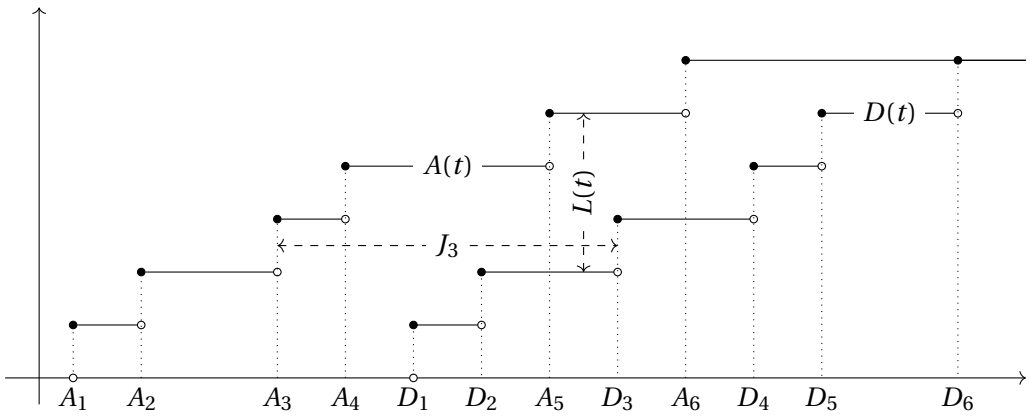


Figure 3.1.2: Relation between the arrival process $\{A(t)\}$, the departure process $\{D(t)\}$, the number in the system $\{L(t)\}$ and the sojourn times $\{J_k\}$. Observe that the arrival and departures times are intertwined.

$$A_k = \min\{t : A(t) \geq k\}, \quad A_0 = 0.$$

And if we have $\{A_k\}$, the inter-arrival times follow from $X_k = A_k - A_{k-1}$. So, with the above recursions and definitions, we can convert into each other the arrival times, the inter-arrival times and the number of arrivals as a function of time. It just depends on what our starting data is to obtain the rest.

Likewise, for the departures $\{D_k\}$ and the *departure process* $\{D(t)\}$ we have the relations

$$D(t) = \max\{k : D_k \leq t\} = \sum_{k=1}^{\infty} \mathbb{1}_{D_k \leq t}, \qquad D_k = \min\{t : D(t) \geq k\}.$$

Once we have the arrival and departure processes, it is easy to compute the *number of jobs in the system* at time t as, see Fig. 3.1.2,

$$L(t) = A(t) - D(t) + L(0), \tag{3.1.6}$$

where $L(0)$ is the number of jobs in the system at time $t = 0$; typically we assume that $L(0) = 0$. Recalling that in a queueing system, a job can either be in queue or in service, we distinguish between the number of jobs in the system $L(t)$, the number of jobs in queue $Q(t)$, and the number in service $L_s(t)$. Clearly, $L(t) = Q(t) + L_s(t)$. Sometimes we write $\tilde{A}(t)$ for the number of jobs that departed from the queue up to time t; it follows that $L_s(t) = \tilde{A}(t) - D(t) = L(t) - Q(t)$.

It is important to realize that the queue length process $\{Q(t)\}$ at general time moments t can be quite different from the queue length process $\{Q(A_k-)\}$ as observed by arriving jobs.[4]

Finally, the *virtual waiting time process* $\{V(t)\}$ is the time a job would have to wait if it would arrive at time t. In other words, the virtual waiting time is the waiting time observed by jobs arriving virtually (but not in reality). To construct[5] $\{V(t)\}$, we simply draw lines that start at points (A_k, J_k) and have slope -1, until the lines hit the x-axis. Once at $y = 0$, the virtual waiting time remains there until an arrival occurs, c.f., Fig. 3.1.1. Here is the expression[6]

[4] *Observe that we write A_k-, and not A_k; we need to be careful about left and right limits at jump epochs.*

[5] *[3.1.5]*

[6] *See [3.1.5].*

$$V(t) = [J_{A(t)} - (t - A_{A(t)})]^+. \tag{3.1.7}$$

Ex 3.1.1. Are the following mappings correct:

$$A_k : \mathbb{N} \to \mathbb{R}, \quad \text{job id (integer) to arrival time (real number)},$$
$$A(t) : \mathbb{R} \to \mathbb{N}, \quad \text{time (real number) to number of jobs (integer)?}$$

Ex 3.1.2. What are the meanings of $A_{A(t)}$, and of $A(A_n)$?

Ex 3.1.3. Can $A(t)$ be defined as $\min\{k : A_k \geq t\}$ or as $\min\{k : A_k > t\}$?

Ex 3.1.4. Show that $L(A_k-) > 0 \iff A_k \leq D_{k-1}$.

Ex 3.1.5. Explain (3.1.7) for the virtual waiting time at time t.

3.2 SIMULATIONS IN CONTINUOUS TIME

With the recursions of the previous section we can simulate simple systems. In this section we implement these recursions in python and show how to build a graph of the system length process L and the virtual waiting time process V. This is not entirely straightward, though: to get around the problems, we need an *event stack*, which is actually a queueing process operating under a specific scheduling rule.[0] More generally, event stacks form the heart of every discrete event simulator you will ever use. In Section 3.4 we will deal with the details of discrete event simultion; here we start simple.

[0] *Thus, we use a queue, i.e., the event stack, to simulate queueing processes.*

THE DYNAMICS OF the single server queue in continuous time is given by the recursions in (3.1.1). The relating code is simple, but there are two small details we should solve. First, suppose we specify the inter-arrival times X as a numpy vector, and we compute the arrival times A as the `cumsum` of X.

```
import numpy as np

X = np.array([2, 3, 4])
A = X.cumsum()
print(A)
```

So `A = [2, 5, 9]`, but this is not what we want, because according to (3.1.2) the arrival times should start with $A_0 = 0$. Moreover, $A_1 = A_0 + X_1$ so we never use X_0. The solution is to set `X[0]=0`, and then take the `cumsum`. The second problem is when we generate a number of service times as a numpy vector. If we were to compute the total offered amount of work, then the natural thing is to call `S.sum()`. However, this includes `S[0]`, while actually job 0 is never served. So, we set `S[0]=0` too. This induces a slight error though: `S.mean()` includes still $S[0]$, so we divide by one job too much; to repair we should compute `S.mean() * len(S) / (len(S) -1)`, but we choose to neglect this small error.

The next code computes first the departure times D, and then the sojourn times J and waiting times W. Note how easy this is. Calling statistics is trivial too.

```python
import numpy as np

rng = np.random.default_rng(3)
labda, mu = 3, 4
num = 10
X = rng.exponential(scale=1 / labda, size=num)
S = rng.exponential(scale=1 / mu, size=num)
X[0] = S[0] = 0
A = X.cumsum()

D = np.zeros_like(X)
for i in range(1, num):
    D[i] = max(A[i], D[i - 1]) + S[i]

J = D - A  # sojourn times
W = J - S  # waiting times

# Statistics
print(J.mean(), J.var())
```

WITH THE RECURSIONS (3.1.1) it is apparently easy to compute the sojourn and waiting times, but it is less simple to compute the number of jobs in queue or in the system at arrival moments, for instance. The problem is that the arrival and departure times are all intertwined, see Fig. 3.1.2. Before discussing how to get around this, we first show how to compute the time-average of the number of jobs in the system, as this is still relatively easy and develops some formulas that we will need for the proof of Little's law, c.f., Section 4.5.

Fig. 3.1.2 clarifies that the number of jobs $L(t)$ in the system at time t is equal to all jobs $A(t)$ that arrived minus the jobs $D(t)$ that departed during $[0,t]$, assuming that $L(0)=0$. Hence, the mean number of jobs during $[0,t]$ must be $t^{-1}\int_0^t L(s)\,\mathrm{d}s$. Another way to look at the integrand $L(s)$ is to count all jobs whose arrival times lie before s and whose departure time lie after s, that is, $L(s)=\sum_{k=1}^n \mathbb{1}_{A_k\le s\le D_k}$, where we assume that the simulation involves n jobs in total. Therefore, the time-average number of L is

$$\frac{1}{t}\int_0^t L(s)\,\mathrm{d}s=\frac{1}{t}\int_0^t\sum_{k=1}^n \mathbb{1}_{A_k\le s\le D_k})\,\mathrm{d}s=\frac{1}{t}\sum_{k=1}^n\int_0^t \mathbb{1}_{A_k\le s\le D_k})\,\mathrm{d}s=\frac{1}{t}\sum_{k=1}^n J_k,$$

because the integral $\int_0^t \mathbb{1}_{A_k\le s\le D_k}\,\mathrm{d}s=J_k$ if $D_k\le t$. In code, to get an estimate for the average system length, just sum all sojourn times and divide by the time the last job leaves the system.

```python
print(J.sum() / D[-1])
```

LET US NEXT turn towards designing an algorithm to compute the number L_k of jobs in the system as seen by job k upon its arrival. Clearly, as time A_k is the arrival time of job k, the number of jobs that arrived during $[0,A_k]$ is precisely k. Thus, if we subtract all jobs that departed before A_k, we have the number of jobs in the system, i..e, $L_k=k-\sum_{i=1}^k \mathbb{1}_{D_i\le A_k}$. Now this formula works, but has a terrible algorithmic performance. To see this,

note that for job k this formula compares A_k with the departure times of all earlier jobs. For k jobs in total, we therefore need $1+2+\cdots k = k(k+1)/2$ comparisons, which has $O(k^2)$ complexity. When k is a million, the square is a really large number indeed.

The next code is much more efficient. We keep a *pivot*, i.e., a pointer, to the last departure before job k. Suppose this pivot is i, in other words, job i was the last job that departed before A_k, then, for L_{k+1} we start searching for departures from job $i+1$ onward.

```
L = np.zeros_like(A)
pivot = 0
for k in range(1, len(A)):
    while D[pivot] < A[k]:
        pivot += 1
    L[k] = k - pivot
```

A MORE FLEXIBLE way to compute $\{L_k\}$ is by means of an *event stack*, but to understand what an event stack is, we first need to discuss *heap queues*.

A heap queue stores elements in a list such that they can be retrieved in accordance to some ordering rule. The following piece of code demonstrates the idea. We *push* (add) a tuple containing an age and an animal name to the *heap*, and we *pop* from the heap to get the animals in order of age. Here is an example; study the output carefully.

```
>>> from heapq import heappop, heappush

>>> heap = []

>>> heappush(heap, (25, "Turtle"))
>>> heappush(heap, (21, "Horse"))
>>> heappush(heap, (20, "Lion"))
>>> heappush(heap, (18, "Cat"))
>>> heappush(heap, (14, "Dog"))

>>> print(heappop(heap)) # Why is the dog?
(14, 'Dog')
>>> print(heappop(heap)) # Why is the cat?
(18, 'Cat')
>>> print("Add Elephant")
Add Elephant
>>> heappush(heap, (23, "Elephant"))

>>> while heap:
...     e = heappop(heap)
...     print(e)
...
(20, 'Lion')
(21, 'Horse')
(23, 'Elephant')
(25, 'Turtle')
```

Note that anytime we pop from the heap, the heap becomes one element shorter, and with pushing the heap becomes longer. Moreover, we

can push animals, here an elephant, at our convenience, without having to wait until the heap is empty. As such, a heap queue is the ideal (and efficient) data structure to support simulation because we can push and pop events in any sequence we like, but we have the guarantee that events are popped in the correct sequence of time.

WITH HEAP QUEUES WE have an appropriate data structure to plot the virtual waiting time and the system length processes in Fig. 3.2.1. In passing we encounter some other useful and generic ideas.

Suppose we set $L = 0$ at time 0, and add 1 to L for each arrival and subtract 1 for each departure, then we have the system length if the arrival times and departures times are sorted in time. For this to work, we form events of type (t, Δ) where t is arrival or departure time and $\Delta \in \{-1, 1\}$ to correspond to a departure or arrival, respectively. To sort the events, we form two lists of events and merge them.

There is technical, but important, detail involved. The function `heapq.merge` returns a generator, not a list, but `heapq.heapify` expects a list instead of a generator. To convert the generator obtained from `heapq.merge` we apply `list` to this generator, and then pass the resulting list to `heapq.heapify`.

```
arrivals = [[t, 1] for t in A[1:]]
departures = [[t, 1] for t in D[1:]]
heap = list(heapq.merge((arrivals, departures)))
heapq.heapify(heap)
```

With this heap we can already plot the system length, but for the virtual waiting time need some additional code. Recall from [2.5.6] that $V(t) = [J_{A(t)} - (t - A_{A(t)})]^+$, and that $A(t) = \sum_{k=1}^{\infty} \mathbb{1}_{A_k \le t}$. The following code would work.

```
k = 1
while A[k] <= t:
    k += 1
```

However, the efficiency of this code is terrible. Instead, we should stick the rule that says 'Thou shalt use binary search to find an element in a sorted sequence', we use `np.searchsorted` to efficiently find the index k in an array A such that $A_{k-1} \le t < A_k$ for some given value t. In [3.2.2] we explain why the next code does the job.

```
def At(t, side='right'):
    """
    The right or left continuous version of A(t), depending on SIDE,
    A(t) = max {k : A_k \geq t}
    A(t-) = A(t) - 1.
    """
    return max(np.searchsorted(A, t, side=side) - 1, 0)
```

Note that we include the option `side` to provide us with the left-continuous version $A(t-)$; $A(t)$ is defined as right-continuous.

The virtual waiting time follows now directly from its recursive definition.

```
def V(t, side='right'):
    """
    The right or left continuous virtual waiting time at t, depending on SIDE
    """
    k = At(t, side)
    return max(J[k] - (t - A[k]), 0)
```

All is in place to make Fig. 3.2.1. We have two panels `ax1` and `ax2` on top of each other, and we set the labels at the axes.

```
fig, (ax1, ax2) = plt.subplots(nrows=2, figsize=(6, 3), sharex=True)
ax1.set_ylabel("Virtual time~$V$")
ax2.set_ylabel("Length~$L$")
ax2.set_xlabel("time")
ax2.set_yticks([0, 1, 2, 3, 4, 5])
```

For the sojourn and waiting times we draw triangles.

```
for i in range(1, num):
    ax1.fill([A[i], A[i], D[i]], [0, J[i], 0], c='green', alpha=0.2)
    ax1.fill([A[i], A[i], A[i] + W[i]], [0, W[i], 0], c='blue', alpha=0.2)
```

Setting the opacity `alpha` to 0.2 gives nice results for overlapping triangles.

Next is the code to plot L and V. Pay attention to the fact that we use $V(t-)$ (in code `V(now, 'left')`). Lines 5 and 7 plot vertical dotted lines for visual clarity. Formally speaking, these lines do not belong to the virtual waiting time or system length processes.

```
L, past = 0, 0
while heap:
    now, delta = heapq.heappop(heap)
    ax1.plot([past, now], [V(past), V(now, 'left')], c='k', lw=0.75)
    ax1.plot([now, now], [V(now, 'left'), V(now)], ":", c='k', lw=0.75)
    ax2.plot([past, now], [L, L], c='k', lw=0.75)
    ax2.plot([now, now], [L, L + delta], ":", c='k', lw=0.75)
    length += delta
    past = now
```

To compile the figure with LaTeX and write to file we follow the same steps as we used to make Fig. 2.5.1.

Ex 3.2.1. Suppose that we want to compute the time-average of L for a time $t < D_n$, i.e., a time that lies before the departure time of the last job of the simulation. How would you compute this?

Ex 3.2.2. Why is $\max\{k : A_{k-1} \le t < A_k\} = 1 + A(t)$?

3.3 PROBABILISTIC ARITHMETIC

When the stochastic system is sufficiently simple that its behavior can be characterized in terms of clean mathematical functions, we can obtain

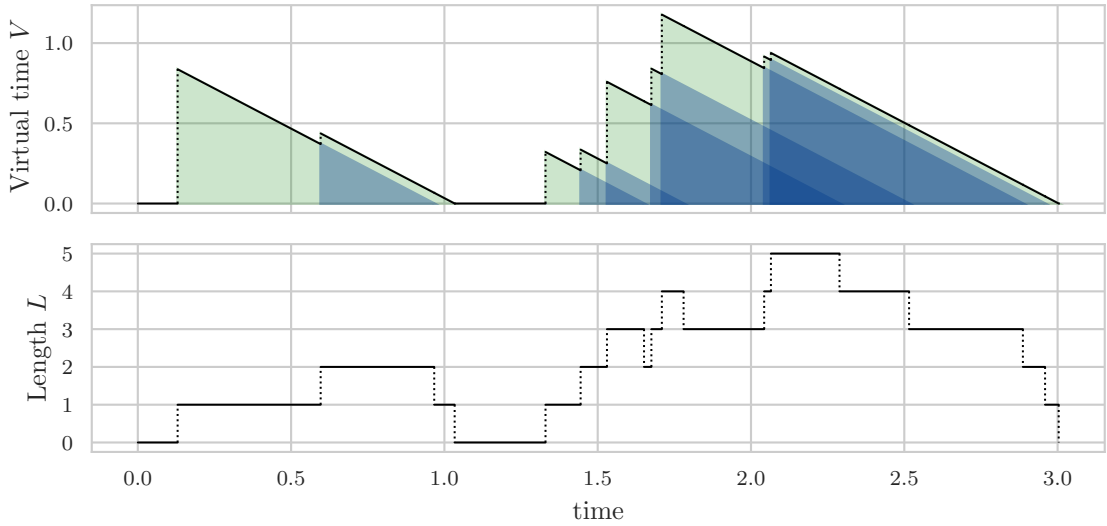


Figure 3.2.1: The top panel shows a graph of the sojourn times and the virtual waiting time, the lower panel the number of jobs in the system. The darker the blue, the more jobs the system contains at that moment.

insight into the probability mass functions of interest by direct algorithmic computations rather than by using simulation. There are two main advantages for this approach. The first is that the direct computations provides us with some insight into how many runs of a simulation we need before obtaining statistically (somewhat) sound data. If we simulate for a too short amount of time, we obtain too few sample paths to get a feeling for the statistical properties of the process we are interested in. However, we also do not want to waste way too many computer cycles (and time) just to be sure we have not missed anything.[0] The second reason is that building code to deal with rvs deepens our understanding of probability and numerical analysis. This will help us prevent making subtle errors; we will point out a few in the main text and in the exercises. We start with an example and then present the code that deals with all the computations.

[0] *As with all things in life, we need to find the right balance.*

WE CAN DETERMINE the waiting time of the nth job in a single-server queue by means of the recursion $W_n = [W_{n-1}S_{n-1} - X_n]^+$. That is, for given random deviates $\{X_i\}$ and $\{S_i\}$ we can compute the *number* W_n. If want to get information about the *distribution* of W_n, we need to do some more work. One is to repeat the recursions many different times with different seeds and make a histogram of the observed values of W_n. There is however, another way that is directly based on arithmetic with the pmfs of X and S. Let us show how to we can do this. Fig. 3.3.1 shows the result of the code below.

Besides the regular libraries we load `pytictoc` which allows us to time how long it takes to run certain parts of the code. The module `random_variables.py` contains our code to handle the probabilistic arithmetic; we discuss that below. We skip the lines that deal with `seaborn` on how to make the LaTeX plots;

```
import numpy as np
import matplotlib.pyplot as plt
from pytictoc import TicToc

import random_variable as rv
```

In the recursions for W we use the $[\cdot]^+$ function. We also instantiate a `tictoc` object and set it.

```
def Plus(x):
    return max(x, 0)


tic = TicToc()
tic.tic()
```

An `RV` object uses a dictionary to make the support and pmf of a rv; the keys of the dict represent possible outcomes, the values the associated probabilities. Observe from the numbers below that the mean service time is larger than the mean inter-arrival time. This is on purpose to see later in the graphs how this affects the distribution of the waiting time and the departure time.

```
X = rv.RV({4: 1 / 3, 5: 1 / 3, 3: 1 / 3})
S = rv.RV({4: 1 / 3, 5: 1 / 3, 7: 1 / 3})
```

The next few lines do the work to compute the distribution of W_{30}. We set the initial arrival and waiting time such that $\mathsf{P}\{A_0 = 0\} = \mathsf{P}\{W_0 = 0\} = 1$. Then, by writing `A += X`, we compute by means of convolution the distribution of the rv $A + X$. This is done implicitly by *overloading* the + operator such that it works on `RV` objects. Below we will discuss operator overloading in a bit more detail. In a similar way we compute $W + S - X$. Finally, we need to apply the 'plus'-function. Note how concise this code is and close it stays to the mathematical formulation of the recursions. This helps much to understand how the maths and the code works. The call to `tic.toc()` prints the time that elapsed between `tic.tic()` and the 'toc'. There is a final remark, we start the recursions at 1. The reason is that we want to compare this result to simulation and the simulation starts necessarily at 1.

```
horizon = 30
A = rv.RV({0: 1})  # arrival time
W = rv.RV({0: 1})  # waiting time
# Mind that we start at 2 instead of 1
A += X
for i in range(2, horizon):
    A += X
    W = rv.apply_function(Plus, W + S - X)

D = A + W + S

tic.toc()
```

Let us now estimate the distribution of W by means of simulation. We compute from $W_{30} = 0$ a number of `num_runs` times and store the outcomes in the array `nth_waiting_time`. In the computations we let the departure times tag along. We use small letters for the variables to avoid a name clash with the capitals like `X` we used earlier.

```
num_runs = 1000
nth_waiting_time = np.zeros(num_runs)
nth_departure = np.zeros(num_runs)
rng = np.random.default_rng(3)
for n in range(num_runs):
    x = X.rvs(horizon, rng)
    s = S.rvs(horizon, rng)
    x[0] = s[0] = 0

    a = x.cumsum()
    w = 0
    for i in range(1, horizon):
        w = Plus(w + s[i - 1] - x[i])
    nth_waiting_time[n] = w
    nth_departure[n] = a[i] + w + s[i]

# check KPIs
print(D.mean(), nth_departure.mean())
print(D.var(), nth_departure.var())

tic.toc()
```

Making the plots follows the same pattern as we did earlier. First we plot the cdfs of the departure times. The keyword `cumulative=True` converts histogram (which is similar to a pmf) to a cdf.

```
fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(6, 3), sharey=True)
ax1.set_title("Departure times")
ax1.set_xlabel("Time")
ax1.hist(
    nth_departure,
    bins=30,
    density=True,
    cumulative=True,
    histtype='step',
    align="right",
    color='k',
    lw=0.5,
    label="sim",
)
ax1.plot(
    D.support(), [D.cdf(k) for k in D.support()], c='k', lw=0.75, label="exact"
)
ax1.legend()
```

And next the waiting times.

```
ax2.set_title("Waiting times")
ax2.set_xlabel("Time")
```

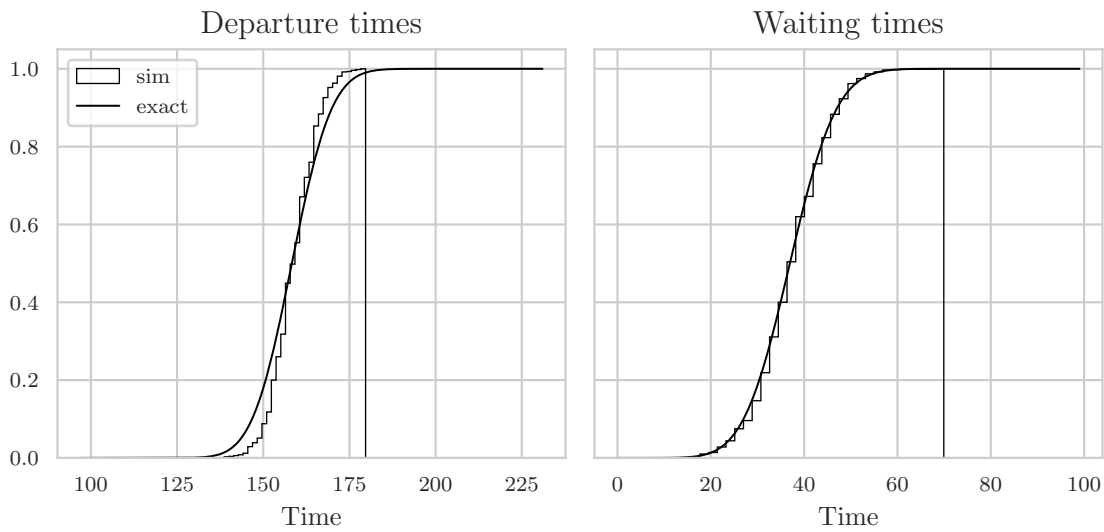


Figure 3.3.1: The cumulative distribution of the departure and waiting time of job 30 with the inter-arrival and service times as mentioned in the text. Observe that the distribution shifts to the right because the queue is not stable. The results of theory and simulation are similar.

```
ax2.hist(
    nth_waiting_time,
    bins=30,
    density=True,
    cumulative=True,
    histtype='step',
    align="right",
    color='k',
    lw=0.5,
    label="sim",
)
ax2.plot(
    W.support(), [W.cdf(k) for k in W.support()], c='k', lw=0.75, label="exact"
)
```

The last step is to save the plot to file.

```
fig.tight_layout()
fig.savefig("../figures/waiting_time_distribution.pdf")
```

Finally, to plot the pmf instead of the cdf, we change the `cdf` in `pmf` and remove the lines with `cumulative=True`. Running the code again gives Fig. 3.3.2.

YOU MIGHT WONDER why we should have a preference for ploting the cdf instead of the pmf of the waiting and departure times; it seems that the pmf is easier to interpret. However, this is not true: the pmf is an ill-behaved object.[1] For instance, let us take the support of X as $3, 4.1, 5$ instead of $3, 4, 5$ as above, and leave the rest of the parameters untouched. Plotting the pmf in the same way as for Fig. 3.3.2, we now obtain Fig. 3.3.3.This looks quite a bit different from the densities in Fig. 3.3.2, to say the least.[2] In general,

[1] *You'll appreciate the wisdom of this only after making some related errors yourself.*

[2] *If you doubt the correctness of the compuations, just run the same code but plot the cdfs instead; the pathology disappear.*

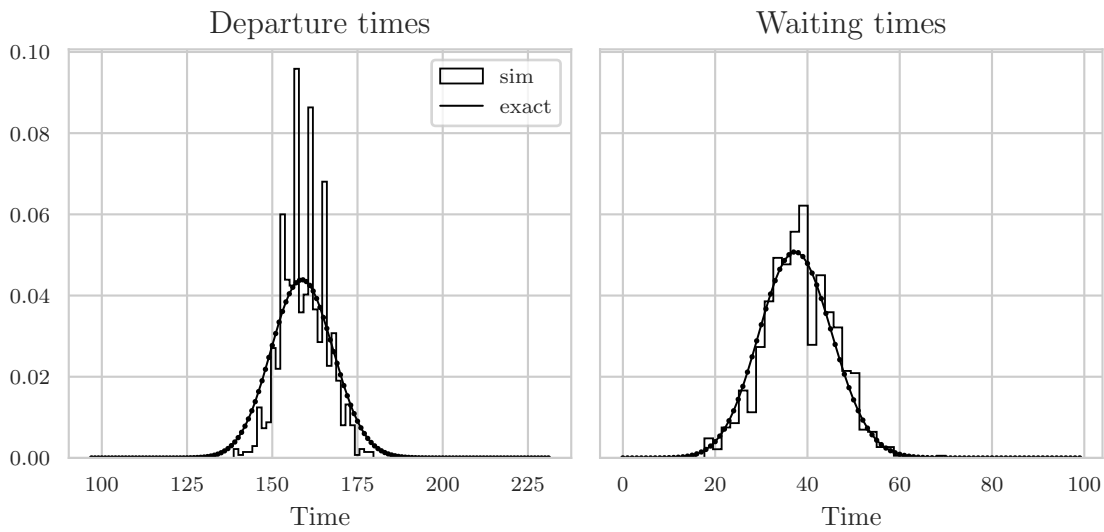


Figure 3.3.2: The pmfs of the departure and waiting time of job 30 with X concentrated on $3, 4, 5$. Compare this to Fig. 3.3.3.

you should remember that plotting the density of a rv is not simple, and I leave the interesting exercise to find out why to you.

Another interesting question is what would happen if we would compute the departure times with the following code, which is the implementation of the recursion $D_n = \max\{D_{n-1}, A_n\} + S_n$.

```
A = rv.RV({0: 1})
D = rv.RV({0: 1})
for i in range(1, horizon):
    A += X
    D = rv.compose_function(max, D, A) + S
```

In fact, this is wrong for a subtle reason. As will become clear from the implementation, the code in `random_variable.py` assumes that the rvs are *independent*! However, D_i and A_{i-1} are dependent.[3] To see this, observe from Fig. 3.3.1 that $\mathsf{P}\{D_{30} \leq 150\} > 0$, but $\mathsf{P}\{D_{30} \leq 150 | A_{30} > 151\} = 0$ while still $\mathsf{P}\{A_{30} > 151\} > 0$.

[3] *Dependent is the same as not independent.*

But why then does it work in the code that implements the recursion $W_i = [W_{i-1} + S_{i-1} - X_i]^+$? This is because W_{i-1} *is* independent of S_{i-1} and X_i so that we can compute $W_{i-1} + S_{i-1} - X_i$ without problems. Keep in mind that stochastic dependence is tricky and can be easily overlooked. To prevent making such mistakes (when it is important) it helps to check the results against simulation. If the results do not agree, there is an error (which may be simple or hard to find).

And this brings us to another conceptual mistake in the code that computes (but in the wrong way as we will see) the distribution of the deparure times with the statement `D = A + W + S`. Let us check Fig. 3.3.1 carefully. The ecdf (empirical cdf) obtained by simulution starts to increases later than the cdf of D but increases faster. The variance of the simulated departure times must therefore be smaller. Increasing the number of samples

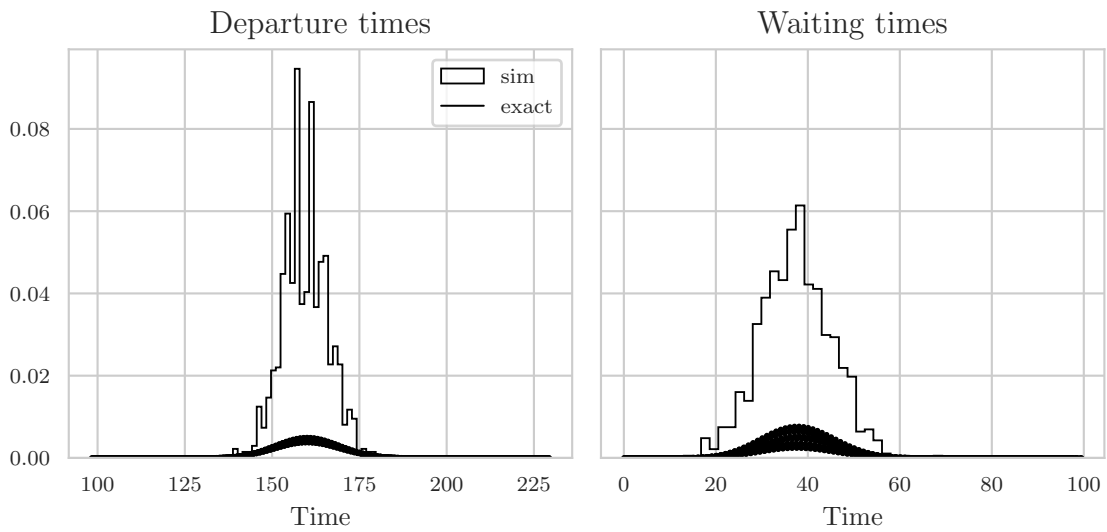


Figure 3.3.3: The pmfs of the departure and waiting time of job 30 with X concentrated on $3, 4.1, 5$ instead of $3, 4, 5$; the rest of the parameters are the same as in Fig. 3.3.1. Note how pathological the pmf of the theoretical distribution behaves. This is not a mistake in the code; when using $3, 4, 5$ as support for X we get Fig. 3.3.2.

does not repair this 'problem'. So what is the matter here? In fact, the arrival time of job 30 and its waiting time are dependent, and theorem 1.2.1 can help to understand why. Suppose that we know that the random sample is such that $N(t) = 30$ and $t \gg S_1 + S_2 + \cdots S_{30}$. theorem 1.2.1 tells us that the arrival times are uniformly distributed, hence in this specific case are probably be spaced out quite a bit. Therefore, the probability must be small that an arrival occurs while another job is in service. Similarly, suppose that $t < S_1$, but $N(t) = 30$. Then all jobs, except the first, wait. In conclusion, A_{30} and W_{30} are dependent, hence D_{30} cannot be computed as simply as the code suggests. In fact, its computation requires a careful build up of the dependency structure of the rvs involved. A nice python package that implements dependence in a correct way is `lea`.

AS PROMISSED ABOVE we now discuss the code that implement the probabilistic arithmetic. By building everything from scratch, our understanding of the workings of libaries such as `lea` or `scipy` increases, but also of probablity theory itself. The code below adds typing information, which means that we specify what type of argument (such as float, or an int) is used.[4]

[4] *In simple scripts I tend not to add typing, but in code that I plan to use later at other places, I find this extra information helpful.*

```
import operator
from fractions import Fraction
from collections import defaultdict
from functools import cache
from typing import Callable

import numpy as np
from numpy.random import default_rng
```

The meaning of the next variables will become clear later.

```
max_denominator = 1_000_000
thres = 1e-16  # Reject probabilities smaller than this.
seed = 3
```

The RV class, which we build in the coming code blocks, stores the pmf and support of a rv, and offers many convenience methods such as `cdf`. We specify the support and pmf of a rv as a dictionary; for instance, `{0: 1 / 3, 1: 2 /3 }` represents a biased coin with support $\{0,1\}$ and pmf $p_0 = 1/3$ and $p_1 = 2/3$. RV use the pmf, support and cdf as private members.

```
class RV:

    """
    A random variable with support concentrated on the keys of a dict and pmfs
    as the values of the dict.
    """

    def __init__(self, pmf: dict[float, float]):
        self._pmf = self.make_pmf(pmf)
        self._support = np.array(sorted(self._pmf.keys()))
        self._cdf = np.cumsum([self._pmf[k] for k in self.support()])
```

Dealing with the pmf as a dictionary requires considerable care. An example helps to understand the problem. I suppose you agree that $1/3 = 1 - 2/3$, so let's see what happens if we run the next code.

```
>>> from collections import defaultdict

>>> res = defaultdict(float)
>>> res[1/3] += 1
>>> res[1 - 2 / 3] += 1
>>> res
defaultdict(<class 'float'>, {0.3333333333333333: 1.0, 0.33333333333333337: 1.0})
```

The dictionary `res` suddenly contains two elements rather than just one. Inspecting the last digit of the keys, we see that in one number the last digit is a 7 and in the other it is a 3. So, for a *computer* $1 - 2/3 \neq 1/3$. This is not something specific to python; the problem lies in the fact that general numbers in $\mathbb{R}$ cannot be represented as floating point numbers.

To get around this, we round the elements of the support to a fixed precision so that we can use fractions instead of floats. This works well as demonstrated by this code.

```
>>> from fractions import Fraction

>>> a = Fraction(1 - 2 / 3).limit_denominator(1000)
>>> b = Fraction(1 / 3).limit_denominator(1000)
>>> a, b, a == b
(Fraction(1, 3), Fraction(1, 3), True)
```

Once you understand this point, the next code is easy. We convert the keys of the dict given as an argument to RV to fractions with a limited

precision. As it may happen that the same key appears twice in the dict, we add the probabilities with the same key. We normalize to ensure that the values of the dict correspond to probabilities.

```
    def make_pmf(self, pmf):
        res: dict[Fraction, float] = defaultdict(float)
        for k, pk in pmf.items():
            if pk >= thres:
                res[Fraction(k).limit_denominator(max_denominator)] += pk
        return self.normalize(res)

    def normalize(self, pmf):
        norm = sum((pmf.values()))
        return {k: pk / norm for k, pk in pmf.items()}
```

To speed up some arithmetic, we need to run over the support such that the elements are decreasing in their pmfs; we use cache to ensure that the result is only computed once.

```
    def support(self) -> np.ndarray:
        return self._support

    @cache
    def sortedsupport(self) -> np.array:
        """Return the support sorted in decreasing order of the pmf."""
        S = sorted(self._pmf.items(), key=operator.itemgetter(1), reverse=True)
        return np.array([k for k, v in S])
```

From the pmf $\{p_k\}$ we want to compute the cdf, the survivor function, mean, variance and standard deviation. All this is easy once we realize that all this can be computed as $\mathsf{E}\left[f(X)\right]$ for suitable chosen functions f. For instance, by taking $f(x) = \mathbb{1}_{x \le y}$, the cdf $F(x) = \mathsf{E}\left[f(x)\right] = \sum_k \mathbb{1}_{k \le x} p_k$. Below is an inefficient method to compute the cdf using `E/`:sidenoteCheck the web on `lambda`.: `lambda x: x <= y`.

```
def cdf_inefficient(self, x: float) -> float:
    return self.E(lambda y: y <= x)
```

Here is the code to compute $\mathsf{E}\left[f(X)\right]$.

```
    def E(self, f: Callable[[float], float]) -> float:
        """Compute E(f(X))"""
        return sum(f(i) * self.pmf(i) for i in self.support())
```

However, computing the cdf with binary search via `np.searchsorted` is much faster.

```
    @cache
    def cdf(self, x: float) -> float:
        if x < self._support[0]:
            return 0
        if x >= self._support[-1]:
```

```
            return 1
        return self._cdf[np.searchsorted(self._support, x)]
```

With the code above, the next convenience methods are easy to implement. As an aside, in more professional code, we should check the input of `pmf`: if it is called with an element not in the support, it should give some proper error messages. To keep the code short, we don't include it.

```
    @cache
    def pmf(self, k: Fraction) -> float:
        return self._pmf[k]

    @cache
    def sf(self, x: float) -> float:
        """Survivor function"""
        return 1 - self.cdf(x)

    @cache
    def mean(self) -> float:
        return self.E(lambda x: x)

    @cache
    def var(self) -> float:
        return self.E(lambda x: x**2) - self.mean() ** 2

    @cache
    def sdv(self) -> float:
        return np.sqrt(self.var())

    def __len__(self):
        return len(self._pmf)
```

Additionally, we like to able to use our `RV` class to generate random deviates that are distributed according to the cdf F. Recall that if $U \sim \text{Unif}([0,1])$, then $F^{-1}(U)$ has the required distribution, because $\mathsf{P}\left\{F^{-1}(U) \le x\right\} = \mathsf{P}\{U \le F(x)\} = F(x)$, where the last equality follows from the fact that U is uniform on $[0,1]$. Binary sort is useful and fast to compute the inverse $F^{-1}(U)$.

```
    def rvs(self, size: int = 1, rng=default_rng(seed)) -> np.ndarray:
        """Generate an array with "size" number of random deviates."""
        U: np.ndarray = rng.uniform(size=size)
        pos: np.ndarray = np.searchsorted(self._cdf, U)
        return self.support()[pos].astype(float)
```

We are nearly finished with the code, but we need to further elements. To add or subtract two instances of `RV`s we utilize the general function `compose_function`, see below, that returns the composition under a function f of two independent random variables. To call the plus and minus operator on two `RV`s we need to tell the class what to do when writing code like `X + Y`. The private methods `__add__` and `__sub__` achieve this. (Note that we build $X - Y$ as $X + (-Y)$.)

```
    def __add__(self, other: 'RV') -> 'RV':
        return compose_function(operator.add, self, other)
```

```python

    def __sub__(self, other: 'RV') -> 'RV':
        rv = RV({-k: other.pmf(k) for k in other.support()})
        return self + rv
```

Finding the pmf of the sum $Z = X + Y$ of two independent rvs X and Y with pmfs p and q requires convolution like so

$$\mathsf{P}\{Z = k\} = \sum_i \sum_j \mathbb{1}_{i+j=k} p_i q_j.$$

In python, we can implement this nicely with a `defaultdict`. Observe that here it becomes clear why we need a support with elements sorted in decreased pmf. Once the probabilities beome too small, we simply break the loop.

```python
def compose_function(f: Callable[[float, float], float], X: RV, Y: RV) -> RV:
    """Make the rv f(X, Y) for the independent rvs X and Y."""
    c: defaultdict[float, float] = defaultdict(float)
    for i in X.sortedsupport():
        for j in Y.sortedsupport():
            p = X.pmf(i) * Y.pmf(j)
            c[f(i, j)] += p
            if p <= thres:
                break
    return RV(c)
```

Finally, we have to be able to compute $f(X)$ for some function f, for instance $[W_i + S_i - X_{i+1}]^+$ applies the function $[\cdot]^+$ to a rv.

```python
def apply_function(f: Callable[[float], float], X: RV) -> RV:
    """Make the rv f(X)"""
    c: defaultdict[float, float] = defaultdict(float)
    for k in X.support():
        c[f(k)] += X.pmf(k)
    return RV(c)
```

3.4 DISCRETE EVENT SIMULATIONS

The queueing and inventory systems in the previous sections were relatively simple to simulate. If, however, we need to analyze more difficult situations we need discrete event simulators, and to demonstrate how that works we will build such a simulator for a single station multi-server queue. We first provide a recursion in mathematical terms, and then discuss its shortcomings. This serves as motivation to set up a simulator with classes and an event stack.

LET US NEXT construct a multiserver FIFO queue in which the service of the first job in line starts when a server becomes free; if a server is free when a job arrives, the job's service starts right away.

Suppose there are c servers available, each with its own waiting line, like in a supermarket. When job k arrives, it sees a waiting time $w_k(i)$ at line i;

write $w_k = (w_k(1), \ldots, w_k(c))$ for the vector of waiting times. Of course, the job selects the line with the shortest waiting time.[0] Thus, it joins the end of line $s_k = \arg\min\{w_k(i) : i = 1, \ldots, c\}$.

[0] *This is not necessarily the same as the shortest queue.*

To formulate a recursion for w_k, let $e(i)$ be the ith unit vector[1], and $\mathbf{1} = (1, \ldots, 1)$. The waiting time of job $k-1$ becomes $W_{k-1} = w_{k-1}(s_{k-1})$, and, in analogy with (3.1.3), the vector w_k updates as

[1] *A 1 at place i and zeros elsewhere.*

$$
\begin{aligned}
s_{k-1} &= \arg\min\{w_{k-1}(i) : i = 1, \ldots, c\}, \\
w_k &= [w_{k-1} + S_{k-1} e(s_{k-1}) - X_k \mathbf{1}]^+,
\end{aligned}
$$

where $[\cdot]^+$ applies element-wise.

It is useful to analyze the algorithmic complexity of this algorithm. For job $k-1$, we need to find the minimum in w_{k-1}, and compute and subtract $X_k \mathbf{1}$. The number of computational operations for this is $2c$, as w_k and $\mathbf{1}$ contain c elements. For a simulation with N jobs, the total amount of operations is therefore $2c \times N$. However, by using a different implementation,[2] the complexity can be reduced to $N \times \log_2 c$, which is considerably faster when c and N are large.

[2] *With event stacks.*

WE NOW TURN to developing a simulator for the multi-server queue step by step. We organize the code by means of classes. There are many reasons for this, but one is that this allows us to represent 'things' of the 'real world' as code which has its own properties and behavior.

The Server is our first class. We implement this as a dataclass because the server is a very simple class, having just two attributes: an id and a service rate so that we can allow for speed differences between servers. We add the method `__lt__` to compare two servers based on their relative rates. With this method we can select a free server based on our preference. Here a lower service rate means that jobs are served at a slower rate; to select a fast server over a slow server, we have to use the inequality as shown.

server.py

```python
from dataclasses import dataclass


@dataclass
class Server:
    id: int
    rate: float

    def __lt__(self, other):
        return self.rate > other.rate
```

The next class is the Servers class and represents a pool of free servers. We subclass Servers from a list so that we can this list for the heapq algoritms to push and pop servers. When a job leaves the queue, Servers will assign a server to the job based on the preferences expressed by the `__lt__` method of the Server class. We push to the deque any server that becomes idle after finishing a job,. Finally, we add the convenience methods `num_free` and `is_server_available` as these are easier to understand and read than their implementations in terms of `len`.

servers.py

```python
from heapq import heappop, heappush


class Servers(list):
    def push(self, server):
        heappush(self, server)

    def pop(self):
        return heappop(self)

    def num_free(self):
        return len(self)

    def is_server_available(self):
        return self.num_free() > 0
```

The Job class speaks mostly for itself. The service time will be the job's load divided by the speed of the server that serves the job. As the service time depends on the server,and since the servers can have different speeds, we cannot compute the service time at the start of the simulation. The job keeps a reference to the server that handles the job. The other attributes are used for the statistics at the end of the simulation. We implement the sojourn time and waiting time as a `property`, as this allows us to address them in the same way as, for instance, the departure time. The `__lt__` method determines which job to select from the queue when a service can start. As it is now, we select jobs in FIFO sequence. If, however, we would prefer a LIFO (last in first out) queue, we only have to reverse the direction of the inequality. Another simple rule is Shortest Processing Time First. To use this, change the inequality to this `self.load < other.load`.

There is one detail. We might want to store jobs in a set or a dict, and this requires to be able to distinguish one job from another. For this reason we give the option `eq=True` to the data class and include a `__hash__` method.

job.py

```python
from dataclasses import dataclass

from server import Server


@dataclass
class Job:
    id: int
    arrival_time: float
    load: float
    service_time: float = 0
    departure_time: float = 0
    queue_length: int = 0
    free_servers: int = 0
    server: Server = None

    @property
    def sojourn_time(self):
        return self.departure_time - self.arrival_time

    @property
```

```
    def waiting_time(self):
        return self.sojourn_time - self.service_time

    def __repr__(self):
        return (
            f"{self.id},  {self.load}, {self.service_time},"
            f"{self.arrival_time}, {self.departure_time}"
        )

    def __hash__(self):
        return self.id

    def __lt__(self, other):
        return self.arrival_time < other.arrival_time
```

The fourth class is the Queue class. This is also a subclass from a list so that we can use the queue as a heap in itself, and use the `__lt__` method of the Job class to select a job from the queue and send that job to the servers to be served.

queues.py

```
from heapq import heappop, heappush


class Queue(list):
    def pop(self):
        return heappop(self)

    def push(self, job):
        heappush(self, job)

    def length(self):
        return len(self)

    def is_empty(self):
        return self.length() == 0
```

An Event is just a time and job attached to it. The `__lt__` method specifies how to order events. Clearly, this must be by time. We subclass the event class to an ArrivalEvent and a DepartureEvent, because during the simulation we need to distinguish the type of event.

event.py

```
from dataclasses import dataclass

from job import Job


@dataclass
class Event:
    time: float
    job: Job

    def __lt__(self, other):
        return self.time < other.time


class ArrivalEvent(Event):
```

```
    pass


class DepartureEvent(Event):
    pass
```

The Statistics class does what its name says. We subclass it from a list, because with a list we can keep the sequence of the jobs in which the jobs depart from the server.

stats.py

```
class Statistics(list):
    def push(self, job):
        self.append(job)

    def num_jobs(self):
        return len(self)

    def mean_waiting_time(self):
        tot = sum(job.waiting_time for job in self)
        return tot / self.num_jobs()

    def mean_sojourn_time(self):
        tot = sum(job.sojourn_time for job in self)
        return tot / self.num_jobs()

    def mean_queue_length(self):
        tot = sum(job.queue_length for job in self)
        return tot / self.num_jobs()

    def mean_servers_free(self):
        tot = sum(job.free_servers for job in self)
        return tot / self.num_jobs()
```

The last class we need is the Simulator itself.[3] We need references to the events, queue, stats and servers. The attribute `now` keeps track of the time; it points to the time of the event the simulator is currently treating. When starting the simulation, we send a list of jobs to the simulator which then pushes an ArrivalEvent for each job to the event stack.

[3] *Study it carefully.*

When a job service starts, the `serve_job` method asks for a free server, and assigns this server to the job. Then it computes the job service time and departure time, and pushes a DepartureEvent to the event stack to inform the simulator later that a job is finished.

The `run` method starts by checking whether there are still events to do, and if so, it pops an event from the event stack. Then it retrieves the time and stores it as `now`, and gets the job attached to the event. If the event is an ArrivalEvent, the job must have just arrived and can be served if server is free, otherwise the job is queued. When the event is a DepartureEvent, the server that served the job becomes free and is pushed to the stack of free servers. The departing job is pushed to the statistics tracker. If there is still work in the queue, a service can start for the job that is at the head of the queue.

simulator.py

```
from event import ArrivalEvent, DepartureEvent

```

```python

class Simulation:
    def __init__(self, events, queue, statistics, servers):
        self.events = events
        self.queue = queue
        self.stats = statistics
        self.servers = servers
        self.now = 0

    def initialize_jobs(self, jobs):
        for job in jobs:
            self.events.push(ArrivalEvent(job.arrival_time, job))

    def serve_job(self, job):
        server = self.servers.pop()
        job.server = server
        job.service_time = job.load / server.rate
        job.departure_time = self.now + job.service_time
        job.queue_length = self.queue.length()
        job.free_servers = self.servers.num_free()
        self.events.push(DepartureEvent(job.departure_time, job))

    def run(self):
        while not self.events.is_empty():
            event = self.events.pop()
            self.now, job = event.time, event.job
            if isinstance(event, ArrivalEvent):
                if self.servers.is_server_available():
                    self.serve_job(job)
                else:
                    self.queue.push(job)
            elif isinstance(event, DepartureEvent):
                self.servers.push(job.server)
                self.stats.push(job)
                if not self.queue.is_empty():
                    job = self.queue.pop()
                    self.serve_job(job)
            else:
                raise ValueError("Unknown event")
```

IT REMAINS TO actually try out the simulation. Later, in Section 5.1, we provide a set of formulas for the so-called $M/M/c$ queue which is a queueing process in which the iid inter-arrival times $X_i \sim \text{Exp}(\lambda)$, the iid service times $S_i \sim \text{Exp}(\mu)$, with $\mu > \lambda$, and there are c identical servers. These formulas give exact results for the expected waiting time and related KPIs, so this model serves as a perfect benchmark for our simulator. We use the next code for the test. The code for the $M/M/c$ queue is contained in a separate file (a module) with the name `mmc.py`; you can find it on github.

For the generation of the data for the simulation we follow the same logic as in Section 3.2.

test-mmc.py

```python
"""This compares the result of the simulation of a multi-server queue
 to the exact results of an M/M/c queue.
"""

import numpy as np

```

```
import mmc
from events import Events
from job import Job
from queues import Queue
from server import Server
from servers import Servers
from simulator import Simulation
from stats import Statistics


def test_mmc():
    num = 10000
    labda, mu = 3, 4

    rng = np.random.default_rng(3)
    X = rng.exponential(scale=1 / labda, size=num)
    X[0] = 0
    A = X.cumsum()
    loads = rng.exponential(scale=1 / mu, size=num)
    jobs = [Job(id=i, arrival_time=A[i], load=loads[i]) for i in range(1, num)]

    servers = Servers()
    for i, rate in enumerate([1, 1]):
        servers.push(Server(id=i, rate=rate))

    sim = Simulation(Events(), Queue(), Statistics(), servers)
    sim.initialize_jobs(jobs)
    sim.run()

    mm2 = mmc.MMC(labda, mu, c=2)
    print("W: ", sim.stats.mean_waiting_time(), mm2.EW)
    print("J: ", sim.stats.mean_sojourn_time(), mm2.EJ)
    print("Q: ", sim.stats.mean_queue_length(), mm2.EQ)


test_mmc()
```

Here is the output. The simulator seems to pass the test.

```
W:  0.044123468095817915 0.04090909090909091
J:  0.2952268030097956 0.2909090909090909
Q:  0.1467146714671467 0.12272727272727273
```

Let us now demonstrate with a few examples how flexible this simulation environment actually is.

SUPPOSE WE HAVE three different servers with rates 1, 0.1 and 0.01, respectively, and we like to select the fastest server whenever multiple servers are free. As it turns out, our simulation supports this already, not a single line has to change. The Servers class acts as a heap, and popping occurs with the preference as expressed by the `__lt__` method of the Server class.

Here are some interesting experiments.

multiple-speeds.py

```
"""Simulate a multi-server queue with different speeds and compare
to an M/M/C queue."""
from collections import Counter

```

```python
import numpy as np

import mmc
from events import Events
from job import Job
from queues import Queue
from server import Server
from servers import Servers
from simulator import Simulation
from stats import Statistics


def multiple_speeds():
    num = 10000
    labda, mu = 3, 3.1

    rng = np.random.default_rng(3)
    X = rng.exponential(scale=1 / labda, size=num)
    X[0] = 0
    A = X.cumsum()
    loads = rng.exponential(scale=1 / mu, size=num)
    jobs = [Job(id=i, arrival_time=A[i], load=loads[i]) for i in range(1, num)]

    servers = Servers()
    for i, rate in enumerate([1, 0.1, 0.01]):
        servers.push(Server(id=i, rate=rate))

    sim = Simulation(Events(), Queue(), Statistics(), servers)
    sim.initialize_jobs(jobs)
    sim.run()

    mm1 = mmc.MMC(labda, mu, c=1)
    print("W: ", sim.stats.mean_waiting_time(), mm1.EW)
    print("J: ", sim.stats.mean_sojourn_time(), mm1.EJ)
    print("Q: ", sim.stats.mean_queue_length(), mm1.EQ)

    count = Counter([j.free_servers for j in sim.stats])
    print(count)


multiple_speeds()
```

This is the output when $\lambda = 3$ and $\mu = 4$.

```
W:  0.4178019854514783 0.75
J:  1.2386226229512027 1.0
Q:  1.2715271527152716 2.25
```

This is interesting, for several reasons. In the multi-server case the second and third server add just a little bit of service capacity. For this reason we compare the result to the $M/M/1$ queue, with just one server working at rate 1. We see that the average queue length is smaller in the multi-server station, but the sojourn time is larger. The reason must be that some jobs end up at a slow server. To see what fraction that is, we use the `Counter`. After simulating, we get this.

```
Counter({0: 8772, 1: 1136, 2: 91})
```

Obviously, a significant number of jobs gets assigned to a slow server.

We might wonder whether we should use the extra capacity offered by the slow servers. Perhaps we should only do this when the queueing times become really large. So, let's increase the load of the hence by setting $\mu = 3.1$, instead of 4. We get the following results.

```
W:  2.0385311552319467 9.677419354838683
J:  2.971938896180861 9.999999999999973
Q:  6.165616561656166 29.03225806451605
Counter({0: 9615, 1: 357, 2: 27})
```

And now it becomes very clear that the extra capacity helps. Interestingly, the fraction of jobs served by the fastest server is *larger* than when the load is smaller.

Another interesting rule to select servers could use the cost of a server. For instance, it may be that some servers use less energy than others, so that an operator has a preference for the cheaper servers. We will that to you to pursue.

IN SOME QUEUEING systems, some jobs have priority over others, like in a hospital. With the next job class we can implement this behavior.. As a priority job is just a job with a priority, we can just subclass the job class from above. The ordering rule is simple: when jobs have the same priority, they should be sorted according to FIFO, otherwise according to priority (here lower priority is better).

priority-job.py

```python
from dataclasses import dataclass

from job import Job
from server import Server


@dataclass(eq=False)
class PriorityJob(Job):
    priority: int = 0

    def __lt__(self, other):
        if self.priority == other.priority:
            return self.arrival_time < other.arrival_time
        else:
            return self.priority < other.priority
```

The queue class is already a heap, so needs no change.

Here is how to use it.

priority-jobs.py

```python
"""The effect of priority service in multi-server queues."""
from collections import defaultdict

import numpy as np

import mmc
from events import Events
from priority_job import PriorityJob
from queues import Queue
```

```python
from server import Server
from servers import Servers
from simulator import Simulation
from stats import Statistics


def priority_jobs():
    num = 100000
    labda, mu = 3, 3.2

    rng = np.random.default_rng(3)
    X = rng.exponential(scale=1 / labda, size=num)
    X[0] = 0
    A = X.cumsum()
    loads = rng.exponential(scale=1 / mu, size=num)
    priorities = rng.binomial(1, 0.1, size=num)
    jobs = []
    for i in range(1, num):
        job = PriorityJob(id=i, arrival_time=A[i], load=loads[i])
        job.priority = priorities[i]
        jobs.append(job)

    servers = Servers()
    for i, rate in enumerate([1]):
        servers.push(Server(id=i, rate=rate))

    sim = Simulation(Events(), Queue(), Statistics(), servers)
    sim.initialize_jobs(jobs)
    sim.run()

    mm1 = mmc.MMC(labda, mu, c=1)
    print("W: ", sim.stats.mean_waiting_time(), mm1.EW)
    print("J: ", sim.stats.mean_sojourn_time(), mm1.EJ)
    print("Q: ", sim.stats.mean_queue_length(), mm1.EQ)

    wait = defaultdict(float)
    nums = defaultdict(float)
    for j in sim.stats:
        nums[j.priority] += 1
        wait[j.priority] += j.waiting_time
    for k in sorted(nums.keys()):
        print(f"Priority: {k}, EW: {wait[k] / nums[k]:.2f}")


priority_jobs()
```

And this is the result.

```
W:  4.6978119398356775 4.6875
J:  5.01121142029712 5.0
Q:  14.089830898308984 14.0625
Priority: 0, EW: 1.92
Priority: 1, EW: 29.38
```

For the population as a whole there is no difference in average waiting time (which is according to our intuition), but the lower priority jobs perceive much longer waiting times.

WITH THIS CODE environment you have all the tools available to simulate and analyze many different queueing (and inventory) systems.

For instance, when jobs have due-dates, we might want to serve the job that has the smallest due-date to minimize the probability of that job being late. Then we have to add a due-date attribute to jobs, and use that in the method `__lt__` to compare jobs. Of course, there are many other different scheduling rules possible.

A more complicated example is how to handle business and economy class customers at a check-in desk at an airport. Ofter there is one server strictly allocated to business class customers, and there are c, say, servers strictly used by the economy class customers. Another way to organize the server allocation could be like this. When the business server is free, this server takes just one economy customer in service (assuming there is such a customer in queue). When, after this service, there is still no business customer in queue, the business server serves another economy customer. Once a business customer arrives, the business server finishes the economy customer (if there is any in service), and then serves the business customer. We use a similar policy for serving business customers if there are free economy servers. The problem is, of course, to analyze the performance of this adapted policy. By how much do the waiting times of the business customers increase, and by how much do the waiting times of the economony class customers decrease? Perhaps we can remove a server, thereby making the tickets cheaper.

It may not be simple to make the necessary adaptions. Different customer queues may be necessary, we might have multiple stations, but the basic interaction with an event stack remains the same.

Ex 3.4.1. Consult the website of RealPython to obtain a better understanding of

1. classes
2. dataclasses
3. heapq
4. deque

In other words Google on 'realpython classes'. Then you'll get to a great page on why classes are such a powerful idea in programming. Then Google on 'realpython dataclasses', and likewise for the other concepts. BTW, you can also ask ChatGPT.

Ex 3.4.2. There are some advantages to subclassing Queue from `deque`. What are these?

4

FUNDAMENTAL TOOLS

With the tools developed in Chapter 2 and Chapter 3 we can *simulate* very general queueing processes. While this is very useful, reasoning about (properties of) queueing systems is simpler with formulas and models, and in this chapter we make a start with that.

Now the mathematical characterization of the *transient behavior* of queueing systems is extremely complicated, so henceforth we focus on the *long-run time average* behavior of queueing systems. For this the arrival and service rate play a crucial role. In particular, the queueing system is only stable when jobs can be served at a higher rate than they arrive. Once we have established the stabilty, we define performance measures such as the long-run average waiting time.

We next derive some results that are fundamental to analyze any stochastic process, of which queueing systems are just examples. These results are based on *sample-paths*, i.e., realizations of the simulation of such systems, and form an elegant and unifying principle between the constructions of Chapter 3 and theoretical considerations. To illustrate the relations between all concepts we provide two mindmaps at the end of the chapter. Here we keep the discussion in these notes mostly at an intuitive level; we refer to El-Taha and Stidham Jr. [1998] for proofs and further background.

In this section we sometimes need some notation to indicate specific types of queueing systems. We discuss this terminology first.

4.1 KENDALL'S NOTATION

As is apparent from Sections 2.1 and 3.1, the construction of a queueing process for a single station involves three main elements: the distribution of job inter-arrival times, the distribution of the service times, the number of servers present to process jobs, and the number (batch) of jobs that arrive and can be served at once.

To characterize the type of queueing process it is common to use *Kendall's abbreviation* $A^B/Y^C/c/K$, where A is the distribution of the iid inter-arrival times, Y the distribution of the iid service times, c the number of servers, and K the system size, i.e., the total number of customers that can be simultaneously present, whether in queue or in service.[0] When at an arrival epoch multiple jobs can arrive simultaneously (like a group of people entering a restaurant), we say that a *batch of jobs* arrives. Likewise, the server can work in batches, for instance, an oven can process multiple jobs at the same time. The letter B denotes the (distribution of the) batch size and C the (distribution of the) size of the service batch. When $B \equiv 1$ or $C \equiv 1$, i.e., single batch arrivals or single batch services, we suppress the B or C in Kendall's formula. Finally, in this notation, it is assumed that the service discipline is FIFO.

[0] *The meaning of K differs among authors. Sometimes it stands for the capacity of the queue, not the entire system. In this book K corresponds to the system's size.*

The most important inter-arrival and service distributions are the exponential distribution, denoted with the shorthand[1] M, and the general distribution[2], denoted with G. We write D for a deterministic (constant) random variable.

[1] *which stands for Markov or Memoryless*

[2] *with the implicit assumption that the first moment is finite*

A FEW IMPORTANT examples are the following queueing processes: the $M/M/1$ queue in which inter-arrival times and service times are exponentially distributed; the $M/G/1$ queue with exponential inter-arrival times and general service times; and the $G/G/c$ queue in which the inter-arrival and service times are not specified.

A model that is often used to determine the number of beds needed in (a ward of) a hospital is the $M/M/c/c$ queue. The motivation is as follows. Practice tells us that patient inter-arrival times are memoryless, hence exponentially distributed. Data of patients treatment times shows that these times are often quite well described by an exponential distribution. Next, there are c beds available. Clearly, as each bed can serve just one patient, when all c beds are occupied, the hospital is 'full'. Thus, the maximal number of jobs in the system is the same as the number of servers available.

We can represent the discrete-time model that corresponds to the recursion (2.1.1) as a $D^B/D^C/1$ queue in which $a_k \sim B$ and $c_k \sim C$.

4.2 RATE, STABILITY AND LOAD

In this section, we develop a number of measures to characterize the performance of queueing systems in steady-state. In particular, we define the load, which is, arguably, the most important performance measure of a queueing system to check.

We first develop the concepts for a queueing process with batch and service sizes of 1, i.e. $B = C = 1$ in Kendall's formula. At the end we present the general formula.

WE FIRST FORMALIZE the arrival rate and departure rate in terms of the arrival and departure processes $\{A(t)\}$ and $\{D(t)\}$, c.f., Section 3.1. The *arrival rate* is the long-run average number of jobs that arrive per unit time along a sample path, i.e.,[0]

[0] *This limit does not necessarily exist if $A(t)$ is some pathological function.*

$$\lambda = \lim_{t\to\infty} \frac{A(t)}{t}.$$

Likewise, the *departure rate* is

$$\delta = \lim_{t\to\infty} \frac{D(t)}{t}. \tag{4.2.1}$$

Henceforth, we assume that both limits exist, are finite, and $0 < \lambda < \infty$.

FOR THE ARRIVAL RATE, suppose we are given a sequence of iid inter-arrival times $\{X_k\}$, each of which is distributed as the common rv X. Then, by the definition of A_n, $\sum_{k=1}^n X_k = A_n$. We prove below that our assumptions imply that the average inter-arrival time $\mathsf{E}[X]$ between two consecutive jobs is equal to the *inverse* of the arrival rate λ, i.e.,

$$\mathsf{E}[X] = \lim_{n\to\infty} \frac{1}{n}\sum_{k=1}^{n} X_k = \lim_{n\to\infty} \frac{A_n}{n} = \lim_{t\to\infty} \frac{t}{A(t)} = \frac{1}{\lambda}.$$

For the service rate of a *single* server, take S_k as the required service time of the kth job served by the server, so that $U_n = \sum_{k=1}^{n} S_k$ is the total service time of the first n jobs. Similar to the definition of $A(t)$, we let $U(t) = \max\{n : U_n \le t\}$ and define the *service*, or *processing*, rate of a single server as

$$\mu := \lim_{t\to\infty} \frac{U(t)}{t}, \quad 0 < \mu < \infty,$$

assuming that this limit exists. This provides us with the relation

$$\mathsf{E}[S] = \lim_{n\to\infty} \frac{1}{n} \sum_{k=1}^{n} S_k = \lim_{n\to\infty} \frac{U_n}{n} = \lim_{t\to\infty} \frac{t}{U(t)} = \frac{1}{\mu}.$$

Thus, the service rate μ of a single server is the *inverse* of $\mathsf{E}[S]$.

THE NEXT CONCEPT of importance is *stability*. To discuss this properly, we distinguish between three counting functions: the number of arrivals $A(t)$, the number of jobs entering the system $\tilde{A}(t)$, and the number of departures $D(t)$. The difference between $A(t)$ and $\tilde{A}(t)$ is relevant when we deal with a queueing system in which the server rejects jobs or arriving customers do not wish to join the queue, for whatever reason.

We require the fundamental property that service is *work-conserving*, that is, the server is not idle when there is one (or more) job in the system. Thus, capacity is not wasted.

Assuming that $\tilde{A}(t)/t \to \tilde{\lambda}$, we consider four (intuitively) obvious situations.

1. $\lambda \ge \tilde{\lambda} > \mu = \delta$. The system overflows because the service capacity cannot keep up with the rate at which jobs enter. Hence, the number of jobs in the system $L(t) \to \infty$. Work-conservation implies that the server is always busy, hence jobs depart at the rate at which they are served.

2. $\lambda \ge \mu > \tilde{\lambda} = \delta$. Jobs must be blocked to prevent the system to explode, but the jobs that enter are (eventually) served.

3. $\mu > \lambda \ge \tilde{\lambda} = \delta$. There is sufficient capacity to serve all jobs that arrive, but still not all arrivals need not enter the system. All accepted jobs are (eventually) served.

4. $\mu > \lambda = \tilde{\lambda} = \delta$. All arrivals enter the system and are (eventually) served.

To prevent all sorts of pathological situations, we do not consider cases in which $\lambda = \mu = \tilde{\lambda}$.

In view of the above inequalities, we say that a system is *rate-stable* if

$$\tilde{\lambda} = \delta.$$

In words: the system is rate-stable whenever in the long run jobs leave the system just as fast as they enter the system.

Henceforth, we *only* consider queueing systems that fall into cases 2–4, i.e., the system is rate stable and $\mu > \tilde{\lambda}$. Moreover, unless explicitly stated, we assume that the fourth case applies. Note that $\lambda = \tilde{\lambda}$ when $A(t) = \tilde{A}(t)$, and then we mean by rate-stability that $\lambda = \delta$.

WITH THE ARRIVAL and service rate, we define for the single server queue the server *load* as

$$\text{Server load } = \rho := \lambda \mathsf{E}[S] = \frac{\lambda}{\mu} = \frac{\mathsf{E}[S]}{EX}.$$

Next, we define the *utilization* as

$$\text{Server utilization } = \tilde{\lambda} \mathsf{E}[S] = \frac{\tilde{\lambda}}{\mu},$$

as this involves the rate at which work enters the system, and by rate-stability, this is the work that is served. When $\lambda = \tilde{\lambda}$, the load ρ and the utilization are all equal, and we will use these terms interchangeably. However, when $\tilde{\lambda} < \lambda$ we need to be careful about the difference between load and utilization. Note that the load can exceed 1, while the utilization is < 1.

WHEN THE STATION contains multiple identical servers, or arrivals occur in batches and/or are served in batches, the work per job arrival is $\mathsf{E}[S]\,\mathsf{E}[B]$. Similarly, when a server serves $\mathsf{E}[C]$ jobs per service time, its service rate is $\mathsf{E}[C]/\mathsf{E}[S]$. Finally, when the station contains c servers, the station's capacity is c times are large as that of a single server. Thus, in the most general case,

$$\rho := \frac{\lambda \mathsf{E}[S]}{c} \frac{\mathsf{E}[B]}{\mathsf{E}[C]}.$$

The utilization has to change accordingly.

IT REMAINS TO establish that

$$\lim_{t\to\infty} \frac{t}{A(t)} = \lim_{n\to\infty} \frac{A_n}{n}. \tag{4.2.2}$$

This equality is not entirely evident because the first limit runs of over all real numbers, while the second runs over the natural numbers.

To show the equality, observe first that the counting funciton $A(t) \to \infty$ as $t \to \infty$ by our assumptions $0 < \lambda < \infty$ and the existence of the limit of $A(t)/t \to \lambda$ as $t \to \infty$. Observe next that $A_{A(t)}$ is the arrival time of the last job *before* time t, hence $A_{A(t)} \le t$. This, in turn, implies that $A_{A(t)+1}$ is the arrival time of the first job *after* time t. Therefore, $A_{A(t)} \le t < A_{A(t)+1}$, which is equivalent to

$$\frac{A_{A(t)}}{A(t)} \le \frac{t}{A(t)} < \frac{A_{A(t)+1}}{A(t)} = \frac{A_{A(t)+1}}{A(t)+1} \frac{A(t)+1}{A(t)}.$$

With regard to the left most term, observe that when $A(t) = n$ then[1] $A_{A(t)} = A_n$, so that $A_{A(t)}/A(t) = A_n/n$. Since $A(t) \to \infty$

[1] [3.1.2]

$$\lim_{t\to\infty} \frac{A_{A(t)}}{A(t)} = \lim_{t\to\infty} \frac{A_{A(t)+1}}{A(t)+1} = \lim_{n\to\infty} \frac{A_n}{n},$$
$$\lim_{t\to\infty} \frac{A(t)+1}{A(t)} = 1.$$

This proves (4.2.2) because the limit in the middle, whose existence we assumed, is wedged between two limits that are the same.

Here is a first idea on how we can use the memoryless property of the inter-arrival times of the $M/G/1$ queue to find show that the expected busy

time of the $M/G/1$ satisfies $\mathsf{E}[U] = \mathsf{E}[S]/(1-\rho)$. During the service time of the first customer that starts a busy time, an expected number $\lambda\,\mathsf{E}[S]$ new jobs arrive. As each of these jobs restarts the busy-period, the expected busy time started by the first jobs must be equal to the expected service time of this job plus all busy periods that are generated by the jobs that arrive during this service time, hence, $\mathsf{E}[U] = \mathsf{E}[S] + \lambda\,\mathsf{E}[S]\,\mathsf{E}[U]$. But from this, $\mathsf{E}[U] = \mathsf{E}[S]/(1-\rho)$, because $\rho = \lambda\,\mathsf{E}[S]$. Realize further that because inter-arrivals times are memoryless for the $M/G/1$ queue, the expected idle time $\mathsf{E}[I] = 1/\lambda$.

Ex 4.2.1. Can you make an arrival process such that $A(t)/t$ does not have a limit?

4.3 (LIMITS OF) EMPIRICAL PERFORMANCE MEASURES

In Section 3.1 we use the arrival process $\{A(t)\}$ and the service times $\{S_k\}$ to construct the waiting times $\{W_k\}$, sojourn times $\{J_k\}$, and the number in the system $\{L(t)\}$. If the queueing system is rate-stable, we can sensibly define several more long-run average performance measures. Recall that these definitions are the most used quantitative measures to express customer satisfaction.

DEFINE THE EXPECTED *waiting time in queue* as

$$\mathsf{E}[W] = \lim_{n\to\infty} \frac{1}{n}\sum_{k=1}^{n} W_k.$$

Note that this is the limit of waiting times as *observed by arriving jobs*: the first job has to wait W_1 in queue, the second W_2, and so on.[0] The *distribution* of the waiting time as seen by arrivals can be found by counting:

$$\mathsf{P}\{W \le x\} = \lim_{n\to\infty} \frac{1}{n}\sum_{k=1}^{n} \mathbb{1}_{W_k \le x}. \tag{4.3.1}$$

[0] *We colloquially say that a statistic based on the sampling of arriving jobs is 'as seen by arrivals'.*

For the sojourn time J we use similar definition. The *average number of jobs* in the system as seen by arrivals is given by

$$\mathsf{E}[L] = \lim_{n\to\infty} \frac{1}{n}\sum_{k=1}^{n} L(A_k-), \tag{4.3.2}$$

since $L(A_k-)$ is the number of jobs in the system just before the arrival epoch of the kth job. Like (4.3.1), for the distribution of L,

$$\mathsf{P}\{L \le m\} = \lim_{n\to\infty} \frac{1}{n}\sum_{k=1}^{n} \mathbb{1}_{L_k \le m}. \tag{4.3.3}$$

A RELATED SET of performance measures follows by tracking the system's behavior over time and taking the *time-average*.[1]

[1] *Now we sometimes say that such performance measures are as 'seen by the server'.*

Assuming the limit exists, we use (3.1.6) to define the *time-average number of jobs* as [2]

$$\mathsf{E}[L] = \lim_{t\to\infty} \frac{1}{t}\int_0^t L(s)\,\mathrm{d}s. \tag{4.3.4}$$

[2] *Even though the symbols are the same, this expectation is not necessarily the same as* (4.3.2).

The *time-average fraction of time the system contains at most m jobs* is defined as

$$\mathsf{P}\{L \le m\} = \lim_{t\to\infty} \frac{1}{t}\int_0^t \mathbb{1}_{L(s)\le m}\,\mathrm{d}s.$$

PROVING THE EXISTENCE of the above limits requires a considerable amount of mathematics.[3] Here we sidestep all such fundamental issues, and simply assume that all is well-defined. The limiting random variables are known as the *steady-state* or *stationary* limits.

[3] *See Asmussen [2003] if you are interested.*

IN THE NEXT chapters we will derive simple formulas for the above performance measures for a variety of queueing systems in steady-state. These formulas, and the relations between them, provide *structural insight* into the behavior of the systems, rather than just numbers. Hence, for many practical purposes, which is (should be?) based on insight, a steady-state analysis suffices.

Ex 4.3.1. Design a queueing system to show that the average number of jobs in the system as seen by the server can be very different from what customers see upon arrival.

Ex 4.3.2. If $L(t)/t \to 0$ as $t \to \infty$, can it still be true that $\mathsf{E}[L] > 0$?

4.4 RENEWAL REWARD THEOREM

In a one-dollar store, each item costs \$1. If every minute a customer arrives, and each customer buys exactly 10 items, this shop earns money at a rate of \$10 per minute. At another shop, customers spend variable amounts but on average \$10 per customer, and customers come in irregularly, but the average inter-arrival time between two customers is 1 minute. This other shop also earns money at rate of \$10 per minute. That both shops have the same earning rate is what the *renewal reward* theorem says. In this section we prove this extremely useful theorem, and in the exercises we show how to apply it to queueing systems.

WE ARE GIVEN a sequence of strictly increasing epochs $0 = T_0 < T_1 < T_2 < \cdots$ such that $T_k \to \infty$ as $k \to \infty$, and a non-decreasing right-continuous (deterministic) function $t \to Y(t)$ with $Y(0) = 0$. Let $N(t) = \sum_{k=1}^{\infty} \mathbb{1}_{T_k \le t}$ count the number of epochs that occurred during $(0, t]$, and $X_k = Y(T_k) - Y(T_{k-1})$ be the increment of $Y(t)$ during $(T_{k-1}, T_k]$.[0] The renewal reward theorem says the following. Assume the limit $\lambda = \lim_{t\to\infty} N(t)/t$ exist with $0 < \lambda < \infty$. Then, the limit $Y = \lim_{t\to\infty} Y(t)/t$ exists iff $X = \lim_{n\to\infty} n^{-1} \sum_{k=1}^{n} X_k$ exist, and in that case, $Y = \lambda X$. Fig. 4.4.1 shows graphically how all concepts relate.

[0] *Here X_k is not necessary an inter-arrival time between two jobs.*

To see why this is true, we copy an idea of Section 4.2. Just as with the relation between $A(t)$ and $A_{A(t)}$, we have here that $T_{N(t)} \le t < T_{N(t)+1}$. As $Y(\cdot)$ is non-decreasing,

$$Y(T_{N(t)}) \le Y(t) < Y(T_{N(t)+1}),$$

hence

$$\frac{N(t)}{t}\frac{Y(T_{N(t)})}{N(t)} \le \frac{Y(t)}{t} < \frac{Y(T_{N(t)+1})}{t}\frac{N(t)+1}{N(t)}.$$

Noting that $Y(T_{N(t)}) = \sum_{k=1}^{N(t)} X_k$, and using the assumption that $N(t)/t \to \lambda$ we conclude that $Y(t)/t \to Y$ iff $(N(t))^{-1} \sum_{k=1}^{N(t)} X_k \to X$, and if (one of the) limits exists, $Y = \lambda X$.

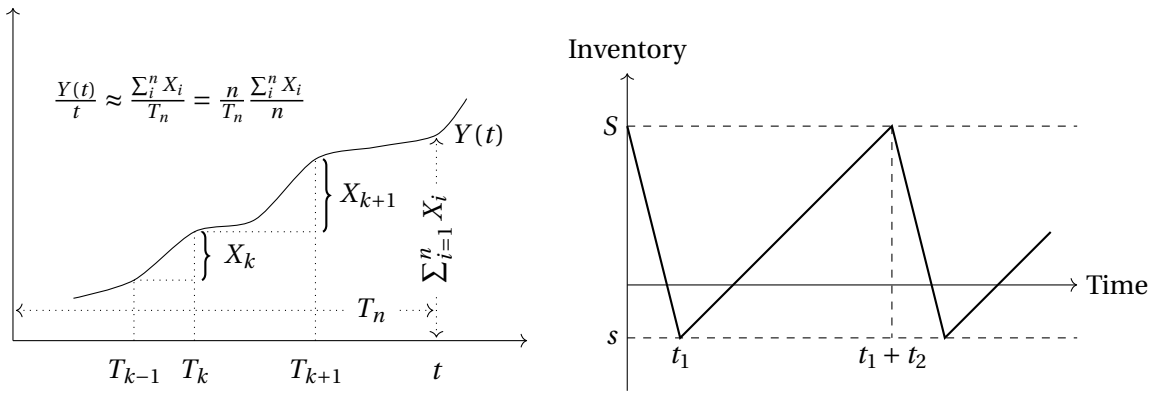


Figure 4.4.1: The left panel provides a graphical 'proof' of the renewal reward equation $Y = \lambda X$. The right panel shows a sample path of an inventory system with constant production capacity and a fixed demand rate. As the inventory is operated under an (s, S)-policy, it is a system that moves from cycle to cycle.

THE RENEWAL REWARD theorem finds an easy application in the analysis of the *Economic Production Quantity* (EPQ) model which is an inventory system in which demand arrives at a fixed rate d and items are produced at constant rate r by a machine that can switch on and off. The inventory is under an (s, S) policy, the leadtime $L = 0$, the holding and backlogging cost are h and b per unit per time, and it costs K to switch on the machine. Fig. 4.4.1 shows a sample path of the inventory level. Let us use the ideas of renewal reward theory to find the policy parameters that minimize the long-run time average cost.

Clearly, the inventory level repeats over time, so it suffices to study just one cycle that starts and stops when the inventory level hits S. From Fig. 4.4.1 it is simple to see that

$$t_1 = \frac{S-s}{d}, \qquad t_2 = \frac{S-s}{r-d}, \qquad T = t_1 + t_2,$$

are, respectively, the time to empty the inventory from level S to s and switch on production, the time needed to replenish the inventory from level s to S and switch off production, and the duration of a full cycle. Writing T_+ (T_-) for the amount of time that the inventory is positive (negative) during a cycle, it follows that

$$\frac{T_+}{T} = \frac{S}{S-s}, \qquad \frac{T_-}{T} = \frac{s}{S-s}, \qquad \frac{1}{T} = \frac{d}{S-s}\left(1 - \frac{d}{r}\right).$$

Using that the holding and backlogging cost are proportional to the areas of the related triangles, the average cost of one cycle becomes

$$g = \frac{K}{T} + b\frac{s}{2}\frac{T_-}{T} + h\frac{S}{2}\frac{T_+}{T}.$$

Substituting the expressions of T_-, T_+, T in terms of s and S, taking the partial derivatives with respect to s and S and solving for the optimal values gives

$$g^* = \sqrt{2Kd}\sqrt{\frac{hb}{h+b}}\sqrt{1 - \frac{d}{r}}, \qquad s^* = \frac{g^*}{b}, \qquad S^* = \frac{g^*}{h}. \tag{4.4.1}$$

Of course we need $r \geq d$ for stability. Interestingly, when $r = d$, the cost $g^* = 0$.[1]

[1] *Why is that so?*

The (perhaps) most important formula for practical inventory management is the Economic Order Quantity (EOQ) which follows from (4.4.1) by taking $r \to \infty$ and $b \to \infty$, which amount to saying that replenishments arrive immediately and backlogging is not allowed. This results in

$$g^*_{EOQ} = \sqrt{2Kdh} \qquad s^* = 0, \qquad Q^* = \sqrt{\frac{2Kd}{h}},$$

where we write Q^* instead of S^* to indicate that we order in fixed quantities. Note that the cost in these limits *increases*.[2] Observe also that the policy that orders the EOQ is an (s, S)-policy, a (Q, r)-policy, and a (T, S)-policy.

[2] *Because we add binding constraints to the optimization problem.*

An interesting extension is to consider a system which has constraints on cycle length and order size. Can you find an optimal policy?

Ex 4.4.1. Take $Y(t) = t$ and $T_k = A_k$, i.e., the arrival time of the kth job. Use this to prove $\lambda = 1/\mathsf{E}[X]$, where X is the inter-arrival time between jobs.

Ex 4.4.2. Without mentioning, we used the renewal reward theorem in Section 6.4 to find an expression for the availability A. How did we apply it to find (6.4.2).

Ex 4.4.3. Use the renewal reward theorem to relate the load $\rho = \lambda \mathsf{E}[S]$ to the limiting fraction of time the server of a stable $G/G/1$ queue is busy.

Ex 4.4.4. We can also derive the relation between utilization and the fraction of busy time of the server of a $G/G/1$ queue by considering the inequalities

$$\sum_{k=1}^{A(t)} S_k \geq \int_0^t \mathbb{1}_{L(s)>0}\, \mathrm{d}s \geq \sum_{k=1}^{D(t)} S_k. \tag{4.4.2}$$

Explain the above and show that by taking appropriate limits we find $\lambda \mathsf{E}[S] \geq \rho \geq \delta \mathsf{E}[S]$.

Ex 4.4.5. Show for the $G/G/1$ queue that $\rho = \mathsf{E}[U]/(\mathsf{E}[U] + \mathsf{E}[I])$ where $\mathsf{E}[B]$ is the expected busy time and $\mathsf{E}[I]$ the expected idle time. (The busy time of a server is the time between starting after an idle time until a new idle time starts.)

4.5 LITTLE'S LAW

There exists a fascinating connection between the average sojourn time of a job in a (queueing) system and the long-run time-average number of jobs within that system. This relationship is known as *Little's law* and stands as one of the most useful results in any input-output system, including queueing systems. To understand the essence of Little's law, let's consider an illustrative example: imagine a highway segment between points A and B spanning 100 km. Every minute, one car enters the highway, and each car maintains a constant speed of 50 km/h. Now, let's determine the number of cars on that highway segment. Since each car spends 2 hours on the highway, there must be 120 cars traversing the distance at any moment in

time. Stated as a formula, if items arrive at rate λ and each item spends a time $\mathsf{E}[J]$ time in the system, then the total number of items in the system is $\mathsf{E}[L] = \lambda\,\mathsf{E}[J]$.

Our objective in this section is to demonstrate that this law also applies to stochastic systems, given a few simple conditions.

LET'S BEGIN BY expressing the sojourn time of a particular job, denoted as k, in terms of an indicator function[0]. When job k is present in the system at time s, then $\mathbb{1}_{A_k \le s < D_k}$ is 1, and 0 otherwise. Consequently, the integral $\int_0^t \mathbb{1}_{A_k \le s < D_k}\,\mathrm{d}s$ represents the total time that job k spends in the system until time t. [1] Hence, we can express the sojourn time of the kth job as follows:

$$J_k = \int_0^\infty \mathbb{1}_{A_k \le s < D_k},\,\mathrm{d}s.$$

Furthermore, it is evident that $L(s) = \sum_{k=1}^\infty \mathbb{1}_{A_k \le s < D_k}$ represents the number of jobs in the system at time s. By combining these two facts, the LHS of the equation

$$\int_0^t L(s)\,\mathrm{d}s = \int_0^t \sum_{k=1}^\infty \mathbb{1}_{A_k \le s < D_k}\,\mathrm{d}s = \sum_{k=1}^\infty \int_0^t \mathbb{1}_{A_k \le s < D_k}\,\mathrm{d}s \tag{4.5.1}$$

can be interpreted as the total time that jobs have spent in the system up to time t.[2]

CONSIDER A SPECIFIC time T at which the system is empty. At this point, any job $k \le A(T)$ must have left the system, implying that $J_k = \int_0^T \mathbb{1}_{A_k \le s < D_k}\,\mathrm{d}s$ when $k \le A(T)$, therefore,

$$\int_0^T L(s)\,\mathrm{d}s = \int_0^T \sum_{k=1}^{A(T)} \mathbb{1}_{A_k \le s < D_k}\,\mathrm{d}s = \sum_{k=1}^{A(T)} \int_0^T \mathbb{1}_{A_k \le s < D_k}\,\mathrm{d}s = \sum_{k=1}^{A(T)} J_k.$$

Suppose now there exists an infinite sequence of times $\{T_i\}$ at which the system is empty, i.e., $A(T_i) = D(T_i)$, and such that $T_i \to \infty$. In that case we can deduce that

$$\frac{1}{T_i}\int_0^{T_i} L(s)\,\mathrm{d}s = \frac{A(T_i)}{T_i}\frac{1}{A(T_i)}\sum_{k=1}^{A(T_i)} J_k.$$

By taking the limit $i \to \infty$ on both sides, provided the limits exist, we arrive at *Little's law*:[3]

$$\mathsf{E}[L] = \lambda\,\mathsf{E}[J].$$

It is important to note that Little's law does not necessarily hold at an arbitrary moment in time, instead, it expresses a relation between *averages*.

IN THE ABOVE proof of Little's law we assumed that there is an increasing sequence of epochs $\{T_k, k = 0, 1, \ldots\}$ at which the system is empty. However, in many practical queueing situations the system is never empty. It can be proven that Little's law still holds under the weaker condition of rate-stability.

Ex 4.5.1. Think of all servers of the (stable) $G/G/c$ queue as being put in a box. Jobs enter the box at rate λ and stay $\mathsf{E}[S]$ in this box. Use Little's law to conclude that the time-average number of busy server is $\mathsf{E}[L_s] = \lambda\,\mathsf{E}[S]$.

[0] *The indicator $\mathbb{1}_A = 0$ if the event A occurs, and 0 otherwise.*

[1] *A helpful analogy is to imagine that a job pays 1 Euro for each unit of time spent in the system, making this integral equivalent to the total payment made by job k up to time t.*

[2] *Alternatively, it's the total payment made by the jobs up to time t.*

[3] *Use the physical dimensions of the components of Little's law to check that $\mathsf{E}[J] \neq \lambda\,\mathsf{E}[L]$.*

Ex 4.5.2. Consider the $M/M/1$ queue when $\lambda = 5/h$ and $\mu = 6/h$. Suppose that, at the moment you join the system, the number of customers in the system is 10. What is your expected waiting time?

Ex 4.5.3. Let's take 'life' itself as an input-output system, and ask how many people are born and die in the Netherlands per week. For ease, assume that in 2023 the Netherlands has 16 M inhabitants, and people have an expected lifetime (i.e., the time they remain in the system called 'life') of 80 years. With Little's law we conclude that the rate λ into the system (i.e., people born) is $16M/80 = 0.2M$ per year. Again for ease, if a year has 50 weeks, there are 4000 births per week. If the size of the population remains about constant, also around 4000 people die per week.

As it turns out, the death rate in 2023 is about 2900 per week. This is much lower than the above estimate, and a more precise computation does not close the gap. Can you explain this paradoxical difference?

Ex 4.5.4. Explain that $\sum_{k=1}^{A(t)} J_k \geq \int_0^t L(s)\,\mathrm{d}s \geq \sum_{k=1}^{D(t)} J_k$.

Ex 4.5.5 (Continuation of [4.5.4])**.** Take suitable limits to show that $\lambda\,\mathsf{E}[J] \geq \mathsf{E}[L] \geq \delta\,\mathsf{E}[J]$. Where do you need the strong law of large numbers?

4.6 POISSON ARRIVALS SEE TIME AVERAGES

Suppose jobs arrive exactly at the start of an hour and require 59 minutes of service. If we sample the server occupation at job arrival times, the server is always free as seen by the arrivals, while if we track the server over time, it is nearly always busy. Thus, what jobs see upon arrival is *in general not equal* to what the server perceives. However, when jobs arrive as a Poisson process, both sampling methods produce the same number, a result known as the *Poisson arrivals see time averages* (*PASTA*) property. Here we will discuss this property in more detail, and in the remainder of the book we will use it time and again.

LET US SAY that the system is in *state* n at time t when $L(t) = n$. Then define[0]

$$A(n,t) := \sum_{k=1}^{\infty} \mathbb{1}_{A_k \leq t}\, \mathbb{1}_{L(A_k-)=n} = \sum_{k=1}^{A(t)} \mathbb{1}_{L(A_k-)=n}$$

as the number of arrivals up to time t that saw the system in state n.[1] Next, let

$$Y(n,t) := \int_0^t \mathbb{1}_{L(s)=n}\,\mathrm{d}s$$

be the total time the system spends in state n during $[0,t]$, and

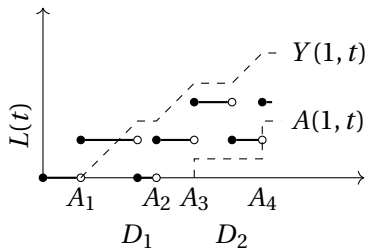
$$p(n,t) := \frac{1}{t}\int_0^t \mathbb{1}_{L(s)=n}\,\mathrm{d}s = \frac{Y(n,t)}{t}, \tag{4.6.1a}$$

be the fraction of time in state n during $[0,t]$. To see the practical relevance of these definitions, suppose it costs w per unit time to have a job in the system. Then, the total cost up to time t is $w\sum_{n=0}^{\infty} nY(n,t)$ and the time-average cost is $w\sum_{n=0}^{\infty} np(n,t)$.

With these definitions, the following two limits, respectively,[2] have the

[0] *Note the A_k-; since $L(t)$ is right-continuous, we concentrate on what an* arrival *sees just before it's arrival*

[1] *[4.6.1],[4.6.2]*



We subtracted $1/2$ from the graph of $A(1,t)$, for otherwise this overlaps with the graph of Y.

[2] *Again assuming they exist.*

interpretations of the arrival rate *while* the system is in state n, and the long-run fraction of time spent in state n, respectively:

$$\lambda(n) := \lim_{t\to\infty} \frac{A(n,t)}{Y(n,t)}, \qquad p(n) = \lim_{t\to\infty} p(n,t).$$

LET US WRITE $A(n,t)/t$ in two ways:

$$\frac{A(n,t)}{Y(n,t)}\frac{Y(n,t)}{t} = \frac{A(n,t)}{t} = \frac{A(t)}{t}\frac{A(n,t)}{A(t)}. \tag{4.6.2}$$

Taking the limit $t \to \infty$, we obtain for the LHS that the rate of arrivals that see the system in state n is equal to

$$\lim_{t\to\infty} \frac{A(n,t)}{t} = \lim_{t\to\infty} \frac{A(n,t)}{Y(n,t)}\frac{Y(n,t)}{t} = \lambda(n)p(n). \tag{4.6.3}$$

For the RHS in (4.6.2), it is clear that $A(t)/t \to \lambda$, so we need to interpret $A(n,t)/A(t)$. First, observe

$$\frac{A(n,t)}{A(t)} = \frac{1}{A(t)} \sum_{k=1}^{A(t)} \mathbb{1}_{L(A_k-)=n}.$$

Then, since $A(t) \to \infty$ as $t \to \infty$, it follows from the renewal reward theorem[3] that the following limit can be defined:

[3] *[4.6.4]*

$$\pi(n) := \lim_{t\to\infty} \frac{1}{A(t)} \sum_{k=1}^{A(t)} \mathbb{1}_{L(A_k-)=n}. \tag{4.6.4}$$

That is, $\pi(n)$ is the long-run fraction of arrivals that observe n customers in the system. Finally, as in (4.6.2) the LHS and RHS are equal, the limits are the same. Hence,

$$\lambda\pi(n) = \lambda(n)p(n). \tag{4.6.5}$$

With this formula we can obtain two useful results. Write $\mathscr{S}$ for all the states the system can attain. When there is no upper bound on $L(t)$, then $\mathscr{S} = \mathbb{N}$, but when the $L(t)$ is bounded by some number K, then $\mathscr{S} = \{0, 1, \ldots, K\}$. Clearly, any arrival must see the system in some state, hence $\sum_{n\in\mathscr{S}} \pi(n) = 1$. Thus, summing over n in (4.6.5), the overall arrival rate λ is such that

$$\lambda = \sum_{n\in\mathscr{S}} \lambda(n)p(n), \tag{4.6.6}$$

from which follows that

$$\pi(n) = \frac{\lambda(n)}{\lambda} p(n).$$

This in turn gives us a second result of utmost importance:

$$\lambda(n) = \lambda \iff \pi(n) = p(n). \tag{4.6.7}$$

In words, if the arrival rate does not depend on the state of the system, i.e., $\lambda(n) = \lambda$, the sample probabilities $\{\pi(n)\}$ are equal to the time-average probabilities $\{p(n)\}$.[4]

[4] *Thus, what arrivals see agrees with what the server sees.*

As the example in the introduction of this section shows, this property is not satisfied in general. However, when the arrival process is Poisson we have[5] that $\lambda(n) = \lambda$, and then, by (4.6.7), $\pi(n) = p(n)$. This result is known as the *PASTA* property.

[5] *A rigorous proof of this is hard, see e.g., El-Taha and Stidham Jr. [1998]*

BY DISCRETIZING TIME we can provide a simple and intuitive way to understand when PASTA does (not) hold. In a discrete-time queueing system, the event $L(A_k-) = m$ has the following interpretation. If in some period n a job arrives, i.e., $a_n = 1$, then the system contains m jobs at the end of the previous period, i.e., $L_{n-1} = m$. Thus, $\pi(m) \approx \mathsf{P}\{L_{n-1} = m | a_n = 1\}$. With Bayes' law:

$$\mathsf{P}\{L_{n-1} = m | a_n = 1\} = \frac{\mathsf{P}\{a_n = 1 | L_{n-1} = m\}\,\mathsf{P}\{L_{n-1} = m\}}{\mathsf{P}\{a_n = 1\}}.$$

If a_n is independent of $\{a_i\}_{i=1}^{n-1}$, hence of L_{n-1} (because L_{n-1} is a function of $\{a_i\}_{i=1}^{n-1}$), $\mathsf{P}\{a_n = 1 | L_{n-1} = m\} = \mathsf{P}\{a_n = 1\}$, and only then

$$\pi(m) \approx \frac{\mathsf{P}\{a_n = 1\}\,\mathsf{P}\{L_{n-1} = m\}}{\mathsf{P}\{a_n = 1\}} = \mathsf{P}\{L_{n-1} = m\} \approx p(n).$$

(For the mathematically interested student: The last $\approx$ holds because, by assumption, $N^{-1}\sum_{n=1}^{N} \mathbb{1}_{L_n=m} \to p(m)$ as $N \to \infty$, and this (strong) convergence implies the (weak) converge $\mathsf{P}\{L_n = m\} \to p(m)$ as $n \to \infty$.)

However, PASTA does not hold when a_n depends on L_{n-1}. For instance, if $a_n = 0$ when $L_{n-1} > 0$, and $c_k \sim \text{Bern}(d)$ for some small probability d, then $\pi(1) = 0$, but $\mathsf{P}\{L_{n-1} = 0\} > 0$ in general.

ACTUALLY, WITH JUST PASTA, Little's law and memorylessness we can already achieve quite a few interesting results for the $M/M/1$ queue.[6] First observe that

$$\mathsf{E}[J] = \mathsf{E}[L]\,\mathsf{E}[S] + \mathsf{E}[S] = \lambda\,\mathsf{E}[J]\,\mathsf{E}[S] + \mathsf{E}[S] = \rho\,\mathsf{E}[J] + \mathsf{E}[S],$$

hence,

$$\mathsf{E}[J] = \frac{\mathsf{E}[S]}{1-\rho}, \tag{4.6.8a}$$

and then step by step,

$$\mathsf{E}[L] = \lambda\,\mathsf{E}[J] = \frac{\lambda\,\mathsf{E}[S]}{1-\rho} = \frac{\rho}{1-\rho}, \tag{4.6.8b}$$

$$\mathsf{E}[W] = \mathsf{E}[L]\,\mathsf{E}[S] = \frac{\rho}{1-\rho}\,\mathsf{E}[S]\,(= \mathsf{E}[J] - \mathsf{E}[S]) \tag{4.6.8c}$$

$$\mathsf{E}[Q] = \lambda\,\mathsf{E}[W] = \frac{\rho^2}{1-\rho}, \tag{4.6.8d}$$

$$\mathsf{E}[L_s] = \mathsf{E}[L] - \mathsf{E}[Q] = \frac{\rho}{1-\rho} - \frac{\rho^2}{1-\rho} = \rho, \tag{4.6.8e}$$

TO LET YOU think a bit deeper about the PASTA property, and statistics in general, consider the next story.[7] The problem is known as the 'inspection paradox'.[8]

In a small Midwestern town there lived a retired railroad engineer named William Johnson. The main line on which he had worked for so many years passed through the town. Mr. Johnson suffered from insomnia and would often wake up at any odd hour of the night and be unable to fall asleep again. He found it helpful, in such cases, to take a walk along the deserted streets of the town, and his way always led him to the railroad

[6] *For the $M/G/1$ queue, things are bit subtler, see [4.6.5] and Section 5.4.*

[7] *This is an interesting riddle from 'Puzzle Math' by G. Gamow and M. Stern.*

[8] *Look it up on the web. You should be aware of such biases in data science.*

crossing. He would stand there thoughtfully watching the track until a train thundered by through the dead of the night. The sight always cheered the old railroad man, and he would walk back home with a good chance of falling asleep.

After a while he made a curious observation; it seemed to him that most of the trains he saw at the crossing were traveling eastward, and only a few were going west. Knowing very well that this line was carrying equal numbers of eastbound and westbound trains, and that they alternated regularly, he decided at first that he must have been mistaken in this reckoning. To make sure, he got his little notebook, and began putting down "E" or "W," depending on which way the first train to pass was traveling. At the end of a week, there were five "E's" and only two "W's" and the observations of the next week gave essentially the same proportion. Could it be that he always woke up at the same hour of night, mostly before the passage of eastbound trains?

Being puzzled by this situation, he decided to undertake a rigorous statistical study of the problem, extending it also to the daytime. He asked a friend to make a long list of arbitrary times such as 9:35 a.m., 12:00 noon, 3:07 p.m., and so on, and he went to the railroad crossing punctually at these times to see which train would come first. However, the result was the same as before. Out of one hundred trains he met, about seventy-five were going east and only twenty-five west. In despair, he called the depot in the nearest big city to find whether some of the westbound trains had been rerouted through another line, but this was not the case. He was, in fact, assured that the trains were running exactly on schedule, and that equal numbers of trains daily were going each way. This mystery brought him to such despair that he became completely unable to sleep and was a very sick man.

Can you reveal the mystery thereby solving the sleeping problems of Mr. Johnson?[9]

[9] *You can find the solution online.*

Ex 4.6.1. Derive (4.6.6).

Ex 4.6.2. If $\lambda > \delta$, can it happen that $\lim_{t\to\infty} A(n,t)/t > 0$ for some (finite) n?

Ex 4.6.3. When $\lambda \neq \delta$, is $\pi(n) \geq \delta(n)$?

Ex 4.6.4. Suppose that the limit $A(n,t)/t \to Y$ exists as $t \to \infty$. Use this and the renewal-reward theorem to explain that $A(n,t)/t \to \lambda\pi(n)$.

Ex 4.6.5. While (4.6.8c) is true for the $M/M/1$, it is *not true for the* $M/G/1$ *queue*. Why is that?

4.7 LEVEL CROSSING AND BALANCE EQUATIONS

Consider a sample path[0] of $L := \{L\}$. We say that the sample path *up-crosses level* n at time t when the number of jobs in the system changes from n to $n+1$ due to an arrival, in other words, when $L(t-) = n$ and $L(t) = n+1$. The sample path *down-crosses level* n at time t when, due to a departure, $L(t-) = n+1$ and $L(t) = n$. Clearly, the number of up-crossings

[0] *A sample path can be seen as the graph of the number of jobs in the system obtained by simulation.*

and down-crossings cannot differ by more than 1 at any time, because it is only possible to down-cross level n after an up-crossing (or the other way around). This simple idea will prove to be a key stepping stone in the analysis of queueing systems.

In this section we make a distinction between jobs that just arrive and jobs that do enter, because we want our arguments to apply to queueing systems in which jobs can be blocked or do not enter for other reasons. Just as in Section 4.2, we write $A(t)$ and $\tilde{A}(t)$ to distinguish between the counting functions.

SIMILAR TO THE definition for $A(n,t)$ in (4.6.1), let

$$\tilde{A}(n,t) := \sum_{k=1}^{\infty} \mathbb{1}_{A_k \le t} \mathbb{1}_{L(A_k)=n+1} = \sum_{k=1}^{A(t)} \mathbb{1}_{L(A_k)=n+1}$$

denote the number of jobs that *enter* the system when the system contains n jobs just before arrival. To see this, if $\mathbb{1}_{L(A_k-)=n} = 1$ then customer k sees n jobs in the system. If customer k does *not* join the system, then $L(A_k-) = L(A_k)$. However, if customer k enters the system, then $L(A_k) = n+1$, because just after its arrival, the system contains $n+1$ jobs. Likewise, let

$$D(n,t) := \sum_{k=1}^{\infty} \mathbb{1}_{D_k \le t} \mathbb{1}_{L(D_k)=n} = \sum_{k=1}^{D(t)} \mathbb{1}_{L(D_k)=n}$$

denote the number of departures up to time t that *leave n customers behind*. Note that we write $L(D_k)$ and not $L(D_k-)$. With this, define

$$\mu(n+1) := \lim_{t\to\infty} \frac{D(n,t)}{Y(n+1,t)},$$

as the departure rate from state $n+1$. As for $\tilde{A}(n,t)$ it is easy to get confused here: to leave n jobs behind, the system must contain $n+1$ jobs just prior to the departure.

AS JOBS ARRIVE and depart as single units, $|L(t) - L(t-)| \le 1$ for all $t \ge 0$. But this implies that[1]

$$|\tilde{A}(n,t) - D(n,t)| \le 1. \tag{4.7.1}$$

[1] *Think about this. This is simple, but has profound consequences. Do you understand why this does not need to hold for $A(n,t)$?*

From this observation it follows immediately that

$$\lim_{t\to\infty} \frac{\tilde{A}(n,t)}{t} = \lim_{t\to\infty} \frac{D(n,t)}{t}. \tag{4.7.2}$$

By defining the rate at which jobs enter when the system contains n jobs as

$$\tilde{\lambda}(n) = \lim_{t\to\times} \frac{\tilde{A}(n,t)}{Y(n,t)},$$

we see that

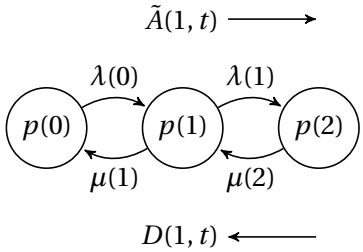
$$\lim_{t\to\infty} \frac{\tilde{A}(n,t)}{t} = \lim_{t\to\infty} \frac{\tilde{A}(n,t)}{Y(n,t)} \frac{Y(n,t)}{t} = \tilde{\lambda}(n)p(n).$$

Similarly, the rate of jobs that leave n jobs behind is

$$\lim_{t\to\infty} \frac{D(n,t)}{t} = \lim_{t\to\infty} \frac{D(n,t)}{Y(n+1,t)} \frac{Y(n+1,t)}{t} = \mu(n+1)p(n+1).$$

Combining the above with (4.7.2) we arrive at the *level-crossing equation*, a result of fundamental importance in queueing (and inventory) theory.

$$\tilde{\lambda}(n)p(n) = \mu(n+1)p(n+1). \tag{4.7.3}$$



SUPPOSE WE CAN specify[2] the arrival and service rates $\tilde{\lambda}(n)$ and $\mu(n)$, then we can easily compute the long-run fraction of time $p(n)$ that the system contains n jobs. To see this, rewrite (4.7.3) as

$$p(n+1) = \frac{\tilde{\lambda}(n)}{\mu(n+1)}p(n). \tag{4.7.4}$$

[2] *In Section 5.1 and onward we will model many queueing situations by making suitable choices for $\tilde{\lambda}(n)$ and $\mu(n)$.*

A straightaway recursion then leads to

$$p(n+1) = \frac{\tilde{\lambda}(n)\tilde{\lambda}(n-1)\cdots\tilde{\lambda}(0)}{\mu(n+1)\mu(n)\cdots\mu(1)}p(0).$$

Thus, $p(n)$, $n \geq 1$, is just $p(0)$ times a constant, and this constant is based on arrival and service rates.

To determine $p(0)$ we can use the fact that the numbers $p(n)$ represent probabilities. Hence, from the normalizing condition $\sum_{n=0}^{\infty} p(n) = 1$, we get $p(0) = G^{-1}$ with G being the *normalization constant*

$$G = 1 + \sum_{n=0}^{\infty} \frac{\tilde{\lambda}(n)\tilde{\lambda}(n-1)\cdots\tilde{\lambda}(0)}{\mu(n+1)\mu(n)\cdots\mu(1)}.$$

Finally, once we have $p(n)$, it is easy to compute the time-average[3] number of jobs in the system and the long-run fraction of time the system contains at least n:

[3] *It is important to realize that this is not necessarily the same as what jobs see upon arrival.*

$$\mathsf{E}[L] = \sum_{n=0}^{\infty} np(n), \qquad \mathsf{P}\{L \geq n\} = \sum_{i=n}^{\infty} p(i).$$

WITH THE DEFINITIONS of $A(n,t)$ and $D(n,t)$, we can establish a nice relation between $\pi(n)$ and $\delta(n)$, i.e., statistics as obtained by the departures. Define, analogous to $\pi(n)$,

$$\delta(n) := \lim_{t\to\infty} \frac{D(n,t)}{D(t)}$$

as the long-run fraction of jobs that leave n jobs *behind*. From (4.7.2) and supposing that $A(n,t) = \tilde{A}(n,t)$,

$$\frac{A(t)}{t}\frac{A(n,t)}{A(t)} = \frac{A(n,t)}{t} = \frac{\tilde{A}(n,t)}{t} \approx \frac{D(n,t)}{t} = \frac{D(t)}{t}\frac{D(n,t)}{D(t)}.$$

Taking limits at the left and right, and using (4.2.1), we obtain for the $G/G/c$ queue[4]

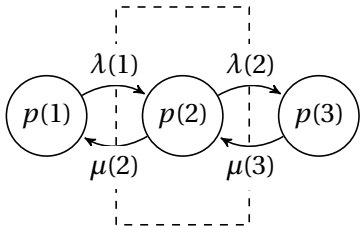
$$\lambda\pi(n) = \delta\delta(n). \tag{4.7.5}$$

[4] *Because customers arrive and leave as single units in a (rate-stable) $G/G/c$ queue.*

Consequently, for the $G/G/c$ queue the statistics obtained by arrivals is the same as statistics obtained by departures, i.e.,

$$\lambda = \delta \iff \pi(n) = \delta(n). \tag{4.7.6}$$

IT IS IMPORTANT to realize that the level-crossing argument cannot always be used, as it is not always possible to split the state space into two disjoint parts by 'drawing a line' between two states. For a more general approach, we focus on a single state and count how often this state is entered and left. Specifically, define $I(n,t) = A(n-1,t) + D(n,t)$ as the number of times the queueing process enters state n either due to an arrival from state $n-1$ or due to a departure leaving n jobs behind. Similarly, $O(n,t) = A(n,t) + D(n-1,t)$ counts how often state n is left either by an arrival (to state $n+1$) or a departure (to state $n-1$).



Just like (4.7.1), it is evident that $|I(n,t) - O(n,t)| \le 1$, and this implies[5]

$$\lambda(n-1)p(n-1) + \mu(n+1)p(n+1) = (\lambda(n) + \mu(n))p(n). \tag{4.7.7}$$

[5] [4.7.5]

These equations hold for any $n \ge 1$ and are known as the *balance equations*. We will use these equations later in the book to analyze queueing networks.

LEVEL CROSSING ARGUMENTS CAN also be applied to continuous systems. With this we can find a closed form expression for the waiting time of the $M/G/1$ queue. However, this is a bit more complicated as the argument hinges on an integral equation. If you don't like developing mathematical skills, you may skip this derivation.

Suppose that the waiting time W of the $M/G/1$ queue has a density f, that is, the cdf $F(x) = \mathsf{P}\{W \le x\}$ has a derivative $f(x)$ for all $x > 0$.[6] (F is only differentiable from the right at 0 because $\mathsf{P}\{W = 0\} > 0$, while $\mathsf{P}\{W \le x\} = 0$ for $x < 0$.) Consider some level $x > 0$. By PASTA, the rate of arriving jobs that see a waiting time $y \in [0,x)$ is equal to $\lambda f(y)$. (Observe that we use the splitting property of Poisson processes.) To up-cross level x from state $y < x$, the service time must be at least $x - y$, the probability of which is $G(x-y) = \mathsf{P}\{S > x - y\}$. Adding up the rates for all possible waiting times below x gives that $\lambda \int_0^x f(y)G(x-y)\,\mathrm{d}y$ is the up-crossing rate of level x. The downcrossing rate is $f(x)$ because a fraction $f(x)$ of waiting time is served per unit time at level x. Equaling these rates gives the integral equation $\lambda \int_0^x f(y)G(x-y)\,\mathrm{d}y = f(x)$. For $x = 0$, we need to consider the atom $\mathsf{P}\{W = 0\}$. In that case, $\lambda\,\mathsf{P}\{W = 0\} = f(0+)$. We can combine all this in the following level crossing equation

[6] *For a real proof we should prove first that f exists.*

$$\lambda \int_0^x G(x-y)\,\mathrm{d}F(y) = f(x).$$

Using this integral equation,

$$\begin{aligned}
\mathsf{E}[W] &= \int_0^\infty x f(x)\,\mathrm{d}x = \lambda \int_0^\infty x \int_0^x G(x-y)\,\mathrm{d}F(y)\,\mathrm{d}x = \\
&= \lambda \int_0^\infty \int_0^\infty x\,\mathbb{1}_{y \le x} G(x-y)\,\mathrm{d}x\,\mathrm{d}F(y) \\
&= \lambda \int_0^\infty \int_0^\infty (u+y)G(u)\,\mathrm{d}u\,\mathrm{d}F(y) \\
&= \lambda \int_0^\infty (\mathsf{E}\left[S^2\right]/2 + y\,\mathsf{E}[S])\,\mathrm{d}F(y) \\
&= \lambda\,\mathsf{E}\left[S^2\right]/2 + \lambda\,\mathsf{E}[S]\,\mathsf{E}[W].
\end{aligned}$$

Bringing $\lambda \mathsf{E}[S]\,\mathsf{E}[W] = \rho\,\mathsf{E}[W]$ to the left, and dividing by $1-\rho$ gives

$$\mathsf{E}[W] = \lambda \frac{\mathsf{E}\left[S^2\right]}{2(1-\rho)}.$$

It is left to compute the second moment of the service time S. We will derive this formula with the renewal reward equation in Section 5.4.

Ex 4.7.1. This first exercise is meant to help you to understand the notation by applying it to a simple case. Consider the following (silly) queueing process. At times $0, 2, 4, \ldots$ customers arrive, each customer requires 1 unit of service, and there is one server.

1. Find an expression for $A(n,t)$, when $L(0) = 0$.
2. Show that $\pi(0) = 1$ and $\pi(n) = 0$, for $n > 0$.
3. Check that $\lambda\pi(n) = \lambda(n)p(n)$.
4. Find an expression for $Y(n,t)$.
5. Compute $p(n)$ and $\lambda(n)$.
6. Compute $D(n,t)$ and $\mu(n+1)$ for $n \geq 0$.
7. Compute $\lambda(n)p(n)$ for $n \geq 0$, and check $\lambda(n)p(n) = \mu(n+1)p(n+1)$.

Ex 4.7.2. Derive $\mathsf{E}[L] = \sum_{n=0}^{\infty} np(n)$ from (4.3.4).

Ex 4.7.3. Show for the $M/G/1$ that with probability ρ a job leaves behind a busy server.

Ex 4.7.4. Consider a single server that serves one queue and serves only in batches of 2 jobs at a time (so never 1 job or more than 2 jobs). At most 3 jobs fit in the system. Single jobs arrive as a Poisson process with λ. Due to blocking, we take $\lambda(n) = \lambda$ for $n < 3$ and $\lambda(n) = 0$ for $n \geq 3$. The batch service times are exponentially distributed with mean $1/\mu$, so that by the memoryless property, $\mu(n) = \mu$.

When level-crossing arguments can be applied, the analysis often becomes quite easy.

What is the acronym for this system?

Make a graph of the state-space and show, with arrows, the transitions that can occur and use level-crossing arguments to express the steady-state probabilities $p(n), n = 0, \ldots, 3$ in terms of λ and μ.

Ex 4.7.5. Show (4.7.7) from $|I(n,t) - O(n,t)| \leq 1$.

4.8 GRAPHICAL SUMMARIES

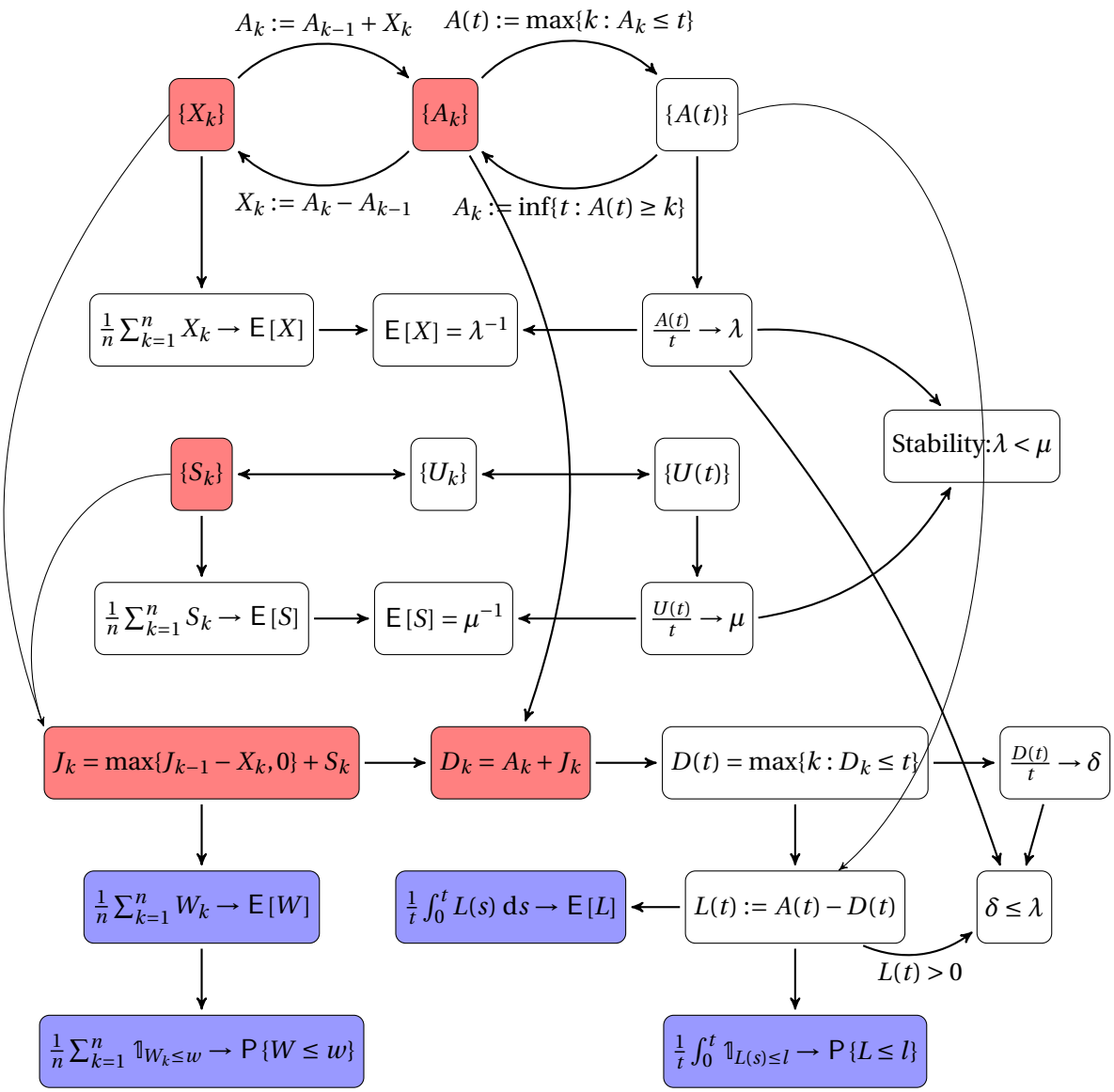


Figure 4.8.1: An overview of the relations between the different types of times (inter-arrivals, arrivals, departures and so on) we use to construct a queueing process in continuous time.

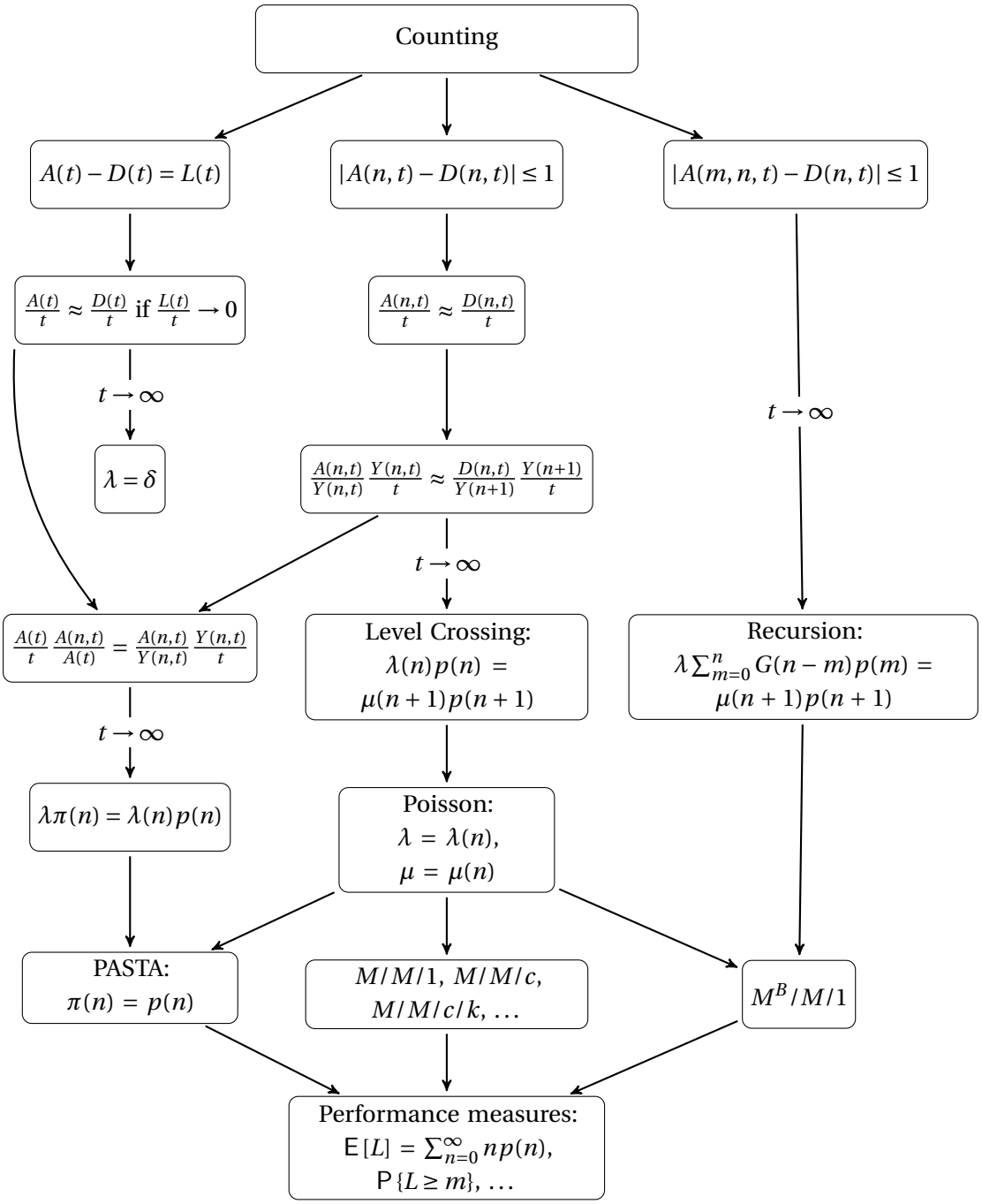


Figure 4.8.2: An overview of the relations that derive from level-crossing arguments and the queueing systems that can be derived from these relations.

5 EXACT MODELS

As we will see, many performance measures for queueing systems depend on the distribution of the inter-arrival times and the service times. To distinguish between the most important different models we present in Section 4.1 some commonly used notation.

In this chapter we use the concepts of Chapter 4 to model and analyze many queueing systems in steady state. With sample-path analysis and level-crossing, we show in Section 5.1 that is simple to derive useful expressions for performance measures of the $M/M/1$ and many of its variations.

In Section 5.2 we combine level-crossing with Little's law and PASTA to compute the most important performance measures for numerous queueing examples.

Next, in Section 5.3 and Section 5.4, We focus on finding the expected waiting time for batch queues and the $M/G/1$ queue. In the last two sections of this chapter we derive expressions for the queue length distributions of the batch queue and the $M/G/1$ queue.

5.1 $M/M/1$ QUEUE AND ITS VARIATIONS

In this section we devevelop many different queueing models by making judicious choices for $\lambda(n)$, $\tilde{\lambda}(n)$ and $\mu(n)$ in the level-crossing equations (4.7.4). It is much more important to understand how to specify the correct level-crossing equations than to derive theexpressions for $\mathsf{E}[L]$, say; the main text therefore only presents the results, and delagates the derivations to the exercises. We first discuss models for which we can obtain closed-form solutions for the performance measures. Then we sketch an algorithm to compute these measures for the general case. We assume that all systems that do not block arrivals in some way are stable; when demand is blocked, the systems is stable anyway.

IN THE MODELS below we allow $\lambda(n)$ and $\mu(n)$ to depend on the number n in the system, but given n, the inter-arrival and service times should be exponentially distributed, that is,[0]

$$X|L = n \sim \text{Exp}(\lambda(n)), \qquad S|L = n \sim \text{Exp}(\mu(n)).$$

[0] *This very general queueing systems is sometimes written as the $M(n)/M(n)/1$ queue.*

However, as in the general case arrivals need not enter, we consider

$$\begin{aligned} \mathsf{P}\{L(t+\Delta t) = n+1 \,|\, L(t) = n\} &= \tilde{\lambda}(n)\Delta t + o(\Delta t), \quad n \geq 0, \\ \mathsf{P}\{L(t+\Delta t) = n-1 \,|\, L(t) = n\} &= \mu(n)\Delta t + o(\Delta t), \quad n \geq 1. \end{aligned}$$

Below, when all arrivals are accepted, we write $\tilde{\lambda} = \lambda$, but otherwise, we use $\tilde{\lambda}$.

THE $M/M/1$ QUEUE has a constant arrival and service rate so that we take

$$\lambda(n) = \lambda, \qquad \mu(n) = \mu.$$

With this, level-crossing equations become

$$p(n+1) = \frac{\lambda(n)}{\mu(n+1)} p(n) = \frac{\lambda}{\mu} p(n) = \rho p(n), \quad \rho = \frac{\lambda}{\mu}.$$

As this holds for any $n \geq 0$, $p(n+1) = \rho^{n+1} p(0)$. Since $p(0) \sum_{n=0}^{\infty} \rho^n = p(0)/(1-\rho)$, the normalization condition $\sum_{n=0}^{\infty} p(n) = 1$ implies that $p(0) = 1-\rho$, hence

$$p(n) = (1-\rho)\rho^n, \quad n \geq 0. \tag{5.1.1}$$

It is now easy to compute the most important performance measures. The load and the utilization are equal since all jobs are accepted, hence served. With a bit of algebra,[1]

[1] [5.1.2]

$$\rho = \frac{\lambda}{\mu}, \qquad \mathsf{E}[L_s] = \rho, \tag{5.1.2}$$

$$\mathsf{E}[L] = \frac{\rho}{1-\rho}, \qquad \mathsf{P}\{L > n\} = \rho^{n+1}, \tag{5.1.3}$$

recall that $\mathsf{E}[L_s]$ is the time-average number of jobs in service. Since $\mathsf{E}[L]$ is the number in the system, we see that for the number of jobs in queue,

$$\mathsf{E}[Q] = \mathsf{E}[L] - \mathsf{E}[L_s] = \frac{\rho^2}{1-\rho}.$$

THE $M/M/c$ QUEUE has c identical servers to process jobs. To model this, set

$$\lambda(n) = \lambda, \qquad \mu(n) = \mu \min\{n, c\}.$$

After some calculations we obtain[2]

[2] [5.1.6]

$$\rho = \frac{\lambda}{c\mu}, \tag{5.1.4a}$$

$$p(n) = \frac{1}{G} \frac{1}{\prod_{k=1}^{n} \min\{c,k\}} (c\rho)^n, \quad n \geq 1, \tag{5.1.4b}$$

$$G = \frac{1}{p(0)} = \sum_{n=0}^{c-1} \frac{(c\rho)^n}{n!} + \frac{(c\rho)^c}{(1-\rho)c!}, \tag{5.1.4c}$$

$$\mathsf{E}[Q] = \sum_{n=c}^{\infty} (n-c)p(n) = \frac{(c\rho)^c}{c!G} \frac{\rho}{(1-\rho)^2} \tag{5.1.4d}$$

$$\mathsf{E}[L_s] = \sum_{n=0}^{\infty} \min\{n,c\} p(n) = \frac{\lambda}{\mu}. \tag{5.1.4e}$$

Note in particular that $\mathsf{E}[L_s] \neq \rho$.

THE $M/M/1/K$ QUEUE blocks jobs when $L \geq K$. We can implement this by taking

$$\tilde{\lambda}(n) = \lambda \mathbb{1}_{n<K}, \qquad \mu(n) = \mu.$$

This gives $\lambda p(n) = \mu p(n+1)$ for $n < K$, and $p(n) = 0$ for $n > K$ because $\tilde{\lambda}(n) = 0$ for $n \geq K$). Using $\tilde{\lambda}(n)p(n) = \mu(n+1)p(n+1)$, it follows right away that

$$\rho = \frac{\lambda}{\mu} \qquad p(n) = \frac{1-\rho}{1-\rho^{K+1}} \rho^n.$$

There is a subtle point here to make. The rate at which jobs arrive at the system is $\lambda(K) = \lambda$ because jobs still arrive as a Poisson process, but the rate at which the jobs enter is $\tilde{\lambda}(K). = 0$ Consequently,

$$\lambda\pi(K) = \lambda(K)p(K) \neq \tilde{\lambda}(K)p(K) = 0.$$

Thus, since $\lambda(K) = \lambda$, we still have that $\pi(K) = p(K)$. By PASTA, the *fraction* of jobs rejected is therefore equal to $p(K)$. Moreover, since jobs are blocked, the utilization is no longer equal to the load; the utilization is now $\lambda(1 - p(K))/\mu \neq \lambda/\mu = \rho$, and

$$\mathsf{E}[L_s] = \sum_{n=0}^{K} \min\{n,1\}p(n) = 1 - p(0) = \frac{\lambda}{\mu}(1 - p(K)). \tag{5.1.5}$$

THE $M/M/c/c$ QUEUE blocks jobs when all of its c servers are occupied.[3] Thus, we take

$$\tilde{\lambda}(n) = \lambda\,\mathbb{1}_{n<c}, \qquad \mu(n) = n\mu.$$

[3] *This queueing model is also known as the Erlang B model.*

This queueing system is often used to determine the number of beds needed by a hospital: the beds act as servers and the patients as jobs. Given the arrival rate of patients, and the average time they occupy a bed[4], the problem is to find the number of beds c such that the probability $p(c)$ to block a patient is less than some threshold, 1% say. For hospitals it is reasonable to assume Poisson arrivals since patients arrive independently from a large population. Then, by PASTA, we conclude that indeed$\pi(c) = p(c)$, in other words, the blocked fraction is equal to the time average $p(c)$ during which all beds are occupied. As for the service times, there are often many patients in a hospital, and they have many different reasons why they occupy a bed. Hence it seems reasonable to model the recovery times as exponentially distributed.

[4] *i.e., the expected service time*

Just as for the $M/M/c$ queue, $\rho = \lambda/(c\mu)$. With some algebra[5] we find

$$p(n) = \frac{1}{G}\frac{(c\rho)^n}{n!}, 0 \leq n \leq c, \qquad G = \sum_{n=0}^{c} \frac{(c\rho)^n}{n!},$$

$$\mathsf{E}[Q] = 0, \qquad \mathsf{E}[L_s] = \frac{\lambda}{\mu}\big(1 - p(c)\big).$$

[5] *[5.1.7]*

The last two results are easy. Jobs either find a free server or are blocked, hence, jobs cannot be in queue. Next, observing that $\lambda(1 - p(c))$ is the rate of accepted jobs[6], the utilization *of the system* is $\lambda(1 - p(c))/\mu$, and this in turn is equal to the average number of busy servers.

[6] *Since, by PASTA, a fraction $p(c)$ is lost.*

THE $M/M/\infty$ QUEUE is simple to analyze, as is has *ample* servers. Any job that arrives finds a free server, hence, its service starts directly aft the arrival of a job. Therefore,

$$\lambda(n) = \lambda, \qquad \mu(n) = n\mu.$$

By taking the limit $c \to \infty$ in the expressions of the $M/M/c$ queue (or the $M/M/c/c$ queue) we get

$$p(n) = e^{-\lambda/\mu}\frac{(\lambda/\mu)^n}{n!}, \qquad \mathsf{E}[L] = \mathsf{E}[L_s] = \frac{\lambda}{\mu}.$$

We see that the number of busy servers in the $M/M/\infty$ queue is Poisson distributed with parameter $\lambda\,\mathsf{E}[S]$. We mention in passing—but do not prove it—that the same results also hold for the $M/G/\infty$ queue.

WITH A FINITE CALLING POPULATION the number of jobs is fixed, N say. This model is useful to analyze repair systems and inventory systems of spare parts. To see this, consider a factory with N machines and c mechanics.[7] A machine can be in one of two states: working or failed. When a machine breaks down, it moves to the repair department and waits until it is repaired. When n machines are in repair, there are $N-n$ machines still working. Thus, if λ is the rate at which a individual machine can fail, $\lambda(N-n)$ is the rate at which any of the working machines can fail, we obtain

[7] *When $c = N$ we obtain the Ehrenfest model of diffusion, which is used to provide some insight into the second law of thermodynamics.*

$$\lambda(n) = \lambda(N-n), \qquad \mu(n) = \mu \min\{n, c\},$$

as jobs are not lost, $\tilde{\lambda}(n) = \lambda(n)$.

In case $c = 1$, the probabilities become [8]

[8] *[5.1.9]*

$$p(n) = \frac{N!}{(N-n)!}\frac{\rho^n}{G}, \qquad G = \sum_{n=0}^{N} \rho^n \frac{N!}{(N-n)!}.$$

As the expression for G cannot be simplified, there is not much point trying to derive simple expressions for $\mathsf{E}[L]$. There exist algorithms to compute $\mathsf{E}[L]$, we don't discuss these here.

A CUSTOMER BALKS when the customer finds the queue too long, hence decides not to join. A simple example model with customer balking is to take,

$$\tilde{\lambda}(n) = \lambda[K-n]^+, \qquad \mu(n) = \mu.$$

Balking is not necessarily the same as blocking. In the latter case, $\tilde{\lambda}(n) = \lambda \mathbb{1}_{n<K}$; in the former case, a fraction of the customers may already choose not the join the system at a lower level than K.

As the steady-state probabilities depend heavily on the form of $\tilde{\lambda}(n)$ and $\mu(n)$, we use the recursion (4.7.4) and the code below to compute the performance measures.

THE $M/M/c/K$ QUEUE is a multiserver queue with blocking and is a simple generalization of the $M/M/c$ and $M/M/1/K$ queue. Now,

$$\tilde{\lambda}(n) = \lambda \mathbb{1}_{n<K}, \qquad \mu(n) = \mu \min\{n, c\}.$$

We can use the code below if we need to analyze such as system.

THE MOST GENERAL case can only be approached with numerical methods. We present example code to deal with a queue with balking; the other models of this section are easy modifications of this. Note that we need to assume that K is finite, although it can be large. For all practical purposes, allowing K in the order of 1000 seems more than large enough.

```
import numpy as np

arrival_rate = 23
service_rate = 6
c, K = 4, 20

```

```python

def labda_tilde(n):
    return arrival_rate * (K > n)


def mu(n):
    return service_rate * min(n, c)


p = np.ones(K + 1, dtype=float)
for n in range(K):
    p[n + 1] = p[n] * labda_tilde(n) / mu(n + 1)
p /= p.sum()  # normalize

Labda = sum(labda_tilde(n) * p[n] for n in range(len(p)))
print(Labda, service_rate * c) # check
EQ = sum(p[n] * max(n - c, 0) for n in range(len(p)))
ELs = sum(p[n] * min(n, c) for n in range(len(p)))
EL = sum(p[n] * n for n in range(len(p)))
print(EL, EQ + ELs) # check
```

Ex 5.1.1. Explain that for the $M/M/1$ queue $\mathsf{E}[Q] = \sum_{n=1}^{\infty}(n-1)\pi(n)$ and use this to find that $\mathsf{E}[Q] = \rho^2/(1-\rho)$.

Ex 5.1.2. Use moment-generating functions to derive $\mathsf{E}[L]$ and $\mathsf{V}[L]$ for the $M/M/1$ queue, and show that $\mathsf{P}\{L > n\} = \rho^{n+1}$.

Ex 5.1.3. Check that the performance measures of the $M/M/c$ queue reduce to those of the $M/M/1$ queue if $c = 1$.

Ex 5.1.4. Show that (5.1.5) is true.

Ex 5.1.5. Show that as $K \to \infty$, the performance measures of the $M/M/1/K$ converge to those of the $M/M/1$ queue.

The remaining exercises ask you to derive the expressions for the various models. You can skip them if you don't like algebra.

Ex 5.1.6. Derive the expressions for the $M/M/c$ queue.

Ex 5.1.7. Derive the expressions for the $M/M/c/c$ queue.

Ex 5.1.8. Derive the expressions for the $M/M/\infty$ queue.

Ex 5.1.9. Find the steady state probabilities for a single-server queue with a finite calling population with N jobs.

Ex 5.1.10. Derive the steady state probabilities $p(n)$ for a queue with a finite calling population with N jobs and N servers. What happens if $N \to \infty$?

5.2 APPLICATIONS OF LEVEL-CROSSING, LITTLE'S LAW AND PASTA

In this section we apply the tools developed above to a number of interesting queueing systems. Specifically, we discuss in detail how to plan the number of cashiers at a supermarket such that the average queue length remains small. Note that in many queueing systems, the manager should balance the cost of the waiting time of customers against the cost of idle servers; recall in queueing systems always somebody has to wait, either a customer in queue or a server being idle.

LET US NEXT discuss the example of cashier planning at a supermarket. Here we keep the models simple, but we include all necessary steps to solve the a realistic planning problem. Note that many service systems, e.g. hospitals, deal with similar personnel planning problems to cover stochastic demand.

We start with specifying the *arrival process*. It is reasonable to model it as a Poisson process, as there are many potential customers, each choosing with a small probability to go to the supermarket at a certain moment in time. Thus, we only have to characterize the arrival rate. Estimating this for a supermarket is easy because the cash registers track all customers payments. This provides us with the departure time of each customer, hence we can use this to estimate the average number of arrivals per hour.

It is common to use a *demand profile* which shows the average number of customers arriving per hour. Then we model the arrivals as a Poisson process with an arrival rate that is constant during a certain hour as specified by the demand profile.

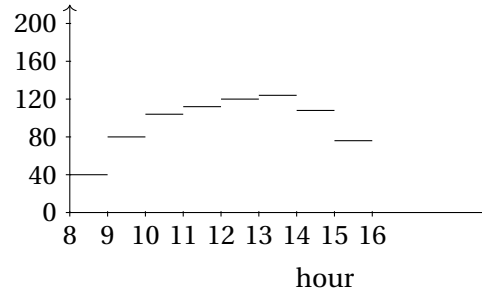


Figure 5.2.1: A demand profile.

It is also easy to find the *service distribution* from the cash registers. The first item scanned after a payment determines the start of a service, and the payment closes the service. For ease, we model the service time distribution as exponential with mean 1.5 minutes.

For the *service objective* we prefer to keep the average number of people in the system such that $\mathsf{E}[L] \le 5$. (Other KPIs are also possible to use.)

Let us model the situation as an $M/M/1$ queue with a fast server, so that it is easy to find the smallest number of servers necessary to prevent the queue from exploding. Since $\mathsf{E}[L] = \rho/(1-\rho)$, and $\rho = \lambda\,\mathsf{E}[S]/c$, solving for c in the inequality $\mathsf{E}[L] \le 5$ gives

$$c \ge \frac{6}{5}\lambda\,\mathsf{E}[S] \approx 0.03\lambda,$$

where, with the above estimate, $\mathsf{E}[S] = 1.5/60$ hour. It's easy to apply this formula. For instance, we see in the demand profile Fig. 5.2.1 that $\lambda = 120$ customers per hour between 12 and 13, hence $c \approx 3.8 = 4$.

With this we can find a formula to convert the demand profile into a *load profile* to specify the minimal number of servers per hour needed to meet the service objective.

THE LAST STEP in the planning process is to *cover the load profile with service shifts*. This is typically not easy since shifts have to satisfy all kinds of restrictions with respect to breaks and durations. Moreover, shifts can also have different costs: evening shifts are typically more expensive per hour.

The usual way to solve such covering problems is by means of an integer problem. For instance, suppose only 4 *shift plans* are available as shown at the right.[0]

0

1. ++−++
2. +++−+
3. ++−+++
4. +++−++,

Shift plans. A + indicates a working hour and − a break of an hour.

For the optimization, let x_i be the number of shifts of type i and c_i the cost of this type. Then the problem is to solve $\min \sum_i c_i x_i$, such that for all hours t the shifts cover the load, i.e., $\sum_i x_i \mathbb{1}_{t \in s_i} \ge 0.03\lambda_t$. (We write $t \in s_i$ if hour t is covered by shift type i.)

FINALLY, WE NEED to address the quality of our approximation: to simplify we used the $M/M/1$ queue with a fast server, while we know that in a

supermarket there are often multiple parallel cashiers. Specifically, we are interested in the ratio $\mathsf{E}[L(M/M/c]$ and $\mathsf{E}[L(M/M/1)]$ where in the $M/M/1$ queue the server works at rate c.

To understand this better, we consider a numerical example. Suppose that we have an $M/M/3$ queue with $\lambda = 5$ per day and $\mu = 2$ per day per server, and we compare $\mathsf{E}[L]$ of this queue to that of an $M/M/1$ queue with the same arrival rate but with a service rate of $\mu = 3 \cdot 2 = 6$.

Fig. 5.2.2 shows the ratio of $\mathsf{E}[L]$ for both queues as a function of ρ. Clearly, when ρ is small, this ratio is about 3. This is reasonable, because the service time in the fast $M/M/1$ is 3 times as small as the service time in the $M/M/3$ queue. Thus, when ρ is small, the time in queue is small, hence $\mathsf{E}[J] \approx \mathsf{E}[S]$. However, when ρ is large, $\mathsf{E}[J] \gg \mathsf{E}[S]$, so that in this case, the ratio between the queue lengths becomes approximately 1.

In our supermarket model, $\rho = 5/6$, hence the approximation is not too bad.

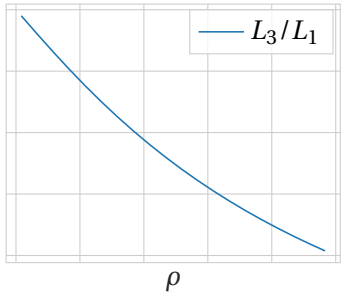


Figure 5.2.2: The ratio of $\mathsf{E}[L]$ for the $M/M/3$ and $M/M/1$ queue.

THE EXERCISES REQUIRE quite some modeling skills. Hence, they may be quite hard even thought the formulas are simple.

Ex 5.2.1. In the town hall of the municipality, people can, for example, renew their driver's license or passport. For ease, assume there is just one employee, and that the arrivals of customers are not planned. Why is it reasonable to model this queueing system as a $M/M/1$ queue?

Measurements show that $\mathsf{E}[W] = 6$ minute, and $\lambda = 5$ per minute. What are the service rate and utilization? Calculate $\mathsf{E}[Q]$, $\mathsf{E}[L]$.

There should be a sufficient number of seatings so that all waiting customers can sit down at least 90 percent of the time. How many seatings should there minimally be? What would you advice to the municipality?

Ex 5.2.2. The smallest barber shop in town works alone and has two chairs for waiting customers. Six jobs arrive per hour, and the barber needs 12 minutes for a hair cut. We model this, for ease, as an $M/M/1/3$ queue. Estimate $\mathsf{E}[Q]$ and the average number of customers served per hour. Then, estimate $\mathsf{E}[Q]$ when the barber hires another location that allows for 3 extra chairs. Quantity the effect on the number of customers served per hour.

Ex 5.2.3. Occasionally there is pizza stand at the corner of the Verlengde Visserstraat and the Westersingel in Groningen. As people (tend to) behave like sheep, the rate at which people join the queue depends on the actual queue length. When there is nobody waiting, people are somewhat hesitant to order: 'When there are no people waiting, most probably the pizzas are not good.'. However, when the queue is long, people prefer to go elsewhere. Measurements show that the number of arrivals per hour is well captured by the array `labda = np.array([10.0, 10.0, 20.0, 20.0, 5.0])`. The pizza oven can contain two pizzas, and it takes 2 minutes to bake a pizza. We model the system as an $M/M/2$ queue.

Calculate the steady state probabilities, and from this the average arrival rate. Then, find $\mathsf{E}[L]$, $\mathsf{E}[Q]$, $\mathsf{E}[J]$ and $\mathsf{E}[W]$.

Ex 5.2.4. Fast food restaurants tend to fry pattys before customers arrive. Like this they can serve customers faster, because when a customer arrives while there is still an inventory of prepared pattys, the server only has to flip a patty in a bun, while otherwise the customer has to wait for the additional frying time of the patty. Assume the restaurant keeps at most 3 pattys on stock. To keep the model simple, assume the restaurant has just one pan to fry a patty and the frying time $S \sim \text{Exp}(\mu)$ with $\mu = 15$ per hour. Customers arrive as a Poisson process with rate $\lambda = 12$ per hour. When there is prepared patty upon arrival, the customer is served immediately, otherwise the customer has to wait for a patty.

Explain how to model this system as an $M/M/1$ system. Then compute the average number of pattys on stock, the average number of people waiting, and the average waiting time of a patty before it ends up in a bun, and the average time a person has to wait for a hamburger.

You should remember this problem. It is of fundamental importance in inventory control and known as an *inventory system with backlogging*; it is (implicitly) used by nearly any company that keeps inventory.

Ex 5.2.5 (Continuation of [5.2.4])**.** What would be the impact of allowing four pattys on stock?

Ex 5.2.6 (Continuation of [5.2.4])**.** Suppose the restaurant does not keep pattys on stock. What is the expected waiting time for customer?

5.3 $M^B/M/1$ QUEUE: EXPECTED WAITING TIME

Sometimes jobs arrive in batches, rather than as single units. For instance, when a car or a bus arrives at a fast-food restaurant, a batch consists of the number of people in the vehicle. When the batches arrive as a Poisson process and the individual items within a batch have exponential service times we use the shorthand $M^X/M/1$ for this queueing model. Here we derive expressions for the load, the expected waiting time and number of jobs in the system.

ASSUME THAT JOBS arrive as a Poisson process with rate λ and each *job* contains multiple *items*. Denote by B_k the number of items, i.e., the batch size, of the kth job. We assume that $\{B_k\}$ is a sequence of iid discrete rvs with common rv B. We write $S_{k,i}$ for the service time of the ith item of batch k, and assume that $\{S_{k,i}\}$ is a sequence of iid rvs with common rv S, and that the service times of the items are independent of the sizes of the batches. The utilization is therefore $\rho = \lambda \mathsf{E}[B]\,\mathsf{E}[S]$; As always we require that $\rho < 1$.

An arriving job joins the end of the queue (if present), and once the queue in front of it has been cleared, it moves in its entirety to the server.[0] Thus, all items in one batch spend the same time in queue. Once the batch moves to the server, the server processes the items one after another until the batch is finished. Write Q^B for the number of batches in queue and L_s^B for the number of items (if any) at the server.

Observe that the average time $\mathsf{E}\left[W^B\right]$ a batch spends in queue is[1]

$$\mathsf{E}\left[W^B\right] = \mathsf{E}\left[L_s^B\right]\mathsf{E}[S] + \mathsf{E}\left[Q^B\right]\mathsf{E}[B]\,\mathsf{E}[S].$$

[0] *This is the same batching model as in Section 6.2.*

[1] *First the items at the server have to be served, then the ones in queue.*

But since $\mathsf{E}\left[L_s^B\right] + \mathsf{E}\left[Q^B\right]\mathsf{E}[B] = \mathsf{E}[L]$,

$$\mathsf{E}\left[W^B\right] = \mathsf{E}[L]\,\mathsf{E}[S] = \mathsf{E}[W],$$

where $\mathsf{E}[W]$ is the average time an *item* spends in queue. We conclude that the average time a job spends in queue is the same as the average time an item spends in queue.

Substituting the equality[2] $\mathsf{E}\left[Q^B\right] = \lambda\,\mathsf{E}\left[W^B\right]$ in the above yields

[2] *Little's law*

$$\mathsf{E}[W] = \mathsf{E}\left[W^B\right] = \frac{\mathsf{E}\left[L_s^B\right]}{1-\rho}\mathsf{E}[S]. \tag{5.3.1}$$

It remains to find an expression for $\mathsf{E}\left[L_s^B\right]$.

For this we can use the renewal reward theorem. With (4.5.1) as inspiration, define $Y(t) := \int_0^t L_s^B(s)\,\mathrm{d}s$, so that $Y(t)/t$ is the time-average number of items at the *server*. By taking D_k as the departure time of the kth batch, let $X_k = Y(D_k) - Y(D_{k-1})$. Suppose the kth job has batch size B_k, then

$$X_k|B_k = \int_{D_{k-1}}^{D_k} L_s^B(s)\,\mathrm{d}s = B_k S_{k,1} + (B_k - 1)S_{k,2} + \cdots + S_{B_k,B_k}.$$

Using Adam's law[3], and that the $S_{k,i}$ are iid,

[3] $\mathsf{E}[\mathsf{E}[Y|X]] = \mathsf{E}[Y]$.

$$\mathsf{E}[X_k|B_k] = B_k\,\mathsf{E}[S] + (B_k-1)\,\mathsf{E}[S] + \cdots \mathsf{E}[S] = B_k(B_k+1)/2\cdot\mathsf{E}[S],$$
$$\mathsf{E}[X_k] = \mathsf{E}[\mathsf{E}[X_k|B_k]] = \mathsf{E}\left[\frac{B(B+1)}{2}\right]\mathsf{E}[S] = \frac{\mathsf{E}\left[B^2\right] + \mathsf{E}[B]}{2}\mathsf{E}[S],$$

as $B_k \sim B$, i.e., distributed as the common rv B. Since $Y = \lim_{t\to\infty} Y(t)/t = \mathsf{E}\left[L_s^B\right]$, $Y = \lambda X$, $\lambda = \delta$, and $\rho = \lambda\,\mathsf{E}[B]\,\mathsf{E}[S]$, we obtain

$$\mathsf{E}\left[L_s^B\right] = \lambda\frac{\mathsf{E}\left[B^2\right]}{2}\mathsf{E}[S] + \frac{\rho}{2}. \tag{5.3.2}$$

We can brush this up by realizing that

$$\frac{\mathsf{E}\left[B^2\right]}{(\mathsf{E}[B])^2} = \frac{\mathsf{E}\left[B^2\right] - (\mathsf{E}[B])^2 + (\mathsf{E}[B])^2}{(\mathsf{E}[B])^2} = \frac{\mathsf{V}[B] + (\mathsf{E}[B])^2}{(\mathsf{E}[B])^2} = C_s^2 + 1.$$

where $C_s^2 = \mathsf{V}[B]/(\mathsf{E}[B])^2$ is the SCV of the batch size distribution. Substituting this into the above gives the final result

$$\mathsf{E}[W] = \frac{1+C_s^2}{2}\frac{\rho}{1-\rho}\mathsf{E}[B]\,\mathsf{E}[S] + \frac{1}{2}\frac{\rho}{1-\rho}\mathsf{E}[S]. \tag{5.3.3}$$

This formula is an important result. It tells us that the load increases when the load increases or the variability C_s^2 of the batch size increases, but also when $\mathsf{E}[B]\,\mathsf{E}[S]$ increases (even when the utilization ρ remains the same). As such, it gives us the most important clues about what we should try to change if we want to improve a queueing system.

RATHER THAN USING the time an entire batch spends at the server, as in the above use of the renewal-reward equation, we can concentrate on the expected time spent by single items at the server. This will provide us with a simple expression for $\mathsf{P}\{L_s = i\}$, from which we can also derive (5.3.2).

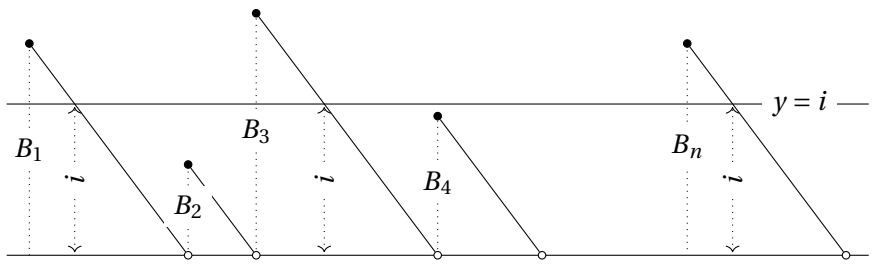


Figure 5.3.1: A batch crosses the line $y = i$ iff it contains at least i items. Thus, during the service of a batch with i or more items, there is precisely one service of an i-th item.

If $L_s(t)$ is the number of items (of the batch in service) at the server at time t, then $Y_i(t) = \int_0^t \mathbb{1}_{L_s(s)=i}\,\mathrm{d}s$ is the total time there are i items at the server. By sampling $Y(\cdot)$ at departure times $\{D_k\}$, we see that

$$X_k = Y_i(D_k) - Y_i(D_{k-1}) = S_{k,i}\,\mathbb{1}_{B_k \geq i},$$

where $S_{k,i}$ is the service time of the ith remaining item of the batch. Since the $\{S_{k,i}\}$ are iid with $\mathsf{E}\left[S_{k,i}\right] = \mathsf{E}[S]$, and the $\{B_k\}$ are iid and independent of the service times of the items, we obtain

$$X = \lim_{n\to\infty} \frac{1}{n} \sum_{k=1}^{n} S_{k,i}\,\mathbb{1}_{B_k \geq i} = \mathsf{E}[S\,\mathbb{1}_{B\geq i}] = \mathsf{E}[S]\,G(i-1),$$

since[4] $G(i-1) = \mathsf{E}[\mathbb{1}_{B\geq i}] = \mathsf{P}\{B > i-1\}$ is the survivor function of B. Since, by construction, $Y_i(t)/t \to \mathsf{P}\{L_s = i\}$, we use the renewal reward theorem and rate stability, $\delta = \lambda$, to conclude that

[4] *Recall:* $\mathsf{P}\{A\} := \mathsf{E}[\mathbb{1}_A]$.

$$\mathsf{P}\{L_s = i\} = \lambda\,\mathsf{E}[S]\,G(i-1) = \rho\frac{G(i-1)}{\mathsf{E}[B]},$$

$$\mathsf{E}[L_s] = \sum_{i=0}^{\infty} i\,\mathsf{P}\{L_s = i\} = \frac{\rho}{\mathsf{E}[B]}\sum_{i=1}^{\infty} iG(i-1).$$

Simplifying[5] the summation yields (5.3.3).

[5] *[5.3.5]*

Ex 5.3.1. A company operates a machine that receives batches of various sizes. Management likes to know how a reduction of the variability of the batch sizes would affect the average queueing time. Suppose, for the sake of an example, that the batch size $\mathsf{P}\{B=1\} = \mathsf{P}\{B=2\} = \mathsf{P}\{B=3\} = 1/3$. Batches arrive at rate $\lambda = 1/h$. The average processing time for an item is 25 minutes. Compute by how much $\mathsf{E}[L]$ would decrease if $B \equiv 2$.

Ex 5.3.2. Show that (5.3.3) reduces to the expression for $\mathsf{E}[W]$ for the $M/M/1$ queue.

Ex 5.3.3. Compute $\mathsf{E}[L]$ for the $M^B/M/1$ queue when B is geometrically distributed with $\mathsf{E}[B] = 1/p$. Check that if $\mathsf{E}[B] = 1$ the $M/M/1$ queue results.

Ex 5.3.4. For a nonnegative discrete random variable X, use indicator functions to prove that

1. $\mathsf{E}[X] = \sum_{k=0}^{\infty} G(k)$,
2. $\sum_{i=0}^{\infty} iG(i) = \mathsf{E}\left[X^2\right]/2 - \mathsf{E}[X]/2$.

Ex 5.3.5. Show that $\sum_{i=1}^{\infty} iG(i-1) = (\mathsf{E}\left[B^2\right] + \mathsf{E}[B])/2$.

5.4 $M/G/1$ QUEUE: EXPECTED WAITING TIME

When the service times are not really well approximated by the exponential distribution, the $M/G/1$ queue may be a more suitable model than the $M/M/1$ queue. In this section we derive with sample path arguments the fundamentally important Pollaczek-Khinchine (PK) formula for the average waiting time of an $M/G/1$ queue.

WE START WITH the simple observation that before an arriving job gets access to the server, it first has to wait until the job in service (if any) completes, and then it has wait for the queue to be cleared. From PASTA it follows that $\mathsf{E}[W] = \mathsf{E}[S_r] + \mathsf{E}[Q]\,\mathsf{E}[S]$, where $\mathsf{E}[S_r]$ is the (time-)average remaining service time of the job in service. With Little's law $\mathsf{E}[Q] = \lambda\,\mathsf{E}[W]$ and writing $\rho = \lambda\,\mathsf{E}[S]$, we find that[0]

$$\mathsf{E}[W] = \frac{\mathsf{E}[S_r]}{1-\rho}.$$

[0] *Compare the derivation for* $\mathsf{E}[W]$ *for the* $M^X/M/1$ *queue, it's similar.*

To make further progress we need to find an expression for $\mathsf{E}[S_r]$ when S is some generally distributed rv.[1] For this, we use the renewal reward theorem, just as in Section 5.3.

[1] *For the* $M/M/1$ *queue,* $\mathsf{E}[S_r] = \rho\,\mathsf{E}[S]$*, but not fora the* $M/G/1$ *queue.*

Consider the kth job of some sample path of the $M/G/1$ queueing process. Let the job's service start[2] at time $\tilde{A}_k$, so that it departs at time $D_k = \tilde{A}_k + S_k$, see Fig. 5.4.1. At time s, the remaining service time of job k is $(D_k - s)\,\mathbb{1}_{\tilde{A}_k \le s < D_k}$. By adding up all these times, we find that the total remaining service time as seen by the server up to time t is given by

[2] *Usually this is later than the arrival time* A_k*.*

$$Y(t) = \int_0^t (D_{D(s)+1} - s)\,\mathbb{1}_{A_{D(s)+1} \le s \le D_{D(s)+1}}\,\mathrm{d}s$$

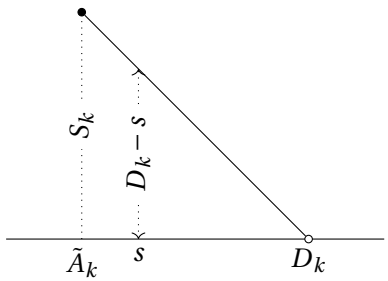


Figure 5.4.1: Remaining service time.

To see this, note that at time s, the number of departures is $D(s)$. Thus, $D(s)+1$ is the first job to depart after time s. The departure time of this job is $D_{D(s)+1}$, hence the remaining service time at time s is $D_{D(s)+1} - s$, provided this job is in service. Then, as $t \to \infty$, $Y(t)/t$ converges to the (time-average) remaining service time $\mathsf{E}[S_r]$.

We also see in Fig. 5.4.1 that $X_k = Y(D_k) - Y(D_{k-1})$ is the area under the triangle. By choosing $T_k = D_k$ as the epochs to inspect $Y(\cdot)$, $X = \lim_{n\to\infty} \frac{1}{n}\sum_{k=1}^n S_k^2/2 = \mathsf{E}\left[S^2\right]/2$. By the renewal-reward theorem, $Y = \delta X$, but $\delta = \lambda$,[3] and therefore

[3] *by rate-stability*

$$\mathsf{E}[S_r] = \frac{\lambda}{2}\mathsf{E}\left[S^2\right].$$

Replacing this expression in the above expression for $\mathsf{E}[W]$, we obtain the *Pollaczek-Khinchine formula*

$$\mathsf{E}[W] = \frac{1}{2}\frac{\lambda\,\mathsf{E}\left[S^2\right]}{1-\rho} = \frac{1+C_s^2}{2}\frac{\rho}{1-\rho}\mathsf{E}[S],$$

where the second equation follows from the observation that

$$\frac{\mathsf{E}\left[S^2\right]}{(\mathsf{E}[S])^2} = \frac{\mathsf{E}\left[S^2\right] - (\mathsf{E}[S])^2 + (\mathsf{E}[S])^2}{(\mathsf{E}[S])^2} = \frac{\mathsf{V}[S] + (\mathsf{E}[S])^2}{(\mathsf{E}[S])^2} = C_s^2 + 1.$$

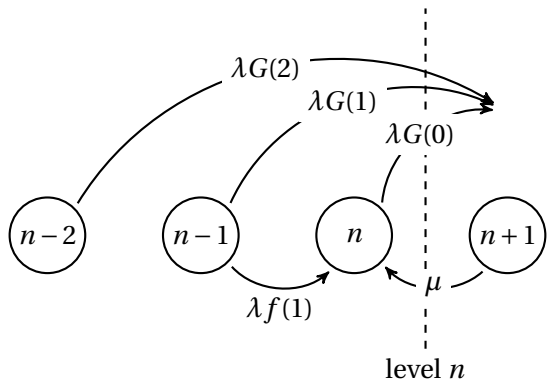


Figure 5.5.1: When $L(t) = n$, the arrival of any batch crosses level n from below, but when $L(t) = n-1$, only a batch larger than 1 ensures a crossing of level n, and so on.

Ex 5.4.1. Consider a workstation with just one machine. We model the job arrival process as a Poisson process with rate $\lambda = 3$ per day. The average service time $\mathsf{E}[S] = 2$ hours, $C_s^2 = 1/2$, and the shop is open for 8 hours per day. Show that $\mathsf{E}[W] = 4.5h$. What would you propose to reduce $\mathsf{E}[W]$ to 2h?

Ex 5.4.2. Show for the $M/G/1$ that $\mathsf{E}[S_r] = \rho\,\mathsf{E}[S_r \mid S_r > 0]$. As consequence, for the $M/M/1$ queue, $\mathsf{E}[S_r] \neq \mathsf{E}[S]$; why not?

Ex 5.4.3. Compare the expected waiting time of the $M/D/1$ queue to that of the $M/M/1$ queue.

Ex 5.4.4. Compute $\mathsf{E}[J]$ for the $M/G/1$ queue with $S \sim U[0,\alpha]$.

Ex 5.4.5. Find an expression in terms of λ and $\mathsf{E}[S]$ for the acceptance and loss probabilities of the $M/G/1/1$ queue. Why is this expression not valid for the $G/G/1/1$ queue.

5.5 $M^B/M/1$ QUEUE LENGTH DISTRIBUTION

In Sections 5.3 and 5.4 we established the Pollaczek-Khinchine formula for the waiting times of the $M^B/M/1$ queue and the $M/G/1$ queue, respectively. To compute more difficult performance measures such as the loss probability $\mathsf{P}\{L > n\}$, we need expressions for the stationary distribution $\pi(n)$, which by PASTA is equal to $p(n)$. Here we present a numerical, recursive, scheme to compute these probabilities.

WE CAN AGAIN use level-crossing arguments to find $\{\pi(n)\}$. However, the reasoning by which we obtained the level-crossing equations $\lambda p(n) = \mu p(n-1)$ needs to be generalized because, at the arrival of a batch containing many items, multiple levels will be up-crossed, see Fig. 5.5.1.

Let us first guess what could be the appropriate generalization of $\lambda\pi(n) = \mu\pi(n+1)$. The down-crossings of level n are easy: since items are served one-by-one, we still must have $\mu\pi(n+1)$, where n refers to the number of items[0] in the system. For the upcrossings of level n we should consider batch arrivals that see m items in the system, with $m \leq n$. Since

[0] *Not the number of batches.*

jobs arrive at rate λ, it follows from PASTA that the arrival rate of jobs that see m upon arrival is $\lambda\pi(m)$, in other words, the process is thinned with a fraction $\pi(m)$. As a further consequence of PASTA, the arrival process $\{N(u), u \geq t\}$ is independent of $\{L(s), s < t\}$, hence this thinned process is also Poisson.

When an arriving job sees m in the system, and level n has to be upcrossed, the batch size of this job must contain more than $n-m$ items. The rate at which such jobs arrive must be $G(n-m)\pi(m)\lambda$ as $G(n-m)$ is the probability that the job size is larger than $n-m$. Noting that the batch sizes are independent of the arrival times, we obtain yet again a Poisson process, but with rate $\lambda\pi(m)G(n-m)$.

Finally, to the compute rate at which level n is upcrossed, we merge all these thinned Poisson processes to form a new Poisson process with rate $\lambda\sum_{m=0}^{n}\pi(m)G(n-m)$. As by level-crossing, the up-crossing and down-crossing rates must match,

$$\lambda \sum_{m=0}^{n} \pi(m)G(n-m) = \mu\pi(n+1). \tag{5.5.1}$$

At the end of this section, we provide some further details for the interested reader.

Moreover, we can use it to derive $\mathsf{E}[L]$.[1] [1] [5.5.5]

IT IS LEFT to find the normalization constant. As (5.5.1) does not lead to a closed form expression for $\pi(n)$, such as (5.1.1), we have to specify a criterion to stop this iterative procedure. We discuss three different strategies to cope with this problem. For actual computations we need to suppose that the batch size B is finite and bounded by some N.

The first method is to prevent the problem altogether by blocking demand above some level M. There are three common policies to decide which items in a batch to accept.

1. Complete acceptance: accept all batches that arrive when the system contains K or fewer items, and reject the entire batch otherwise.[2] [2] [5.5.2]

2. Partial acceptance: accept whatever fits of a batch, and reject the rest.[3] [3] [5.5.3]

3. Complete rejection: if a batch does not fit entirely into the system, it will be rejected completely.[4] [4] [5.5.4]

In the exercises we will derive recursions similar to (5.5.1) by which we can obtain $\{\pi(n)\}$. Since the recursions stop after some level M, there is a finite number of recusions, hence we can solve them numerically without much problems.

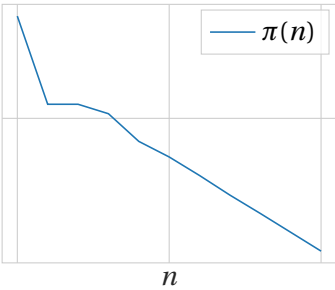
A second method is to set $\pi(0) = 1$ to start the iteration, then compute the first M iterates until $\pi(M) \ll 1$, and take $\sum_{i=0}^{M}\pi(i)$ as the normalization constant. Note that as $B \leq N$, it must be that $G(n) = 0$ for $n \geq N$. With this we can save some computer cycles by simplify (5.5.1) to

$$\mu\pi(n+1) = \lambda \sum_{m=0}^{\min\{n,N-1\}} \pi(n-m)G(m).$$

Below we show how to code this. We implement p as a dictionary, because we do not yet know how many terms we need before satisfying the stopping criterion.

```
import numpy as np

labda, mu = 1, 3
f = np.array([0, 1, 1, 1])
f = f / f.sum()


def compute_pi_2(f, eps):
    F = np.cumsum(f)
    G = np.ones_like(F) - F

    pi, n = {}, 0
    pi[n] = 1

    while pi[n] > eps:
        pi[n + 1] = sum(pi[n - m] * G[m] for m in range(min(n + 1, len((G)))))
        pi[n + 1] *= labda / mu
        n += 1

    norm = sum(pi[n] for n in pi.keys())
    return {n: pi[n] / norm for n in pi.keys()}


pi = compute_pi_2(f, eps=0.001)
EL = sum(n * pi[n] for n in pi.keys())
```

When running this code, we get $\mathsf{E}[L] = 3.304$ and we stopped at $M = 29$. Whether $\epsilon = 0.001$ is small enough is moot: the second digit is wrong when comparing it to the exact formula (5.3.3).



THE THIRD METHOD relies on the fact that the numbers $\pi(n)$ decrease geometrically fast for n sufficiently large. Let us check this for the numerical example we studied above. In the figure at the right we plot $\log(\pi(n))$ as a function of n. Clearly, $\pi(n), n \geq 5$ seems to decrease as a straight line.

To provide some intuition for this fact, observe that for $N < n \leq 2N-1$,

$$\pi(n+1) = \frac{\lambda}{\mu}\sum_{m=0}^{N-1}\pi(n-m)G(m) < \frac{\lambda}{\mu}\sum_{m=0}^{N-1}G(m) = \rho < 1.$$

as $\pi(n) < 1$ for $n \leq N$. But then, for $2N < n \leq 3N-1$,

$$\pi(n+1) = \frac{\lambda}{\mu}\sum_{m=0}^{N-1}\pi(n-m)G(m) < \frac{\lambda}{\mu}\sum_{m=0}^{N-1}G(m)\rho = \rho^2,$$

and so on.

For the interested reader, we note that we can do better by substituting the form $\pi(n) = C\alpha^n$ into (5.5.1) for $k > N$. Then α should be a root of the equation $\lambda\sum_{m=0}^{N-1}\alpha^m G(N-1-m) = \mu\alpha^N$. Now this is a polynomial (in α) of degree N, which has N roots in general. It is an interesting challenge to motivate why the α we need is the largest root.

Ex 5.5.1. Show that (5.5.1) reduces to $\lambda\pi(n) = \mu\pi(n+1)$ for the $M/M/1$ case.

Ex 5.5.2. Derive a set of recursions analogous to (5.5.1) to compute $\pi(n)$ for the $M^B/M/1/K$ queue with complete acceptance.

Ex 5.5.3. Derive a set of recursions for the $M^B/M/1/K$ queue with partial acceptance.

Ex 5.5.4. Derive a set of recursions for the $M^B/M/1/K$ queue with complete rejection.

Ex 5.5.5. Use (5.5.1) in $\mathsf{E}[L] = \sum_{n=0}^{\infty} n\pi(n)$ to derive (5.3.3).

This is an important check both on (5.3.3) and (5.5.1).

HERE WE FORMALIZE the above reasoning; you can skip it if you understand the details of level crossing. Define

$$A(m,n,t) := \sum_{k=1}^{A(t)} \mathbb{1}_{L(A_k-)=m}\,\mathbb{1}_{B_k>n-m}$$

as the number of jobs up to time t that see m in the system upon arrival and have batch size larger than $n-m$. From Fig. 5.5.1 we see that $A(n,m,t)$ counts the number of up-crossings of level n up to time t.

As earlier, we are interested in the limit $A(n,m,t)/t$ as $t \to \infty$. If we follow the reasoning of Section 4.6, we obtain by multiplying and dividing,

$$\frac{A(m,n,t)}{t} = \frac{A(t)}{t}\,\frac{A(m,t)}{A(t)}\,\frac{A(m,n,t)}{A(m,t)}. \tag{5.5.2}$$

We already know that $A(t)/t \to \lambda$ and $A(m,t)/A(t) \to \pi(m)$; so, let us provide an interpretation for the third term.

If we fill in the definitions, we get

$$\frac{A(m,n,t)}{A(m,t)} = \frac{\sum_{k=1}^{A(t)} \mathbb{1}_{L(A_k-)=m,B_k>n-m}}{\sum_{k=1}^{A(t)} \mathbb{1}_{L(A_k-)=m}}.$$

In words, this is the fraction of those jobs that see m in the system upon arrival and have size $> n-m$. But recall that we assumed that $\{B_k\}$ are independent of what jobs see upon arrival. Thus, we can just as well count all jobs of size $> n-m$ along a sample path instead of those that see m upon arrival; in the limit the statistics must be the same. Therefore

$$\lim_{t\to\infty} \frac{\sum_{k=1}^{A(t)} \mathbb{1}_{L(A_k-)=m,B_k>n-m}}{\sum_{k=1}^{A(t)} \mathbb{1}_{L(A_k-)=m}} = \lim_{t\to\infty} \frac{\sum_{k=1}^{A(t)} \mathbb{1}_{B_k>n-m}}{A(t)} = G(n-m).$$

By level-crossing $\left|\sum_{m=0}^{n} A(m,n,t) - D(n,t)\right| \le 1$, from which we obtain (5.5.2) by combining this with the above, dividing by t, and taking the limit $t \to \infty$.

5.6 N-POLICIES FOR THE $M/G/1$ QUEUE

In the queueing systems we analyzed up to now, the server is always present to start serving jobs at the moment they arrive. However, in cases in which there is a cost associated with changing the server from idle to busy, this condition is typically not satisfied. For instance, the cost to heat up an oven after being idle can be quite significant; in other cases, the operator of a machine has to move from one machine to another, which takes time.

TO REDUCE THE average cost, we can use an N-policy[0], which works as follows. As soon as the system becomes empty,[1] the server switches off. Then it waits until N jobs have arrived,[2] and then it switches on. The server processes jobs until the system is empty again, then switches off, and remains idle until N new jobs have arrived, and so on, cycle after cycle.

[0] *[2.2.2]*

[1] *and the server idle*

[2] *or N or more items in case of batch arrivals*

Note that under an N-policy, even though the load remains the same, the server has longer busy and idle times. In fact, some type of servers might use such policies to their advantage. At hospitals, for instance, doctors might prefer to let patients wait a bit until the waiting room is quite full. Like this, the doctor (server) does not have to wait for short times for late patients, but instead can collect idle times into one long stretch, and do something (they find more) useful instead.

Suppose it costs K Euro to set up the server, independent of the time it has been idle, and each job charges h Euros per unit time while in the system. Then it makes sense to first build up a queue of N jobs right after the server becomes idle,[3] and after some time switch on the server to process jobs until the system is empty again.[4] The problem is to find a switching threshold N^* that minimizes long-run average cost.

[3] *To reduce time-average setup cost.*

[4] *To reduce time-average queueing costs of jobs.*

In this section we solve this problem for the $M/G/1$ queue, and, in passing, we obtain yet another way to compute the time-average number $\mathsf{E}[L]$ of jobs in the system. Throughout we consider the queueing process at moments at which services start.

AS A FIRST STEP, we concentrate on the expected time $T(q)$ it takes to clear the system when the server *starts working* on a job and there are q jobs in the system. We present three different ideas to obtain $T(q)$.

The first heuristic idea is this. We know that jobs arrive at rate λ and they are served at rate $\mu > \lambda$, where $\mu = 1/\mathsf{E}[S]$. Clearly, the net 'drain rate' of the queue is $\mu - \lambda$; hence we guess that[5]

[5] *By analogy: when you have a 'queue' of q km to cycle, and your speed is v km/h, then it takes q/v h to complete the trip.*

$$T(q) = \frac{q}{\mu - \lambda} = \frac{\mathsf{E}[S]}{1 - \lambda\,\mathsf{E}[S]}\,q = \frac{\mathsf{E}[S]}{1-\rho}\,q. \tag{5.6.1}$$

Observe, however, that this reasoning completely ignores the stochasticity in arrival times and services, hence, we have no guarantee that the result is correct.

For the second idea, consider a regular[6] $M/G/1$ queue for the moment. When a job arrives to an empty system, it takes a busy time $\mathsf{E}[U]$ to get rid of this job and all jobs that arrive during the service of this first job. In other words, it takes $\mathsf{E}[U]$ time on average to move from 1 job in the system to 0, i.e., one less, jobs. But then, when there are q jobs in the system, it must take $T(q) = q\,\mathsf{E}[U]$ units of time to move from state q to state 0. By [4.4.5], $\mathsf{E}[U] = \mathsf{E}[S]/(1-\rho)$, so we again obtain (5.6.1), but now by proper reasoning.

[6] *i.e., not controlled by an N-policy.*

The third idea is the most powerful. Write Y for the number of jobs that arrive during a service time. Then, $T(q)$ satisfies the relation

$$T(q) = \mathsf{E}[S] + \mathsf{E}\big[T(q+Y-1)\big], \quad q \geq 1, \tag{5.6.2}$$

because first the job in service must leave, and then, when $Y = k$, it takes $T(q+k-1)$ to clear the system. To solve this equation, we substitute the

guess $T(q) = aq + b$ and solve for a and b. It is clear that $T(0) = 0$, hence $b = 0$.[7] Substituting aq into (5.6.2) gives $qa = \mathsf{E}[S] + aq + a\mathsf{E}[Y] - a$. Noting that $\mathsf{E}[Y] = \lambda\mathsf{E}[S]$,[8] and solving for a we find ((5.6.1)). We note that the solution of (5.6.2) is unique once $T(0)$ is fixed. To see this, observe that in (5.6.2), $T(q+1)$ is a function of $T(q)$ and $T(q-1)$. Thus, $T(1)$ follows from $T(0)$, $T(2)$ from $T(1)$ and $T(0)$, and so on.

[7] *If the system is empty, it takes no time to clear it.*

[8] (6.4.3)

TO FIND THE cost to clear the queue, we first concentrate on the expected queueing cost $U(q)$ that accrue during a service that starts with q jobs in the system. The expected cost for the q jobs already present is $hq\mathsf{E}[S]$, i.e., h times the area of the rectangle with base $\mathsf{E}[S]$ and height q. The expected cost for new arrivals during the service is $h\mathsf{E}\left[\int_0^S N(s)\,\mathrm{d}s\right]$, where $N(s) \sim \mathrm{Pois}(\lambda s)$. Conditional on $S = t$, $\mathsf{E}\left[\int_0^t N(s)\,\mathrm{d}s\right] = \int_0^t \mathsf{E}[N(s)]\,\mathrm{d}s = \lambda t^2/2$, hence, $\mathsf{E}\left[\int_0^S N(s)\,\mathrm{d}s\right] = \mathsf{E}\left[\lambda S^2/2\right]$. The total cost is therefore

$$U(q) = hq\mathsf{E}[S] + \frac{1}{2}\lambda h\mathsf{E}\left[S^2\right].$$

Let $V(q)$ be the expected queueing costs to clear the system just after a service starts and q jobs are in the system. By analogy with (5.6.2), $V(q)$ must be the solution of

$$V(q) = U(q) + \mathsf{E}\left[V(q+Y-1)\right], \quad q \geq 1. \tag{5.6.3}$$

Now note that as in (5.6.6), $U(q)$ has a term linear in q and a constant term. We substitute the form $V(q) = aq^2 + bq + c$, assemble terms with the same power in q, and solve for a, b and c. Of course $c = 0$ since $V(0) = 0$. For a and b we need to do some work to arrive at[9]

[9] *[5.6.8]–[5.6.9]*

$$V(q) = \frac{h}{2}\frac{\mathsf{E}[S]}{1-\rho}q^2 + h\frac{1+\rho C_s^2}{2}\frac{\mathsf{E}[S]}{(1-\rho)^2}q.$$

IT REMAINS TO analyze the long-run average cost under a general N-policy. As we already have expressions for the time and cost while the server is on, we only have to consider the time and cost while the server is off. Clearly, right after the server switches off, we need N independent inter-arrival times to reach level N, which takes N/λ units of time in expectation. As a result, the expected cycle duration is

$$C(N) = N/\lambda + T(N).$$

For the cost during the build up the queue, we use again a recursive procedure. Write $W(q)$ for the accumulated queueing cost[10] from the moment the server becomes idle up to the arrival time of the qth job (the job that sees $q-1$ jobs in the system upon arrival). Then,[11]

[10] *Here W is not the waiting time in queue.*

[11] *[5.6.3]*

$$W(q) = W(q-1) + h\frac{q-1}{\lambda} = h\frac{q(q-1)}{2\lambda}.$$

By combining all the above cost factors and using the renewal reward theorem, we find for the long-run average cost[12]

[12] *[5.6.12]*

$$\frac{W(N)+K+V(N)}{C(N)} = h\frac{1+C_s^2}{2}\frac{\rho^2}{1-\rho} + h\rho + h\frac{N-1}{2} + K\frac{\lambda(1-\rho)}{N}. \tag{5.6.4}$$

From this, the PK-formula follows directly.[13]

[13] [5.6.11]

Finding the optimal N is easy. Taking the derivative with respect to N, just as if it is a continuous variable, setting the derivative to zero and solving for N gives[14]

[14] *The optimal value is* ± 1 *this estimate.*

$$N^* \approx \sqrt{\frac{2\lambda(1-\rho)K}{h}}.$$

The expression for N^*, called the *Economic Production Quantity (EPQ)*, is a famous result in inventory theory. Specifically, it is the optimal production quantity for a production-inventory system in which a machine switches on at cost K and items pay holding cost h per unit time. By taking $\mathsf{E}[S] \to 0$, hence $\rho \to 0$, we see that N^* reduces to the *Economic Order Quantity (EPQ)* $\sqrt{2\lambda K/h}$.

Ex 5.6.1. Explain intuitively that the system is rate-stable for any N.

Ex 5.6.2. Why doesn't the utilization ρ depend on N?

Ex 5.6.3. Explain the recursion for $W(q)$ and solve it.

Ex 5.6.4. Use the memoryless property for the $M/M/1$ queue to show that

$$T(q) = \frac{1}{\lambda+\mu} + \frac{\lambda}{\lambda+\mu}T(q+1) + \frac{\mu}{\lambda+\mu}T(q-1). \tag{5.6.5}$$

Ex 5.6.5. Show that for the $M/M/1$ queue, $V(q)$ satisfies a relation like this:

$$V(q) = h\frac{q}{\lambda+\mu} + \frac{\lambda}{\lambda+\mu}V(q+1) + \frac{\mu}{\lambda+\mu}V(q-1). \tag{5.6.6}$$

Ex 5.6.6. Explain for the $M/M/1$ that $V(1)/C(1) = h\mathsf{E}[L]$, where

$$C(1) = \mathsf{E}[I] + \mathsf{E}[U] = 1/\lambda + T(1).$$

Ex 5.6.7. Show for the $M/M/1$ queue that $V(1)/C(1) = h\rho/(1-\rho)$.

Ex 5.6.8. Simplify $aq^2 + bq = a\mathsf{E}\left[(q+Y-1)^2\right] + b\mathsf{E}\left[q+Y-1\right] + U(q)$, and assemble powers in q to obtain:

$$a = \frac{h}{2}\frac{\mathsf{E}[S]}{1-\mathsf{E}[Y]} = \frac{h}{2}\frac{\mathsf{E}[S]}{1-\rho},$$
$$b(1-\mathsf{E}[Y]) = a(\mathsf{E}\left[Y^2\right] - 2\mathsf{E}[Y] + 1) + \frac{1}{2}h\lambda\mathsf{E}\left[S^2\right].$$

Ex 5.6.9. Derive the expression for $V(q)$ with the previous exercise.

In a sense, this is trivial, as it is just algebra, but it is hard to get the details right.

Ex 5.6.10. Check that $V(q)$ reduces to that of the $M/M/1$ queue.

Ex 5.6.11. Derive the PK formula from (5.6.4).

Ex 5.6.12. Derive (5.6.4).

5.7 INVENTORY CONTROL, ANALYTIC RESULTS

In this section we establish a set of formulas by which we can compute several performance measures for the basestock, the (Q, r) and the (T, S) inventory models as functions of the policy parameter(S). Thus, with these formulas we can avoid simulation to obtain insight into the policy performance. We will not deal with the (s, S) policy because that is a bit too difficult for these notes.[0]

[0] *For those interested in a proof of the optimality of (s, S) poliicies and an algorithm to compute the optimal policy parameters see Van Foreest and Kilic [2023].*

The period demands $\{D_i\}$ are assumed to be iid rvs. Write

$$D(i, j) = \sum_{k=i+1}^{j-1} D_k, \qquad D[i, j] = D_i + D(i, j, \qquad D[i, j] = D[i, j) + D_j,$$

and likewise notation for $Q(i, j]$.

FOR THE BASETOCK model in discrete time, recall from (2.3.2) that the start-of-period inventory position is always equal to the order-up-to level $S = s + 1$, i.e., $P_t = S$, and that the end-of-period $P'_t = S - D_t$.

Lemma 5.7.1. Under a basestock policy with reorder level s, if $P_1 = S$, then for all periods t,

$$D[1, t) = Q(1, t],$$

that is, all demand is either delivered or outstanding.

Proof. Using the recursion for the inventory position under the basestock policy repeatedly,

$$\begin{aligned} P_t &= P_{t-1} - D_{t-1} + Q_t \\ &= P_{t-2} - D_{t-2} + Q_{t-1} - D_{t-1} - Q_t \\ &= P_1 - D[1, t) + Q(1, t]. \end{aligned}$$

The proof is finished by realizing that $P_t = S$ for all t. □

We can use this lemma to obtain a useful result.

Lemma 5.7.2. Under a basestock policy with reorder level s, if $I_1 = P_1 = S$ and $Q_t = 0$ for $t \le 1$, then for all periods t,

$$I_t = S - D[t - L, t).$$

hence,

$$I'_t = I_t - D_t = S - D[t - L, t].$$

In words, items not on-hand must be on order.

Proof. Using the recursion for the inventory level and the assumptions,

$$\begin{aligned} I_t &= I_{t-1} - D_{t-1} + Q_{t-L} \\ &= I_{t-2} - D_{t-2} + Q_{t-L-1} - D_{t-1} + Q_{t-L} \\ &= I_1 - D[1, t) + Q(1, t - L] \\ &\overset{1}{=} S - D[1 - L, t), \end{aligned}$$

where we apply Lemma 5.7.1 to $Q(1, t - L]$ in step 1. □

Since the period demand are iid rvs, the rvs $\{D(t-L,t]\}$, while not independent, do have the same distribution. Necessarily the rvs $\{I_t\}$ have the same distribution. We can associate a rv I to this distribution, and we call this rv the stochastic inventory level $I(S)$, and likewise for the end-of-period inventory level $I'(S)$; we include the reference to the basestock level S for ease. Let us write F for the cdf common to the rvs $\{D[t-L,t]]$ and let $X \sim F$ be the associated rv. The cdf F of X follows from straightforward convolution[1], and from F we obtain easiy the pmf f and the survivor function $G = 1 - F$.

[1] *We show in code how to do this.*

In Section 2.3 we express the performance measures in terms of the of-period inventory level. The ready rate follows directly from the relation $I' = S - X$:

$$\alpha(S) = \mathsf{P}\left\{I' \geq 0\right\} = \mathsf{P}\{X \leq S\} = F(S). \tag{5.7.1}$$

The two other service levels can only be computed numerically.[2] For the mean inventory level,

[2] *To the extent, I know.*

$$\mathsf{E}\left[I'(S)\right] = s + 1 - \theta = s + 1 - (L+1)\,\mathsf{E}[D].$$

Noting that $I'^{+}(S) - I'^{-}(S) = I'(S)$,

$$\mathsf{E}\left[I'^{+}(S)\right] = \mathsf{E}\left[I'(S)\right] + \mathsf{E}\left[I'^{-}(S)\right] = S - \theta + \mathsf{E}\left[I'^{-}(S)\right].$$

Since $I'^{-}(S) = [-I'(S)]^{+} = [X - S]^{+}$, we obtain for the average backorder level

$$\mathsf{E}\left[I'^{-}(S)\right] = \sum_{k=0}^{\infty} (k-S)^{+} f(k). \tag{5.7.2}$$

Clearly, this summation runs to ∞, which is problematic for numerical purposes. It turns out that it can be rewritten to a simpler form.

Lemma 5.7.3. The average end-of-period backlog is equal to

$$\mathsf{E}\left[I'^{-}(S)\right] = \theta - \sum_{j=0}^{s} G(j).$$

Mind, the s in the RHS is the reorder level $s = S - 1$, not S.

Proof. Since $\sum_{j=0}^{\infty} \mathbb{1}_{j<k-r-1} = k - r - 1$, from (5.7.2)

$$\begin{aligned}\sum_{k=S}^{\infty} (k-S) f(k) &= \sum_{k=S}^{\infty}\sum_{j=0}^{\infty} \mathbb{1}_{j<k-S} f(k) = \sum_{j=0}^{\infty}\sum_{k=S}^{\infty} \mathbb{1}_{k>j+S} f(k) \\ &= \sum_{j=0}^{\infty} G(j+S) = \sum_{j=0}^{\infty} G(j) - \sum_{j=0}^{s} G(j).\end{aligned}$$

Because $\sum_{t=0}^{\infty} G(i) = \theta$ the claim follows. □

With these expressions the average inventory cost because

$$c(S) = h\,\mathsf{E}\left[I^{+}(S)\right] + b\,\mathsf{E}[I^{-}(S)] = h\,\mathsf{E}[I(S)] + (h+b)\,\mathsf{E}[I^{-}(S)].$$

Minimize the average cost is simple.

GIVES A THRESHOLD POLICY.

FOR THE (s, Q) MODEL we can also find some nice closed form expressions. There is a nice way to relate the (Q, r) system to a number of basestock systems. To see this, imagine two animals, a quoll that observes the (Q, r) system, and a bear that only wakes up when the inventory position right at the start of the interval equals i where $k \in \{r+1, \ldots r+Q\}$ and hibernates otherwise. Clearly, from the bear's point of view, the system behaves as a basestock system with order-up-to level k. More generally, we can associate a different bear to each different order-up-to level, so that always exactly one bear is awake, and the rest sleeps. To combine the statistics of the bears into the statistics as observed by the quoll, we need to know for each k the average fraction of periods the inventory position $P = k$. More specifically, the quoll needs to know[3]

$$\pi_k = \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} \mathbb{1}_{P_t = k},$$

because with this, the quoll can retrieve the ready rate $\sum_{k=r+1}^{Q+r} \pi_k \alpha(k)$ from the ready rates $\alpha(k)$ of the bears.

[3] *We assume that the limit over all sample paths $\{P_t\}$ exists almost surely, so that we can invoke the strong law of large numbers.*

Theorem 5.7.1. The probabilities $\pi_k = \mathsf{P}\{P = k\}$, $k = r+1, \ldots, r+Q$, under the (Q, r) are all equal, that is, $\pi_k = 1/Q$.

Proof. For ease take $Q = 3, r = 0$, so that the inventory position P cycles between the levels $1, 2$ and 3.[4] We can use level crossing arguments as follows. Write $p_i = \mathsf{P}\{D \in \{i, 3+i, 6+i, \ldots\}\}$ for the probability that the period demand is a multiple of 3 offset by i, $i = 0, 1, 2$. Suppose that, at the start of period t, the position $P_t = 3$. Since we order in multiples of $Q = 3$, the probability $\mathsf{P}\{P_{t+1} = 3 | P_t = 3\} = p_0$, i.e., equal to the probability that D_t is a multiple of 3. Likewise, $\mathsf{P}\{P_{t+1} = 2 | P_t = 3\} = p_1$, and $\mathsf{P}\{P_{t+1} = 1 | P_t = 3\} = p_2$. However, again because we order in multiples of Q, $\mathsf{P}\{P_{t+1} = 1 | P_t = 2\} = p_1$, and similarly for the other possibities. Therefore, we can use level-crossing to conclude that on the long run, $(p_1 + p_2)\pi_3 = p_2\pi_2 + p_1\pi_1$. But, because of the (Q, r) ordering rule, we can cyclically change π_1, π_2 and π_3 is this equality. The only solution that is compatible with this is that $\pi_i = 1/3$ for all i. □

[4] *The proof is the same for general Q and r, but needs a bit more notation.*

With the above theorem, the ready rate for the (Q, r) model can be computed with the formula

$$\alpha(Q, r) = \frac{1}{Q} \sum_{k=r}^{r+Q-1} \alpha(k), \tag{5.7.3}$$

where $\alpha(k)$ is the ready rate of the basestock model with reorder level k. Interestingly, the ready rate of the (Q, r) policy can be expressed in terms of the average backlog of the basestock model.

Lemma 5.7.4. The ready rate of the (Q, r) model is equal to

$$\alpha(Q, r) = 1 - \frac{1}{Q}(\mathsf{E}[I^-(r-1)] - \mathsf{E}[I^-(r+Q-1)]). \tag{5.7.4}$$

Proof. From (5.7.1), and the fact that $G(k) = 1 - F(k)$,

$$\begin{aligned}\sum_{k=r+1}^{r+Q} \alpha(k) &= \sum_{k=r+1}^{r+Q} F(k) = \sum_{k=0}^{r+Q} F(k) - \sum_{k=0}^{r} F(k) \\ &= Q - \sum_{k=0}^{r+Q} G(k) + \sum_{k=0}^{r} G(k).\end{aligned}$$

Dividing by Q and using Lemma 5.7.3 gives the result. □

For the average inventory level we find

$$\begin{aligned}\mathsf{E}\left[I'(Q,r)\right] &= \frac{1}{Q}\sum_{t=r+1}^{r+Q} \mathsf{E}\left[I'(i)\right] = \frac{1}{Q}\sum_{t=r+1}^{r+Q} (i-\theta) \\ &= \frac{Q+1}{2} + r - \theta,\end{aligned}$$

and for the expected number of back-orders

$$\mathsf{E}\left[I'^{-}(Q,r)\right] = \frac{1}{Q}\sum_{k=r+1}^{r+Q} \mathsf{E}\left[I'^{-}(k)\right],$$

From this, $\mathsf{E}\left[I'^{+}(Q,r)\right] = \mathsf{E}\left[I'(Q,r)\right] + \mathsf{E}\left[I'^{-}(Q,r)\right]$. Finally, with h and b the holding and backorder cost per item per period, the total average cost becomes

$$K\frac{\mathsf{E}[D]}{Q} + h\,\mathsf{E}\left[I'(Q,r)\right] + (b+h)\,\mathsf{E}\left[I'^{-}(Q,r)\right].$$

With regard to the first term, note that $\mathsf{E}[D]$ is average demand per period, so $\mathsf{E}[D]/Q$ is the order frequency per period.

It remains to find good values for r and Q. One way would be to carry out a full grid search over a (reasonable) set of pairs of r and Q. Another simple heuristic is to take $Q = \sqrt{2\,\mathsf{E}[D]\,K/h}$, i.e., the EOQ value, and then search for r such that the total average cost is minimal, or such that some service level criterion is met. and take for r the optimal basestock level, as if $Q = 1$.

For the (s,S) model we can take s to be the same as for the (s,Q) model, and then set $S = s + Q$.

HERE IS A WORKED example: we implement the above formulas for the Lighthouse Company that we simulated earlier in Section 2.3 under the (s,S) rule. Here we consider the (Q,r) rule with $Q = 3$ and $r = 10$.

Ex 5.7.1. Here is a subtle problem, that you should think about, and *memorize*. In these notes so far, we have been concerned with *periodic-review* systems: at the end of period i, we check the inventory I_t. Hence, if $I_t \geq 0$, all demand must have been satisfied.

In continuous-review systems, the notation I_t corresponds to the inventory level as perceived by the ith customer (observe that this is very different from the meaning of the I_t for period-review systems).

Why is the ready rate $F(S)$ (and not $F(S) = F(s+1)$ as in (4.6.5)) for continuous-review systems?

6

APPROXIMATE MODELS

In the previous two chapters we learned how to construct and simulate queueing processes. Simulation is a powerful tool but one of its limitations is that it does not easily provide insight into structural behavior of systems. For this we need theoretical models, and the derivation of such models form the contents of the remainder of the book.

IN THIS CHAPTER we discuss two formulas that might be considered as the most important formulas to understand the behavior of queueing systems. The first is Sakasegawa's formula that approximates the expected queueing time in a $G/G/c$ queue; the second characterizes the propagation of variability through a tandem network of $G/G/c$ queues. With a bit of exaggeration, we can say that the entire philosophy behind lean manufacturing and the world-famous Toyota production system are based on the principles that can be derived from these two formulas.

Here we take these formulas for granted, but focus on the insights they provide into the performance of queueing systems and how to use them to guide improvement procedures for production and service systems.

In Section 6.1 we introduce Sakasegawa's formula and discuss the main insights it offers. Then we illustrate how to use this formula to estimate waiting times in three queueing settings in which the service process is interrupted. In the first case, Section 6.2, the server has to produce jobs from different families, and there is a change-over time required to switch from one production family to another. As such setups reduce the time the server is available, the load must increase. In fact, to reduce the load, the server produces in batches of fixed sizes. In the second case, in Section 6.3, the server sometimes requires small adjustments, for instance, to prevent the production quality to degrade below a certain level. Clearly, such adjustments are typically not required during a job's service; however, they can occur *between* any two jobs. As a consequence, the number of jobs served between two such adjustments (or setups) is not constant, hence different from batch production where batch sizes are constant. In the third example, in Section 6.4, quality problems or break downs can occur *during* a job's service. These make job service times more variable, which leads to longer expected queueing times. In the final Section 6.5, we concentrate on tandem queues.

In passing, we use some interesting results of probability theory and the Poisson process.

6.1 $G/G/c$ QUEUE: SAKASEGAWA'S FORMULA

While there is no expression available to compute the exact expected waiting time for the $G/G/c$ queue, Sakasegawa's formula provides a reasonable approximation. This takes the form

$$\mathsf{E}[W] = \frac{C_a^2 + C_s^2}{2} \frac{\rho^{\sqrt{2(c+1)}-1}}{1-\rho} \frac{\mathsf{E}[S]}{c}, \tag{6.1.1}$$

where $C_a^2 = \mathsf{V}[X]/(\mathsf{E}[X])^2$ and $C_s^2 = \mathsf{V}[S]/(\mathsf{E}[S])^2$ are the square coefficient of variation (SCV) of the inter-arrival time and service time, respectively, and the utilization of the station[0] is $\rho = \lambda \mathsf{E}[S]/c$, where λ is the rate at which jobs arrive at the system, $\mathsf{E}[S]$ is the expected service time, and c the number of servers. It is evident from this formula that $\rho \in [0,1)$, hence the load and utilization are the same.

[0] *Here is a subtle point. When there are multiple machines, the utilization of each machine need not be equal to ρ. For instance, if there is a preference to choose the 'left-most' machine whenever it is free, then the utilization of this machine is larger than ρ. Only when the machines have the same speed, and the* routing *is such that each machine receives a fraction λ/c of jobs, the machines have the same utilization.*

Note that Sakasegawa's formula for the $G/G/c$ queue reduces to the PK formula for $\mathsf{E}[W]$ for the $M/G/1$ queue; just fill in $c = 1$ and set $C_a^2 = 1$. Part of the intuition behind Sakasegawa's formula is based on the following idea. To generalize the formula for $\mathsf{E}[W]$ from the $M/M/1$ queue to that of the $M/G/1$ queue, we replace $(1+1)/2$ by $(1+C_s^2)/2$; the result remains exact. Now, to go from the $M/G/1$ queue to the $G/G/1$ queue, replace $(1+C_s^2)/2$ by $(C_a^2+C_s^2)/2$; this is no longer exact, but the approximation remains good for most practical purposes. For the details behind the power with c, we refer to Sakasegawa's paper.

IT IS CRUCIAL to memorize the insights into the performance of queueing systems that this formula offers. Even though (6.1.1) is an approximation, it proves to be exceedingly useful when designing queueing systems and analyzing the effect of certain changes.

First, we see that $\mathsf{E}[W] \sim (1-\rho)^{-1}$. Consequently, when ρ is large, the waiting time is (very) large. And, not only is $\mathsf{E}[W]$ large, it is also extremely *sensitive* to the actual value of ρ. Clearly, such situations must be avoided, and therefore, when trying to improve a queueing system, the first focus should be on reducing ρ.

Second, $\rho = \lambda \mathsf{E}[S]/c$. Thus, when ρ must be made smaller, we have just three options.[1] We can reduce the arrival rate λ of jobs, for instance by blocking demand, or sending it elsewhere such as to another machine.[2] We can make $\mathsf{E}[S]$ smaller by replacing a server with a faster one or by shifting some of the processing steps of a job to other servers, thereby making job sizes smaller. Finally, we can add servers, which reduces ρ quite significantly when c is small.[3] If technically possible, adapting c is a very effective mechanism to control waiting times.

[1] *And not more!*

[2] *And if customers have a choice, they will take their own measures, simply by going elsewhere.*

[3] *This is precisely what you see in supermarkets.*

Third, $\mathsf{E}[W] \sim C_a^2$ and $\mathsf{E}[W] \sim C_s^2$, which implies that when job inter-arrival or service times are very variable, $\mathsf{E}[W]$ is large. Thus, after ensuring that ρ is sufficiently small, it becomes important to concentrate on reducing on C_a^2 and C_s^2.[4]

[4] *It works the other way too: systems with low variability can deal with higher load.*

Finally, $\mathsf{E}[W] \sim \mathsf{E}[S]/c$. This says that, from the perspective of a job in queue, average job service times are c times as short as 'its own service time'.

Ex 6.1.1. In a manufacturing setting, the Poisson process is not always a suitable model for the arrival process of jobs at a production station. Can you provide an example to see why this is the case?

Ex 6.1.2. Consider a single-server queue at which every minute a customer arrives, precisely at the first second and $S \equiv 50$ s. What are ρ, $\mathsf{E}[L]$, C_a^2, and C_s^2?

This is an important example to memorize.

Ex 6.1.3. Consider the same single-server system as in [6.1.2], but now the customer service time is stochastic: with probability 1/2 a customer requires 1 minute and 20 seconds of service, and with probability 1/2 the customer requires only 20 seconds of service. What are ρ, C_a^2, and C_s^2?

Ex 6.1.4. In a workplace, when a mechanic has used a specific high precision tool, the tool has to be inspected by a microscope for potential failures. The inter-arrival times of the tools at the microscope is $X \sim \text{Unif}(3,9)$], and the inspection time S is such that $\mathsf{E}[S] = 5$ minutes and $\mathsf{V}[S] = 4$ minutes squared. Use Sakasegawa's formula to estimate $\mathsf{E}[W]$. Next, when tools would arrive as a Poisson process, what would be $\mathsf{E}[W]$. Discuss your findings.

Ex 6.1.5. A machine serves two types of jobs. The processing time of jobs of type i, $i = 1,2$, is exponentially distributed with parameter μ_i. The type T of a job is random and independent of anything else, and such that $\mathsf{P}\{T = 1\} = p = 1 - q = 1 - \mathsf{P}\{T = 2\}$. (An example is a desk serving adults and children, both requiring different average service times, and p is the probability that the customer in service is a adult.) Show that the expected processing time and variance are given by

$$\begin{aligned}\mathsf{E}[S] &= p\,\mathsf{E}[S_1] + q\,\mathsf{E}[S_2]\\ \mathsf{V}[S] &= p\,\mathsf{V}[S_1] + q\,\mathsf{V}[S_2] + pq(\mathsf{E}[S_1] - \mathsf{E}[S_2])^2.\end{aligned}$$

Thus, even if $\mathsf{V}[S_1] = \mathsf{V}[S_2] = 0$, still $\mathsf{V}[S] > 0$ if $\mathsf{E}[S_1] \neq \mathsf{E}[S_2]$. Mixing different jobs types increases variability, hence queueing times.

6.2 SERVER SETUPS

In some cases, machines have to be setup before they can start producing items. Consider, for instance, a machine that paints red and blue items.[0] When the machine requires a color change, it may be necessary to clean up the machine, which takes time. Another example is an oven that needs a temperature change when different item types require different production temperatures. Service operations form another setting with setup times: when servers (personnel) have to move from one part of a building to another, the time spent moving cannot be spent on serving customers.[1]

[0] In realistic problems there can be tens of families.

In all such cases, the setups consume a significant amount of time; in fact, setup times of an hour or longer are not uncommon. To prevent overloading the server, it is necessary to produce in batches such that a server first processes a batch of jobs of one type[2], then performes a setup to serve a batch of another type, and so on.

[1] Often, the setup time depends on the sequence in which the families are produced, for example, a switch in color from white to black might take less cleaning time than from black to white. The problem then becomes to determine a production sequence that minimizes the sum of the setup times to produce the families, and then find suitable batch sizes to minimize the average waiting times.

[2] Producing one type of job is often called a 'run'.

Here we use Sakasegawa's formula to model the effect of change-over, or setup, times on the average sojourn time of jobs. In the main text we provide a list of elements required to compute $\mathsf{E}[J]$, in [6.2.1] we illustrate in detail how to carry out the computations.

ASSUME A SINGLE machine produces runs of red jobs and blue jobs, and needs a setup for each color change. Jobs arrive at rate λ_r and λ_b. The SCV of both job types inter-arrival times is given by C_a^2. Once the server is setup for the correct color, we assume for simplicity that jobs of both type have the same average *net processing time* $\mathsf{E}[S_0]$ and variance $\mathsf{V}[S_0]$. It is a simple exercise to generalize the notation to allow service times to depend on color. Finally, setup times are iid and have mean $\mathsf{E}[R]$ and variance $\mathsf{V}[R]$, and are assumed to be independent of job service times.

A job's sojourn time in the *entire* system is built up as follows. First, we assemble jobs into batches of the same color. For simplicity, we take B constant and the same for both colors.[3] Once B jobs of one color arrived, the batch is complete, and the *batch enters* a queue; thus we consider a queue with batches, not single jobs. After some time the batch reaches the head of the queue, and its service starts as soon as the machine becomes free. To serve a batch, the machine first performs a setup, and then processes each job individually until the batch is finished. Once a job's service is completed, it leaves the server.

[3] *In general, B should depend on the arrival rate of the job type when the arrival rates vary considerably between the famlilies.*

It remains to make a quantitative model of this queueing system.

WHEN A JOB of color i arrives, the expected time for its batch to be formed is given by[4]

$$\frac{B-1}{2\lambda_i}, \quad i \in \{r, b\}. \tag{6.2.1}$$

[4] *[6.2.2]*

WHEN THE BATCH is complete, the batch joins the queue, so we next compute the average time batches spend in queue.[5] For this we can use Sakasegawa's formula, and this formula makes us easy because we only have to find expressions for each of its components.

[5] *Note that we shift interpretation: batches (not individual items) form the queue.*

The total arrival rate of jobs is $\lambda = \lambda_b + \lambda_r$. Thus, $\lambda_B = \lambda / B$ is the arrival rate of *batches*.

The expected service time of a batch is

$$\mathsf{E}[S_B] = \mathsf{E}[R] + B\,\mathsf{E}[S_0]. \tag{6.2.2}$$

With the batch arrival rate and expected batch service time, the load becomes $\rho = \lambda_B \mathsf{E}[S_B]$

It is essential that B is sufficiently large to ensure that $\rho < 1$. With (6.2.2) this leads to the condition that B must be larger than some minimal batch size, i.e.,

$$B > B_m = \frac{\lambda \mathsf{E}[R]}{1 - \lambda \mathsf{E}[S_0]}.$$

Now that we have identified the arrival rate and service times of batches, it remains to find expressions for the SCV of the batch inter-arrival times $C_{a,B}^2$ and the batch service times $C_{s,B}^2$. It's easy to derive that[6]

[6] *[6.2.3] [6.2.4]*

$$C_{a,B}^2 = \frac{C_a^2}{B}, \qquad C_{s,B}^2 = \frac{\mathsf{V}[S_B]}{(\mathsf{E}[S_B])^2}, \tag{6.2.3}$$

where

$$\mathsf{V}[S_B] = \mathsf{V}[R] + B\,\mathsf{V}[S_0].$$

Observe that we now have all components for Sakasegawa's formula.

It can useful to convert the effects of the setup time into an *effective processing time* of individual jobs. By dividing by B we see

$$\mathsf{E}[S] = \frac{\mathsf{E}[S_B]}{B} = \mathsf{E}[S_0] + \frac{\mathsf{E}[R]}{B}, \qquad \mathsf{V}[S] = \mathsf{V}[S_0] + \frac{\mathsf{V}[R]}{B}. \tag{6.2.4}$$

Note in particular that the variance of the effective service times is larger than the variance of the net processing times.

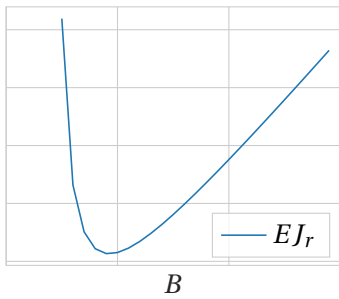
IT IS LEFT to find a rule to determine what happens to an item after it has been processed. If the job has to wait until all jobs in the batch are served, the expected time it spends at the server is $\mathsf{E}[R] + B\,\mathsf{E}[S_0]$. However, if the item can leave right after being served, the expected time at the server is[7]

[7] [6.2.5]

$$\mathsf{E}[R] + \frac{B+1}{2}\mathsf{E}[S_0]; \tag{6.2.5}$$

the first component is the average time a job has to wait before its service starts, the second is its service time. Our model is complete!

Sojourn time of red jobs



WE CAN OBTAIN A NUMBER of important insights from the above model. Using the code from [6.2.1] we plot the sojourn time $\mathsf{E}[J_r]$ of a red job for various values of B in the figure at the right.

First, as B increases, we see a sharp decline of $\mathsf{E}[J_r]$. The reason for this is that the load ρ decreases as a function of B. Since $\mathsf{E}[W] \sim (1-\rho)^{-1}$, it is essential to stay away from critically loading the server.

When B becomes quite large, we see that the sojourn time increases linearly. This follows right away from (6.2.1) and (6.2.5), because the time to (dis)assemble batches is linear in B.

Finally, observe that the graph is not symmetric around the minimum. It is much worse to take B too small than too large. More generally, when designing systems, we can use such graphs to understand how sensitive the system is to variation and measurement errors.

For the purpose of making this simple, but useful, figure, we took B and (the distribution of) S_0 the same for the red and blue jobs. Allowing for different batch sizes and net processing times per color is very simple indeed: just label the relevant symbols with and carry out the algebra.

Before dealing with technical derivations, let us first see how to apply the model.

Ex 6.2.1. Jobs arrive at $\lambda = 3$ per hour at a machine with $C_a^2 = 1$; service times are exponential with an average of 15 minutes. Assume $\lambda_r = 0.5$ per hour, hence $\lambda_b = 2.5$ per hour. Between any two batches, the machine requires a cleanup of 2 hours, with a standard deviation of 1 hour. First find the smallest batch size that can be allowed, then compute the average time a red job spends in the system in case $B = 30$ jobs.

Ex 6.2.2. Show that the average time a job has to wait to fill the batch (to which this job belongs) is given by (6.2.1).

Ex 6.2.3. Explain that the SCV of the batch inter-arrival times is given by (6.2.3).

Ex 6.2.4. Show that $C^2_{s,B}$ takes the form as in (6.2.3).

Ex 6.2.5. Show that, when items can leave right after being served, the time at the server is given by (6.2.5)

6.3 SERVER ADJUSTMENTS

In Section 6.2 we study the effect of setup times between batches of jobs. There are, however, other reasons to interrupt the server from serving jobs that are in queue. For instance, when a knife in a cutting machine becomes blunt, an operator has to stop the machine to replace the knife. Another example is a medical doctor who has to make a number of unexpected phone calls between seeing two patients. All such small (maintenance) tasks can be carried out in between jobs, but it is often hard to predict when they are required. Therefore, the number of jobs[0] served between any two tasks[1] is no longer constant. But we know from Sakasegawa's formula that randomness in service times affects queueing times, and we also know from (6.2.4) that the batch size affects the service time. Hence, such unplanned tasks must have a negative effect on sojourn times.

[0] *Compare this to a batch of jobs.*

[1] *And compare a task to a setup.*

We refer to this type of interruption as a server *adjustment*. It is important to realize that setups and adjustments are *non-preemptive outages*, i.e., they occur *between* jobs, not *during* job service times. Section 6.4 deals with the latter set of outages.

In this section we develop a simple model to understand the impact of adjustments on job sojourn times; we use the same notation as in Section 6.2 and follow the same line of reasoning. With this model, we can analyze a number of trade-offs, such doing fewer, but longer adjustments, or planning adjustments instead just waiting until it becomes necessary at an unexpected moment.[2] With the model we can make graphs of the sojourn time as a function of adjustment rate, so that we can optimize for the adjustment rate.

[2] *[6.3.2]*

LET US WRITE F_i for the Bernouilli rv that, when equal to 1, indicates that job i requires an adjustment, and, when 0, the adjustment is not necessary. We assume that $\{F_i\}$ is a set of iid rvs, distributed as the common rv F such that $\mathsf{P}\{F = 1\} = p = 1 - \mathsf{P}\{F = 0\}$. Thus, with constant probability p an adjustment occurs between any two jobs. As a consequence, the number of jobs served between two consequtive adjustments is a geometric rv B such that $\mathsf{E}[B] = 1/p$, and B is memoryless.[3] We further assume that the adjustment times $\{R_i\}$ are iid rvs, distributed as the common rv R with mean $\mathsf{E}[R]$ and variance $\mathsf{V}[R]$. Finally, we assume that the net processing times $\{S_{0,i}\}$, $\{R_i\}$ and $\{F_i\}$ are independent.

[3] *In practice, we measure the average number of jobs $\mathsf{E}[B]$ served between a number of adjustments, and take $p = 1/\mathsf{E}[B]$.*

TO COMPUTE $\mathsf{E}[W]$ with Sakasegawa's formula, we only have to find out how the adjustments affect the mean $\mathsf{E}[S]$ and SCV C^2_s of the effective processing time. As the adjustments have no influence on the job arrival process, λ and C^2_a remain the same.

In the exercises we show that the average effective processing time is

$$\mathsf{E}[S] = \mathsf{E}[S_0] + p\,\mathsf{E}[R] = \mathsf{E}[S_0] + \frac{\mathsf{E}[R]}{\mathsf{E}[B]}, \tag{6.3.1}$$

and its variance is

$$\begin{aligned} \mathsf{V}[S] &= \mathsf{V}[S_0] + p\mathsf{V}[R] + p(1-p)(\mathsf{E}[R])^2 \\ &= \mathsf{V}[S_0] + \frac{\mathsf{V}[R]}{\mathsf{E}[B]} + (\mathsf{E}[R])^2 \frac{C_B^2}{\mathsf{E}[B]}, \end{aligned} \tag{6.3.2}$$

where C_B^2 is the SCV of the runlength B. By dividing $\mathsf{V}[S]$ by $(\mathsf{E}[S])^2$ we obtainx C_S^2.

Assuming there is one server, we can now fill in Sakasegewa's formula!

BEFORE CONSIDERING SOME examples, let us compare the results of this model to those of Section 6.3. The expected effective service time is the same: an amount $\mathsf{E}[R]/\mathsf{E}[B]$ gets added to the net processing time $\mathsf{E}[S_0]$. However, the impact on the variance is different. By comparing (6.2.4) to (6.3.2) we see that the latter has an extra (positive) term. This supports our intuition: Unexpected interruptions have a larger effect on variability than expected (planned) interruptions.

The first few exercises demonstrate how to do apply the above, the rest are concerned with deriving (6.3.1) and (6.3.2).

Ex 6.3.1. Jobs arrive as a Poisson process with rate $\lambda = 9$ per working day. The machine works two 8 hour shifts a day. Work not processed on a day is carried over to the next day. Job service times are 1.5 hours, on average, with standard deviation of 0.5 hours. Outages occur on average between 30 jobs. The average duration of an outage is 5 hours and has a standard deviation of 2 hours. Compute $\mathsf{E}[J]$.

Ex 6.3.2. In the setting of [6.3.1] we can perhaps choose to do an adjustment after every 20 jobs. For simplicity, assume that then adjustments never occur at random. Also assume that an adjustment takes less time, for instance 4.5 hour and are constant. What is $\mathsf{E}[J]$ now?

Ex 6.3.3. The easiest way to derive the expressions for $\mathsf{E}[S]$ and $\mathsf{V}[S]$ is to use Adam's and Eve's rule[4]. Try it.

[4] $\mathsf{V}[Y] = \mathsf{E}[\mathsf{V}[Y|X]] + \mathsf{V}[\mathsf{E}[Y|X]]$.

6.4 SERVER FAILURES

In Sections 6.2 and 6.3 we assumed that servers are never interrupted while serving a job. However, this assumption is not always satisfied, for instance, a machine may fail in the midst of processing of a job. In this section, we develop a model[0] to compute the influence on the mean waiting time of such *preemptive outages*, again based on Sakasegawa's formula for the $G/G/1$ queue.

[0] *There is not much point in combining the models of Section 6.2–Section 6.4 into one large model. For instance, when a machine can be setup, while a part is being repaired, there may not be time lost on the setup. All depends on the specific strategies to combine outages.*

Just as in Section 6.3, we only have to derive expressions for the expectation and variance of the effective processing time S. The other components of Sakasegawa's formula are not affected.

We assume that a break down can occur at any moment during a job's service time. The repair times $\{R_i\}$ are a set of iid rvs, distributed as the common rv E and have mean $\mathsf{E}[R]$ and finite second moment $\mathsf{E}[R^2]$. Supposing that N interruptions occur during the net job service time S_0, the effective service time becomes[1]

[1] *In an insurance context, if we interpret $\{R_i\}$ as a set of claims, then $\sum_{i=1}^{N} R_i$ is the total claim size of N claims. Likewise, in an inventory system, this sum is the total demand of N customers.*

$$S = S_0 + S_N = S_0 + \sum_{i=1}^{N} R_i. \tag{6.4.1}$$

In general, N is a random number, so we need to adjust for this in the computation of the mean and variance of S.

IT IS COMMON to assume that the time between two interruptions is memoryless with mean $1/\lambda_f$, where λ_f is the *failure rate*.

Defining the mean time to fail as $m_f = 1/\lambda_f$, then the *availability* is given by

$$A := \frac{m_f}{m_f + \mathsf{E}[R]},$$

from which follows easily that[2]

[2] [6.4.2]

$$A = (1 + \lambda_f \mathsf{E}[R])^{-1}. \tag{6.4.2}$$

Next, because by assumption the time between two failures is $\sim \text{Exp}(\lambda_f)$ so that $N \sim \text{Pois}(\lambda S_0)$ if S_0 is given,

$$\mathsf{E}[N] = \mathsf{E}[\mathsf{E}[N|S_0]] = \mathsf{E}\left[\lambda_f S_0\right] = \lambda_f \mathsf{E}[S_0]. \tag{6.4.3}$$

Therefore,

$$\begin{aligned}\mathsf{E}[S] &= \mathsf{E}[S_0] + \mathsf{E}[N]\,\mathsf{E}[R] = \mathsf{E}[S_0] + \lambda_f \mathsf{E}[S_0]\,\mathsf{E}[R] \\ &= \mathsf{E}[S_0]\,(1 + \lambda_f \mathsf{E}[R]) = \mathsf{E}[S_0]\,/\,A.\end{aligned}$$

With this expression for the expectted effective servie, the load becomes

$$\rho = \lambda \mathsf{E}[S] = \lambda \frac{\mathsf{E}[S_0]}{A}.$$

Clearly, $A \in (0,1)$, hence the load increases due to failures.

WITH SOME WORK, we obtain[3] from (6.4.1),

[3] [6.4.3]

$$\mathsf{E}[S] = (1 + \lambda_f \mathsf{E}[R])\,\mathsf{E}[S_0], \qquad \mathsf{V}[S] = \frac{\mathsf{V}[S_0]}{A^2} + \lambda_f \mathsf{E}\left[R^2\right]\mathsf{E}[S_0]. \tag{6.4.4}$$

By assuming the repair times are exponentially distributed, it is possible to find a neat expression for the SCV of the service times[4]

[4] [6.4.5]

$$C_s^2 = C_0^2 + 2A(1-A)\frac{\mathsf{E}[R]}{\mathsf{E}[S_0]}, \tag{6.4.5}$$

where C_0^2 is the SCV of S_0.

Ex 6.4.1. Suppose we have a machine with memoryless failure behavior, with a mean-time-to-fail of $m_f = 3$ hours. Regular service times are deterministic with an average of $S_0 = 10$ minutes, jobs arrive as a Poisson process with rate of $\lambda = 4$ per hour. Repair times are exponential with a mean duration of $\mathsf{E}[R] = 30$ minutes. What is the average sojourn time?

Ex 6.4.2. Derive (6.4.2).

Ex 6.4.3. Derive (6.4.4) using Adam's and Eve's law.

Ex 6.4.4. Show that

$$C_s^2 = \frac{\mathsf{V}[S]}{(\mathsf{E}[S])^2} = C_0^2 + \frac{\lambda_f \mathsf{E}\left[R^2\right] A^2}{\mathsf{E}[S_0]},$$

Ex 6.4.5. With the above assumption that R is exponentially distributed, show (6.4.5).

6.5 $G/G/c$ QUEUES IN TANDEM

Consider two $G/G/1$ stations in tandem.[0] Suppose we have the financial means to reduce the variability of the processing times at one of the stations, but not at both. Then we like to improve the one that has the most impact on the total sojourn time in the line.

[0] *'In tandem' means 'in line', i.e., one station after the other.*

For the waiting time of the first machine, we can use Sakasegawa's formula, but to apply this to the second machine, we need $C^2_{a,2}$, i.e., the SCV of the inter-arrival times at the second station. Now, noting that the output of the first machine forms the input of the second machine, it is clear that $C^2_{a,2} = C^2_{d,1}$, where $C^2_{d,1}$ is the SCV of the *inter-departure* times of *first* station.

Let us consider the inter-departure times of a $G/G/1$ queue. Suppose that the utilization ρ is very high. Then the server will seldom be idle, so that most of the inter-departure times are equal to the service times. However, if the utilization is low, the server will be idle most of the time, and the inter-departure times must be approximately equal to the inter-arrival times.

We obtain an approximation for the SCV C^2_d of the inter-departure times by interpolating between these two extremes

$$C^2_d \approx (1-\rho^2)C^2_a + \rho^2 C^2_s. \tag{6.5.1}$$

For the $G/G/c$, i.e., a multiserver queue, there is the generalization

$$C^2_d \approx 1 + (1-\rho^2)(C^2_a - 1) + \frac{\rho^2}{\sqrt{c}}(C^2_s - 1).$$

It is simple to see that this reduces to (6.5.1) for the $G/G/1$ queue.

Combining Sakasegawa's formula with this expression provides us with a very useful insight for a line of queues. If we reduce $C^2_{s,1}$, i.e., the SCV of service times at the first station, $\mathsf{E}[W_1]$ and $C^2_{d,1}$ become smaller. Since $C^2_{a,2} = C^2_{d,1}$, $\mathsf{E}[W_2]$ becomes lower too, but also $C^2_{d,2}$, and so on. In other words, the entire chain benefits from an improvement in service variability at the first station.[1]

[1] *In other words, try to improve from the start of the chain.*

Ex 6.5.1. Consider two $G/G/1$ stations in tandem. Suppose $\lambda = 2$ per hour, $C^2_{a,1} = 2$, $C^2_{s,1} = C^2_{s,2} = 0.5$, and $\mathsf{E}[S_1] = 20$ minutes and $\mathsf{E}[S_2] = 25$ minutes. Compute $\mathsf{E}[J] = \mathsf{E}[J_1] + \mathsf{E}[J_2]$.

7 HINTS

h.1.2.2. Why is $M_{A_i}(t) = \mathsf{E}\left[e^{tA_i}\right] = \prod_{k=1}^{i} \mathsf{E}\left[e^{tX_k}\right]$?

h.1.2.3. LOTUS: $\mathsf{E}\left[\alpha^N\right] = \sum_{k=0}^{\infty} \alpha^k \mathsf{P}\{N = k\}$.

h.1.2.4. $\{\min\{X, S\} > x\} = \{X > x\} \cap \{S > x\}$.

h.1.2.6. Use that $\mathsf{P}\{S > X\} = \mathsf{E}[\mathsf{E}[\mathbb{1}_{S>X}|X]]$.

h.1.2.8. Dropping the dependence of N on t for the moment for notational convenience, consider the random variable

$$Y = \sum_{i=1}^{N} Z_i,$$

with $N \sim P(\lambda)$ and $Z_i \sim B(p)$ and $\{Z_i\}$ iid. Show that the moment-generating function of Y is equal to the moment-generating function of a Poisson random variable with parameter λp.

h.2.1.1. Modify d_k in (2.1.1) to incorporate the changed service behavior. Then, substitute d_k in the expression for L_k.

h.2.1.3. Compute the number of jobs that depart from queue 1. Subtract the used capacity for these jobs from the total capacity to get the capacity remaining for queue 2.

h.2.2.2. Introduce a variable $I_k \in \{0, 1\}$ to keep track of the state of the server. Then, $I_{k+1} = \mathbb{1}_{L_k \geq N} + I_k \mathbb{1}_{0<L_k<N}$ implements the N-policy.

h.2.2.4. Introduce an extra variable p_k that specifies the production state of the server. If $p_k = 0$, the server serves the first queue, if $p_k = 1$ it moves from queue 1 to queue 2, if $p_k = 2$ it serves queue 2. Finally, if $p_k = 3$ it moves from queue 2 to queue 1. Thus, the production state cycles from $0 \to 1 \to 2 \to 3 \to 0$.

h.2.5.3. When the machine makes n items, and each of these is faulty with probability p, then the number of faulty items is $\mathrm{Bin}(n, p)$.

h.2.5.4. Make a queue for the new and repaired items and use [2.1.3].

h.3.1.3. Compare this to Definition (3.1.5), or use a numerical example.

h.3.1.4. Use that $L(A_k-) > 0$ means that the system contains at least one job at the time of the kth arrival, and that $A_k- < D_{k-1}$ means that job k arrives (almost surely) before job $k-1$ departs.

h.3.1.5. Make a plot of the function $t - A_{A(t)}$.

h.3.2.2. $\max\{k : A_{k-1} \leq t < A_k\} = \max\{k : A_{k-1} \leq t\}$ because the epochs A_k are ordered and separated in time (almost surely).

h.4.2.1. As a start, the function $\sin(t)$ does not have a limit as $t \to \infty$. However, the time-average $\sin(t)/t \to 0$. Now you need to make some function whose time-average does not converge, hence it should grow fast, or fluctuate wilder and wilder.

For the mathematically interested, we seek a function whose Cesàro limit does not exist.

h.4.3.1. Consider a queueing system with constant service and constant inter-arrival times.

h.4.4.2. Define suitable moments $T_1 < T_2 < \ldots$ to inspect the system, and try to find a suitable function $Y(t)$ that captures what we want to measure.

h.4.4.3. Observe that $Y(t) = \int_0^t \mathbb{1}_{L(s)>0}\,\mathrm{d}s$ is the total amount of time the server is busy during $[0, t]$, hence $Y(t)/t$ is the fraction of time busy. Then take $T_k = D_k$ as the epochs at which to inspect $Y(t)$, and realize that $Y(D_k) = \sum_{i=1}^k S_i$.

h.4.4.4. For the direction of the inequalities (4.4.2), observe that t can lie half way a service interval, and $A(t) \geq D(t)$. Divide by t in (4.4.2). Then for the LHS, multiply by $A(t)/A(t)$ and take appropriate limits. Similar for the RHS but multiply by $D(t)/D(t)$.

h.4.5.1. Recall that $\lim_{t\to\infty} t^{-1} \int_0^t L_s(s)\,\mathrm{d}s$ is the time-average number of servers busy up to t.

h.4.5.3. What happens if the population increases?

h.4.6.1. Show that $A(t) = \sum_{n=0}^\infty A(n, t)$.

h.4.6.3. Use that $\lambda \geq \delta$ always holds. Thus, when $\lambda \neq \delta$, it must be that $\lambda > \delta$. What are the consequences of this inequality; how does the queue length behave as a function of time?

h.4.6.4. Check that the conditions of the renewal reward theorem are satisfied in the above proof of (4.6.4). Then define

$$Y(t) := A(n, t) = \sum_{k=1}^{A(t)} \mathbb{1}_{L(A_k-)=n}$$
$$X_k := Y(A_k) - Y(A_{k-1}) = A(n, A_k) - A(n, A_{k-1}) = \mathbb{1}_{L(A_k-)=n}.$$

h.4.7.3. Apply PASTA and (4.7.6).

h.5.1.2. First show that $M_L(s) = (1-\rho)\sum_n e^{sn}\rho^n$, then use the familiar expression of the sum over a geometric series. Similarly, $\mathsf{P}\{L \geq n\} = \sum_{k\geq n} p(k)$.

h.5.1.3. Fill in $c = 1$. Realize that this is a check on the formulas.

h.5.1.5. Use that $\sum_{i=0}^n x^i = (1 - x^{n+1})/(1 - x)$. BTW, is it necessary for this expression to be true that $|x| < 1$? What should you require for $|x|$ when you want to take the limit $n \to \infty$?

h.5.1.7. Realize that the $M/M/c/c$ queue is similar to the $M/M/c$ queue. However, there cannot be more than c jobs in the system.

h.5.1.8. Use that for any x, $x^n/n! \to 0$ as $n \to \infty$.

h.5.2.1. $\mathsf{E}[Q]$ follows right away from an application of Little's law. For the other quantities we need to find $\mathsf{E}[S]$. Use the expression for $\mathsf{E}[W]$ for the $M/M/1$ queue to solve for ρ. Then, since λ is known, $\mathsf{E}[S]$ follows.

h.5.2.4. Let T be the number of pattys waiting and G the number of persons waiting. There cannot be a patty and a person waiting at the same time, hence $T \cdot G = 0$. Take L as the number of jobs in an $M/M/1$ queue. If $L = 0$, then there are 3 pattys on stock, when $L = 1$ there is one patty less $T = 2, G = 0$; when $L = 2$, we have two pattys less, so $T = 1, G = 0$, when $L = 3$, there are no pattys on stock, but also no people waiting, hence $T = 0, G = 0$. When $L = 4$, we ran out of pattys and one person must be waiting, hence $T = 0, G = 1$.

With this reasoning we see a pattern: at all moments in time, $3 = T + L - G$. Now map this system to an $M/M/1$ queue.

h.5.2.5. How is the *invariant* $3 = T + L - G$ affected?

h.5.3.2. For the $M/M/1$ queue, we have that $B \equiv 1$, i.e., constant.

h.5.3.3. $\mathsf{P}\{B = k\} = q^{k-1}p$ with $q = 1 - p$. Look up the expectation and variance of a geometric rv.

h.5.3.4. Write For 1: $\sum_{k=0}^{\infty} G(k) = \sum_{k=0}^{\infty} \sum_{m=k+1}^{\infty} \mathsf{P}\{X = m\}$, reverse the summations. Then realize that $\sum_{k=0}^{\infty} \mathbb{1}_{k<m} = m$. For 2: $\sum_{i=0}^{\infty} iG(i) = \sum_{n=0}^{\infty} \mathsf{P}\{X = n\} \sum_{i=0}^{\infty} i\,\mathbb{1}_{n \geq i+1}$,and reverse the summations.

h.5.3.5. Use [5.3.4].

h.5.4.2. Use the PASTA property. Realize that when estimating $\mathsf{E}[S_r]$ along a sample path, $S_r = 0$ for jobs that arrive at an empty system.

h.5.4.3. Use that $\mathsf{V}[S] = 0$ for the $M/D/1$ queue and $C_s^2 = 1$ for the $M/M/1$ queue.

h.5.4.5. The rate of accepted jobs is $\lambda\pi(0)$, hence $\rho = \lambda\pi(0)\,\mathsf{E}[S]$. But also $\rho = 1 - \pi(0)$. Now solve for $\pi(0)$.

h.5.5.5. Show first that

$$\mu\,\mathsf{E}[L] = \mu \sum_{n=0}^{\infty} n\pi(n) = \lambda \frac{\mathsf{E}\left[B^2\right]}{2} + \lambda\,\mathsf{E}[B]\,\mathsf{E}[L] + \lambda \frac{\mathsf{E}[B]}{2}.$$

h.5.6.2. Use the argumentation that leads to (4.4.2).

h.5.6.3. Use that $\sum_{n=0}^{N} \alpha^n = \frac{1-\alpha^{N+1}}{1-\alpha}$.

h.5.6.6. What is the cost of one cycle? What is the duration of one cycle? Use the renewal reward theorem.

h.5.6.8.

$$\begin{aligned} aq^2 &= aq^2, \\ bq &= 2aq\,\mathsf{E}[Y] - 2aq + bq + hq\,\mathsf{E}[S], \\ 0 &= a\,\mathsf{E}\left[Y^2\right] - 2a\,\mathsf{E}[Y] + a + b\,\mathsf{E}[Y] - b + \frac{1}{2}\lambda h\,\mathsf{E}\left[S^2\right]. \end{aligned}$$

h.5.6.11. Take $K = 0$ and $N = 1$, and realize that the LHS is $h\mathsf{E}[L]$.

h.6.1.1. Think about the inter-departure time of a machine that produces items every 10 minutes. What inter-arrival times will a down-stream machine see?

h.6.1.4. What is λ? What is C_a^2 in the $G/G/1$ setting; what is it in the $M/G/1$ setting?

h.6.1.5. Let S be the processing (or service) time at the server, and S_i the service time of a type i job. Then,

$$S = \mathbb{1}_{T=1} S_1 + \mathbb{1}_{T=2} S_2.$$

h.6.2.2. An arbitrary job has to wait half the time it takes to form a batch.

h.6.3.1. Get the units right. Compute the load, and then the rest.

h.6.3.2. Realize that we now deal with setups and batch processing.

h.6.4.1. Mind to work in a consistent set of units, e.g., hours. It is easy to make mistakes.

h.6.4.2. Observe that $m_f = 1/\lambda_f$ and $m_r = \mathsf{E}[R]$.

h.6.4.3. Condition on S_0 and N.

h.6.4.4. Just realize that $\mathsf{E}[S] = \mathsf{E}[S_0]/A$, and use the above.

8

SOLUTIONS

s.11.

$$M_X(t) = \mathsf{E}\left[\exp(tX)\right] = \int_0^\infty e^{tx} f(x)\,\mathrm{d}x = \int_0^\infty e^{tx}\lambda e^{-\lambda x}\,\mathrm{d}x = \frac{\lambda}{\lambda - t}.$$

This last integral only converges when $\lambda - t > 0$. Next, $M'_X(t) = \lambda/(\lambda - t)^2 \implies M'_X(0) = 1/\lambda$, $M''_X(t) = 2\lambda/(\lambda - t)^3 \implies \mathsf{E}\left[X^2\right] = 2\lambda^{-2}$. Thus, $\mathsf{V}[X] = \mathsf{E}\left[X^2\right] - (\mathsf{E}[X])^2 = \frac{2}{\lambda^2} - (\frac{1}{\lambda})^2 = \lambda^{-2}$.

s.1.2.2. Using the iid property of the $\{X_i\}$,

$$M_{A_i}(t) = \mathsf{E}\left[e^{tA_i}\right] = \mathsf{E}\left[\exp\left(t\sum_{k=1}^{i} X_k\right)\right] = \prod_{k=1}^{i} \mathsf{E}\left[e^{tX_k}\right] = \left(\frac{\lambda}{\lambda - t}\right)^i.$$

From a table of moment-generating functions it follows immediately that $A_i \sim \Gamma(i, \lambda)$, i.e., A_i is Gamma distributed.

s.1.2.3. Check the hint.

$$\sum_{k=0}^{\infty} \alpha^k \mathsf{P}\{N = k\} = \sum_{k=0}^{\infty} \alpha^k \frac{(\lambda)^k}{k!} e^{-\lambda} = e^{-\lambda} \sum_{k=0}^{\infty} \frac{(\alpha\lambda)^k}{k!} = \exp(\lambda(\alpha - 1)).$$

Take $\alpha = e^s$ in [1.2.3].

$$M'_{N(t)}(s) = \lambda t e^s \exp(\lambda t(e^s - 1)).$$

Hence $\mathsf{E}[N(t)] = M'_{N(t)}(0) = \lambda t$. Next, $M''_{N(t)}(s) = (\lambda t e^s + (\lambda t e^s)^2)\exp(\lambda t(e^s - 1))$, hence $\mathsf{E}\left[(N(t))^2\right] = M''(0) = \lambda t + (\lambda t)^2$, and thus, $\mathsf{V}[N(t)] = \mathsf{E}\left[(N(t))^2\right] - (\mathsf{E}[N(t)])^2 = \lambda t + (\lambda t)^2 - (\lambda t)^2 = \lambda t$.

s.1.2.4. $\mathsf{P}\{Z > x\} = \mathsf{P}\{\min\{X, S\} > x\} = \mathsf{P}\{X > x, S > x\} = \mathsf{P}\{X > x\}\,\mathsf{P}\{S > x\} = e^{-\lambda x}e^{-\mu x}$, as X and S independent.

s.1.2.5. With the above:

$$\begin{aligned}\mathsf{P}\left\{N_\lambda(t) = 1 \,\middle|\, N_\lambda(t) + N_\mu(t) = 1\right\} &= \frac{\mathsf{P}\{N_\lambda(t) = 1\}\,\mathsf{P}\left\{N_\mu(t) = 0\right\}}{\mathsf{P}\left\{N_{\lambda+\mu}(t) = 1\right\}} \\ &= \frac{\lambda t \exp(-\lambda t)\exp(-\mu t)}{((\lambda+\mu)t)\exp(-(\lambda+\mu)t)} = \frac{\lambda t \exp(-(\lambda+\mu)t)}{((\lambda+\mu)t)\exp(-(\lambda+\mu)t)} = \frac{\lambda}{\lambda+\mu}.\end{aligned}$$

s.1.2.6. With conditioning, $\mathsf{P}\{S > X | X = t\} = \mathsf{P}\{S > t\} = e^{-\mu t}$. With the fundamental bridge and conditional expectation, $\mathsf{E}[\mathbb{1}_{S>X}|X] = e^{-\mu X}$, hence $\mathsf{E}[\mathbb{1}_{S>X}] = \mathsf{E}\left[e^{-\mu X}\right]$. But this last formula is equal to $M_X(-\mu)$, so taking $t = -\mu$ in the MGF of X we obtain $\lambda/(\lambda + \mu)$.

s.1.2.7.

$$\begin{aligned}M_{N_\lambda(t)+N_\mu(t)}(s) &= M_{N_\lambda(t)}(s) \cdot M_{N_\mu(t)}(s) = \exp(\lambda t(e^s - 1)) \cdot \exp(\mu t(e^s - 1)) \\ &= \exp((\lambda + \mu)t(e^s - 1)).\end{aligned}$$

s.1.2.8. Use the hint. Then,

$$\mathsf{E}\left[e^{sZ}\right] = e^0\,\mathsf{P}\{Z=0\} + e^s\,\mathsf{P}\{Z=1\} = (1-p) + e^s p,$$

from which

$$\mathsf{E}\left[e^{s\sum_{i=1}^n Z_i}\right] = \left(\mathsf{E}\left[e^{sZ}\right]\right)^n = \left(1+p(e^s-1)\right)^n,$$

where we use that the Z_i are iid. Thus, $\mathsf{E}\left[e^{sY}|N=n\right] = \mathsf{E}\left[e^{s\sum_{i=1}^n Z_i}\right] = \alpha^n$, where we write $\alpha = 1+p(e^s-1)$ for ease. Now with Adam's law and [1.2.3], we get

$$\mathsf{E}\left[e^{sY}\right] = \mathsf{E}\left[\alpha^N\right] = e^{\lambda(\alpha-1)} = \exp(\lambda p(e^s-1)).$$

This is the MGF of a Poisson rv with rate λp.

s.2.1.1.

$$\begin{aligned}
d_k &= \min\{L_{k-1}+a_k, c_k\},\\
L_k &= L_{k-1}+a_k-d_k = L_{k-1}+a_k-\min\{L_{k-1}+a_k, c_k\}\\
&\overset{1}{=} -\min\{c_k-L_{k-1}-a_k, 0\} \overset{2}{=} \max\{L_{k-1}+a_k-c_k, 0\},
\end{aligned}$$

where 1 uses that $x-\min\{x,y\} = -\min\{0, y-x\}$ and 2 that $-\min\{x,0\} = \max\{-x,0\}$.

s.2.1.2. Take $c_k = c\,\mathbb{1}_{L_{k-1}>=N}$.

s.2.1.3.

$$d_k^1 = \min\{L_{k-1}^1, c_k\}, \qquad c_k^2 = c_k - d_k^1, \qquad d_k^2 = \min\{L_{k-1}^2, c_k^2\}, \qquad L_k^i = L_{k-1}^i + a_k^i - d_k^i.$$

s.2.1.4. Let c_k^i be the capacity allocated to queue i in period k. The fair rule gives that

$$\begin{aligned}
c_k^1 &= \text{round}\frac{L_{k-1}^1}{L_{k-1}^1+L_{k-1}^2}, & c_k^2 &= c_k - c_k^1,\\
d_k^i &= \min\{L_{k-1}^i, c_k^i\}, & L_k^i &= L_{k-1}^i + a_k^i - d_k^i.
\end{aligned}$$

In general, rules to distribute capacity c_k over the queues can be based on game-theoretic ideas, such as the principle of *equal division of the contested sum*, see the work of Aumann and Maschler.

s.2.1.5. Queue 2 minimally needs $c_k^2 = \min\{L_{k-1}^2, r^2\}$, and with this,

$$d_k^1 = \min\{L_{k-1}^1, c_k - c_k^2\}, \qquad d_k^2 = \min\{L_{k-1}^2, c_k - d_k^1\}.$$

s.2.2.1. Here are the recursion in python. Learn how to read codde.

```python
import numpy as np
from icecream import ic

rng = np.random.default_rng(3)

num_weeks = 100
```

```
a1 = rng.poisson(11, size=num_weeks)
a2 = rng.poisson(3, size=num_weeks)
c = rng.poisson(15, size=num_weeks)

r1, r2 = 9, 1

L1 = np.zeros(num_weeks)
L2 = np.zeros(num_weeks)
for k in range(1, num_weeks):
    d1 = min(L1[k - 1], c[k] - r2)
    c2 = c[k] - min(r1, d1)
    d2 = min(L2[k - 1], c2)
    L1[k] = L1[k - 1] + a1[k] - d1
    L2[k] = L2[k - 1] + a2[k] - d2


ic(L1.mean(), L1.std())
ic(L2.mean(), L2.std())
```

s.2.2.2. Think about the hint. Understanding the state variable I_k is the hardest part.

```
import numpy as np
import matplotlib.pyplot as plt


rng = np.random.default_rng(3)

num_weeks = 40
a = rng.poisson(1, size=num_weeks)
c = rng.poisson(5, size=num_weeks)
N = 20

L = np.zeros(num_weeks)
I = np.zeros(num_weeks)  # on or not

for k in range(1, num_weeks):
    I[k] = (L[k - 1] >= N) + I[k - 1] * (0 < L[k - 1] < N)
    c[k] *= I[k]
    d = min(L[k - 1], c[k])
    L[k] = L[k - 1] + a[k] - d


xx = range(num_weeks)
plt.step(xx, L, where="pre")
plt.step(xx, N * I, ":", where="pre")
plt.savefig("N-policy.pdf")
```

Note that when arrivals cannot be served in the period in which they arrive, it might be that the system is never empty. For instance, if $a_k \sim \text{Unif}(4, 10)$, then $L_k \geq a_k \geq 4$ for all k.

s.2.2.3. The last line is the next code most interesting. Ask chatgpt to explain it to you if you find it hard (I did it, to check. The answer is real good.)

```
beta = 0.5
K = 50
```

```
h = 1


cost = beta * I.sum()
cost += h * L.sum()
cost += K * (I[1:] > I[:-1])
```

s.2.2.4. Read the hint, in particular think about the cycling. This is implemented with the indicators on p_k. Next, multiplying the potential server capacity by an indicator on the state of the server gives the actual service capaity.

```
import numpy as np
import matplotlib.pyplot as plt


rng = np.random.default_rng(3)

num_weeks = 40
a1 = rng.poisson(1, size=num_weeks)
a2 = rng.poisson(3, size=num_weeks)
c1 = rng.poisson(3, size=num_weeks)
c2 = rng.poisson(5, size=num_weeks)

L1 = np.zeros(num_weeks)
L2 = np.zeros(num_weeks)
p = np.zeros(num_weeks, dtype=int)  # production state

for k in range(1, num_weeks):
    p[k] = p[k - 1]
    p[k] += (p[k - 1] == 0) * (L1[k - 1] == 0) + (p[k - 1] == 1)
    p[k] += (p[k - 1] == 2) * (L2[k - 1] == 0) - 3 * (p[k - 1] == 3)
    d1 = min(L1[k - 1] + a1[k], c1[k] * (p[k] == 0))
    d2 = min(L2[k - 1] + a2[k], c2[k] * (p[k] == 2))
    L1[k] = L1[k - 1] + a1[k] - d1
    L2[k] = L2[k - 1] + a2[k] - d2

xx = range(num_weeks)
plt.step(xx, L1, where="pre", label="L1")
plt.step(xx, L2, where="pre", label="L2")
plt.step(xx, p, ":", where="pre", label="P")
plt.legend()
plt.savefig("nurses.pdf")
```

s.2.3.1. Only P' triggers the orders and only I depends on the leadtime L. When $L = 0$, the inventory position and level coincide.

s.2.3.2. In a queueing system, an item is not produced before the demand for it arrives. This is a cost effective way to produce when items are very expensive, hence holding cost is high, or customer specific. Besides, services (which are not products) cannot be put on stock. For instance, keeping an invetory of haircuts seems impossible.

Note that in a queueing system the server has a finite capacity. In the inventory models above, the production capacity is assumed infinite.

A basestock model with $s = -1$ is appropriate. When a customer arrives, the server switches on, and keeps on producing until there are no further jobs in queue. Note also that a job in queue is the same as a customer in backlog.

s.2.3.3. We want all service levels to be zero. If $h \gg b$ for some inventory system, the cost structure expresses that we dislike stock on hand. In other words, it expresses that we want produce according to MTO.

s.2.3.4. Note the differences between these schemes. In the (s, S) the order level is fixed, but in case of stochastic demand, the specific period in which an order is triggered is not fixed. The order quantity (which is not the same as the order-up-to level) may also differ from period to period. In the (Q, r) policy, the order quantity is fixed, but not the period. The (T, S) policy fixes the period, but leaves the order quantity free. The (s, S) policy must be ablle to achieve the lowest average cost, because it is the least constrained of the policies.

s.2.3.5. Since at the start of period t, the inventory present to serve is I_t, the rules for a loss system become

$$\begin{aligned}
L_t &= (I_t - D_t)^- = I_t'^- && \text{lost demand} \\
A_t &= I_t - L_t && \text{accepted demand} \\
P_t' &= P_t - A_t, && \text{subtract the accepted demand} \\
I_t' &= I_t - A_t, && \text{subtract the accepted demand.}
\end{aligned}$$

s.2.5.1. The (Q, r) rule with $Q = 10$ and $r = s = 20$.

s.2.5.2. Here is the python code.

```python
import numpy as np

rng = np.random.default_rng(3)
a = rng.integers(3, 8, size=100)
c = 2 * np.ones_like(a)
L = np.zeros_like(a)
l = np.zeros_like(a)  # store the lost jobs

K = 8  # loss level

for k in range(1, len(a)):
    d = min(L[k - 1], c[k])
    Lp = L[k - 1] + a[k] - d  #  without loss
    L[k] = min(Lp, K)  #  chop off at K
    l[k] = Lp - L[k]  #  lost

print(sum(l) / sum(a))  # fraction lost.
```

s.2.5.3. Here is the code.

```python
import numpy as np

```

```
rng = np.random.default_rng(3)
a = rng.integers(3, 8, size=100)
c = 7 * np.ones_like(a)
L = np.zeros_like(a)
d = np.zeros_like(a)
p = 0.1

for k in range(1, len(a)):
    produced = min(L[k - 1], c[k])
    faulty = rng.binomial(produced, p)
    d[k] = produced - faulty
    L[k] = L[k - 1] + a[k] - d[k]

print(L.mean())
```

Observe that faulty items do not leave the system.

An interesting challenge: can you use these recursions to *prove* that the long-run average service capacity $n^{-1}\sum_{i=1}^{n} c_i$ must be larger than $\gamma/(1-p)$, where $\gamma = \lim_{n\to\infty} n^{-1}\sum_{k=1}^{n} a_k$ is the arrival rate of new jobs?

s.2.5.4. In code:

```
1  a = [0, 4, 8, 2, 1]
2  c = [3] * len(a)
3  L_R = [0] * len(a)  # repair jobs
4  d_R = [0] * len(a)  # departing repair jobs
5  L_N = [0] * len(a)  # new jobs
6  d_N = [0] * len(a)  # departing new jobs
7  p = 0.2
8
9  L_R[0] = 2
10 L_N[0] = 8
11
12 for k in range(1, len(a)):
13     l = L_R[k - 1]
14     if l % 2 == 1:
15         l -= 1
16     d_R[k] = min(l, 2 * c[k])
17     c_N = c[k] - d_R[k] / 2  # capacity left for new jobs
18     d_N[k] = min(L_N[k - 1], c_N)
19     a_R = int(p * d_N[k] + 0.5)  # rounding
20     a_N = a[k]
21     L_R[k] = L_R[k - 1] + a_R - d_R[k]
22     L_N[k] = L_N[k - 1] + a_N - d_N[k]
23
24 print(L_R)
25 print(L_N)
```

We need a trick to get around the division by 2 in line 17 (because of $L_R[k]$ is odd we might end up with 0.5). For this, we should check whether $L_R[k]$ is odd or not. If so, it must be at least be 1, and then we can subtract 1 to make l even. This l can be used to compute the number of departures.

Here is a general observation: the number of ways to organize the repairs is countless. Do we serve them with priority or not, do we bin them and make new items instead, do we do the

repairs on another, separate, station? Such differences need to be reflected in the simulation model.

s.2.5.5. Using [2.2.1] with $d_k^1 = \min\{L_{k-1}^1, c_k^1, M - L_{k-1}^2\}$ is nearly correct. But, what if $L_{k-1}^2 > M$ for the first few periods? Therefore, take instead $d_k^1 = \min\{L_{k-1}^1, c_k^1, [M - L_{k-1}^2]^+\}\}$.

```
a1 = [0, 2, 3, 8, 0, 9]
c1 = [3] * len(a1)
c2 = [2] * len(a1)
L1 = [0] * len(a1)
L2 = [0] * len(a1)

M2 = 10

if L2[0] > M2:
    print("The starting value of L2 is too large")
    exit(0)

for k in range(1, len(a1)):
    d1 = min(L1[k], c1[k], M2 - L2[k - 1])
    L1[k] = L1[k - 1] + a1[k] - d1
    d2 = min(L2[k], c2[k])
    L2[k] = L2[k - 1] + d1 - d2
```

s.2.5.6. Take $a_k^C = d_k^A + d_k^B$.

s.2.5.7. The idea is like this. The dictionary `L_count` counts the number of jobs that see 0, 1, and so on, in the system.

Here is the code. As an aside, it was hard to make it this simple.

See the web what defaultdict does.

```
from collections import defaultdict

L_count = defaultdict(int)

a = [0, 2, 5, 1, 2]
c = [0, 1, 1, 1, 1]

d = [0] * len(a)
L = [0] * len(a)

for k in range(1, len(a)):
    d[k] = min(L[k - 1], c[k])
    L[k] = L[k - 1] + a[k] - d[k]
    for i in range(a[k]):
        L_count[L[k - 1] - d[k] + i] += 1


# normalize
tot = sum(L_count.values())
L_dist = {k: v / tot for k, v in L_count.items()}

print(L_count)
print(L_dist)
```

s.3.1.1. Yes, it is true.

s.3.1.2. $A(t)$ is the number of arrivals during $[0,t]$. Suppose that $A(t)=n$. This nth job arrived at time A_n. Thus, $A_{A(t)}$ is the arrival time of the last job that arrived before or at time t. In a similar vein, A_n is the arrival time of the nth job. Thus, the number of arrivals up to time A_n, i.e., $A(A_n)$, must be n.

s.3.1.3. Suppose $A_3=10$ and $A_4=20$. Take $t=15$. Then $\min\{k: A_k \geq 15\}=4$ since $A_3<t=15<A_4$. On the other hand $\max\{k: A_k \leq t\}=3$. And, indeed, at time $t=15$, 3 jobs arrived, not 4. So defining $A(t)$ as $\min\{k: A_k \geq t\}$ is not OK. This example also shows that in general $A(t) \neq \min\{k: A_k > t\}$. So, neither definition is correct.

s.3.1.4. In a sense, the claim is evident, for, if the system contains a job when job k arrives, it cannot be empty. But if it is not empty, then at least the last job that arrived before job k, i.e., job $k-1$, must still be in the system. That is, $D_{k-1} \geq A_k$. A more formal proof proceeds along the following lines. Using that $A(A_k-)=k-1=D(D_{k-1})$,

$$\begin{aligned} L(A_k-)>0 &\iff A(A_k-)>D(A_k-) \iff A(A_k-) \geq D(A_k) \\ &\iff k-1 \geq D(A_k) \iff D(D_{k-1}) \geq D(A_k) \iff D_{k-1} \geq A_k, \end{aligned}$$

where the last relation follows from the fact that $D(t)$ is a counting process, hence monotone non-decreasing.

s.3.1.5. Suppose that $A_{n+1}=A_n+\epsilon$, i.e., the arrival time A_{n+1} is just a tiny bit larger than A_n. Then job $n+1$ would have to wait $J_n-\epsilon$ before it can get access to the server. Now, recall from a previous exercise that if $A(t)=n$, then A_n is the arrival time of the nth job, so that the function $A_{A(t)}$ provides us with arrival times as a function of t. Between arrival moments, the virtual waiting time decreases with slope 1, until it hits 0.

s.3.2.1. First compute the sum of the sojourn times of all jobs that departed before t; in a formula $\sum_{k=1}^n J_k \mathbb{1}_{D_k \leq t}$. Then add to this the times of the jobs arrived before t but that have yet departed: $\sum_{k=1}^n (t-A_k)\mathbb{1}_{A_k \leq t < D_k}$. Divide the total by t.

s.3.2.2. See the hint, and check out the definition of $A(t)$.

$$\max\{k: A_{k-1} \leq t < A_k\} = \max\{A_{k-1} \leq t\} = \sum_{k=1}^{\infty} \mathbb{1}_{A_{k-1} \leq t} = \sum_{k=0}^{\infty} \mathbb{1}_{A_k \leq t} = 1 + A(t).$$

s.3.4.1. Type on the Google prompt: 'realpython dataclasses', click on the link that appears, and then read and think hard.

s.3.4.2. Give this string to ChatGPT: 'What is the advantage of a python deque over a heapq. Discuss the algorithmic efficiency too.'

s.4.2.1. If $A(t)=3t^2$, then clearly $A(t)/t=3t$. This does not converge to a limit.

Another example, let the arrival rate $\lambda(t)$ be given as follows:

$$\lambda(t) = \begin{cases} 1 & \text{if } 2^{2k} \leq t < 2^{2k+1} \\ 0 & \text{if } 2^{2k+1} \leq t < 2^{2(k+1)}, \end{cases}$$

for $k=0,1,2,\ldots$. Let $A(t)=\lambda(t)t$. Then $A(t)/t$ does not have a limit. Of course, these examples are quite pathological, and are not representable for 'real life cases'. (Although this is also quite vague: what is actually a real-life case?)

s.4.3.1. Take $L(0) = 0$, $X_k = 10$ and $S_k = 10 - \epsilon$ for some tiny $\epsilon > 0$. Then $L(t) = 1$ nearly all the time. In fact, $\lim_{t\to\infty} t^{-1} \int_0^t L(t)\,\mathrm{d}t = 1 - \epsilon/10$. However, $L(A_k-) = 0$ for all k.

s.4.3.2.

$$\mathsf{E}[L] = \lim_{t\to\infty} \frac{1}{t} \int_0^t L(s)\,\mathrm{d}s \neq \lim_{t\to\infty} \frac{L(t)}{t}.$$

If $L(t) = 1$ for all t, $\mathsf{E}[L] = 1$, but $L(t)/t \to 0$.

s.4.4.1. Note that $Y(t)/t = 1$. Next, since $T_K = A_k$ and $Y(t) = t$,

$$X_k = Y(T_k) - Y(T_{k-1}) = Y(A_k) - Y(A_{k-1}) = A_k - A_{k-1},$$

thus, X_k is the inter-arrival time between jobs $k-1$ and k. Also, $N(t) = \sum_k \mathbb{1}_{T_k \le t} = \sum_k \mathbb{1}_{A_k \le t}$, hence $N(t)/t \to \lambda$ by assumption. By renewal reward, $\lambda\,\mathsf{E}[X] = Y = 1 \implies \lambda = 1/\mathsf{E}[X]$.

s.4.4.2. Take T_k as the end of the kth repair, and $Y(t) = \int_0^t u(s)\,\mathrm{d}s$, where $u(s) = 1$ if the server is up (operational) and 0 if the server is down (broken). Thus $Y(t)$ is the total amount of time the system has been up during $[0, t]$, and $Y(t)/t$ becomes the fraction of up time (which is the availability) in the limit. As for X_k, this is $X_k = Y(T_k) - Y(T_{k-1})$, which is the up time in $[T_{k-1}, T_k]$. Therefore, in the limit, X is the average uptime between two break downs, hence $X \sim m_f$. Finally, the time between two inspections consists of an uptime and break down. Hence, the rate at which such inspection epochs occur is $1/(m_f + \mathsf{E}[R])$.

s.4.4.3. Take the hint serious. Then, as $k/T_k \to \delta$, it follows from the renewal reward theorem that $Y(t)/t \to \delta\,\mathsf{E}[S]$. By rate-stability, $\delta = \lambda$, so that $\rho = \lambda\,\mathsf{E}[S] = \delta\,\mathsf{E}[S] = Y$, from which the claim follows.

s.4.4.4. For the LHS, as $A(t) \to \infty$ as $t \to \infty$,

$$\lim_{t\to\infty} \frac{1}{t} \sum_{k=1}^{A(t)} S_k = \lim_{t\to\infty} \frac{A(t)}{t} \frac{1}{A(t)} \sum_{k=1}^{A(t)} S_k = \lim_{t\to\infty} \frac{A(t)}{t} \cdot \lim_{t\to\infty} \frac{1}{A(t)} \sum_{k=1}^{A(t)} S_k = \lambda\,\mathsf{E}[S].$$

Apply similar limits to the RHS. The limit of the middle term gives the fraction of time the server is busy.

s.4.4.5. Take $Y(t) = \int_0^t \mathbb{1}_{L_s(s)=1}\,\mathrm{d}t$. Then, when U_k is the kth busy time and I_k the kth idle time, let $X_k = U_k$ and $T_k = U_k + I_k$. Then $Y(t)/t \to \rho$, $X_k/k \to \mathsf{E}[U]$, and $(I_k + U_k)/k \to \mathsf{E}[I] + \mathsf{E}[U] = \lambda^{-1}$, i.e., the inverse of the λ we use in the renewal-reward theorem.

s.4.5.1. Since the queue is stable by assumption, jobs depart at rate λ, hence any job enters and leaves the box that contains the c servers. The average time a job spends in the box is $\mathsf{E}[S]$. By Little's law, the average number in the box is $\mathsf{E}[L_s] = \lambda\,\mathsf{E}[S]$.

s.4.5.2. When you arrive, you first have to wait for the job in service to finish and then for the 9 jobs in queue. By the memoryless property, at the moment you arrive, the remaining service time of the job in service is still $\mathsf{E}[S]$ Thus, you have to wait $10/\mu = 5/3h \neq 5/6h$.

s.4.5.3. Let's make a very simple model to clarify the problem. We start with an initial population size of A. Then assume that each and every individual dies after exactly 80 years (and does not live a day longer). Moreover, assume that the population increases by 1% per year. Then, after 79 years, the size of the population is $A\sum_{k=0}^{79}(1.01)^k$. If we assume (as we do when we apply Little's law) that each year contains the same amount of people, then the number of people per year equals:

$$\frac{A}{80}\sum_{k=0}^{79}(1.01)^k = \frac{A}{80}\frac{(1.01)^{80}}{1.01-1} = 1.5A.$$

However, after 80 years, just A people die, not $1.5A$. Interestingly, $4000/1.5 = 2700$, which is quite a bit closer to 2900 per week.

In conclusion, we should apply Little's law with caution; in particular, when we just consider stretches of time in which we know that the population increases (or decreases) on average, Little's law does not hold.

s.4.5.4. Intuitively, the left term is all the work that arrived up to time t, the middle term is all the work that has been processed, and the right term all the work that left. Any job that is half-way its service counts for full at the left, for half in the middle expression, and not in the right.

More formally, for any job k and time t, we have $J_k \mathbb{1}_{A_k\le t} \ge \int_0^t \mathbb{1}_{A_k\le s<D_k}\,\mathrm{d}s \ge J_k\mathbb{1}_{D_k\le t}$. (To see this, fix k, and check the three cases $t<A_k, A_k\le t<D_k, D_k<t$.) Then,

$$\sum_{k=1}^{\infty} J_k\mathbb{1}_{A_k\le t} \ge \int_0^t \sum_{k=1}^{\infty}\mathbb{1}_{A_k\le t<D_k}\,\mathrm{d}s \ge \sum_{k=1}^{\infty} J_k\mathbb{1}_{D_k\le t}.$$

Finally, note that $\sum_{k=1}^{\infty} J_k\mathbb{1}_{A_k\le t} = \sum_{k=1}^{A(t)} J_k$ and $\sum_{k=1}^{\infty} J_k\mathbb{1}_{D_k\le t} = \sum_{k=1}^{D(t)} J_k$, and use the definition of $L(s)$.

s.4.5.5.

$$\lim_{t\to\infty}\frac{A(t)}{t}\frac{1}{A(t)}\sum_{k=1}^{A(t)} J_k \ge \lim_{t\to\infty}\frac{1}{t}\int_0^t L(s)\,\mathrm{d}s \ge \lim_{t\to\infty}\frac{D(t)}{t}\frac{1}{D(t)}\sum_{k=1}^{D(t)} J_k.$$

We use the strong law of large numbers to conclude that the limits converges to $n^{-1}\sum_{k=1}^{n} J_k \to \mathsf{E}[J]$, and we assume that $\{J_k, k\ge N\}$ is a sequence of iid random variables for N sufficiently large.

s.4.6.1.

$$A(t) = \sum_{k=1}^{\infty}\mathbb{1}_{A_k\le t} = \sum_{k=1}^{\infty}\sum_{n=0}^{\infty}\mathbb{1}_{L(A_k-)=n}\mathbb{1}_{A_k\le t} = \sum_{n=0}^{\infty} A(n,t).$$

Divide by t, then use (4.6.3).

s.4.6.2. If $\lambda>\delta$, then $L(t)\to\infty$. But then there must be a last time, s say, that $L(s)=n+1$, and $L(t)>n+1$ for all $t>s$. Hence, after time s no job will see the system with n jobs. Thus $A(n,t)=A(n,s)$ for all $t>s$. As this is finite, $\lim_{t\to\infty}A(n,t)/t=0$.

s.4.6.3. The assumptions lead us to conclude that $\lambda>\delta$. As a consequence, the queue length must increase in the long run (jobs come in faster than they leave). Therefore, $A(n,t)/t\to 0$ for all n, and also $D(n,t)/t\to 0$. Consequently, $\pi(n)=\delta(n)=0$, which is the only sensible reconciliation with (4.7.5).

s.4.6.4. First we check the conditions. The counting process here is $\{A(t)\}$ and the epochs at which $A(t)$ increases are $\{A_k\}$. By assumption, $A_k \to \infty$, hence $A(t) \to \infty$ as $t \to \infty$. Moreover, by assumption $A(t)/t \to \lambda$. Also, $A(n,t)$ is evidently non-decreasing and $A(n,t) \to \infty$ as $t \to \infty$.

From the definitions in the hint,

$$X = \lim_{m\to\infty} \frac{1}{m} \sum_{k=1}^{m} X_k = \lim_{m\to\infty} \frac{1}{m} \sum_{k=1}^{m} \mathbb{1}_{L(A_k-)=n} = \pi(n).$$

Since $Y = \lim_{t\to\infty} Y(t)/t = \lim_{t\to\infty} A(n,t)/t$ it follows from the renewal reward theorem that

$$Y = \lambda X \implies \lim_{t\to\infty} \frac{A(n,t)}{t} = \lambda X = \lambda\pi(n).$$

s.4.6.5. By the memoryless property of the (exponential) distributed service times of the $M/M/1$ queue, the duration of a job in service, if any, is $\mathrm{Exp}(\mu)$ also at an arrival moment. Therefore, at an arrival moment, all jobs in the system (whether in service or not) have the same expected duration. Hence, the expected time to spend in queue is the expected number of jobs in the system times the expected service time of each job, i.e., $\mathsf{E}[J] = \mathsf{E}[L]\,\mathsf{E}[S]$. Note that we use PASTA to see that the expected number of jobs in the system at an arrival is $\mathsf{E}[L]$. For the $M/G/1$ queue, the job in service (if any) does not have the same distribution as a job in queue. Hence, for the $M/G/1$ queue, the *expected time in queue is not* $\mathsf{E}[L]\,\mathsf{E}[S]$.

s.4.7.1.

1. $A_k = 2(k-1)$, $k = 1,2,\ldots$, as jobs arrive at $t = 0,2,4,\ldots$ Thus, $A(t) = 1 + \lfloor t/2 \rfloor$ for $t \geq 0$. (BTW, $A(t)/t \to 1/2$ as $t \to \infty$.) We also know that $L(s) = 1$ if $s \in [2k, 2k+1)$ and $L(s) = 0$ for $s \in [2k-1, 2k)$ for $k = 0,1,2,\ldots$. Thus, $L(A_k-) = L(2k-) = 0$. Therefore, $A(0,t) = A(t)$ for $t \geq 0$, and $A(n,t) = 0$ for $n \geq 1$.

2. All arrivals see an empty system. Hence, $A(0,t)/A(t) \approx (t/2)/(t/2) = 1$, and $A(n,t) = 0$ for $n > 0$. Thus, $\pi(0) = \lim_{t\to\infty} A(0,t)/A(t) = 1$ and $\pi(n) = 0$ for $n > 0$. Recall from the other exercises that $p(0) = 1/2$. Hence, statistics as obtained via time averages are not necessarily the same as statistics obtained at arrival moments (or any other point process).

3. $\lambda = \lim_{t\to\infty} A(t)/t = 1/2$. $\lambda(0) = 1$, $p(0) = 1/2$, and $\pi(0) = 1$. Hence,

$$\lambda\pi(0) = \lambda(0)p(0) \implies \frac{1}{2} \times 1 = 1 \times \frac{1}{2}.$$

   For $n > 0$ it's easy, everything is 0.

4. Observe that the system never contains more than 1 job. Hence, $Y(n,t) = 0$ for all $n \geq 2$. Then we see that $Y(1,t) = \int_0^t \mathbb{1}_{L(s)=1}\,\mathrm{d}s$. Now observe that for our queueing system $L(s) = 1$ for $s \in [0,1)$, $L(s) = 0$ for $s \in [1,2)$, $L(s) = 1$ for $s \in [2,3)$, and so on. Thus, when $t < 1$, $Y(1,t) = \int_0^t \mathbb{1}_{L(s)=1}\,\mathrm{d}s = \int_0^t 1\,\mathrm{d}s = t$. When $t \in [1,2)$,

$$L(t) = 0 \implies \mathbb{1}_{L(t)=0} \implies Y(1,t) \text{ does not change.}$$

   Continuing to $[2,3)$ and so on gives

$$Y(1,t) = \begin{cases} t & t \in [0,1), \\ 1 & t \in [1,2), \\ 1 + (t-2) & t \in [2,3), \\ 2 + (t-4) & t \in [4,5), \end{cases}$$

and so on. With this, we see that the general formula must be

$$Y(t,1) = \begin{cases} 1 + \lfloor t/2 \rfloor + t - \lfloor t \rfloor, & \text{if } \lfloor t \rfloor \text{ is even,} \\ 1 + \lfloor t/2 \rfloor, & \text{if } \lfloor t \rfloor \text{ is odd.} \end{cases}$$

Since $Y(n,t) = 0$ for all $n \geq 2$, $L(s) = 1$ or $L(s) = 0$ for all s, therefore,

$$Y(0,t) = t - Y(1,t).$$

5.

$$\begin{aligned} \lambda(0) &\approx \frac{A(0,t)}{Y(0,t)} \approx \frac{t/2}{t/2} = 1, \\ \lambda(1) &\approx \frac{A(1,t)}{Y(1,t)} \approx \frac{0}{t/2} = 0, \\ p(0) &\approx \frac{Y(0,t)}{t} \approx \frac{t/2}{t} = \frac{1}{2}, \\ p(1) &\approx \frac{Y(1,t)}{t} \approx \frac{t/2}{t} = \frac{1}{2}. \end{aligned}$$

For the rest $\lambda(n) = 0$, and $p(n) = 0$, for $n \geq 2$.

6. $D(0,t) = \sum_{k=1}^{\infty} \mathbb{1}_{D_k \leq t, L(D_k)=0}$. From the graph of $\{L(s)\}$ we see that all jobs leave an empty system behind. Thus, $D(0,t) \approx t/2$, and $D(n,t) = 0$ for $n \geq 1$. With this, $D(0,t)/Y(1,t) \sim (t/2)/(t/2) = 1$, and so,

$$\mu(1) = \lim_{t\to\infty} \frac{D(0,t)}{Y(1,t)} = 1,$$

and $\mu(n) = 0$ for $n \geq 2$.

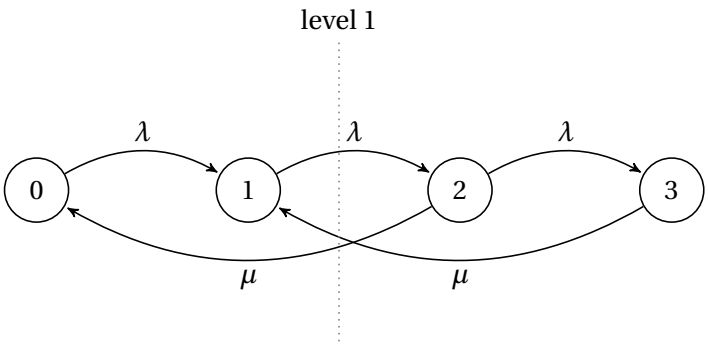
7. $\lambda(0)p(0) = 1 \cdot 1/2 = 1/2$, $\lambda(n)p(n) = 0$ for $n > 1$, as $\lambda(n) = 0$ for $n > 0$. Next, $\mu(1) = 1$, hence $\mu(1)p(1) = 1 \cdot 1/2 = 1/2$. Moreover, $\mu(n) = 0$ for $n \geq 2$. Clearly, for all n we have $\lambda(n)p(n) = \mu(n+1)p(n+1)$.

s.4.7.2. Noting that $L(s) = \sum_{n=0}^{\infty} n\, \mathbb{1}_{L(s)=n}$, we see that

$$\frac{1}{t}\int_0^t L(s)\,\mathrm{d}s = \frac{1}{t}\int_0^t \left(\sum_{n=0}^{\infty} n\, \mathbb{1}_{L(s)=n}\right) \mathrm{d}s = \sum_{n=0}^{\infty} \frac{n}{t}\int_0^t \mathbb{1}_{L(s)=n}\,\mathrm{d}s.$$

Now apply (4.6.1a).

s.4.7.3. $\rho = \sum_{i=1}^{\infty} p(i) = \sum_{i=1}^{\infty} \pi(i) = \sum_{i=1}^{\infty} \delta(i)$.



s.4.7.4.

With level-crossing:

$$\begin{aligned}
\lambda p(0) &= \mu p(2), \quad \text{the level between 0 and 1,}\\
\lambda p(1) &= \mu p(2) + \mu p(3), \quad \text{see level 1,}\\
\lambda p(2) &= \mu p(3), \quad \text{the level between 2 and 3.}
\end{aligned}$$

Solving this in terms of $p(0)$ gives $p(2) = \rho p(0)$, $p(3) = \rho p(2) = \rho^2 p(0)$, and

$$\lambda p(1) = \mu(p(2) + p(3)) = \mu(\rho + \rho^2)p(0) = (\lambda + \lambda^2/\mu)p(0),$$

hence $p(1) = p(0)(\mu + \lambda)/\mu$.

s.4.7.5.

$$\lim_{t\to\infty} \frac{I(n,t)}{t} = \lim_{t\to\infty} \frac{A(n-1,t)}{t} + \lim_{t\to\infty} \frac{D(n,t)}{t} = \lambda(n-1)p(n-1) + \mu(n+1)p(n+1)$$

and

$$\lim_{t\to\infty} \frac{O(n,t)}{t} = \lim_{t\to\infty} \frac{A(n,t)}{t} + \lim_{t\to\infty} \frac{D(n-1,t)}{t} = \lambda(n)p(n) + \mu(n)p(n)$$

s.5.1.1. The fraction of time the system contains n jobs is $\pi(n)$ (by PASTA). When the system contains $n > 0$ jobs, the number in queue is one less, i.e., $n-1$.

$$\mathsf{E}[Q] = \sum_{n=1}^{\infty} (n-1)\pi(n) = \sum_{n=1}^{\infty} n\pi(n) - \sum_{n=1}^{\infty} \pi(n) = \mathsf{E}[L] - (1 - \pi(0)) = \mathsf{E}[L] - \rho.$$

s.5.1.2.

$$M_L(s) = \mathsf{E}\left[e^{sL}\right] = \sum_{n=0}^{\infty} e^{sn} p(n) = (1-\rho)\sum_n e^{sn}\rho^n = \frac{1-\rho}{1-e^s\rho},$$

where we assume that s is such that $e^s\rho < 1$. Then,

$$\begin{aligned}
M_L'(s) &= (1-\rho)\frac{1}{(1-e^s\rho)^2}e^s\rho \implies \mathsf{E}[L] = M_L'(0) = \frac{\rho}{1-\rho},\\
\mathsf{E}\left[L^2\right] &= M''(0) = \frac{2\rho^2}{(1-\rho)^2} + \frac{\rho}{1-\rho},\\
\mathsf{V}[L] &= \mathsf{E}\left[L^2\right] - (\mathsf{E}[L])^2 = \frac{\rho(1+\rho)}{(1-\rho)^2} - \frac{\rho^2}{(1-\rho)^2} = \frac{\rho}{(1-\rho)^2},\\
\mathsf{P}\{L \geq n\} &= \sum_{k=n}^{\infty} p(k) = \sum_{k=n}^{\infty} p(0)\rho^k = (1-\rho)\sum_{k=n}^{\infty}\rho^k = (1-\rho)\rho^n\sum_{k=0}^{\infty}\rho^k = (1-\rho)\rho^n\frac{1}{1-\rho} = \rho^n.
\end{aligned}$$

s.5.1.3. Take $c = 1$

$$\begin{aligned}
p(0) &= \frac{1}{G}\frac{(c\rho)^0}{0!} = \frac{1}{G},\\
p(n) &= \frac{1}{G}\frac{c^c\rho^n}{c!} = \frac{1}{G}\frac{1^1\rho^n}{1!} = \frac{\rho^n}{G}, \quad n = 1,2,\ldots\\
G &= \sum_{n=0}^{c-1}\frac{(c\rho)^n}{n!} + \frac{(c\rho)^c}{(1-\rho)c!} = \sum_{n=0}^{0}\frac{\rho^0}{0!} + \frac{\rho}{(1-\rho)} = 1 + \frac{\rho}{1-\rho} = \frac{1}{1-\rho},\\
\mathsf{E}[L] &= \frac{(c\rho)^c}{c!G}\frac{\rho}{(1-\rho)^2} = \frac{\rho}{1/(1-\rho)}\frac{\rho}{(1-\rho)^2} = \frac{\rho^2}{1-\rho},\\
\mathsf{E}[L] &= \sum_{n=0}^{c} np(n) + \sum_{n=c+1}^{\infty} cp(n) = p(1) + 1\sum_{n=2}^{\infty} p(n) = 1 - p(0) = \rho.
\end{aligned}$$

Everything is in accordance to the formulas we derived earlier for the $M/M/1$ queue.

s.5.1.4. With the other results for the $M/M/1/K$ queue,

$$1-p(0) = 1-\frac{1-\rho}{1-\rho^{K+1}} = \frac{1-\rho^{K+1}-1+\rho}{1-\rho^{K+1}} = \rho\frac{1-\rho^K}{1-\rho^{K+1}},$$
$$\rho(1-p(K)) = \rho\frac{1-\rho^{K+1}-\rho^K+\rho^{K+1}}{1-\rho^{K+1}} = \rho\frac{1-\rho^K}{1-\rho^{K+1}}.$$

s.5.1.5. To take the limit $K\to\infty$—mind, not the limit $n\to\infty$—, write

$$G = \frac{1-\rho^{K+1}}{1-\rho} = \frac{1}{1-\rho} - \frac{\rho^{K+1}}{1-\rho}.$$

Since $\rho^{K+1}\to 0$ as $K\to\infty$ (recall, $\rho<1$), we get

$$G \to \frac{1}{1-\rho},$$

as $K\to\infty$. Therefore, $p(n) = \rho^n/G \to \rho^n(1-\rho)$, and the latter are the steady-state probabilities of the $M/M/1$ queue. Finally, if the steady-state probabilities are the same, the performance measures (which are derived from $p(n)$) must be the same.

s.5.1.6. For $n\geq 1$,

$$\begin{aligned}
p(n) &= \frac{\lambda(n-1)}{\mu(n)}p(n-1) = \frac{\lambda}{\min\{c,n\}\mu}p(n-1) = \frac{1}{\min\{c,n\}}(c\rho)p(n-1)\\
&= \frac{1}{\min\{c,n\}\min\{c,n-1\}}(c\rho)^2 p(n-2)\\
&= \frac{1}{\prod_{k=1}^n \min\{c,k\}}(c\rho)^n p(0).
\end{aligned}$$

To obtain the normalization constant $G = 1/p(0)$,

$$\begin{aligned}
1 &= \sum_{n=0}^{\infty} p(n) = p(0) + \sum_{n=1}^{c-1} p(n) + \sum_{n=c}^{\infty} p(n)\\
&= p(0) + p(0)\sum_{n=1}^{c-1}\frac{(c\rho)^n}{n!} + p(0)\sum_{n=c}^{\infty}\frac{c^c}{c!}\rho^n\\
&= p(0)\sum_{n=0}^{c-1}\frac{(c\rho)^n}{n!} + p(0)\sum_{n=c}^{\infty}\frac{(c\rho)^c}{c!}\rho^{n-c}\\
&= p(0)\sum_{n=0}^{c-1}\frac{(c\rho)^n}{n!} + p(0)\frac{(c\rho)^c}{c!}\sum_{n=0}^{\infty}\rho^n\\
&= p(0)\sum_{n=0}^{c-1}\frac{(c\rho)^n}{n!} + p(0)\frac{(c\rho)^c}{c!(1-\rho)}.
\end{aligned}$$

Next, for $n\geq c$, note that $p(n) = (c\rho)^n/(c^{n-c}c!) = c^c\rho^n/c!$, so that

$$\begin{aligned}
\mathsf{E}[Q] &= \sum_{n=c}^{\infty}(n-c)p(n) = \sum_{n=c}^{\infty}(n-c)\frac{c^c}{c!}\rho^n p(0)\\
&= \frac{c^c\rho^c}{Gc!}\sum_{n=c}^{\infty}(n-c)\rho^{n-c} = \frac{c^c\rho^c}{Gc!}\sum_{n=0}^{\infty}n\rho^n\\
&= \frac{c^c\rho^c}{Gc!}\frac{\rho}{(1-\rho)^2}.
\end{aligned}$$

The derivation of the expected number of jobs in service becomes easier if we pre-multiply the normalization constant G:

$$\begin{aligned}
G\mathsf{E}[L_s] &= G\left(\sum_{n=0}^{c} np(n) + \sum_{n=c+1}^{\infty} cp(n)\right) \\
&= \sum_{n=1}^{c} n\frac{(c\rho)^n}{n!} + \sum_{n=c+1}^{\infty} c\frac{c^c\rho^n}{c!} = \sum_{n=1}^{c}\frac{(c\rho)^n}{(n-1)!} + \frac{c^{c+1}}{c!}\sum_{n=c+1}^{\infty}\rho^n \\
&= \sum_{n=0}^{c-1}\frac{(c\rho)^{n+1}}{n!} + \frac{(c\rho)^{c+1}}{c!}\sum_{n=0}^{\infty}\rho^n = c\rho\left(\sum_{n=0}^{c-1}\frac{(c\rho)^n}{n!} + \frac{(c\rho)^c}{c!(1-\rho)}\right).
\end{aligned}$$

Observe that the RHS is precisely equal to ρcG, so that $\mathsf{E}[L_s] = c\rho$.

s.5.1.7. In the expressions for the $M/M/c$ queue, just neglect the parts that deal with states $n > c$. We see that $p(n) = p(0)(c\rho)^n/n!$. For the normalization $1 = \sum_{n=0}^{c} p(n) = p(0)\sum_{n=0}^{c}\frac{(c\rho)^n}{n!}$. Next,

$$\begin{aligned}
\mathsf{E}[L_s] &= \sum_{n=0}^{c} np(n) = \sum_{n=1}^{c} np(n) \\
&= G^{-1}\sum_{n=1}^{c} n\frac{(\lambda/\mu)^n}{n!} = G^{-1}\sum_{n=1}^{c}\frac{(\lambda/\mu)^n}{(n-1)!} \\
&= \frac{\lambda}{\mu G}\sum_{n=0}^{c-1}\frac{(\lambda/\mu)^n}{n!} = \frac{\lambda}{G\mu}\left(G - \frac{(\lambda/\mu)^c}{c!}\right) \\
&= \frac{\lambda}{\mu}\left(1 - \frac{1}{G}\frac{(\lambda/\mu)^c}{c!}\right) \\
&= \frac{\lambda}{\mu}\big(1 - p(c)\big).
\end{aligned}$$

s.5.1.8. By taking the limit $c \to \infty$, note first that in (5.1.4c),

$$\frac{(c\rho)^c}{(1-\rho)c!} = \frac{(\lambda/\mu)^c}{(1-\rho)c!} \to 0, \quad \text{as } c \to \infty.$$

Hence

$$G = \sum_{n=0}^{c-1}\frac{(c\rho)^n}{n!} + \frac{(c\rho)^c}{(1-\rho)c!} \to \sum_{n=0}^{\infty}\frac{(c\rho)^n}{n!} = e^{\lambda/\mu}.$$

Next, for any fixed n, eventually $c > n$, and then, as $\rho = \lambda/(\mu c)$,

$$p(n) = \frac{1}{G}\frac{(c\rho)^n}{n!} = \frac{1}{G}\frac{(\lambda/\mu)^n}{n!} \to e^{-\lambda/\mu}\frac{(\lambda/\mu)^n}{n!}, \quad \text{as } c \to \infty.$$

s.5.1.9.

$$\begin{aligned}
p(n+1) &= \frac{(N-n)\lambda}{\mu}p(n) = \rho(N-n)p(n) \\
&= \rho^2(N-n)(N-(n-1))p(n-1) \\
&= \rho^3(N-n)(N-(n-1))(N-(n-2))p(n-2) \\
&= \rho^{n+1}(N-n)(N-(n-1))\cdots(N-(0))p(0) \\
&= \rho^{n+1}\frac{N!}{(N-(n+1))!}p(0).
\end{aligned}$$

s.5.1.10. Take $\lambda(n) = (N-n)\lambda$ and $\mu(n) = n\mu$. Then

$$\begin{aligned} p(n+1) &= \frac{\lambda(n)}{\mu(n+1)} p(n) = \frac{(N-n)\lambda}{(n+1)\mu} p(n) = \frac{(N-n)(N-(n-1))}{(n+1)n} \frac{\lambda^2}{\mu^2} p(n-1) \\ &= \frac{N!}{(N-(n+1))!} \frac{1}{(n+1)!} \rho^{n+1} p(0) = \binom{N}{n+1} \rho^{n+1} p(0). \end{aligned}$$

Therefore, $G = \sum_{k=0}^{N} \rho^k \binom{N}{k}$.

s.5.2.1. As people arrive without making appointments, it is reasonable to say that the inter-arrival times are memoryless. As for the services, the employee deals with many different questions, some are short, some are long. So, we model service times as exponential too.

With Little's law:

```
>>> labda = 5. # per minute
>>> EW = 6.
>>> EQ = labda * EW
>>> EQ
30.0
```

We can use the expression for $\mathsf{E}[W]$ to solve for ρ, but we can also do a simple search.

```
>>> from scipy.optimize import bisect

>>> EW = 6.

>>> def find_W(rho):
...     # return W -1 for given rho
...     ES = rho / labda
...     return rho / (1 - rho) * ES - EW
...

>>> rho = bisect(find_W, 0, 0.999)
>>> rho
0.968719422671953
>>> ES = rho / labda
>>> ES
0.1937438845343906
>>> J = EW + ES
>>> J
6.19374388453439
>>> L = labda*J
>>> L
30.968719422671953
```

Next, find n such that $\sum_{j=1}^{n} p_j > 0.9$. We start counting at 1, because while the system contains one job, this customer is in service, hence stands at desk of the employee.

```
>>> n, p = 0, 1 - rho
>>> total = p
>>> while total <= 0.9:
```

```
...     p *= rho
...     total += p
...     n += 1
...
>>> total
0.9017224011105635
>>> n - 1
71
```

Note that the while loop breaks when n is one too large.

The average queue is huge: hire an extra employee.

s.5.2.2. This is an $M/M/1/3$ queue: there is room for 1 customer in service and two in queue.

We import `numpy` and convert the lists to arrays is to fix the output precision to 3, otherwise we get long floats in the output.

First the case with $b = 2$.

```
>>> import numpy as np
>>> np.set_printoptions(precision=3)

>>> labda, mu, c = 6.0,  5.0, 1
>>> rho = labda / mu
>>> K = c + 2


>>> p = np.array([rho ** n for n in range(K + 1)]) # range(n) is up to n
>>> G = sum(p)
>>> p /= G  # normalize
>>> p
array([0.186, 0.224, 0.268, 0.322])
>>> L = sum(n * p[n] for n in range(len(p)))
>>> L
1.725782414307004
>>> Q = sum(max(n - c,0) * p[n] for n in range(len(p)))
>>> Q
0.9120715350223545
>>> lost = labda * p[-1]  # the last element of p
>>> labda - lost # accepted, hence served
4.06855439642325
```

Now with three extra chairs.

```
>>> K += 3
>>> K
6
>>> p = np.array([rho ** n for n in range(K + 1)]) # range(n) is up to n
>>> G = sum(p)
>>> p /= G  # normalize
>>> lost = labda * p[-1]  # the last element of P
>>> labda - lost # accepted, hence served
4.612880368265357
```

We see that since the server is overloaded, the acceptance is not much affected by increasing the number of chairs. We need an extra server.

s.5.2.3. The computations.

```
>>> import numpy as np
>>> np.set_printoptions(precision=3)

>>> labda = np.array([10.0, 10.0, 20.0, 20.0, 5.0])
>>> mu = np.array([0., 30., 60., 60., 60., 60.])
>>> c = 2

>>> p = np.ones_like(mu)
>>> for i in range(len(labda)):
...     p[i+1] = labda[i] *p[i]/ mu[i+1]
...
>>> p /= p.sum()
>>> p
array([7.072e-01, 2.357e-01, 3.929e-02, 1.310e-02, 4.365e-03, 3.638e-04])
>>> labdaBar = sum(labda[n] * p[n] for n in range(len(labda)))
>>> labdaBar
10.498363041105856
>>> L = sum(n * p[n] for n in range(len(p)))
>>> L
0.37286285922153506
>>> Q = sum(max(n - c,0) * p[n] for n in range(len(p)))
>>> Q
0.022917424518006546
>>> J = L / labdaBar
>>> J
0.035516285516285516
>>> W = Q / labdaBar
>>> W
0.002182952182952183
```

s.5.2.4. From the hint, the rv L behaves like the number of jobs in an $M/M/1$ queue. If we have L, then with the two properties $TG = 0$ and $3 = T + L - G$, we can retrieve (uniquely at all moments in time) the number of pattys T on stock and the number of people waiting G. In inventory terms, G represents the number of *backlogged* pattys.

With this mapping, the expected number of pattys on stock is $\mathsf{E}[T] = \sum_{l=0}^{3}(3-l)p(l)$.

```
>>> labda = 12.0  # per hour
>>> mu = 15.0  # per hour
>>> rho = labda / mu
>>> max_pattys = 3

>>> ET, p = 0, 1 - rho
>>> for l in range(max_pattys):
...     ET += (max_pattys - l) * p
...     p *= rho
...
>>> ET
1.0479999999999998
```

For the expected number of people waiting, use that $3 = T - L + G$ always. Taking expectations, we see that $3 = \mathsf{E}[T] + \mathsf{E}[L] - \mathsf{E}[G]$, where $\mathsf{E}[G]$ is the expected number of people.

```
>>> EL = rho/(1-rho)
>>> EG = ET + EL - max_pattys
>>> EG
2.048000000000001
```

Computing the waiting times is tricky. For the pattys, the rate at which 'jobs' arrive, is the arrival rate of people. For the people, the rate at which 'jobs' arrive is the arrival rate of pattys.

```
>>> ET/labda # Waiting for pattys
0.08733333333333332
>>> EG/mu # waiting time for people
0.1365333333333334
```

What would be the impact of allowing 4 pattys maximally?

s.5.2.5. The *invariant* $3 = T + L - G$ becomes $4 = T + L - G$.

Estimating the impact of allowing maximally four pattys is simple with the code we already have.

```
>>> max_pattys = 4

>>> ET, p = 0, 1 - rho
>>> for l in range(max_pattys):
...     ET += (max_pattys - l) * p
...     p *= rho
...
>>> ET
1.6383999999999999
>>> EG = ET + EL - max_pattys
>>> EG
1.6384000000000007
```

s.5.2.6. The *invariant* $3 = T + L - G$ becomes $0 = T + L - G$. This implies that $T = 0$ and $L = G$. Hence $\mathsf{E}[G] = \mathsf{E}[L]$.

s.5.3.1. Start with the simple case. $B \equiv 2 \implies \mathsf{V}[B] = 0 \implies C_s^2 = 0$, $\rho = \lambda\,\mathsf{E}[B]\,\mathsf{E}[S] = 1{\cdot}2{\cdot}25/60 = 5/6$. Hence,

$$\mathsf{E}[L] = \frac{1}{2}\frac{5/6}{1/6}2 + \frac{1}{2}\frac{5/6}{1/6} = 5 + \frac{5}{2}.$$

Now the other case. $\mathsf{E}\left[B^2\right] = (1+4+9)/3 = 14/3$. Hence, $\mathsf{V}[B] = 14/3 - 4 = 2/3$. Hence, $C_s^2 = \frac{1}{6}$. And thus,

$$\mathsf{E}[L] = \frac{1+1/6}{2}\frac{5/6}{1/6}2 + \frac{1}{2}\frac{5/6}{1/6} = \frac{7}{6}5 + \frac{5}{2}.$$

The ratio between $\mathsf{E}[L]$ is $10/9$: the average waiting time can be reduced by about 10% by working in fixed batch sizes.

s.5.3.2. $B \equiv 1 \implies \mathsf{V}[B] = 0 \implies C_s^2 = 0$ and $\mathsf{E}[B] = 1$. Filling the formula for $\mathsf{E}[S]$ is now trivial.

s.5.3.3. Filling the formula for $\mathsf{E}[W]$,

$$\mathsf{E}[B] = \frac{1}{p}, \quad \mathsf{V}[B] = \frac{1}{p^2} - \frac{1}{p}, \quad C_s^2 = \frac{\mathsf{V}[B]}{(\mathsf{E}[B])^2} = p^2\left(\frac{1}{p^2} - \frac{1}{p}\right) = 1 - p, \quad (1 + C_s^2)/2 = 1 - p/2,$$

$$\mathsf{E}[L] = \left(1 - \frac{p}{2}\right)\frac{\rho}{1-\rho}\frac{1}{p} + \frac{1}{2}\frac{\rho}{1-\rho} = \frac{\rho}{1-\rho}\frac{1}{p}.$$

For the $M/M/1$ queue, $\mathsf{E}[B] = 1 \implies p = 1 \implies \mathsf{E}[L] = \rho/(1-\rho)$.

If you can't find the expectation and variance of a geometric rv, then here is the derivation.

$$\begin{aligned}
M_B(s) &= \mathsf{E}\left[e^{sB}\right] = \sum_{k=0}^{\infty} e^{sk}\,\mathsf{P}\{B = k\} \\
&= \sum_{k=0}^{\infty} e^{sk} p q^{k-1} = \frac{p}{q}\sum_{k=0}^{\infty}(qe^s)^k = \frac{p}{q}\frac{1}{1-qe^s}, \\
\mathsf{E}[B] &= M_B'(0) = \frac{p}{q}\frac{q}{(1-qe^s)^2}\bigg|_{s=0} = \frac{p}{(1-q)^2} = \frac{1}{p}, \\
\mathsf{E}\left[B^2\right] &= M_B''(0) = \frac{2}{p^2} - \frac{1}{p}, \\
\mathsf{V}[B] &= \mathsf{E}\left[B^2\right] - (\mathsf{E}[B])^2 = \frac{2}{p^2} - \frac{1}{p} - \frac{1}{p^2} = \frac{1}{p^2} - \frac{1}{p},
\end{aligned}$$

s.5.3.4. For 1, observe first that $\sum_{k=0}^{\infty} \mathbb{1}_{m>k} = m$, since $\mathbb{1}_{m>k} = 1$ if $k < m$ and $\mathbb{1}_{m>k} = 0$ if $k \geq m$. With this,

$$\begin{aligned}
\sum_{k=0}^{\infty} G(k) &= \sum_{k=0}^{\infty}\mathsf{P}\{X > k\} = \sum_{k=0}^{\infty}\sum_{m=k+1}^{\infty}\mathsf{P}\{X = m\} \\
&= \sum_{k=0}^{\infty}\sum_{m=0}^{\infty}\mathbb{1}_{m>k}\,\mathsf{P}\{X = m\} = \sum_{m=0}^{\infty}\sum_{k=0}^{\infty}\mathbb{1}_{m>k}\,\mathsf{P}\{X = m\} \\
&= \sum_{m=0}^{\infty} m\,\mathsf{P}\{X = m\} = \mathsf{E}[X].
\end{aligned}$$

There are some technical details with respect to the interchange of the summations. Since the summands are positive, this is allowed.

For 2,

$$\begin{aligned}
\sum_{i=0}^{\infty} iG(i) &= \sum_{i=0}^{\infty} i \sum_{n=i+1}^{\infty}\mathsf{P}\{X = n\} = \sum_{n=0}^{\infty}\mathsf{P}\{X = n\}\sum_{i=0}^{\infty} i\,\mathbb{1}_{n\geq i+1} \\
&= \sum_{n=0}^{\infty}\mathsf{P}\{X = n\}\sum_{i=0}^{n-1} i = \sum_{n=0}^{\infty}\mathsf{P}\{X = n\}\frac{(n-1)n}{2} \\
&= \sum_{n=0}^{\infty}\frac{n^2}{2}\mathsf{P}\{X = n\} - \frac{\mathsf{E}[X]}{2} = \frac{\mathsf{E}\left[X^2\right]}{2} - \frac{\mathsf{E}[X]}{2}.
\end{aligned}$$

s.5.3.5.

$$\sum_{i=1}^{\infty} iG(i-1) = \sum_{i=0}^{\infty}(i+1)G(i) = \sum_{i=0}^{\infty} iG(i) + \sum_{i=0}^{\infty} G(i) = (\mathsf{E}\left[B^2\right] - \mathsf{E}[B])/2 + \mathsf{E}[B].$$

s.5.4.1. $\rho = \lambda \mathsf{E}[S] = (3/8)\cdot 2 = 3/4$, $\mathsf{E}[W] = 4.5$ h. If we were able to reduce all service variability, i.e., $C_s^2 = 0$, then still $\mathsf{E}[W] = 3$h. Hence, we have to increase capacity, or reduce $\mathsf{E}[S]$. Another possibility is to plan the arrival of jobs such that $C_a^2 = 0$. However, typically this is not possible. Would you accept that the supermarket plans your visits?

s.5.4.2. The probability to find the server busy upon arrival is ρ, and only jobs that find the server occupied see a positive remaining service time $S_r > 0$.

$$\mathsf{E}[S_r] = \rho\,\mathsf{E}[S_r \mid S_r > 0] + (1-\rho)\,\mathsf{E}[S_r \mid S_r = 0] = \rho\,\mathsf{E}[S_r \mid S_r > 0].$$

For the $M/M/1$, service times are memoryless, hence, $\mathsf{E}[S_r \mid S_r > 0] = \mathsf{E}[S]$, and this implies that $\mathsf{E}[S_r] = \rho\,\mathsf{E}[S]$ for the $M/M/1$ queue.

s.5.4.3. $\mathsf{V}[S] = 0 \implies C_s^2 = 0 \implies \mathsf{E}[W_{M/D/1}] = \mathsf{E}[W_{M/M/1}]/2$.

s.5.4.4.

$$\begin{aligned}
\mathsf{E}[S] &= \alpha/2, & \mathsf{E}\left[S^2\right] &= \int_0^\alpha x^2\,\mathrm{d}x/\alpha = \alpha^2/3,\\
\mathsf{V}[S] &= \alpha^2/3 - \alpha^2/4 = \alpha^2/12, & C_s^2 &= (\alpha^2/12)/(\alpha^2/4) = 1/3,\\
\rho &= \lambda\alpha/2, & & \\
\mathsf{E}[W] &= \frac{1+C_s^2}{2}\,\frac{\lambda\alpha/2}{1-\lambda\alpha/2}\,\frac{\alpha}{2}, & \mathsf{E}[J] &= \mathsf{E}[W] + \frac{\alpha}{2}.
\end{aligned}$$

s.5.4.5.

$$\lambda\pi(0)\,\mathsf{E}[S] = 1-\pi(0) \iff \pi(0) = \frac{1}{1+\lambda\,\mathsf{E}[S]} \iff 1-\pi(0) = \frac{\lambda\,\mathsf{E}[S]}{1+\lambda\,\mathsf{E}[S]}.$$

There is another way to derive this. The system can contain at most 1 job. Necessarily, if the system contains a job, this job must be in service. All jobs that arrive while the server is busy are rejected. Just after a departure, the average time until the next arrival is $1/\lambda$, and then a new service starts with an average duration of $\mathsf{E}[S]$. After this departure, a new cycle starts. Thus, $\rho = \mathsf{E}[S]/(1/\lambda + \mathsf{E}[S]) = \lambda\,\mathsf{E}[S]/(1+\lambda\,\mathsf{E}[S])$.

For the $G/G/1/1$ queue the PASTA property does not hold, hence $p(0) \neq \pi(0)$ in general.

s.5.5.1. In the $M/M/1$ queue, $G(0) = 1$ and $G(1) = G(2) = \cdots = 0$. Thus, $\sum_{m=0}^{n} G(n-m)\pi(m) = G(0)\pi(n) = \pi(n)$.

s.5.5.2. The complete-acceptance policy is actually quite simple. As any batch will be accepted when $n \le K$, the queue length is not bounded. Only when the number of items in the system is larger than K, we do not accept jobs.

$$\mu\pi(n+1) = \begin{cases} \lambda\sum_{m=0}^{n}\pi(m)G(n-m), & \text{for } n \le K,\\ \lambda\sum_{m=0}^{K}\pi(m)G(n-m), & \text{for } n > K.\end{cases}$$

s.5.5.3. For the partial acceptance case, any job is accepted, but the system only admits whatever fits. As level $n \in 0,1,\ldots,K-1$ is still up-crossed by any batch of size at least $n-m$ when the system is in state m, the formula for the up-crossing rate is identical to the case without this acceptance policy. Hence, $\mu\pi(n+1) = \lambda\sum_{m=0}^{n}\pi(m)G(n-m)$, for $n = 0,1,\ldots,K-1$.

s.5.5.4. Suppose a batch of size k arrives when the system contains m items. When $m+k \le K$, the batch can be accepted since the entire batch will fit into the queue, otherwise it will be rejected. Further, level $n, 0 \le n < K$, can only be crossed when $m+k > n$. Thus, for $n = 0, \ldots, K-1$,

$$\begin{aligned}\mu\pi(n+1) &= \lambda \sum_{m=0}^{n} \pi(m)\,\mathsf{P}\{n-m < B \le K-m\}\\ &= \lambda \sum_{m=0}^{n} \pi(m)[G(n-m) - G(K-m)],\end{aligned}$$

Let us check this for some simple cases. First, when $K \to \infty$, then $G(K-m) \to 0$, so we get our earlier result. Second, take $n = K$. Then $G(n-m) - G(K-m) = 0$ for all m, so that the RHS is 0, as it should. Third, take $n = 0$ and $K = 1$, then $\mu\pi(1) = \lambda\pi(0)$. This also makes sense.

s.5.5.5. We use that $\mu\pi(n) = \lambda \sum_{i=0}^{n-1} \pi(i)G(n-1-i)$ and the results of the exercises of Section 5.3 to see that

$$\begin{aligned}\mu\,\mathsf{E}[L] &= \sum_{n=0}^{\infty} n\,\mu\pi(n), \quad \text{now substitute for } \mu\pi(n) \text{ the recursion (5.5.1)},\\ &= \lambda \sum_{n=0}^{\infty} n \sum_{i=0}^{n-1} \pi(i)G(n-1-i) = \lambda \sum_{n=0}^{\infty} n \sum_{i=0}^{\infty} \mathbb{1}_{i<n}\pi(i)G(n-1-i)\\ &= \lambda \sum_{i=0}^{\infty} \pi(i) \sum_{n=0}^{\infty} \mathbb{1}_{i<n} nG(n-1-i) = \lambda \sum_{i=0}^{\infty} \pi(i) \sum_{n=i+1}^{\infty} nG(n-1-i)\\ &= \lambda \sum_{i=0}^{\infty} \pi(i) \sum_{n=0}^{\infty} (n+i+1)G(n) = \lambda \sum_{i=0}^{\infty} \pi(i) \left[\sum_{n=0}^{\infty} nG(n) + (i+1)\sum_{n=0}^{\infty} G(n)\right]\\ &= \lambda \sum_{i=0}^{\infty} \pi(i) \sum_{n=0}^{\infty} nG(n) + \lambda\,\mathsf{E}[B] \sum_{i=0}^{\infty} \pi(i)(i+1)\\ &= \lambda \sum_{i=0}^{\infty} \pi(i) \frac{\mathsf{E}\left[B^2\right] - \mathsf{E}[B]}{2} + \lambda\,\mathsf{E}[B]\,(\mathsf{E}[L]+1)\\ &= \lambda \frac{\mathsf{E}\left[B^2\right] - \mathsf{E}[B]}{2} + \lambda\,\mathsf{E}[B]\,\mathsf{E}[L] + \lambda\,\mathsf{E}[B]\\ &= \lambda \frac{\mathsf{E}\left[B^2\right]}{2} + \lambda\,\mathsf{E}[B]\,\mathsf{E}[L] + \lambda \frac{\mathsf{E}[B]}{2}.\end{aligned}$$

Dividing both sides by μ and using that $\lambda\,\mathsf{E}[B]/\mu = \rho$, we obtain

$$EL = \lambda \frac{\mathsf{E}\left[B^2\right]}{2}\,\mathsf{E}[S] + \rho\,\mathsf{E}[L] + \frac{\rho}{2}.$$

By bringing $\rho\,\mathsf{E}[L]$ to the LHS, the RHS becomes equal to (5.3.2).

s.5.6.1. When we switch on the server, the queue 'drains' at rate $\mu - \lambda > 0$, with $\mu = 1/\mathsf{E}[S]$. Consequently, no matter how large N, $T(N) < \infty$. And, whenever the system is empty, the stochastic process restarts. As such cycles start over and over again, and the queue length can never 'escape to infinity'.

s.5.6.2. The total number $A(t)$ of job that arrive during $[0, t]$ does not depend on N. Thus, in (4.4.2), $\sum_{k=1}^{A(t)} S_k$ does not depend on N. Now use rate-stability.

s.5.6.3. The cost up to the qth job is the cost $W(q-1)$ up to the arrival of job $q-1$ plus the cost while there are $q-1$ jobs in the system. The time between the arrival of job $q-1$ and q is $1/\lambda$. When there are $q-1$ jobs in the system, the expected cost is therefore $h(q-1)/\lambda$ during the arrival of job $q-1$ and job q. And thus, $W(q) = h\sum_{i=1}^{q}(i-1)$.

s.5.6.4. Consider an arbitrary moment in time at which $q > 0$ and the server is busy. Now either of two events happens first: a new job enters the system, or the job in service leaves. The probability of an arrival to occur first is $\alpha = \lambda/(\lambda+\mu)$,[0] the probability of a departure to occur first is $\beta = 1-\alpha = \mu/(\lambda+\mu)$. Moreover, the expected time to either an arrival or a departure, whichever is first, is $1/(\mu+\lambda)$.[1] In words, the system stays in state q for an expected time $1/(\lambda+\mu)$ until an arrival or departure occurs. Then, it moves to state $q+1$ or $q-1$, and from there it takes $T(q+1)$ or $T(q-1)$ until the system is empty. Observe that this reasoning depends crucially on the memoryless property.

[0] [1.2.6]

[1] [1.2.4]

s.5.6.5. Note that the queueing cost is hq per unit time when there are q jobs in the system, it costs $hq/(\lambda+\mu)$ until an arrival or departure occurs. For the rest we follow the reasoning by which we derive the recursion for $T(q)$ for the $M/M/1$ queue.

s.5.6.6. Consider a cycle that results under the $N = 1$ policy. A cycle starts when the server becomes empty. Then we wait until a job arrives, the server switches on (since $N = 1$), and a busy period U starts. Once the system becomes empty again, the server switches off again, and the cycle is complete. The $C(1)$ as defined in the problem description is therefore the expected duration of the cycle.

The cost of the jobs in the system during one cycle must be $V(1)$. By the renewal-reward theorem, the time-average cost is then $V(1)/C(1)$.

If the time-average number of jobs in the system is $\mathsf{E}[L]$, and each job pays h per unit time, the time-average cost must be $h\mathsf{E}[L]$.

The result follows.

s.5.6.7.

$$\begin{aligned}\frac{V(1)}{1/\lambda+T(1)} &= \frac{a+b}{1/\lambda+1/(\mu-\lambda)} = \left(\frac{h}{2}\frac{1}{\mu-\lambda}+\frac{h}{2}\frac{\lambda+\mu}{(\mu-\lambda)^2}\right)\frac{\lambda(\mu-\lambda)}{\mu}\\ &= \frac{h}{2}\rho\left(1+\frac{\lambda+\mu}{\mu-\lambda}\right) = \frac{h}{2}\rho\frac{2\mu}{\mu-\lambda} = h\frac{\rho}{1-\rho}.\end{aligned}$$

s.5.6.8. In the hint, the first equation is superfluous. In the second, bq cancels at both sides, by which we find a. The third now follows.

s.5.6.9. For b, using the expressions for $\mathsf{E}[Y]$ and $\mathsf{E}\left[Y^2\right]$,

$$\begin{aligned}
b(1-\mathsf{E}[Y]) &= a(\mathsf{E}\left[Y^2\right]-2\mathsf{E}[Y]+1)+\frac{1}{2}h\lambda\mathsf{E}\left[S^2\right] \\
&= \frac{h\mathsf{E}[S]}{2(1-\mathsf{E}[Y])}(\mathsf{E}\left[Y^2\right]-2\mathsf{E}[Y]+1)+\frac{1}{2}h\lambda\mathsf{E}\left[S^2\right] \\
&= \frac{h\mathsf{E}[S]}{2(1-\lambda\mathsf{E}[S])}\left(\lambda^2\mathsf{E}\left[S^2\right]+\lambda\mathsf{E}[S]-2\lambda\mathsf{E}[S]+1+\lambda\mathsf{E}\left[S^2\right]\frac{1-\lambda\mathsf{E}[S]}{\mathsf{E}[S]}\right) \\
&= \frac{h\mathsf{E}[S]}{2(1-\lambda\mathsf{E}[S])}\left(\lambda^2\mathsf{E}\left[S^2\right]-\lambda\mathsf{E}[S]+1+\frac{\lambda\mathsf{E}\left[S^2\right]}{\mathsf{E}[S]}-\lambda^2\mathsf{E}\left[S^2\right]\right) \\
&= \frac{h\mathsf{E}[S]}{2(1-\lambda\mathsf{E}[S])}\left(1+\frac{\lambda\mathsf{E}\left[S^2\right]}{\mathsf{E}[S]}-\lambda\mathsf{E}[S]\right) \\
&= \frac{h\mathsf{E}[S]}{2(1-\lambda\mathsf{E}[S])}\left(1+\frac{\lambda\mathsf{E}\left[S^2\right]}{\mathsf{E}[S]}-\lambda\frac{(\mathsf{E}[S])^2}{\mathsf{E}[S]}\right) \\
&= \frac{h\mathsf{E}[S]}{2(1-\lambda\mathsf{E}[S])}\left(1+\lambda\frac{\mathsf{V}[S]}{\mathsf{E}[S]}\right) \\
&= \frac{h\mathsf{E}[S]}{2(1-\lambda\mathsf{E}[S])}\left(1+\lambda\frac{\mathsf{V}[S]}{(\mathsf{E}[S])^2}\mathsf{E}[S]\right) \\
&= \frac{h\mathsf{E}[S]}{2(1-\lambda\mathsf{E}[S])}\left(1+\rho C_s^2\right).
\end{aligned}$$

Divide now both sides by $1-\mathsf{E}[Y]$.

s.5.6.10. For a, multiply the numerator and denominator by $\mu = 1/\mathsf{E}[S]$. For b, multiply by $\mu^2 = 1/(\mathsf{E}[S])^2$, use that $C_s^1 = 1$ because the service times are exponentially distributed, and note that

$$\frac{1+\rho}{1-\rho} = \frac{\mu+\lambda}{\mu-\lambda}.$$

s.5.6.11. $h\mathsf{E}[L] = h\mathsf{E}[Q] + h\mathsf{E}[L_s]$. $\mathsf{E}[Q] = \lambda\mathsf{E}[W]$ and $\mathsf{E}[L_s] = \lambda\mathsf{E}[S]$.

s.5.6.12. Note first that $C(N) = N(1/\lambda + \mathsf{E}[S]/(1-\rho)) = N/(\lambda(1-\rho))$. Then,

$$\begin{aligned}
\frac{V(N)+K+W(N)}{C(N)} &= \left(aN^2+bN+K+hN(N-1)/2\lambda\right)\frac{\lambda(1-\rho)}{N} \\
&= \frac{h}{2}\rho N+\frac{h}{2}\frac{\rho}{1-\rho}(1+\rho C_s^2)+\frac{h}{2}(N-1)(1-\rho)+K\frac{\lambda(1-\rho)}{N} \\
&= \frac{h}{2}\frac{\rho}{1-\rho}(1+\rho C_s^2)+\frac{h}{2}(N-1+\rho)+K\frac{\lambda(1-\rho)}{N} \\
&= \frac{h}{2}\frac{\rho}{1-\rho}(\rho+\rho C_s^2+1-\rho)+\frac{h}{2}(N-1+\rho)+K\frac{\lambda(1-\rho)}{N} \\
&= \frac{h}{2}\frac{\rho^2}{1-\rho}(1+C_s^2)+\frac{h}{2}\rho+\frac{h}{2}(N-1+\rho)+K\frac{\lambda(1-\rho)}{N} \\
&= \frac{h}{2}\frac{\rho^2}{1-\rho}(1+C_s^2)+h\rho+h\frac{N-1}{2}+K\frac{\lambda(1-\rho)}{N}.
\end{aligned}$$

s.5.7.1. In continuous time, when $I_t = 0$, the ith demand 'sees' an empty inventory. Hence, the ith demand cannot be met from on-hand stock when $I_t = 0$. Thus, only when $I_t > 0$, this demand can be served. Therefore,

$$\begin{aligned}\alpha(s) &= \mathsf{P}\{I_t > 0\}\\ &= \mathsf{P}\{r+1-D[i-L,i] > 0\}\\ &= \mathsf{P}\{D[i-L,i] < r+1\}\\ &= \mathsf{P}\{D[i-L,i] \le r\}\\ &= F(r).\end{aligned}$$

s.6.1.1. When processing times at a station are nearly constant, and the jobs of this station are sent to a second station for further processing, the inter-arrival times at the second station must be roughly equal. But then the inter-arrival times are not well approximated by the exponential distribution, consequently, the arrival process is not well described by a Poisson process.

s.6.1.2. $\rho = \lambda\,\mathsf{E}[S] = 1/60 \cdot 50 = 5/6$. Since job arrivals do not overlap any job service, the number of jobs in the system is 1 for 50 seconds, then the server is idle for 10 seconds, and so on. Thus $\mathsf{E}[L] = 1 \cdot 5/6 = 5/6$. There is no variance in the inter-arrival times, and also not in the service times, thus $C_a^2 = C_s^2 = 0$. Also $\mathsf{E}[W] = 0$ since $\mathsf{E}[Q] = 0$.

s.6.1.3. Again $\mathsf{E}[S]$ is 50 seconds, so that $\rho = 5/6$. Also $C_a^2 = 0$. For the C_s^2 we have to do some work.

$$\begin{aligned}\mathsf{E}[S] &= \frac{20}{2} + \frac{80}{2} = 50\\ \mathsf{E}\left[S^2\right] &= \frac{400}{2} + \frac{6400}{2} = 3400\\ \mathsf{V}[S] &= \mathsf{E}\left[S^2\right] - (\mathsf{E}[S])^2 = 3400 - 2500 = 900\\ C_s^2 &= \frac{\mathsf{V}[S]}{(\mathsf{E}[S])^2} = \frac{900}{2500} = \frac{9}{25}.\end{aligned}$$

s.6.1.4. First the $G/G/1$ case. Observe that in this case, the inter-arrival time $X \sim U[3,9]$, that is, never smaller than 3 minutes, and never longer than 9 minutes.

```
>>> a = 3.0
>>> b = 9.0
>>> EX = (b + a) / 2.0  # expected inter-arrival time
>>> EX
6.0
>>> labda = 1.0 / EX  # per minute
>>> labda
0.16666666666666666
>>> VA = (b - a) * (b - a) / 12.0
>>> CA2 = VA / (EX * EX)
>>> CA2
0.08333333333333333

>>> ES = 5.0
>>> VS = 4
>>> CS2 = VS / (ES * ES)
```

```
>>> CS2
0.16

>>> rho = labda * ES
>>> rho
0.8333333333333333

>>> W = (CA2 + CS2) / 2.0 * rho / (1.0 - rho) * ES
>>> W
3.041666666666665
```

Now the $M/G/1$ case. In that case $C_a^2 = 1$.

```
>>> W = (1.0 + CS2) / 2.0 * rho / (1.0 - rho) * ES
>>> W
14.499999999999993
```

The arrival process with uniform inter-arrival times is much more regular than a Poisson process. In the first case, tool arrivals are spaced in time at least with 3 minutes.

s.6.1.5. With the hint,

$$\begin{aligned}
\mathsf{E}[S] &= \mathsf{E}[\mathbb{1}_{T=1} S_1] + \mathsf{E}[\mathbb{1}_{T=2} S_2] \\
&= \mathsf{E}[\mathbb{1}_{T=1}]\,\mathsf{E}[S_1] + \mathsf{E}[\mathbb{1}_{T=2}]\,\mathsf{E}[S_2], \quad \text{by the independence of } T, \\
&= \mathsf{P}\{T=1\}\,\mathsf{E}[S_1] + \mathsf{P}\{T=2\}\,\mathsf{E}[S_2] \\
&= p\,\mathsf{E}[S_1] + q\,\mathsf{E}[S_2].
\end{aligned}$$

For the variance, we need some algebra. Since,

$$\mathbb{1}_{T=1}\,\mathbb{1}_{T=2} = 0 \text{ and } \mathbb{1}_{T=1}^2 = \mathbb{1}_{T=1},$$

we get

$$\begin{aligned}
\mathsf{V}[S] &= \mathsf{E}\left[S^2\right] - (\mathsf{E}[S])^2 \\
&= \mathsf{E}\left[(\mathbb{1}_{T=1} S_1 + \mathbb{1}_{T=2} S_2)^2\right] - (\mathsf{E}[S])^2 \\
&= \mathsf{E}\left[\mathbb{1}_{T=1} S_1^2 + \mathbb{1}_{T=2} S_2^2\right] - (\mathsf{E}[S])^2 \\
&= p\,\mathsf{E}\left[S_1^2\right] + q\,\mathsf{E}\left[S_2^2\right] - (\mathsf{E}[S])^2 \\
&= p\,\mathsf{V}[S_1] + p(\mathsf{E}[S_1])^2 + q\,\mathsf{V}[S_2] + q(\mathsf{E}[S_2])^2 - (\mathsf{E}[S])^2 \\
&= p\,\mathsf{V}[S_1] + p(\mathsf{E}[S_1])^2 + q\,\mathsf{V}[S_2] + q(\mathsf{E}[S_2])^2 - p^2(\mathsf{E}[S_1])^2 - q^2(\mathsf{E}[S_2])^2 - 2pq\,\mathsf{E}[S_1]\,\mathsf{E}[S_2] \\
&= p\,\mathsf{V}[S_1] + q\,\mathsf{V}[S_2] + pq(\mathsf{E}[S_1])^2 + pq(\mathsf{E}[S_2])^2 - 2pq\,\mathsf{E}[S_1]\,\mathsf{E}[S_2], \quad \text{as } p = 1-q \\
&= p\,\mathsf{V}[S_1] + q\,\mathsf{V}[S_2] + pq(\mathsf{E}[S_1] - \mathsf{E}[S_2])^2.
\end{aligned}$$

s.6.2.1. First check the load.

```
>>> labda = 3  # per hour
>>> ES0 = 15.0 / 60  # hour
>>> ES0
0.25
```

```
>>> ER = 2.0
>>> Bmin = labda * ER / (1 - labda * ES0)
>>> Bmin
24.0
```

```
>>> B = 30
>>> ES = ES0 + ER / B
>>> rho = labda * ES
>>> rho
0.95
```

The time to form a red batch is

```
>>> labda_r = 0.5
>>> EW_r = (B - 1) / (2 * labda_r)
>>> EW_r  # in hours
29.0
```

And the time to form a blue batch is

```
>>> labda_b = labda - labda_r
>>> EW_b = (B - 1) / (2 * labda_b)
>>> EW_b  # in hours
5.8
```

The time a batch spends in queue.

```
>>> Cae = 1.0
>>> CaB = Cae / B
>>> CaB
0.03333333333333333
>>> Ce = 1.0  # SCV of service times
>>> VS0 = Ce * ES0 * ES0
>>> VS0
0.0625
>>> VR = 1.0 * 1.0  # Var setups is sigma squared
>>> VS = B * VS0 + VR
>>> VS
2.875
>>> ESb = B * ES0 + ER
>>> ESb
9.5
>>> CeB = VS / (ESb * ESb)
>>> CeB
0.03185595567867036
>>> EW = (CaB + CeB) / 2 * rho / (1 - rho) * ESb
>>> EW
5.8833333333333275
```

The time to unpack the batch, i.e., the time at the server.

```
>>> Eunpack = ER + (B - 1) / 2 * ES0 + ES0
>>> Eunpack
5.875
```

The overall time red jobs spend in the system.

```
>>> total = EW_r + EW + Eunpack
>>> total
40.758333333333326
```

s.6.2.2. Suppose a batch is just finished. The first job of a new batch needs to wait, on average, $B-1$ inter-arrival times until the batch is complete, the second $B-2$ inter-arrival times, and so on. The last job does not have to wait at all. Thus, the total time to form a batch is $(B-1)/\lambda_r$. An arbitrary job can be anywhere in the batch, hence its expected time is half the total time.

s.6.2.3. The variance of the inter-arrival time of batches is B times the variance of job inter-arrival times. The inter-arrival times of batches is also B times the inter-arrival times of jobs. Thus,

$$C_{a,B}^2 = \frac{B\,\mathsf{V}[X]}{(B\,\mathsf{E}[X])^2} = \frac{\mathsf{V}[X]}{(\mathsf{E}[X])^2}\frac{1}{B} = \frac{C_a^2}{B}.$$

s.6.2.4. The variance of a batch is $\mathsf{V}\left[R+\sum_{i=1}^{B} S_{0,i}\right] = \mathsf{V}[R] + B\,\mathsf{V}[S_0]$, since the normal service times $S_{0,i}$, $i = 1,\ldots,B$, of the jobs are independent, and also independent of the setup time R of the batch.

s.6.2.5. First, wait until the setup is finished, then wait (on average) for half of the batch (minus the job itself) to be served, and then the job has to be served itself, that is, $\mathsf{E}[R] + \frac{B-1}{2}\mathsf{E}[S_0] + \mathsf{E}[S_0]$.

s.6.3.1. First we determine the load.

```
>>> EB = 30
>>> p = 1 / EB
>>> ES0 = 1.5
>>> labda = 9.0 / (2 * 8)  # arrival rate per hour
>>> ER = 5.0
>>> ES = ES0 + p * ER
>>> ES
1.6666666666666667
>>> rho = labda * ES
>>> rho
0.9375
```

So, at least the system is stable.

```
>>> VS0 = 0.5 * 0.5
>>> VR = 2.0 * 2.0
>>> VS = VS0 + p * VR + p * (1 - p) * ER * ER
>>> VS
```

```
1.1888888888888887
>>> Ce2 = VS / (ES * ES)
>>> Ce2
0.4279999999999999
```

And now we can fill in the waiting time formula.

```
>>> Ca2 = 1  # Poisson arrivals
>>> EW = (Ca2 + Ce2) / 2 * rho / (1 - rho) * ES
>>> EW
17.849999999999998
>>> EJ = EW + ES
>>> EJ
19.516666666666666
```

s.6.3.2. We can use the model of Section 6.2.

```
>>> B = 20
>>> ER = 4.5
>>> VR = 0
>>> ES = ES0 + ER / B
>>> ES
1.725
>>> rho = labda * ES
>>> rho
0.9703125
>>> VS = VS0 + VR / B
>>> Ce2 = VS / (ES * ES)
>>> Ce2
0.08401596303297626
>>> EW = (Ca2 + Ce2) / 2 * rho / (1 - rho) * ES
>>> EW
30.55855263157897
>>> EJ = EW + ES
>>> EJ
32.28355263157897
```

Comparing this to the results of [6.3.1], we see that the load becomes somewhat higher. Since ρ becomes close to one, doing adjustments regularly is not a good idea.

s.5. Applying adam's and Eve's is straightforward, but the details require attention. Using independence where necessary (and allowed by our assumptions),

$$\begin{aligned}
\mathsf{E}[S|F] &= \mathsf{E}[S_0 + RF|F] = \mathsf{E}[S_0] + F\,\mathsf{E}[R]\,,\\
\mathsf{E}[S] &= \mathsf{E}[\mathsf{E}[S|F]] = \mathsf{E}[S_0] + \mathsf{E}[F]\,\mathsf{E}[R] = \mathsf{E}[S_0] + p\,\mathsf{E}[R]\,,\\
\mathsf{V}[\mathsf{E}[S|F]] &= \mathsf{V}[\mathsf{E}[S_0] + F\,\mathsf{E}[R]] = (\mathsf{E}[R])^2\,\mathsf{V}[F] = (\mathsf{E}[R])^2 p(1-p),\\
\mathsf{V}[S|F] &= \mathsf{V}[S_0 + RF|F] = \mathsf{V}[S_0] + F\,\mathsf{V}[R]\,,\\
\mathsf{E}[\mathsf{V}[S|F]] &= \mathsf{V}[S_0] + p\,\mathsf{V}[R]\,,\\
\mathsf{V}[S] &= \mathsf{E}[VS|F] + \mathsf{V}[\mathsf{E}[S|F]] = \mathsf{V}[S_0] + p\,\mathsf{V}[R] + p(1-p)(\mathsf{E}[R])^2.
\end{aligned}$$

s.6.4.1. Let's first check that $\rho < 1$.

```
>>> labda = 4.0
>>> ES0 = 10.0 / 60  # in hours
>>> labda_f = 1.0 / 3
>>> ER = 30.0 / 60  # in hours
>>> A = 1.0 / (1 + labda_f * ER)
>>> A
0.8571428571428571
>>> ES = ES0 / A
>>> ES
0.19444444444444445
>>> rho = labda * ES
>>> rho
0.7777777777777778
```

```
>>> Ca2 = 1.0
>>> C02 = 0.0  # deterministic service times
>>> Ce2 = C02 + 2 * A * (1 - A) * ER / ES0
>>> Ce2
0.7346938775510207
>>> EW = (Ca2 + Ce2) / 2 * rho / (1 - rho) * ES
>>> EW
0.5902777777777779
>>> EW + ES  # = EJ
0.7847222222222223
```

s.6.4.2. The time to fail is the time in between two interruptions. By assumption, the failure times are $\mathrm{Exp}(\lambda_f)$, hence $m_f = 1/\lambda_f$. The expected duration of an interruption is $\mathsf{E}[R]$. With this

$$A = \frac{m_f}{m_f + \mathsf{E}[R]} = \frac{1/\lambda_f}{1/\lambda_f + \mathsf{E}[R]} = (1 - \lambda_f \mathsf{E}[R])^{-1}.$$

s.6.4.3. When doing the computations by hand, I worked from bottom to top. However, for the solutions, the current sequence is perhaps easier understand.

$$\begin{aligned}
\mathsf{E}[S|S_0, N] &= \mathsf{E}\left[S_0 + \sum_{i=1}^{N} R \middle| S_0, N\right] = S_0 + N\mathsf{E}[R], \text{ since } \{R_i\} \text{ are iid,} \\
\mathsf{E}[S|S_0] &= \mathsf{E}[\mathsf{E}[S|S_0, N]|S_0] = \mathsf{E}[S_0 + N\mathsf{E}[R]|S_0] = S_0 + \lambda_f S_0 \mathsf{E}[R] \\
\mathsf{E}[S] &= \mathsf{E}[\mathsf{E}[S|S_0]] = \mathsf{E}\left[S_0(1 + \lambda_f \mathsf{E}[R])\right] = \mathsf{E}[S_0](1 + \lambda_f \mathsf{E}[R]) = \mathsf{E}[S_0]/A. \\
\mathsf{V}[S|S_0, N] &= \mathsf{V}\left[S_0 + \sum_{i}^{N} R_i | S_0, N\right] = N\mathsf{V}[R], \text{ as } \mathsf{V}[S_0|S_0] = 0, \\
\mathsf{V}[S|S_0] &= \mathsf{E}[\mathsf{V}[S|S_0, N]|S_0] + \mathsf{V}[\mathsf{E}[S|S_0 N]|S_0] = \lambda_f S_0 \mathsf{V}[R] + \mathsf{V}[S_0 + N\mathsf{E}[R]|S_0] \\
&= \lambda_f S_0 \mathsf{V}[R] + (\mathsf{E}[R])^2 \mathsf{V}[N|S_0] = \lambda_f S_0 \mathsf{V}[R] + (\mathsf{E}[R])^2 \lambda_f S_0, \\
\mathsf{V}[S] &= \mathsf{E}[\mathsf{V}[S|S_0]] + \mathsf{V}[\mathsf{E}[S|S_0]] = \lambda_f (\mathsf{V}[R] + (\mathsf{E}[R])^2)\mathsf{E}[S_0] + \mathsf{V}\left[(1 + \lambda_f \mathsf{E}[R])S_0\right] \\
&= \lambda_f \mathsf{E}\left[R^2\right]\mathsf{E}[S_0] + (1 + \lambda_f \mathsf{E}[R])^2 \mathsf{V}[S_0] = \lambda_f \mathsf{E}\left[R^2\right]\mathsf{E}[S_0] + \mathsf{V}[S_0]/A^2.
\end{aligned}$$

s.6.4.4.

$$C_s^2 = \frac{\mathsf{V}[S]}{(\mathsf{E}[S])^2} = \frac{V(S)A^2}{(\mathsf{E}[S_0])^2} = \frac{\mathsf{E}\left[S_0^2\right] + \lambda_f \mathsf{E}\left[R^2\right] \mathsf{E}[S_0] A^2 - (\mathsf{E}[S_0])^2}{(\mathsf{E}[S_0])^2}$$
$$= \frac{\mathsf{E}\left[S_0^2\right] - (\mathsf{E}[S_0])^2}{(\mathsf{E}[S_0])^2} + \frac{\lambda_f \mathsf{E}\left[R^2\right] \mathsf{E}[S_0] A^2}{(\mathsf{E}[S_0])^2} = C_0^2 + \frac{\lambda_f \mathsf{E}\left[R^2\right] A^2}{\mathsf{E}[S_0]}.$$

s.6.4.5. When repair times are exponentially distributed with mean $\mathsf{E}[R]$ we have that $\mathsf{E}\left[R^2\right] = 2(\mathsf{E}[R])^2$. Since $A = 1/(1+\lambda_f \mathsf{E}[R])$,

$$\lambda_f \mathsf{E}\left[R^2\right] A^2 = 2\lambda_f (\mathsf{E}[R])^2 A^2 = 2\frac{\lambda_f \mathsf{E}[R]}{1+\lambda_f \mathsf{E}[R]} A \mathsf{E}[R]$$
$$= 2\left(1 - \frac{1}{1+\lambda_f \mathsf{E}[R]}\right) A \mathsf{E}[R] = 2(1-A)A\mathsf{E}[R].$$

s.6.5.1. First station 1.

```
>>> labda = 2.0
>>> S1 = 20.0 / 60
>>> rho1 = labda * S1
>>> ca1 = 2.0
>>> cs1 = 0.5
>>> EW1 = (ca1 + cs1) / 2 * rho1 / (1 - rho1) * S1
>>> EJ1 = EW1 + S1
>>> EJ1
1.1666666666666665
```

Now station 2. We first need to compute C_{d1}^2.

```
>>> cd1 = (1 - rho1 ** 2) * ca1 + rho1 ** 2 * cs1
>>> cd1
1.3333333333333335
>>> labda = 2
>>> S2 = 25.0 / 60
>>> rho2 = labda * S2
>>> ca2 = cd1  # here we use our formula
>>> cs2 = 0.5
>>> EW2 = (ca2 + cs2) / 2 * rho2 / (1 - rho2) * S2
>>> EJ2 = EW2 + S2
>>> EJ2
2.3263888888888897
>>> EJ1 + EJ2
3.4930555555555562
```

BIBLIOGRAPHY

S. Asmussen. *Applied Probability and Queues.* Springer-Verlag, Berlin, 2003.

M. El-Taha and S. Stidham Jr. *Sample-Path Analysis of Queueing Systems.* Kluwer Academic Publishers, 1998.

W.J. Hopp and M.L. Spearman. *Factory Physics.* Waveland Press, Inc., 3rd edition, 2008.

N. D. Van Foreest and O.A. Kilic. An intuitive approach to inventory control with optimal stopping. *European Journal of Operational Research*, 311(3):921–924, 2023.

A.A. Yushkevich and E.B. Dynkin. *Markov Processes: Theorems and Problems.* Plenum Press, 1969.

NOTATION

a_k = Number of arrivals in kth period
$A(t)$ = Number of arrivals in $[0, t]$
A_k = Arrival time of kth job
$\tilde{A}_k$ = Start of service of kth job
B = General batch size
B_k = Batch size at kth arrival time
c_k = Service/production capacity in kth period
d_k = Number of departures in kth period
c = Number of servers
C = General batch size during a service time
C_a^2 = Squared coefficient of variation of inter-arrival times
C_s^2 = Squared coefficient of variation of service times
$D(t)$ = Number of departures in $[0, t]$
D_k = Departure time of kth job
δ = Departure rate
F = Distribution of service time of a job
I = Idle time of single server
J_k = Sojourn time of kth job
$L(t)$ = Number of customers/jobs in system at time t
L_k = Number in system as end of kth period
$L_s(t)$ = Number of customers/jobs in service at time t
λ = Arrival rate
μ = Service rate
$N(t)$ = Number of arrivals in $[0, t]$
$N(s, t)$ = Number of arrivals in $(s, t]$
$p(n)$ = Long-run time average that system contains n jobs
$\pi(n)$ = Stationary probability that an arrival sees n jobs in system
$Q(t)$ = Number of customers/jobs in queue at time t
Q_k = Queue length as seen by kth job, or at end kth period
ρ = utilization of a single server
S = Generic service time
S_k = Service time of kth job
U = Busy time of single server
W = Generic waiting time (in queue)
W_k = Waiting time in queue of kth job
X = Generic inter-arrival time between two consecutive jobs
X_k = Inter-arrival time between job $k-1$ and job k